Appunti di Networking

Alessio Novel

Internet and computer networks (introduction)....5
What is internet?....5
The nuts-and-bolts view....5
The services view....6
What is a protocol?....6
Network edge....7
Access network....7
Physical media....9
Network core....11
Packet switching....11
Circuit switching....12
Network of networks....14
Performance (delay, loss and throughput) in packet switched networks....17
Delay....17
Packet loss....19
Throughput....20
Security....21
Packet sniffing....21
Fake identity....22
Denial of Service....22
Lines of defense....23
Protocol layers and their service models....23
Application layer....24
Transport layer....25
Network layer....25
Link layer....25
Physical layer....25
Encapsulation....26
History....27
The development of packet switching: 1961-1972....27
Proprietary Networks and Internetworking: 1972–1980....28
A Proliferation of Networks and new protocols: 1980–1990....29
The Internet Explosion: The 1990s....29
The new millennium....30
Application layer....30
Principles of Network Applications....31
Network Application Architecture....31
Processes Communicating....33
Transport Services Available to Applications....34

Transport Services Provided by the Internet....35
The Web and HTTP....37
Overview of HTTP....37
Non-Persistent and Persistent Connections....38
HTTP message format....40
User-Server Interaction: Cookies....43
Web caching....44
Electronic Mail in the Internet....49
SMTP....50
Comparison with HTTP....52
Mail message format....52
Mail access protocols....53
DNS, The Internet's Directory Service....54
Service provided by DNS....55
Overview of how DNS works....56
DNS Records and Messages....59
DNS security....61
Peer-to-Peer File Distribution....61
Video Streaming and Content Distribution Networks....66
Internet video....66
HTTP Streaming and DASH....67
Content Distribution Networks....69
Socket programming: Creating Network Applications....70
Socket programming with UDP....71
Socket programming with TCP....73
Transport Layer....74
Introduction and Transport-Layer Services....74
Relationship Between Transport and Network Layers....75
Overview of the Transport Layer in the Internet....76
Multiplexing and Demultiplexing....77
Connectionless Transport: UDP....80
UDP Segment Structure....82
UDP Checksum....83
Principles of Reliable Data Transfer....84
Building a Reliable Data Transfer Protocol....85
rdt1.0....86
rdt2.0....86
rdt2.1....88
rdt2.2....89
rdt3.0....89
Pipelined Reliable Data Transfer Protocol....91
Go-Back-N (GBN)....93
Selective Repeat (SN)....95
Connection-Oriented Transport: TCP....97

The TCP connection....98
TCP segment structure....98
Round-Trip Time Estimation and Timeout....101
Reliable Data Transfer....102
Flow control....106
TCP connection management....109
Principles of congestion control....113
The Causes and the Costs of Congestion....113
Approaches to Congestion Control....119
TCP congestion control....121
Fairness....127
Explicit Congestion Notification (ECN): Network-assisted Congestion Control....129
Evolution of transport-layer functionality....130
The Network Layer: Data Plane....132
Overview of Network Layer....132
Forwarding and Routing: The Data and Control Planes....133
Network Service Model....135
What's Inside a Router?....136
Input Port Processing and Destination-Based Forwarding....138
Switching....140
Output Port Processing....142
Where Does Queueing Occur?....143
Packet scheduling....146
Network Neutrality....149
The Internet Protocol (IP): IPv4, Addressing, IPv6, and More....150
IPv4 Datagram Format....150
IPv4 Datagram Fragmentation....153
IPv4 Addressing....154
Network Address Translation (NAT)....161
IPv6....164
Generalized Forwarding and SDN....168
Match....170
Action....171
The Link Layer and LANs....172
Introduction to the Link Layer....172
The Services Provided by the Link Layer....173
Where Is the Link Layer Implemented?....174
Error-Detection and -Correction Techniques....176
Parity Checks....177
Checksumming methods....178
Cyclic Redundancy Check (CRC)....179
Multiple Access Links and Protocols....180
Channel Partitioning Protocols....182
Random Access Protocols....184

Taking-Turns Protocols....190
DOCSIS: The Link-Layer Protocol for Cable Internet Access....191
Switched Local Area Networks....192
Link-Layer Addressing and ARP....193
Ethernet....199
Link-Layer switches....202
Data Center Networking....206

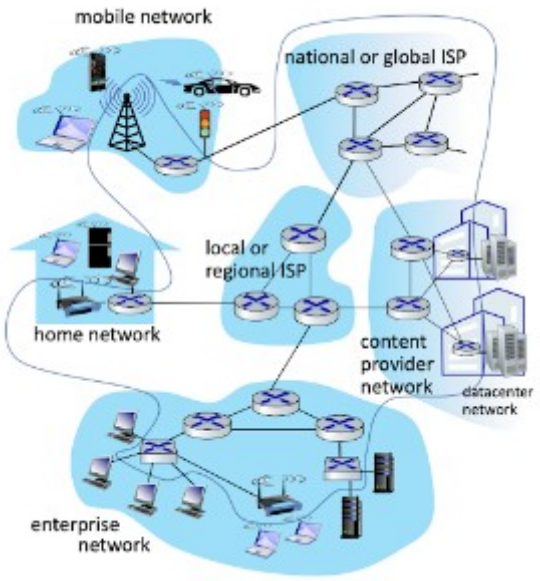
Internet and computer networks (introduction)

What is internet?

The nuts-and-bolts view

At the beginning of the “internet” there were only computers in the network, named in fact **computer network**, nowadays there are many types of device in this network and the name “computer network” is a bit dated.
The fundamental components of internet are:

- Billions of connected **devices**
    - **Hosts**, the end systems
    - **Packet switches**, the devices that forward the informations (router, switch)
- The devices running **network applications** at internet’s edge
- The **communication links**, that are the physical connections like fiber, radio, satellite
    - Different links can transmit data at different rates, the transmission rate (or **bandwidth**) are measured in bits/second
- The **networks**, a collection of devices, routers and links **managed by an organization**

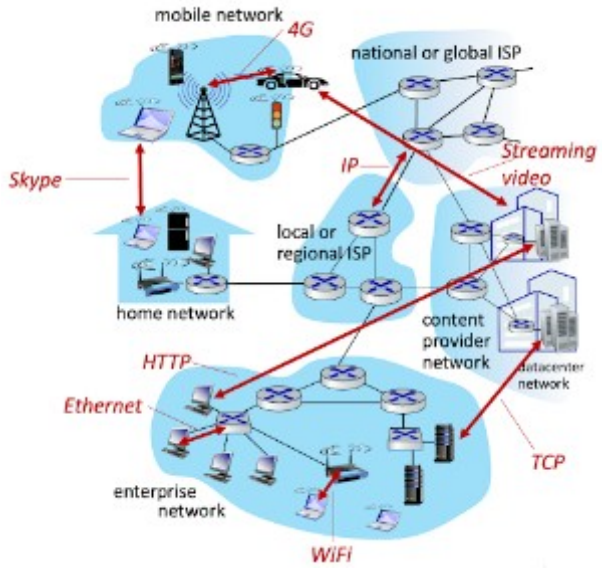


When a end system has data to send to another end system, data is added for each layer and the result is called **packet** (chunks of data)
When you want to share a video on the internet, you have to transmit a sequence of undefined length of bits and this isn’t possible, so the bits are divided in sequences of a defined length and are inserted into a packet

- **ISP**, internet service provider, that not always have their own network but may use the network of other provider (telephone company, universities, etc)
  The end systems access to internet through an ISP
- **Protocols** that manage and specify how to do the things, they control the sending and receiving of the information through the internet.

You can’t connect to internet without using protocols (some example of protocols are Skype, HTTP, TCP, IP, WiFi, Ethernet)
The end device must follow the protocol if they want to communicate in the internet

- The entity that formalizes the protocols, that make **internet standards**, is the **IETF** (Internet Engineering Task Force). Their standard documents are called **request for comments** (the name came from their first scope, request comments to resolve problems with the internet)

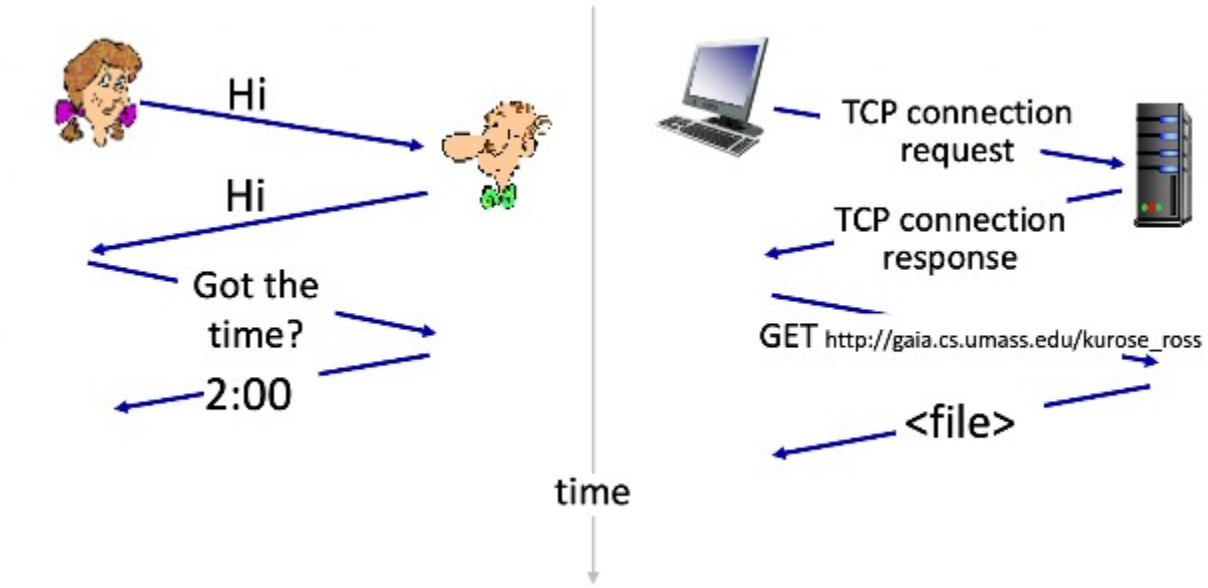


The services view

We can see the internet also with a “services” view, as an infrastructure that provides services to applications
The applications are said to be distributed applications, since they involve multiple end systems that exchange data with each other.
Important: Internet applications run on end devices and not on the packet switches

What is a protocol?

A protocol is a defined set of rules, to understand better let’s make an example with human protocol.
For example when we have to ask for the time, we start with an “Hi”, and only if the other person shows his availability responding “Hi” we can continue with asking the time, otherwise we stop because he doesn’t “say” to us that he’s ready/available. A network protocol is similar to a human protocol, except that the entities exchanging messages and taking actions are hardware or software components of some device



The **protocols** are based on control messages and specific actions that have to be taken when a message arrives, so they establish the procedures.
Protocols define the format, order of messages sent and received among network entities, and actions taken on message transmission/receipt

Network edge

In the previous section we talked about a high-level overview of the internet and network protocols, in this chapter we will see a bit more deeply the components of a computer network, starting with the edge: clients and servers (or both called **hosts**)
Servers are often not close to the client, but they're located in data centers.
We have seen that computers, servers, mobile phones and a lot of other devices (IoT) are also called end systems, that's because they sit at the edge of the network.

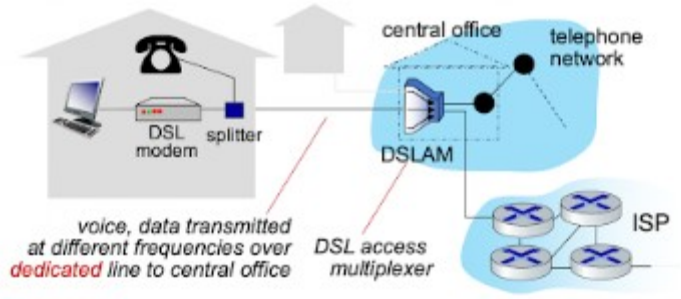
Access network

Let's now consider the **access network**, that is to say the network that physically connects an end system to the first router (also known as the "**edge router**").

We can connect an end system to the edge router in different ways:

- Residential access nets
- Institutional access network
- Mobile access network (WiFi, 4G/5G)

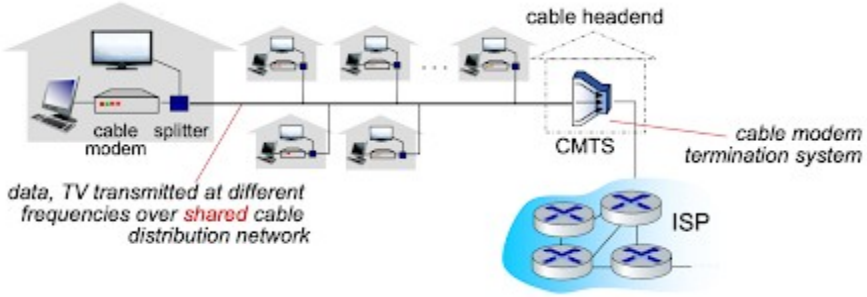
Today the two prevalent types of broadband residential access are **DSL** (digital subscriber line) and **cable-based**. The first use the cable of the telephone and the second the cable of the television



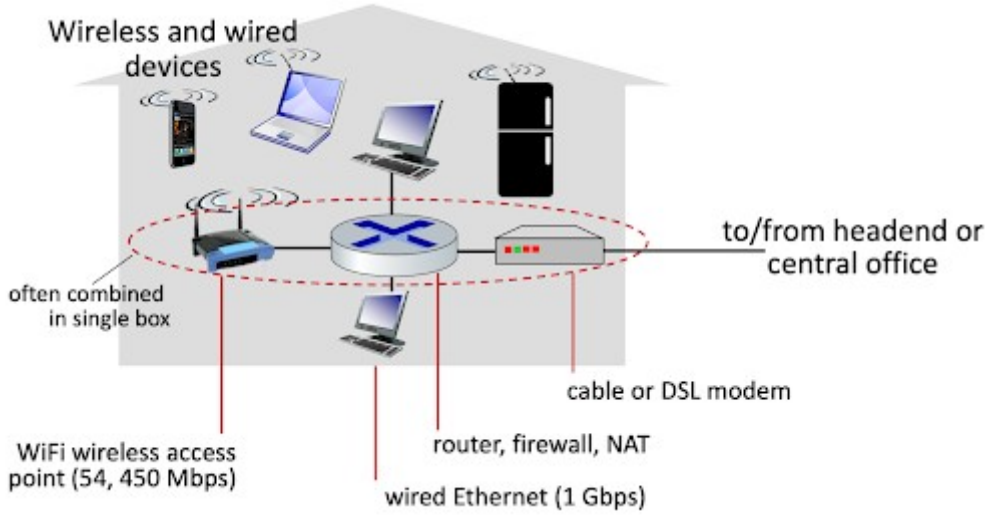
The DSL modem uses the existing telephone line to exchange data with the DSLAM (digital subscriber line access multiplexer) located in the telephone company's local central office.
The DSLAM separates the data and phone signals and sends the data into the Internet.
For transmit both data and traditional telephone signals simultaneously, the signals are transmitted at different frequencies:

- A high-speed downstream channel, in the 50 kHz to 1 MHz band (24-52 Mbps)
- A medium-speed upstream channel, in the 4 kHz to 50 kHz band (3.5-16 Mbps)
- An ordinary two-way telephone channel, in the 0 to 4 kHz band

The access is said to be asymmetric, because upstream and downstream have different rates.



In this case there is a hybrid solution, because there is fiber till the cable headend and then there is the coaxial cable, so it's called **HFC** (hybrid fiber coax).
The CMTS is similar to DSLAM, obviously it does what the DSLAM does with the phone signals but with the television signals.

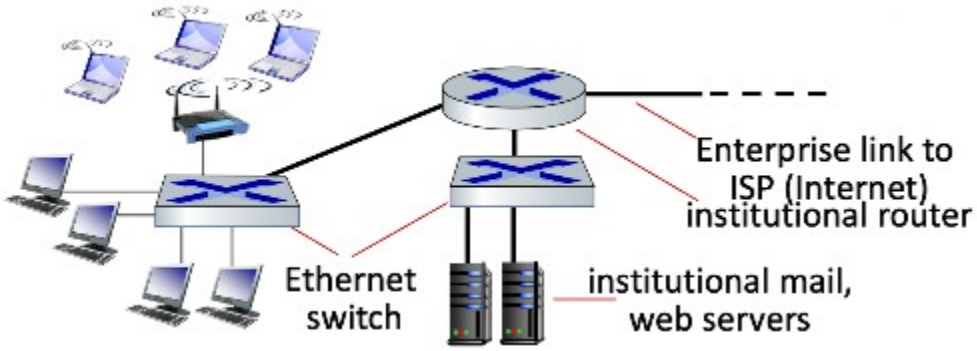


In the red circle we have in order:

- The base station (the wireless access point)
- The router, that gives cabled connection
- The cable/DSL modem

The smartphone can also connect to an edge router via wide-area cellular network, the device needs to be within 10km (instead of 100m of the WLAN) but it will have lower transmission rate

In the university we have an **enterprise network** because we have switches and routers and a mix of wired and wireless link technologies



These schemes is used by big enterprises
In the **data center networks** there are connected thousands of server together, with high-bandwidth links (100s Gbps)

How does the host communicate with the network? The hosts sends **packet** of data:

1. They take the application message
2. They divide it into smaller chunks (the packets) of a fixed length, L bits (usually one thousand)
3. Then they transmit this packets into the access network at transmission rate R (or link capacity, or link bandwidth)

So if we want to know the packet transmission delay, that is to say the time needed to transmit L-bit packet into a link, is $\frac{L}{R}$

Physical media

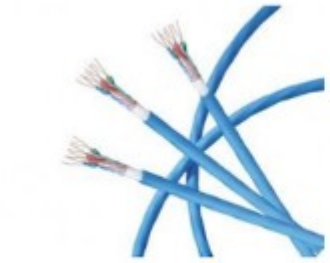
So, we have a bit that must be propagated between a transmitter/receiver pair.
The **physical link** is what lies between the transmitter and the receiver, we can divide them in two categories:

- Guided media
  When the signals propagate in solid media like copper, fiber, coax
- Unguided media
  When the signals propagate freely, like the radio or the WiFi

The cost of the physical media is very lower than the cost of installation of them, for this reason many builders install the physical link in every room of a building, even if only one media is initially used.

Let’s list some type of physical media:

The first media used for wired interconnection of devices was **Twisted pair** that is composed by two insulated **copper wires** (fili di rame) and the bandwidth is 10 Gbps with Ethernet
This is the least expensive and most commonly used guided transmission (it’s the same of telephone networks)
It exits also UTP (Unshielded Twisted Pair)




Then we have the **coaxial cable** that consists of two **copper conductors**. It's quite common in cable television systems, in the network where all the end systems are connected to the coaxial cable, every end systems receive every message from the others.
It’s a broadband and has multiple frequencies channel on cable, every channel reach 100s Mbps



Then we have **fiber optic cable**, that is composed of glass fiber carrying light pulses, each pulse is a bit. It’s very high speed (100s Gbps) and has **low error rate** (is immune to electromagnetic noise)
Light on=1, light off=0, the fiber optic can transmit over long distances without distortion, unlike the coaxial cable for example, that for the long distance have a lot of distorsions



Then we have **wireless radio** links, where the signal is carried in various **bands** in the electromagnetic spectrum, it is half-duplex (only sender to receiver) but we have a lot of problems with the environment effects like reflection, obstruction by objects and interference/noise.
The electromagnetic spectrum goes from 1 KHz to 300 GHz
Some radio links are:

- Terrestrial radio channels

    - Bluetooth - short distances, limited rates
    - Wireless LAN (WiFi) - 100’s Mbps, 10’s meters
    - Wide-area (4G) - 10’s Mbps, 10 Km
    - Terrestrial microwave - Point to point, 45 Mbps per channel
- Satellite - a little delay (270ms), 45 Mbps per channel

The main difference between cable propagation and wireless propagation is the affidability of the propagation, in a cable one is well-known but in a wireless one you could have some obstacles that make the signal worse. So in the wireless connection the propagation is variable and is not known at priory

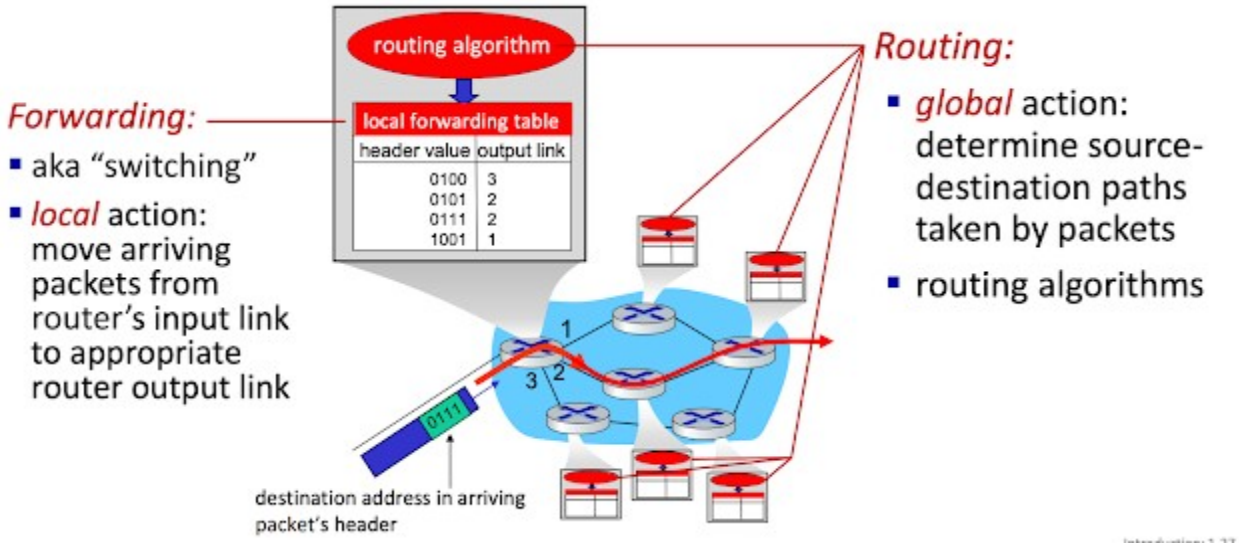
Network core

The part that the normal users will never see because it is placed “behind the scenes”.

Packet switching

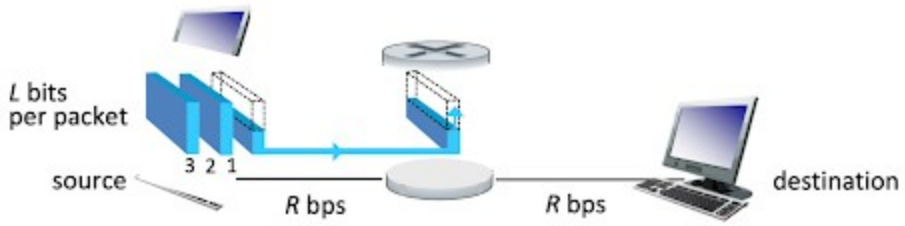
There is a mesh of **interconnected routers** and there is the **packet switching**, so every host “breaks” the application-layer messages into **packets** that travel through the **packet switches**.
The “two key” network-core functions are:

- **Forwarding** (or switching)
  It is a **local** action, means that it is done inside the router, moving arriving packet in an input line to appropriate output line
- **Routing**
  It is a **global** action, it consists in determining source-destination paths taken by packets (there exists a lot of routing algorithms)

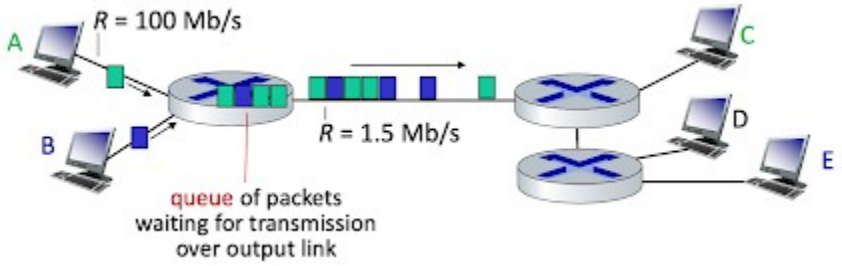


The routing algorithm decides for a router input link which is the best router output link for every router
The **store-and-forward** transmission means that the packet switch must receive the entire packet before it can begin to transmit the first bit of the packet to the output link.

Obviously the operation of forwarding requires some amount of time, that is $\frac{L}{R}$ seconds to transmit L-bit packet into link at R bps

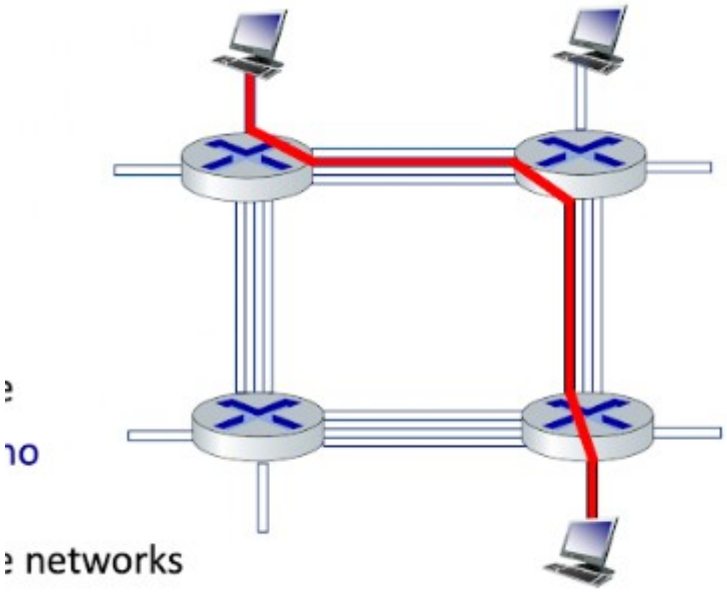


Because of this time, it is normal that in a packet switcher there forms a **queue**, because work arrives faster than it can be serviced (more precisely the arrival rate in bps is higher than transmission rate for some period of time), this queue is called **output buffer**. When the output buffer is completely full of other packets, the **packet loss** will occure.



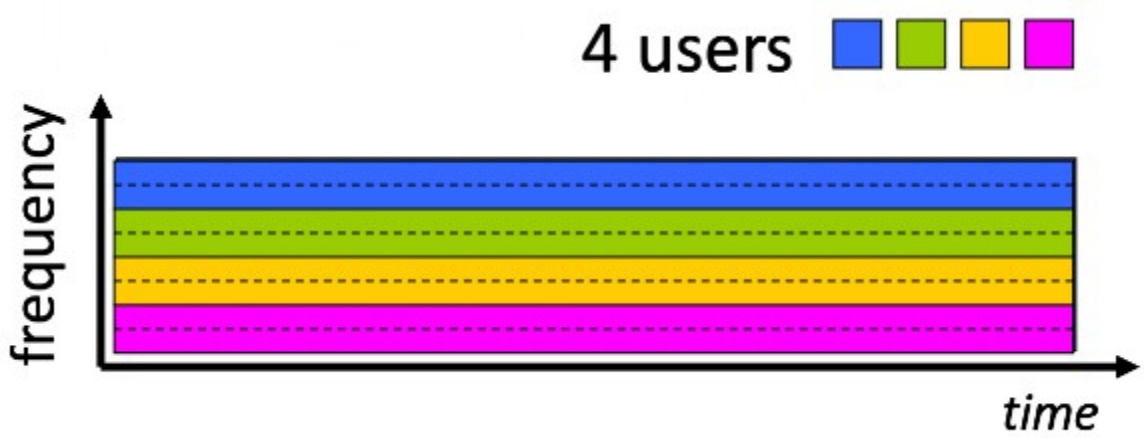
Circuit switching

In this approach, the provided resources needed along a path for the communication are **reserved** for the duration of the session. An example of circuit switching connection is the traditional telephone, where the network reserves a constant transmission rate only for that conversation.
This approach has guaranteed performance and in fact does have no queue, but the circuit segment idle if it is not used by call (**waste of resources**)



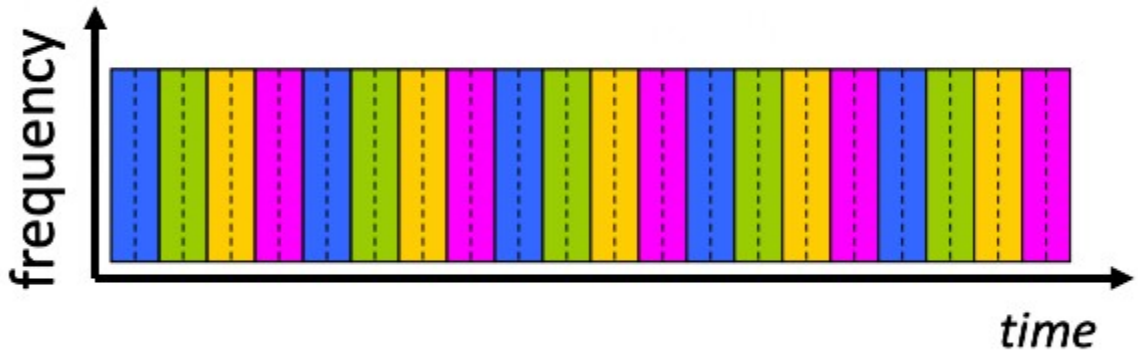
To implement circuit switching into a link we can use two techniques:

- **FDM** (Frequency Division Multiplexing)
  The spectrum of the link is divided up among the connections across the link. The link dedicates a frequency band to each connection for the duration of the connection, and the width of the band is called bandwidth (for example each radio station has his own specific frequency band for his transmission)



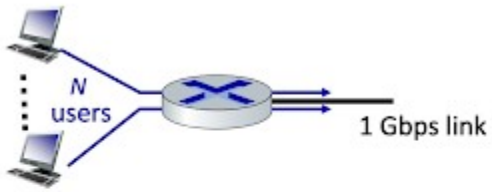
- **TDM** (Time Division Multiplexing)
  The time is divided into frames of fixed duration, based on the number of connections the frame is divided into slots, and in every slot the connection can use all of the wider

bandwidth to transmit



The packet switching is better than the circuit switching, speaking about waste of resources, in fact in the circuit switching the dedicated circuits are idle during the silent periods (when one person in a telephone call stops talking for example).
But for example for real-time services the circuit switching is the best.
It depends on the case, let's see an example of when is better packet switching, because in this case you could make 10 separate line with circuit switching, but the probability of having more than 10 users online at the same time is near to 0, so with the using of packet switching we can raise the number of total user (35 for example has a good probability)

example:
- 1 Gb/s link
- each user:
  - 100 Mb/s when "active"
  - active 10% of time



Q: how many users can use this network under circuit-switching and packet switching?

- *circuit-switching:* 10 users
- *packet switching:* with 35 users, probability > 10 active at same time is less than .0004 *

Q: how did we get value 0.0004?

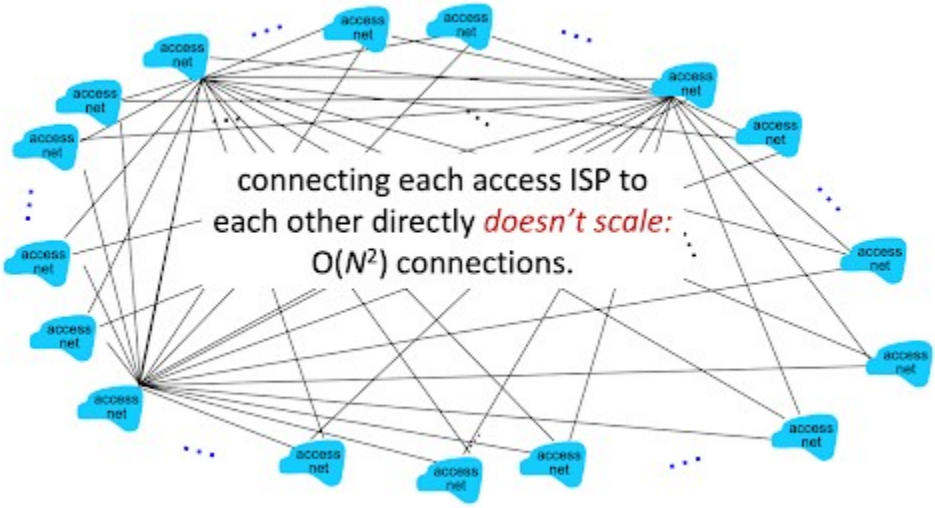
A: HW problem (for those with course in probability only)

So, packet switching is better for "bursty" data (sometimes has data to send but often not), but we risk a delay and the packet loss (we need some protocol for reliable data transfer and congestion control)

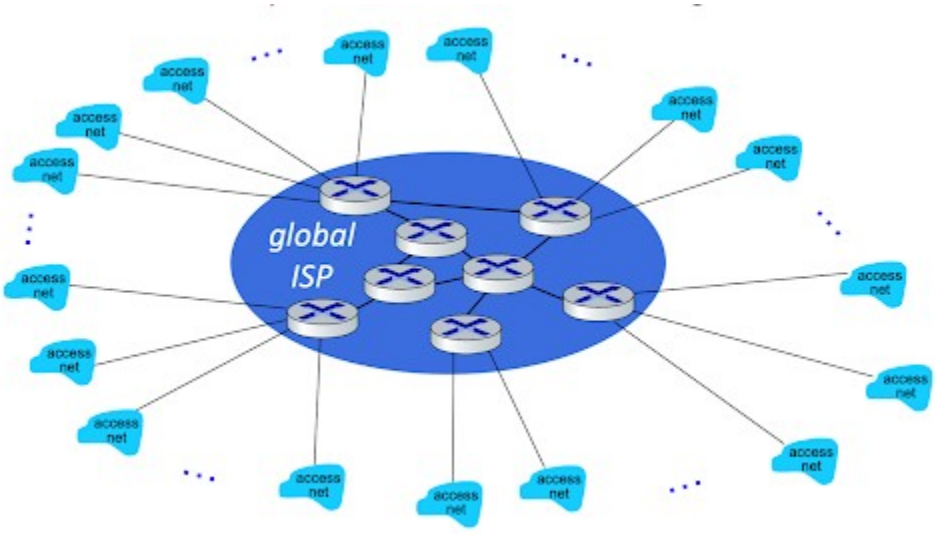
Network of networks

We saw that end systems connect into the Internet via an access ISP. But two end systems that access 2 different ISPs can communicate? The answer is obviously yes and to do it the ISPs

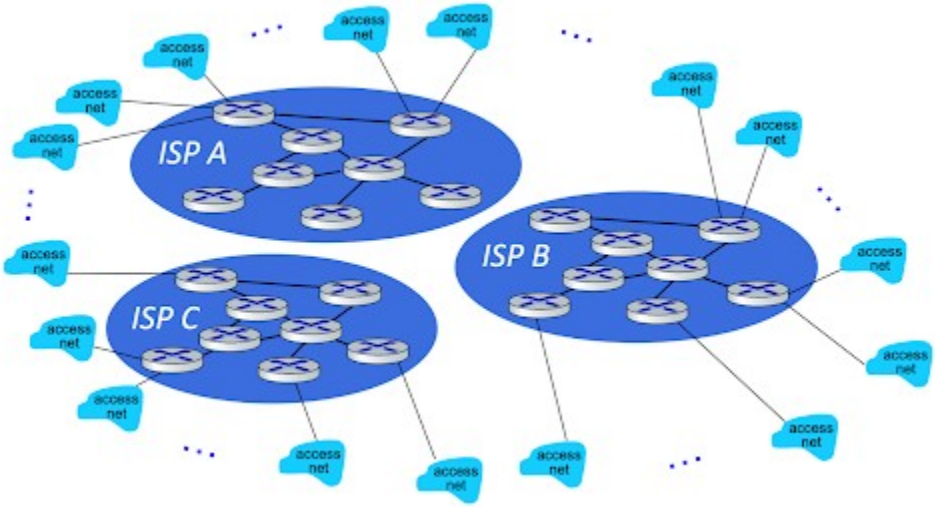
must be interconnected, forming a **network of networks** that is very complex (the evolution of this network was/is driven by economics and national policies).
We can think that each access ISP is connected to each other, but this is also impossible with the actual number of access ISPs.



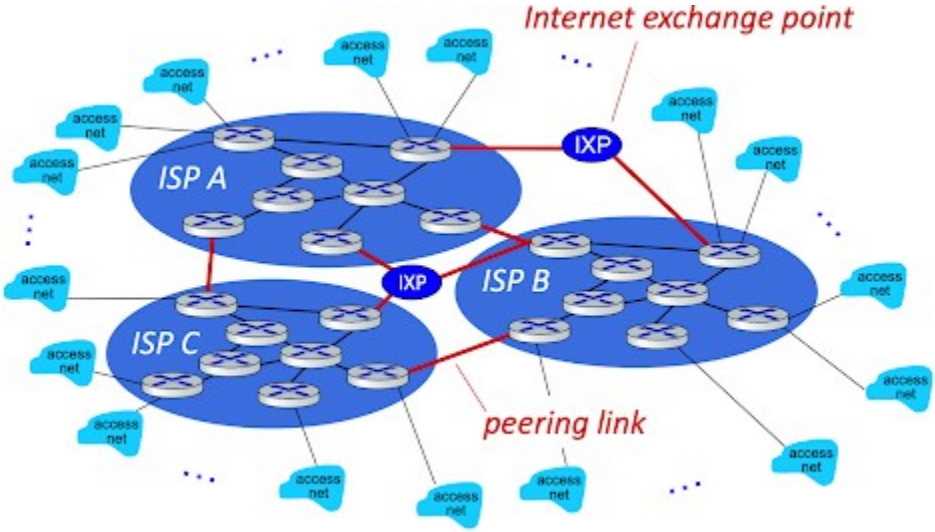
So we can think to a single global transit ISP that interconnects all the access ISPs, with economic agreement with access ISP(customer) and global ISP(provider) (but it would be very costly and difficult to manage)



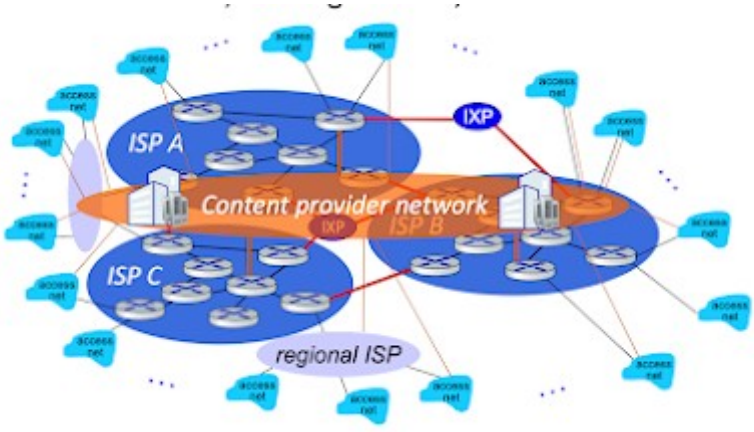
But if a global ISP is a viable business, then other company will build their own global ISP to compete the business



So now the various global ISPs are competitors, but they need to be interconnected to satisfy the hosts needs, so they are competitors who want to be connected.



Now content provider networks may run their own network to bring services close to end users



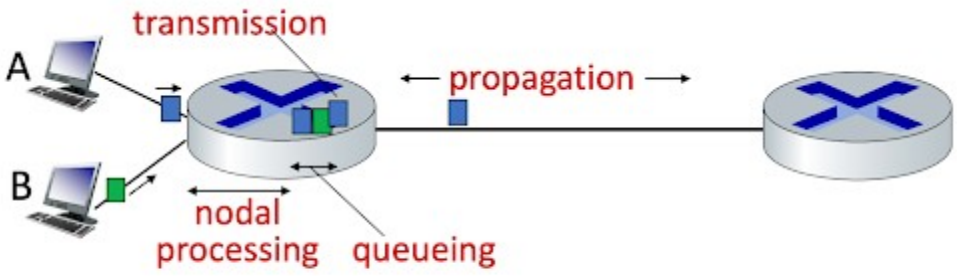
Performance (delay, loss and throughput) in packet switched networks

Ideally, we would like Internet services to be able to move as much data as we want between any two end systems, instantaneously, without any loss of data. But this scenario is impossible due to the **physical limits** of our world.
We have three parameters to calculate the performance of a network: delay, loss and throughput.

Delay

As the packet travels from one node to another one till the destination, it suffers from several types of **delays** at each node **along the path**
The most important of these delays are:

- **nodal processing** delay
  Time required to examine the packet's header, to determine output link, to check bit errors.
  After this nodal processing, the router directs the packet to the queue that precedes the output link to the next node
- **queueing** delay
  The waiting of the packet to be transmitted onto the link. The length of the queueing delay depends on the earlier-arriving packets (congestion) that are waiting for the transmission onto the same link. The delay is zero if there are no packets in the queue
- **transmission** delay
  The amount of time required to push (that is, transmit) all of the packet's bits into the link
  It is deterministic: if the packet is $L \ \ bits$ length and the transmission rate from the actual node to the next is $R \ \ bits/sec$, the transmission delay will be $\frac{L}{R} \ \ secs$
- **propagation** delay
  The time required to propagate from the beginning of the link to the next router, it depends on the physical medium of the link. The propagation speed is a little less than the speed of light. In the fiber the propagation speed is not exactly the speed of the light, but it is divided by a reflection factor.
  The propagation delay is the distance between two node (source and destination) divided by the propagation speed $\frac{d}{s}$ (distance divided by the speed of the link)

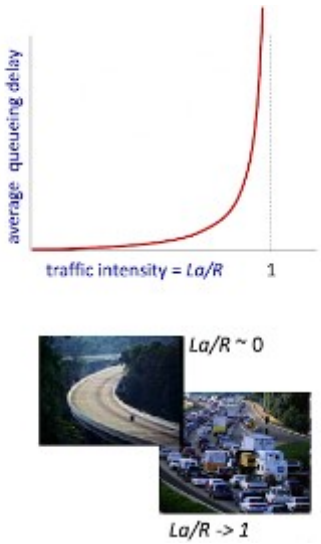


$$d_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$$

Together these delays accumulate to give a total **nodal delay**.
The transmission and the propagation delay are very different (caravan analogy)
The processing delay is often negligible, however it strongly influences a router's maximum **throughput**, which is the maximum rate at which a router can forward packets.

Unlike the other three delays, the queuing delay can vary from packet to packet (if ten packets arrive, the first will have 0 delay, and the last a large queueing delay). So to calculate it, statistical measures are used.
So, let's denote with $a \ \ (packets/secs)$ the average rate at which packets arrive at the queue, recalling $R \ \ (bits/secs)$ the transmission rate and let's suppose that all the packets consist of $L \ \ bits$
Then the average rate at which bits arrive at the queue is $L \cdot a$, and finally assume that the queue is very big, so that it can hold an infinite number of bits.
The ratio $\frac{L \cdot a}{R}$ is the **traffic intensity**:

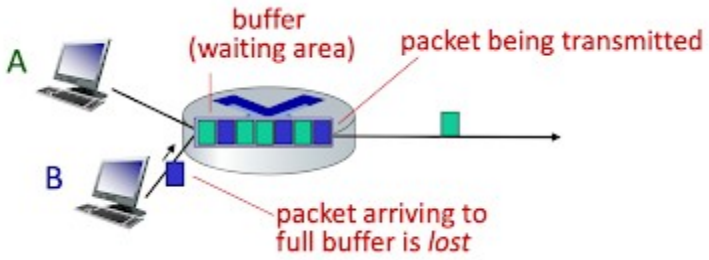
- $\frac{L \cdot a}{R} > 1$
  The average rate at which bits arrive at the queue exceeds the rate at which the bits can be transmitted from the queue (average delay infinite)
- $\frac{L \cdot a}{R} \simeq 1$
  The queueing delay is large
- $\frac{L \cdot a}{R} \simeq 0$
  The queueing delay is small



One important aspect of the figure is the fact that as the traffic intensity approaches 1, the average queuing delay increases rapidly.

Packet loss

In our discussions above, we have assumed that the queue is capable of holding an infinite number of packets. In reality a queue preceding a link has finite capacity, and the packet that arrives at a full queue will be dropped (lost).
Therefore, performance at a node is often measured not only in terms of delay, but also in terms of the probability of packet loss.
As we'll discuss in the subsequent chapters, a lost packet may be retransmitted on an end-to-end basis in order to ensure that all data is eventually transferred from source to destination (TCP). The packet can be retransmitted by the source or by a general previous node
In general the propagation delay is usually larger if the distance is very long



The program "traceroute" is an open source program that provides some delay measurement. For every node i, the program sends 3 packets that will reach i, and the node i will return the packets to the sender. When the packets come back the program will show the time interval between transmission and reply if the packet arrives, or the message of a lost packet.

Real Internet delays and routes

traceroute: gaia.cs.umass.edu to www.eurecom.fr

3 delay measurements from gaia.cs.umass.edu to cs-gw.cs.umass.edu

3 delay measurements to border1-rt-fa5-1-0.gw.umass.edu

trans-oceanic link

looks like delays *decrease*! Why?

* means no response (probe lost, router not replying)

```
1  cs-gw (128.119.240.254)  1 ms  1 ms  2 ms
2  border1-rt-fa5-1-0.gw.umass.edu (128.119.3.145)  1 ms  1 ms  2 ms
3  cht-vbns.gw.umass.edu (128.119.3.130)  6 ms 5 ms 5 ms
4  jn1-at1-0-0-19.wor.vbns.net (204.147.132.129)  16 ms 11 ms 13 ms
5  jn1-so7-0-0-0.wae.vbns.net (204.147.136.136)  21 ms 18 ms 18 ms
6  abilene-vbns.abilene.ucaid.edu (198.32.11.9)  22 ms  18 ms  22 ms
7  nycm-wash.abilene.ucaid.edu (198.32.8.46)  22 ms  22 ms  22 ms
8  62.40.103.253 (62.40.103.253)  104 ms 109 ms 106 ms
9  de2-1.de1.de.geant.net (62.40.96.129)  109 ms 102 ms 104 ms
10 de.fr1.fr.geant.net (62.40.96.50)  113 ms 121 ms 114 ms
11 renater-gw.fr1.fr.geant.net (62.40.103.54)  112 ms  114 ms  112 ms
12 nio-n2.cssi.renater.fr (193.51.206.13)  111 ms  114 ms  116 ms
13 nice.cssi.renater.fr (195.220.98.102)  123 ms  125 ms  124 ms
14 r3t2-nice.cssi.renater.fr (195.220.98.110)  126 ms  126 ms  124 ms
15 eurecom-valbonne.r3t2.ft.net (193.48.50.54)  135 ms  128 ms  133 ms
16 194.214.211.25 (194.214.211.25)  126 ms  128 ms  126 ms
17 * * *
18 * * *
19 fantasia.eurecom.fr (193.55.113.142)  132 ms  128 ms  136 ms
```

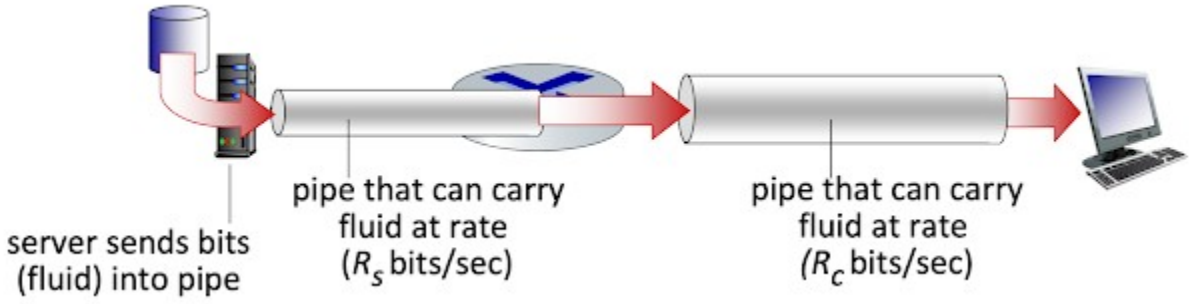
* Do some traceroutes from exotic countries at www.traceroute.org

Throughput

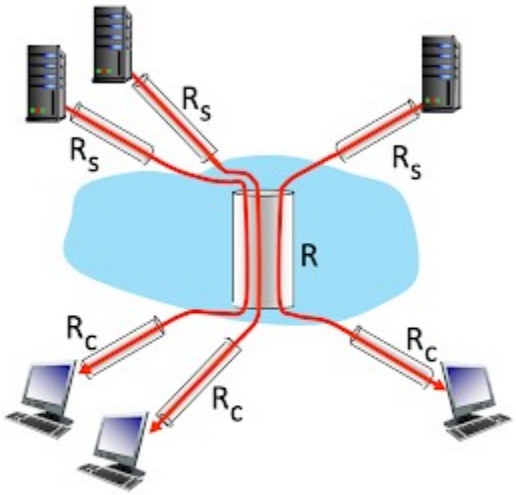
In addition to delay and packet loss, another critical performance measure in computer networks is end-to-end **throughput**.

The **instantaneous throughput** at any instant of time is the rate (in bits/sec) at which a host B is receiving the file from host A. (Many applications, including many P2P file sharing systems, display the instantaneous throughput during downloads in the user interface).

The **average throughput** of the file transfer is $\frac{F}{T}$ in bits/sec, where F is the bits of the file and T is the seconds taken for the transfer of all F bits.



The throughput of a system is the minimum of the throughput of the single links (**bottleneck link**).



10 connections (fairly) share backbone bottleneck link *R* bits/sec

- per-connection end-end throughput: $min(R_c, R_s, R/10)$
- in practice: R_c or R_s is often bottleneck

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/

In a case of shared link, the transmission rate is divided by the number of connections alive in that moment.

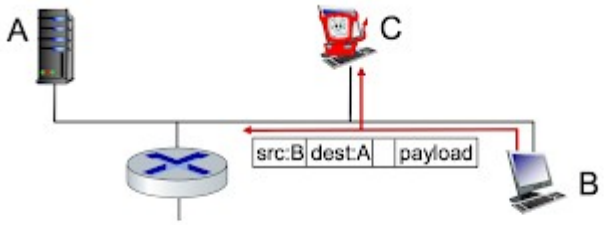
Security

The field of network security is about **how the bad guys can attack computer networks** and about how we can **defend networks against those attacks**, or better yet, design **new architectures** that are **immune to such attacks** in the first place.
Originally there wasn't security in mind, because the network was small and only the person who could access the machines were allowed to access the network, and the original vision was: “a group of mutually trusting users attached to a transparent network”, a model in which there is no need of security.

Unfortunately, along with all that good stuff comes malicious stuff, collectively known as **malware**, that can also enter and infect our devices. Much of the malware out there today is **self-replicating** (viruses (some sort of user interaction is needed) and worms (without any need of user interaction)).

Packet sniffing

Many users today access the Internet via broadcast media, like the wireless ones. While ubiquitous Internet access is extremely convenient and enables marvelous new applications for mobile users, it also creates a major security vulnerability, by placing a passive receiver in the vicinity of the wireless transmitter, that receiver, called **packet sniffer**, can obtain a copy of every packet that is transmitted!

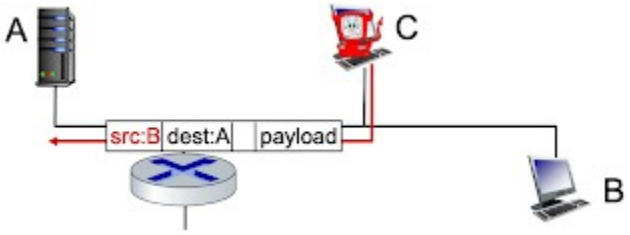


Wireshark software used for our end-of-chapter labs is a (free) packet-sniffer

Because packet sniffers are passive (that is, they do not inject packets into the channel) they are difficult to detect.

Fake identity

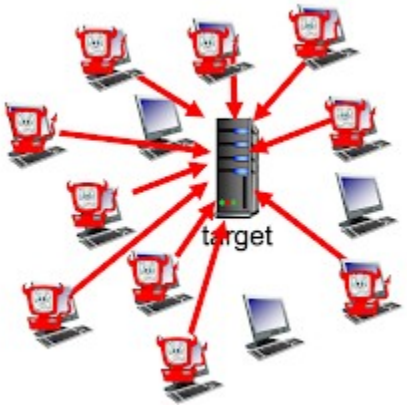
It is surprisingly easy to create a packet with an arbitrary source address, packet content, and destination address and then transmit this hand-crafted packet into the Internet.
Imagine the unsuspecting receiver (say an Internet router) who receives such a packet, takes the (false) source address as being truthful, and this is known as **IP spoofing**, and is one of many ways in which one user can masquerade as another user.



Denial of Service

Another broad class of security threats are known as **denial-of-service** (**DoS**) **attacks**. As the name suggests, a DoS attack renders a network, host, or other piece of infrastructure unusable by legitimate users. Some examples are **vulnerability attack** (to the user), **bandwidth flooding** (filling in the link) and **connection flooding** (establish a large number of TCP connections)

1. select target
2. break into hosts around the network (see botnet)
3. send packets to target from compromised hosts



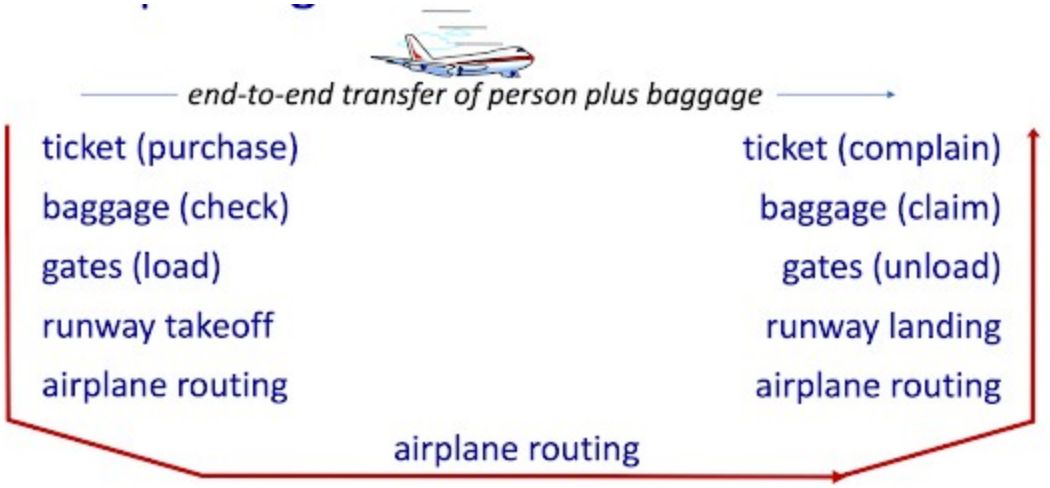
Lines of defense

There are several types of defense, for every type of threat.
Let's see the most common and important:

- **Authentication**
  Proving you are who you say you are (with the SIM for example)
- **Confidentiality**
  Via data encryption
- **Integrity checks**
  With digital signatures to prevent/detect tampering (manomissioni)
- **Access restriction**
  With password or with protected VPNs
- **Firewalls**
  Which are specialized "middleboxes" in access and core networks (filter incoming packets, detecting DoS attacks)

Protocol layers and their service models

So, we saw that the Internet is a global network with billions of devices. The question is… is there any hope of organizing a network architecture?
Before attempting to organize our thoughts on Internet architecture, let's think about the airline system, which is a complex system with different phases.



So we defined a complex system, such as an airplane travel, with a series of **steps**, involving many services. It's important to underline that the baggage check can't be done without a ticket.
We can now structure this scheme into a layers scheme:



We can notice that each layer implements a **service** relying on services provided by **layer below**.

A layered architecture allows us to discuss a well-defined and specific part of a large and complex system, and with the **modularity**, it is much easier to change the implementation of the service provided by a layer. In fact, as long as the layer reaches the same result, we can change the functions inside it.

A similar approach is used also for the Internet:

- *application:* supporting network applications
  - HTTP, IMAP, SMTP, DNS
- *transport:* process-process data transfer
  - TCP, UDP
- *network:* routing of datagrams from source to destination
  - IP, routing protocols
- *link:* data transfer between neighboring network elements
  - Ethernet, 802.11 (WiFi), PPP
- *physical:* bits "on the wire"



Each protocol belongs to one of the layers. A protocol layer can be implemented in software, in hardware, or in a combination of the two, and, like before, a change in a layer is transparent for the rest of the system.
There are two potential drawback of the layering:

- one layer may duplicate lower-layer functionality (for example the reliability)
- functionality at one layer may need information that is present only in another layer (violation of separation of layers)

Let's take a top-down approach to analyze the Internet protocol stack

Application layer

This layer is the closest to the user and is where network applications (and their protocols) reside.
Some examples of protocols at this layer are HTTP, SMTP, FTP and DNS

An application layer application is used in end systems and it uses the protocol to exchange packets of information with the application on another system.
We'll refer to this packet of information at the application layer as a **message**.

Transport layer

The Internet's transport layer transports application-layer messages between application endpoints. There are two transport protocols: **TCP** and **UDP**.
The first is connection-oriented and includes guaranteed delivery, the second instead is connectionless and it isn't reliable.
These protocols **encapsulate** application-layer messages and they add the transport header.
We'll refer to a transport-layer packet as a **segment**.

Network layer

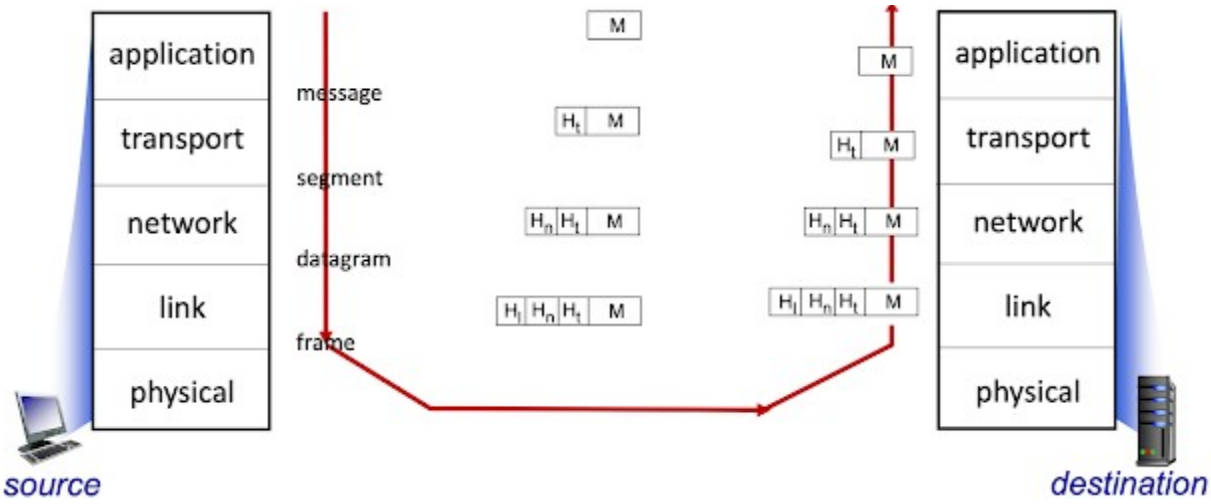
We'll refer to a network-layer packet as a **datagram**. This layer is responsible for moving transport layer packets from one host to another.
This layer includes the IP protocol and routing protocols, in fact this layer is also known as the "IP layer".

Link layer

To move a packet from one node to another, the network layer relies on the services of the link layer.
In this layer there are some protocols that provide reliable delivery, but it is very different to the TCP, because TCP is based on end systems, these ones are based on the singolar node connection.
Some examples of this protolcs are Ethernet, WiFi. A datagram typically traverses different links with different link layer protocols.
We'll refer to a link-layer packet as a **frame**.

Physical layer

While the job of the link layer is to move the entire frames, the physical layer deals with the individual bits, transmitting them by electromagnetic/wireless/fiber transmission.
For example Ethernet has many physical layers protocols: one for twisted-copper wire, one for the coaxial cable, one for the fiber etc..

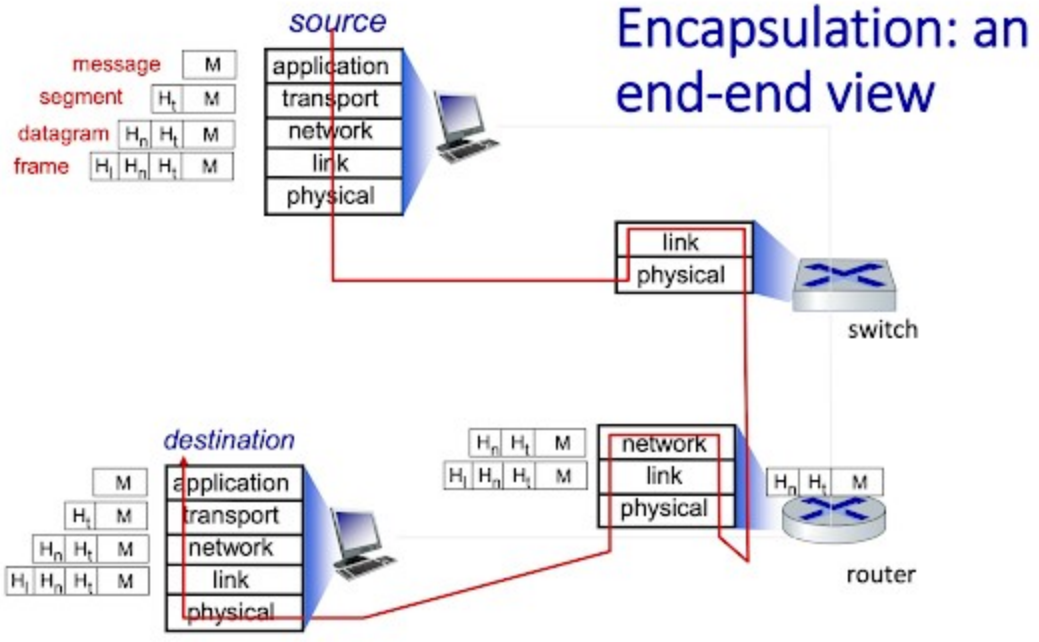


We have to remark two very important factors:

- Every layer relies to the below layer for the transmission of the information
- The header of every level is fundamental for the operation of that level

We discussed a protocol stack, but this isn't the only protocol stack for the internet.

Encapsulation



We can notice that some devices don't implement all of the layers we've seen before, but only the bottom layers, that's because those devices aren't interested in the information of the higher levels, so there would be no point in unpacking at high levels. We can see that the switch implements only layer 1 and 2 (so not the IP protocol), and the router implements layer 1,2 and 3.

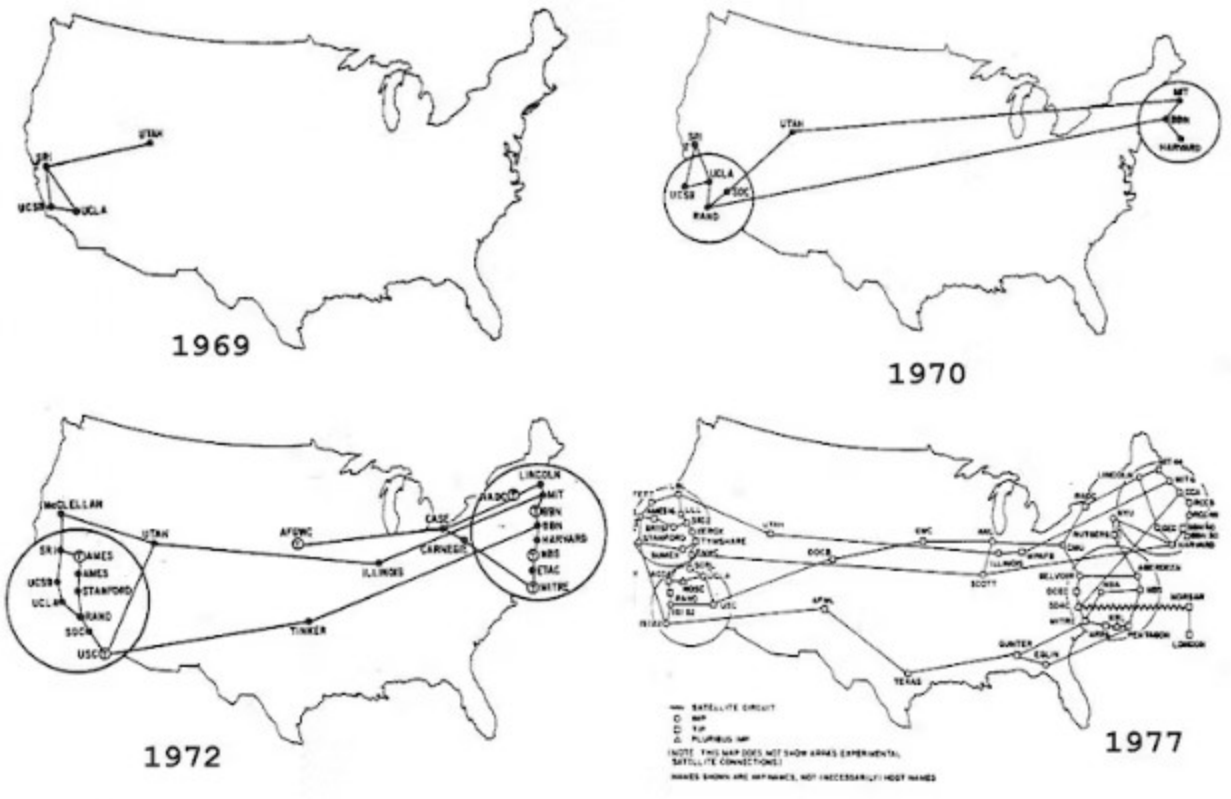
A very important concept is the **encapsulation**, that consists in the adding of information at every level. But it is very complex, for example a large message could be divided into multiple transport layer segments

History

If we understand the history of the internet, we can understand how the internet was born, how it grew up and how it became structured in the way we mentioned.
We can divide the Internet history into periods, each one with a title, that represents the main develop of that period

The development of packet switching: 1961-1972

In the early 1960s the world's dominant communication network was the telephone network. The telephone network was implemented by a circuit switching system, but with the increasing importance of computers it was natural to consider how to hook these devices, where the traffic was likely to be bursty.
So, in **1961** it was published the first work on packet switching by Leonard Kleinrock, who's "invented" the queueing theory, that demonstrated the effectiveness of packet switching approach for busty traffic sources, but still nothing practical.
In **1964** Paul Baran had begun investigating the use of packet switching for secure voice over military networks for the US army, connecting the computers of the US defense systems, in reliable and safe manners.
The Internet has also a long history of a *let's-build-it-and-demonstrate-it* attitude
In **1967** Lawrence Roberts published an overall plan for the ARPAnet (Advanced Research Projects Agency net), the first packet-switched computer network, always for the US army
In **1969** the first packet switch was installed at UCLA, under Kleinrock supervision, shortly other three additional packet switches were installed.
By **1972** ARPAnet had grown to approximately 15 nodes and was given its first public demonstration by Robert Kahn. The first host-to-host protocol between ARPAnet end systems, known as the network-control protocol (NCP), was completed. Ray Tomlinson wrote the first email program



Proprietary Networks and Internetworking: 1972–1980

The initial ARPAnet was a single, closed network: only the host connected could communicate.
In **1970s** anothers packet switching networks besides ARPAnet came into being:

- ALOHAnet, a microwave network linking universities on the Hawaiian islands
- DARPA's packet-satellite
- Telenet, a BBN commercial packet-switching network based on ARPAnet technology
- Cyclades, a French packet-switching network pioneered by Louis Pouzin
- others

The ALOHA protocol, in **1970** by Abramson, was the first multiple-access protocol, allowing geographically distributed users to share a single broadcast communication medium (a radio frequency).
In **1976**, Metcalfe and Boggs built on Abramson's multiple-access protocol work when they developed the Ethernet protocol for wire-based shared broadcast networks.

The number of networks was growing, so in **1974** Vinton Cerf and Robert Kahn started the internetting, a term coined to describe the study of the architecture of a network of networks. In those days there was only one principal protocol: TCP (who's the mix of the actual TCP and IP). But many factors led to the separation of IP out of TCP and the development of the UDP protocol. The three key Internet protocols that we see today (TCP, UDP, and IP) were conceptually in place by the end of the 1970s.
Cerf and Kahn had some internetworking principles:

- minimalism, autonomy; no internal changes required to interconnect networks

- best-effort service model
- stateless routing
- decentralized control

These principles are still today defining the actual internet architecture, centralized control for Internet is impossible

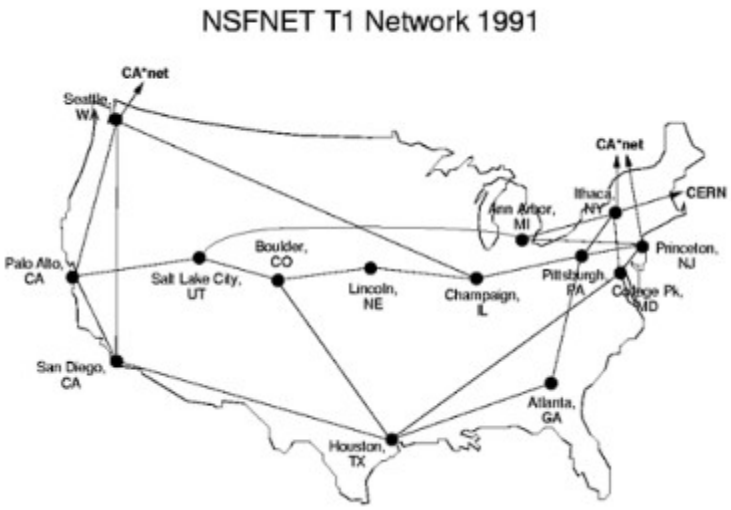
A Proliferation of Networks and new protocols: 1980–1990

By the end of the 1970s, approximately two hundred hosts were connected to the ARPAnet. By the end of the 1980s the number of hosts connected to the public Internet, a confederation of networks looking much like today's Internet, would reach a hundred thousand. The 1980s would be a time of tremendous growth.

January 1, **1983** saw the official deployment of TCP/IP as the new standard host protocol for ARPAnet (replacing the NCP protocol). From that day, all hosts were required to transfer to TCP/IP:

- **1982**, SMTP
- **1983**, DNS
- **1985**, FTP
- **1988**, TCP congestion protocol

In this period new national networks were developed: Minitel (French), CSnet, BITnet, NSFnet (successor of ARPAnet),…



The Internet Explosion: The 1990s

In early 1990s ARPAnet, the progenitor of the Internet, ceased to exist.
In 1991, NSFNET lifted its restrictions on the use of NSFNET for commercial purposes.
The main event of the 1990s was the emergence of the World Wide Web application, which brought the Internet into the homes and businesses of millions of people worldwide.
In the early 1990s the Web was invented:

- hypertext (Bus 1945, Nelson's 1960s)
- Berners-Lee initial versions of HTML, HTTP, a Web server, and a browser (the four key components of the Web).
- Mosaic (later Netscape), a web browser in **1994** (then Microsoft won this war)

Late in the 1990s:

- more killer apps (indispensabili): email, instant messaging, peer-to-peer file sharing
- network security to forefront
- backbone links running at Gbps

The period from 1995 to 2001 was a roller-coaster ride for the Internet in the financial markets, many companies were valued in the billions of dollars without having any significant revenue streams.

The new millennium

Innovation in computer networking continues at a rapid pace. Advances are being made on all fronts, including deployments of faster routers and higher transmission speeds in both access networks and in network backbones. But the following developments merit special attention:

- Aggressive deployment of broadband Internet access to homes (10-100's MBPS)
- The increasing ubiquity of high-speed (54 Mbps and higher) public WiFi networks and medium-speed (tens of Mbps) Internet access via 4G cellular telephony networks
- Online social networks have created massive people networks on top of the Internet.
- Online service providers (Google and Microsoft) have deployed their own extensive private networks, which not only connect together their globally distributed data centers, but are used to bypass the Internet as much as possible by peering directly with lower-tier ISPs. As a result, Google provides search results and e-mail access almost instantaneously, as if their data centers were running within one's own computer.
- Many Internet commerce companies are now running their applications in the "cloud"

Application layer

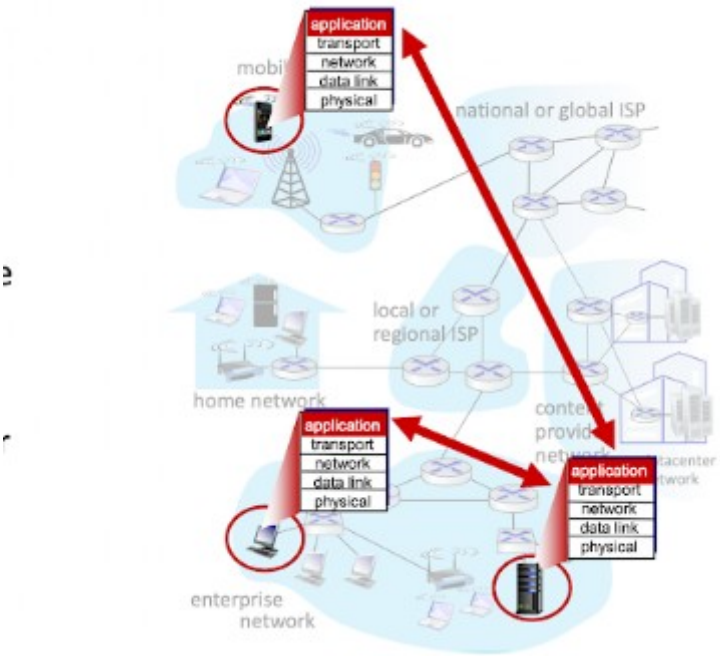
Since the Internet's inception, numerous useful and entertaining applications have indeed been created.
These applications have been the driving force behind the Internet's success, motivating people in homes, schools, governments, and businesses to make the Internet an integral part of their daily activities.
P2P (peer-to-peer) file sharing was very diffused on the network, but this type of application was increasing the violation of copyright and after years the mass media companies stopped by law the file sharing. Now all the p2p applications are under payment.

Telephone companies tried to oppose Skype and others voice over IP, but Skype is totally legal so they had to co-exist with them.
After the pandemic explosion, services like real-time video conferencing (Zoom) became very popular, mostly for distance learning.

Netscape was the best web browser and Internet Explorer (Microsoft) was less efficient, but on Microsoft PCs was installed Microsoft Explorer by default so the users started (out of laziness) to use it.

Principles of Network Applications

To write a network application you should write programs for different end systems, because these applications run only on end systems that communicate with others over the network, and in the network core there isn't anything about the application.
For example, in the Web application there are two distinct programs (web browser on the client and web server on the server), in the P2P file sharing both hosts have the same program
So we don’t need to write software for network-core devices. Even if you wanted to write application software for these network-core devices, you wouldn’t be able to do so (as we’ve seen in the first chapter, these devices don’t include the application layer).

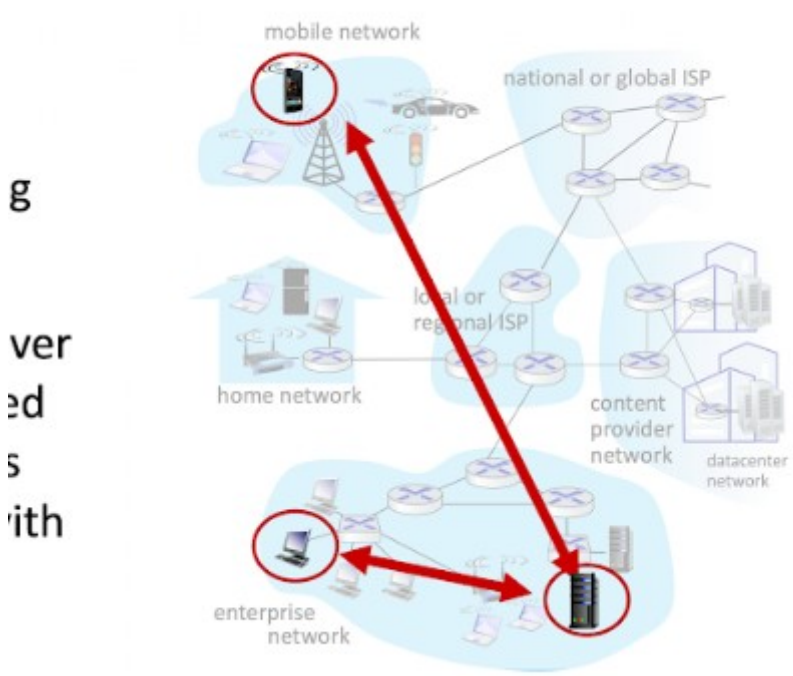


Network Application Architecture

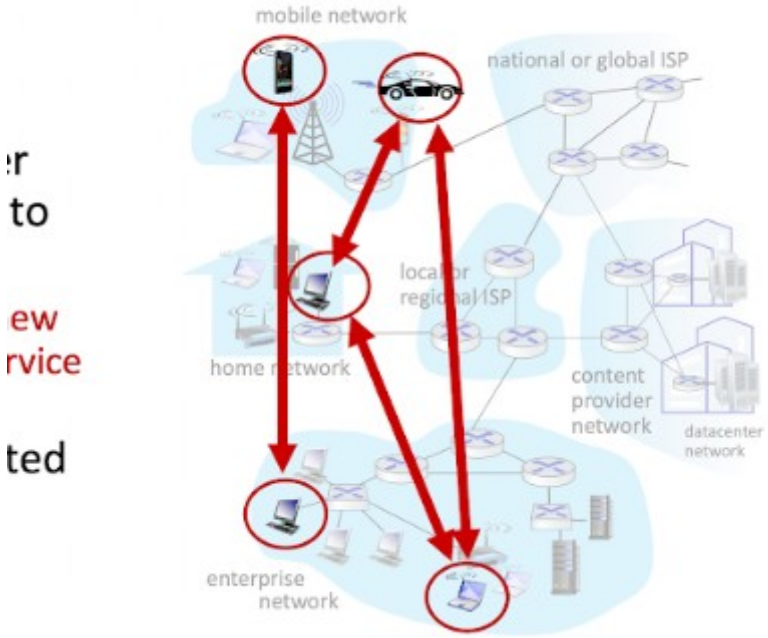
Before diving into software coding, you should have a broad architectural plan for your application. Keep in mind that an application’s architecture is distinctly different from the network architecture.
The application architecture is designed by the application developer and dictates how the application is structured over the various end systems. There are two predominant architectural paradigms used in modern network applications:

- the client-server architecture
- the peer-to-peer architecture

In a client-server architecture, there is an always-on host, called the **server**, which services requests from many other hosts, called **clients**. An example of a client-server application is the Web application, where the host requires the server to send an object, and the server responds with the object (in absence of errors).
In this architecture, clients do not communicate directly with each other.
Another important characteristic of this architecture is that the server has a fixed well-known IP address.
Other examples of this architecture are FTP, Telnet, e-mail, etc
Often a single-server host is incapable of keeping up with all the requests from clients. For this reason, a **data center**, housing a large number of hosts, is often used to create a powerful virtual server. A data center can have hundreds of thousands of servers, which must be powered and maintained.



In a P2P architecture, there is minimal (or no) reliance on dedicated servers in data centers. Instead the application exploits direct communication between pairs of intermittently connected hosts, called peers. Some examples of these applications are file sharing (like BitTorrent) and Internet telephony and video conference (like Skype).
One of the most compelling features of P2P architectures is their **self-scalability**. In fact peers request service from other peers, and provide service in return to other peers.
One disadvantage of this architecture is that the peer change the IP address and this requires a complex management



We mention that some applications have hybrid architectures, combining both client-server and P2P elements. For example, for many instant messaging applications, servers are used to track the IP addresses of users, but user-to-user messages are sent directly between user hosts (without passing through intermediate servers).

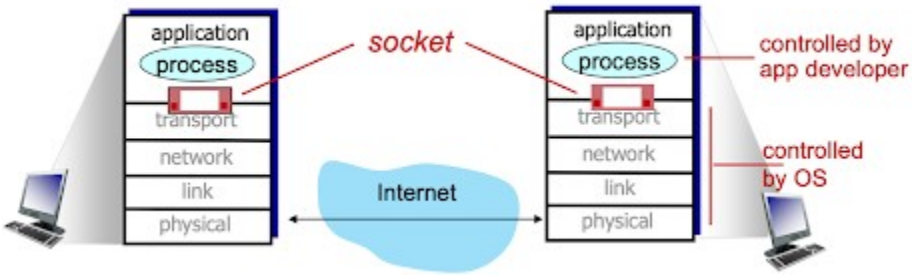
Processes Communicating

In the jargon of operating systems, it is not actually programs but processes that communicate. A process can be thought of as a program that is running within an end system.
In the same host, processes communicate with each other using rules that are governed by the end system's operating system.
But in different hosts, processes communicate by exchanging **messages** across the computer network. A sending process creates and sends messages into the network; a receiving process receives these messages and possibly responds by sending messages back.

For each pair of communicating processes (both client-server and peer-to-peer), we typically label one of the two processes as the **client** and the other process as the **server**.
You may have observed that in some applications, such as in P2P file sharing, a process can be both a client and a server.
In the context of a communication session between a pair of processes, the process that initiates the communication (that is, initially contacts the other process at the beginning of the session) is labeled as the client. The process that waits to be contacted to begin the session is the server.

A process sends messages into, and receives messages from, the network through a software interface called a **socket**. A process is analogous to a house and its socket is analogous to its door:

- When a process wants to send a message to another process on another host, it shoves the message out its door (socket).
- This sending process assumes that there is a transportation infrastructure on the other side of its door that will transport the message to the door of the destination process.
- Once the message arrives at the destination host, the message passes through the receiving process's door (socket), and the receiving process then acts on the message.



As shown in this figure, a socket is the interface between the application layer and the transport layer within a host. The application developer has control of everything on the application-layer side of the socket but has little control of the transport-layer side of the socket (only the transport protocol and the size of buffer and segments)

In order for a process running on one host to send packets to a process running on another host, the receiving process needs to have an address. To identify the process we need two informations: the IP address of the host and an identifier of the process (this information is needed because in general a host could be running many network applications in different processes).
A destination **port number** serves this purpose. Some internet applications, the most popular, has a specific port number (web server 80, mail server 25, etc)

An application layer protocol defines:

- Types of messages exchanged
  Message of request, of response etc
- Message syntax
  What fields in messages and how fields are delineated
- Message semantics
  Meaning of information in fields
- Rules for when and how process send and response to messages

There exist open protocols, where everyone has access to protocol definition (HTTP, SMTP, etc), but there also exist proprietary protocols (like Skype, Zoom, etc)

Transport Services Available to Applications

Many networks, including the Internet, provide more than one transport-layer protocol. When you develop an application, you must choose one of the available transport-layer protocols.

What are the services that a transport-layer protocol can offer to applications invoking it? We can broadly classify the possible services along four dimensions:

- **Reliable data transfer**
  Some packets could be lost due to the overflow of a buffer or because it has some bits corrupted. For many applications data lost can have devastating consequences
  If a protocol provides such a guaranteed data delivery service, it is said to provide **reliable data transfer**.
  If a protocol doesn't provide reliable data transfer, some of the data sent by the sending process may never arrive, this may be accessible for loss-tolerant applications
- **Throughput**
  We studied the sharing of the bandwidth, so we can think of a protocol that can provide a guaranteed available throughput at some specified rate.
  If the transport protocol cannot provide this throughput, it can transmit at a lower rate or give up.
  Applications that have throughput requirements are said to be **bandwidth-sensitive applications**. Otherwise the application is said to be an **elastic application**
- **Timing**
  For example guarantee that a bit arrives at the receiver's socket no more than 100 msec
- **Security**
  A transport protocol can encrypt all data transmitted by the sending processes (and then decrypt before delivering to the receiver process). Also data integrity is a part of the security

application	data loss	throughput	time sensitive?
file transfer/download	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5Kbps-1Mbps video:10Kbps-5Mbps	yes, 10's msec
streaming audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	Kbps+	yes, 10's msec
text messaging	no loss	elastic	yes and no

Transport Services Provided by the Internet

The Internet (more generically the TCP/IP networks) makes two transport protocols available to applications, **UDP** and **TCP**. When you create a new network application, one of the first decisions you have to make is whether to use UDP or TCP.

The principal services of TCP are:

- **Connection-oriented service**
  The client and the server exchange transport layer control information **before** the application level messages begin to flow, this is called **handshake**. After the handshaking phase, a TCP connection is said to exists between the sockets of the two processes
- **Reliable data transfer service**
  The communicating processes can totally rely on TCP to deliver all data sent without error and in the proper order
- **Congestion-control mechanism**
  It throttles sender when network is overloaded
- **Flow control**
  The sender won't overwhelm the receiver

The UDP protocol provides the minimal needed service:

- **Connectionless**
- **Unreliable data transfer service**
  No guarantee that the message will ever reach the receiver, and if it reach it, it's not guaranteed the order of the packets
- **Not** include a **congestion-control** mechanism

Neither TCP nor UDP provide timing, minimum throughput guarantee or security, in fact neither TCP or UDP provides any encryption, so the Internet community has developed an enhancement for TCP, called Secure Sockets Layer (SSL) (now there is TLS that is more reliable).

Why doesn't it use only TCP? Because the parameters that TCP guarantees requires time, and some applications doesn't have time, so they sacrifice safety for speed

application	application layer protocol	transport protocol
file transfer/download	FTP [RFC 959]	TCP
e-mail	SMTP [RFC 5321]	TCP
Web documents	HTTP 1.1 [RFC 7320]	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or proprietary	TCP or UDP
streaming audio/video	HTTP [RFC 7320], DASH	TCP
interactive games	WOW, FPS (proprietary)	UDP or TCP

It is important to distinguish between network applications and application-layer protocols. An application-layer protocol is only one piece of a network application

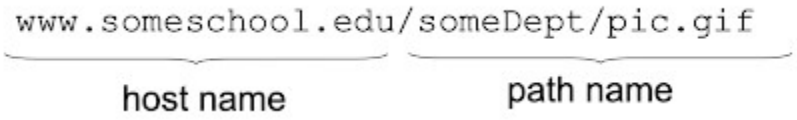
The Web and HTTP

Until early 1990s the Internet was used primarily by academics and researchers, then a major new application arrived con the scene: the World Wide Web, the first application that caught the general public's eye
Perhaps what appeals the most to users is that the Web operates on demand. Users receive what they want, when they want it. This is unlike traditional broadcast radio and television, which force users to tune in when the content provider makes the content available.
Everyone can become a publisher at extremely low cost

Overview of HTTP

The HyperText Transfer Protocol (HTTP), the Web's application-layer protocol, is at the heart of the Web. It is implemented in two programs: client and server, and they exchange information with HTTP messages

A **Web page** (also called a document) consists of objects. An **object** is simply a file (such as an HTML file, a JPEG image, a Java applet, or a video clip) that is addressable by a **single URL**.
Most web pages consist of a base HTML file (HyperText Markup Language, used for writing web pages) and several referenced objects.
Hyper Text are text with links to other texts or objects
Each URL has two components: the hostname of the server that houses the object and the object's path name.



The client (or Web browser) requests and receives (using HTTP) the Web objects, and then it displays them.
The server, the place where the objects are contained, sends Web objects (always using HTTP) in response to the client requests.



HTTP uses TCP as its underlying transport protocol, so the client first initiates a TCP connection with the server (port 80). The sockets are the door between the processes and the TCP connection.
We can already see an advantage of the layered approach, because HTTP need not to worry about lost data or the details of how TCP recovers from loss or reordering of data within the network

HTTP is said to be a **stateless** protocol because the server sends requested files to clients without storing any state information about the client. (Protocols that maintain the state are very complex, they need to store the past history)

Non-Persistent and Persistent Connections

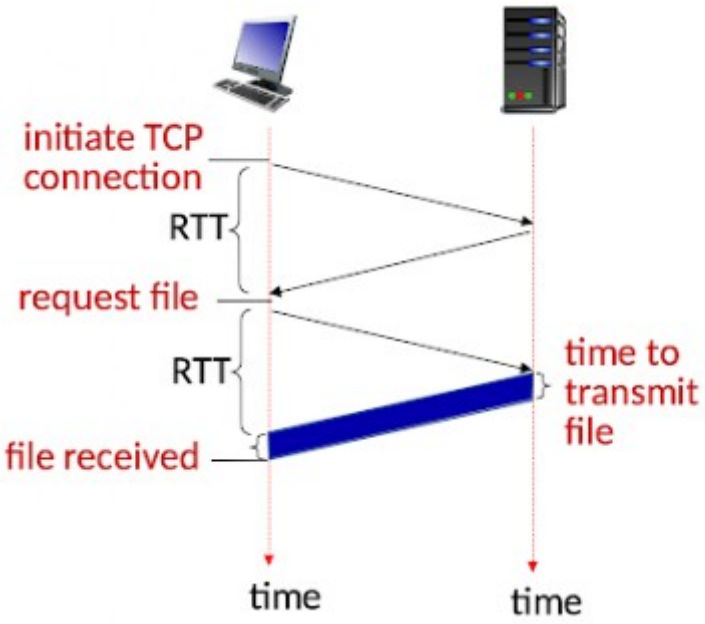
In many Internet applications, the client and server communicate for an extended period of time, with the client making a series of requests and the server responding to each of the requests. When this client-server interaction is taking place over TCP, the application developer needs to make an important decision: should each request/response pair be sent over a separate TCP connection, or should all of the requests and their corresponding responses be sent over the same TCP connection?
So we have two approaches: **persistent** HTTP connections and **non-persistent** HTTP connections.
HTTP can use both, but it uses persistent connections on its default mode.

For understanding the advantages and disadvantages of these approaches let’s make an example.
If a Web page is composed of one HTML file and 10 images, the client must open a connection for the HTML file, and then repeat all the steps for each image.

1. The HTTP client process initiates a TCP connection to the server `www.someSchool.edu` on port number 80, which is the default port number for HTTP. Associated with the TCP connection, there will be a socket at the client and a socket at the server.
2. The HTTP client sends an HTTP request message to the server via its socket. The request message includes the path name `/someDepartment/home .index`. (We will discuss HTTP messages in some detail below.)
3. The HTTP server process receives the request message via its socket, retrieves the object `/someDepartment/home.index` from its storage (RAM or disk), encapsulates the object in an HTTP response message, and sends the response message to the client via its socket.
4. The HTTP server process tells TCP to close the TCP connection. (But TCP doesn't actually terminate the connection until it knows for sure that the client has received the response message intact.)
5. The HTTP client receives the response message. The TCP connection terminates. The message indicates that the encapsulated object is an HTML file. The client extracts the file from the response message, examines the HTML file, and finds references to the 10 JPEG objects.
6. The first four steps are then repeated for each of the referenced JPEG objects.

There is also the possibility to open different parallel TCP connections at the same time.

Let's do a back-of-the-envelope calculation to estimate the amount of time that elapses from when a client requests the base HTML file until the entire file is received by the client. To this end, we define the round-trip time (RTT), which is the time it takes for a small packet to travel from client to server and then back to the client.



So in a non-persistent HTTP connection, the response time for each file is

$$2 \cdot RTT + file\ \ transmission\ \ time$$

In general $2 \cdot RTT > file\ \ transmission\ \ time$, so $2 \cdot RTT + file\ \ transmission\ \ time \simeq 2 \cdot RTT$

In a persistent HTTP connection (HTTP 1.1) the server leaves the TCP connection open after sending a response.
Subsequent requests and responses between the same client and server can be sent over the same connection. In particular, an entire Web page (in the example above, the base HTML file and the 10 images) can be sent over a single persistent TCP connection.
Typically, the HTTP server closes a connection when it isn't used for a certain time (a configurable timeout interval).
The response time is as little as one RTT for all the referenced objects (cutting response time in half)

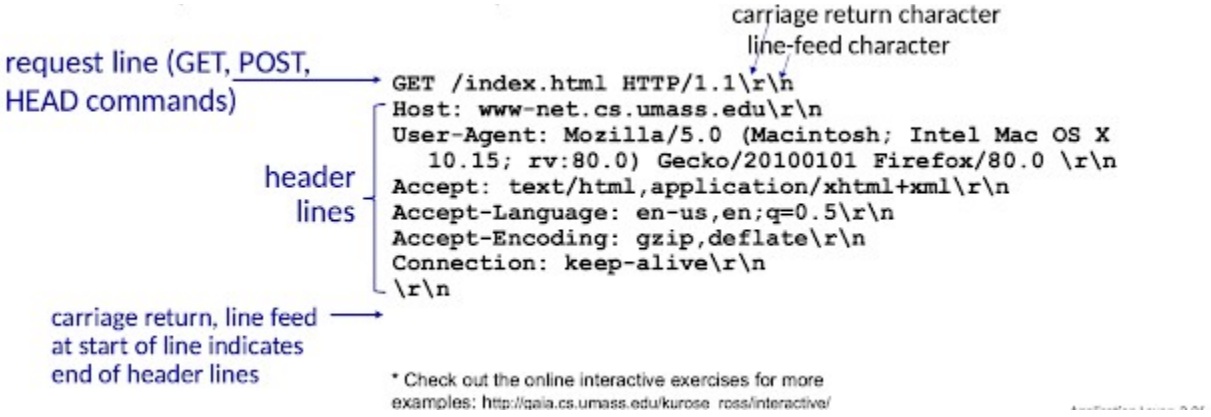
So, with non-persistent HTTP for each TCP connection, at most one object can be sent, instead in persistent HTTP multiple objects can be sent over a single TCP connection between client, and that server.
The persistent HTTP is used in modern applications.

HTTP message format

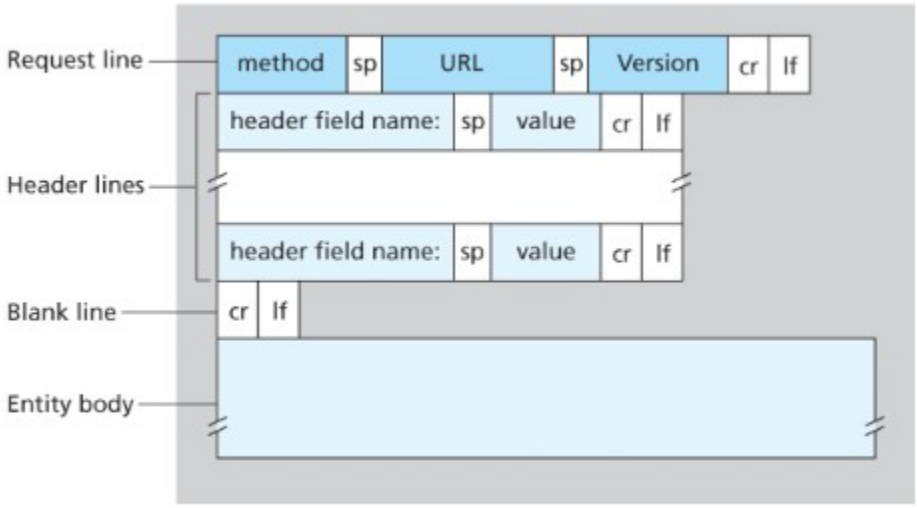
The HTTP specifications include the definitions of the HTTP message formats. There are two types of HTTP messages, **request** messages and **response** messages, both of which are discussed below.

These messages are written in ordinary ASCII text. Every line finishes with a carriage return character and a line-feed character.

The first line of a HTTP **request message** is called **request line**, the subsequent lines are called **header lines**.
The request line has three fields: the method field, the URL field, and the HTTP version field. The method field can take on several different values, including GET, POST, HEAD, PUT, and DELETE. The great majority of HTTP request messages use the GET method. The GET method is used when the browser requests an object, with the requested object identified in the URL field.

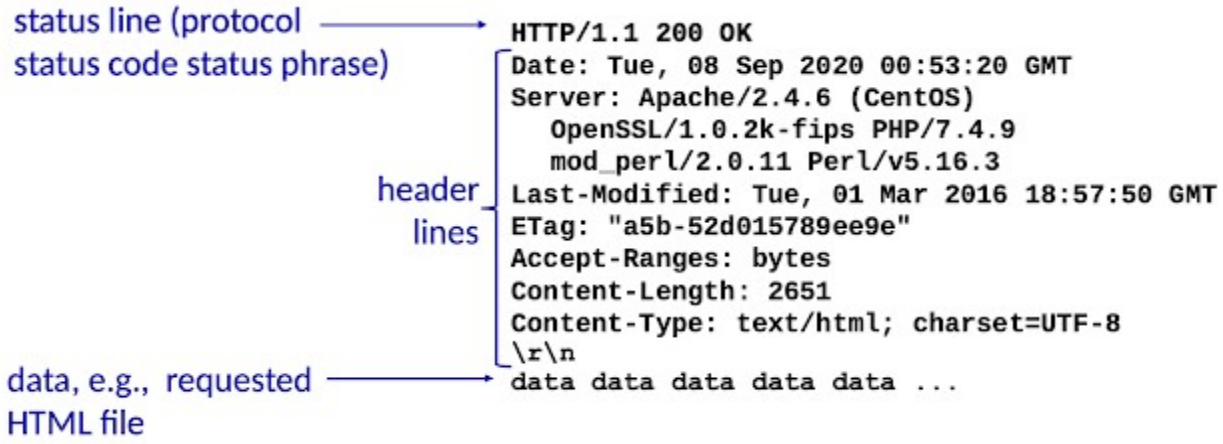


The header line **Host** specifies the host on which the object requested should reside (this is unnecessary, as there is already TCP, but some web proxy caches require it).
By including the **Connection**: keep-alive header line, the browser is telling the server that it wants to bother with persistent connections; otherwise if it is 'close', it wants the server to close the connection after sending the requested object.
The **User-agent**: header line specifies the user agent, that is, the browser type that is making the request to the server (the server could send different versions of objects to different types of user agents).
The **Accept-language**: header indicates that the user prefers to receive a French version of the object, if such an object exists on the server



The Entity Body in the GET requests is empty, but it is used in **POST** requests, when the user has to compile a form to require specific contents on the Web page (but you can use GET and add the data in the URL).
The **HEAD** method is similar to the GET, but the server will leave out the requested object and include only the headers (it is usually used for debugging)
The **PUT** method allows the user to upload an object to a specific path (directory) on a specific web server.
The **DELETE** method allows the user to delete an object on a Web server.

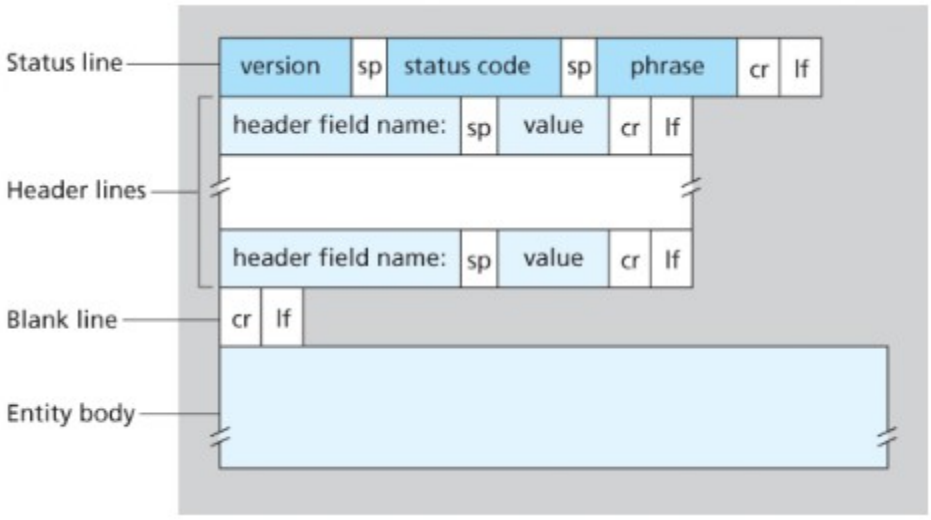
Let's see an example of a HTTP response message (from the server obviously)



It has three sections: an **initial status line**, seven **header lines**, and then the **entity body**.
The entity body is the meat of the message, it contains the object itself.
The status line has three fields: the protocol version field, a status code, and a corresponding status message.
The status code, and the associated message, indicate the result of the request:

- 200 OK
  Request succeeded and the information is returned in the response.
- 301 Moved Permanently
  Requested object has been permanently moved; the new URL is specified in Location: header of the response message.
- 400 Bad Request
  This is a generic error code indicating that the request could not be understood by the server.
- 404 Not Found
  The requested document does not exist on this server.
- 505 HTTP Version Not Supported
  The requested HTTP protocol version is not supported by the server.

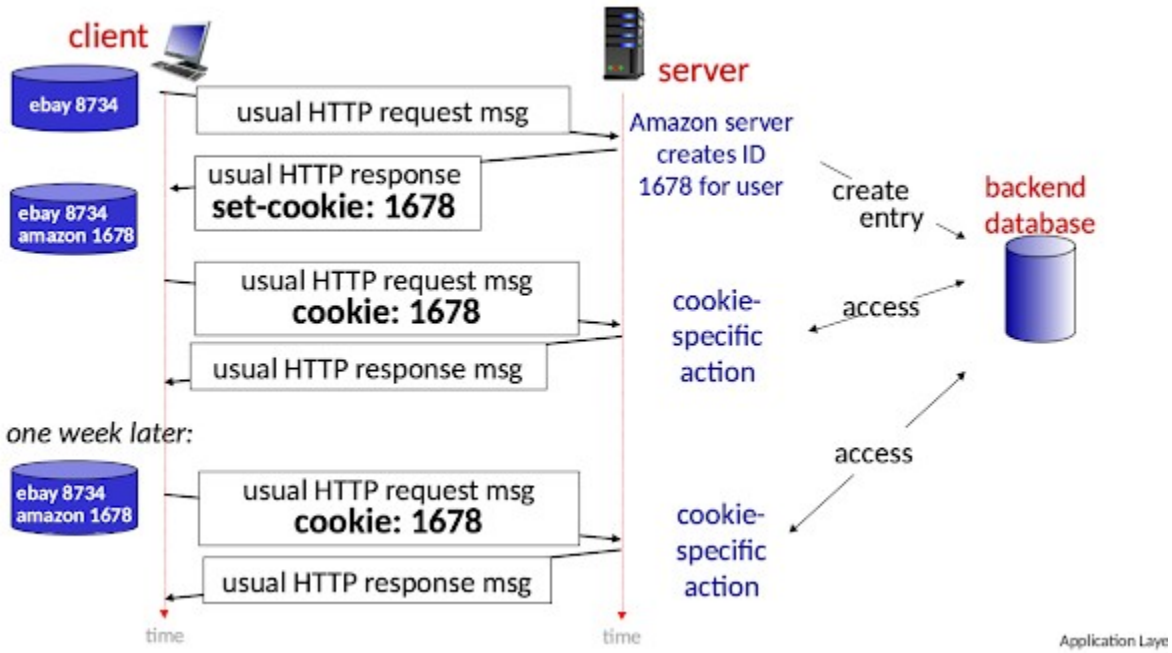
Now let's look at the header lines.
The **Date** indicates the time and date when the HTTP response was created and sent.
The **Server** indicates what server generated the message (it is analogous to the User agent)
The **Last-modified** indicates the time and date when the object was created or last modified (it is critical for object caching).
The **Content-length** indicates the number of bytes in the object being sent.
The **Content-Type** indicates that the object in the entity body is HTML text.



The HTTP specification defines many, many more header lines that can be inserted by browsers, Web servers, and network cache servers. We have covered only a small number of the totality of header lines.

User-Server Interaction: Cookies

We mentioned above that an HTTP server is stateless. This simplifies server design and has permitted engineers to develop high-performance Web servers that can handle thousands of simultaneous TCP connections.
However, it is often desirable for a Web site to identify users. For these purposes, HTTP uses cookies. Cookies allow sites to keep track of users. Most major commercial Web sites use cookies today.



As we can see in the image, cookies have four components:

- a cookie header line in the HTTP request message

- a cookie header line in the HTTP response message
- a cookie file kept on the user's end system and managed by the user's browser
- a back-end database at the Web site

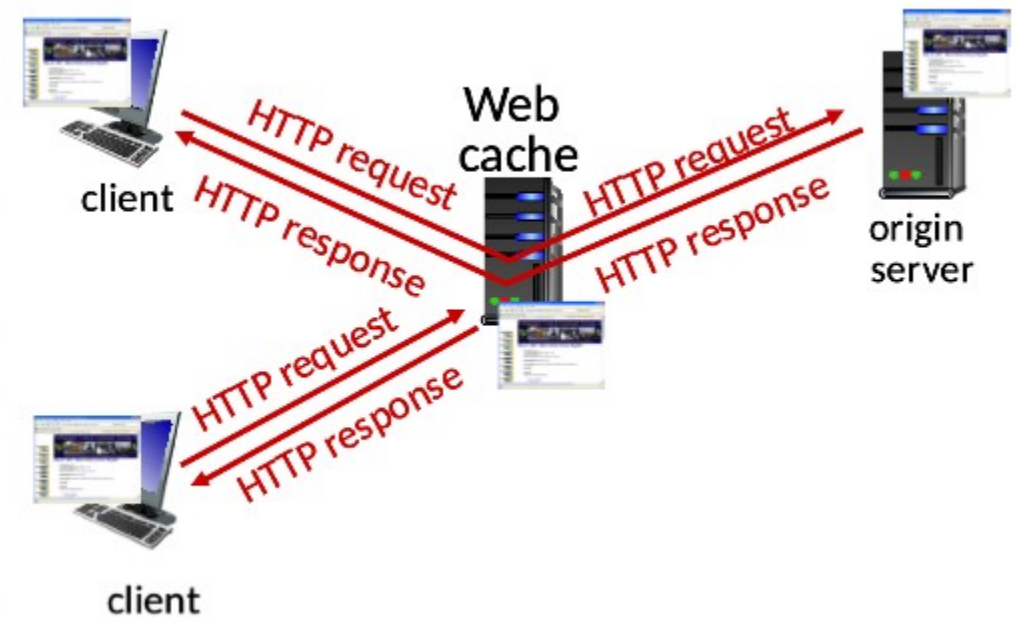
Let's walk through an example of how cookies work.
The client, who has already visited the ebay site in the past, contacts Amazon.com for the first time. The server creates an unique identification number and saves it in a database, responding to the client with the header Set-cookie set on this number.
When the browser receives the response, it sees that there is the Set-cookie and adds a line in a special cookie file that it manages. From now on, every time that Susan requests a web page from Amazon.com, her browser consults her cookie file, extracts her identification number and puts Cookie header in the request.
Amazon, for example, uses cookies to provide its shopping cart service
If Susan returns to Amazon's site, say, one week later, her browser will continue to put the header line Cookie: 1678 in the request messages.
Then, Susan can create an account registering her name, e-mail address, postal address, and credit card information and Amazon can add this information to its database and Susan doesn't need to re-enter her name, credit card number, or address in next purchases

Cookies can thus be used to create a user session layer on top of stateless HTTP.
Although cookies often simplify the Internet shopping experience for the user, they are controversial because they can also be considered as an invasion of privacy. As we just saw, using a combination of cookies and user-supplied account information, a Web site can learn a lot about a user and potentially sell this information to a third party.

Web caching

A **web cache** (or also called **proxy server**) is a network entity that satisfies HTTP requests on the behalf of an origin Web server. The Web cache has its own disk storage and keeps copies of recently requested objects in this storage.
So once the user's browser is configured to point to a (local) Web cache, all the user's HTTP requests are first directed to the Web cache and there are two cases:

1. The Web cache has a copy of the object in its memory (object in cache)
   In this case the Web cache returns the object with an HTTP response message.
2. The Web cache does not have a copy of the object (object not in cache)
   In this case the Web cache makes an HTTP request to the origin server and it receives the object in a HTTP response from him. Then it stores it in its memory (in cache) and then it continues with point 1.



From this image we can see that the first client asks for an object, the Web cache doesn't have it in its memory so it has to make another request to the origin server.
When the second client asks for the same object, the Web cache gives it the object because it already has it in its memory.

The Web caches are also called **proxy servers** because they act as both **server** and **client** at the same time; it is a server when it receives requests from a client and sends a response to it, it is a client when it sends requests to the origin server and receives responses from it.

Typically a web cache server is purchased and installed by an ISP. The origin server can tell a Web cache if the object is allowable to caching, the max time where the object is "valid" and other things.

```
Cache-Control: max-age=<seconds>
```

```
Cache-Control: no-cache
```

Web caching has seen deployment in the Internet for two reasons:

1. It can substantially reduce the response time for a client request
   The object is closer to the client, and particularly in the situation where the bandwidth connection between the client and the cache is high-speed
2. It can substantially reduce traffic on an institution's access link to the Internet
   By reducing traffic, the institution does not have to upgrade bandwidth as quickly, thereby reducing costs. Web caches can reduce traffic in the Internet as a whole, improving performances for all applications

The Internet is dense with caches precisely for these advantages.

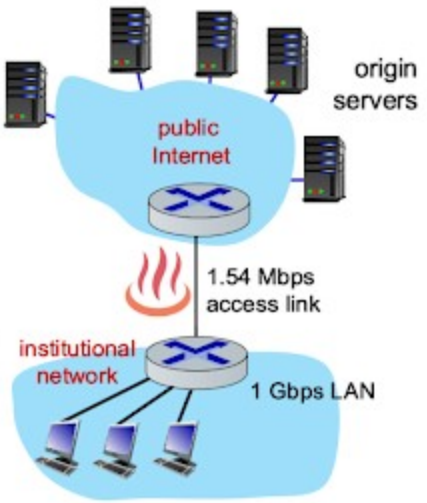
Let's see an example of the advantages of using Web caches:

Scenario:

- access link rate: 1.54 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
  - avg data rate to browsers: 1.50 Mbps

Performance:

- access link utilization = .97 — *problem:* large queueing delays at high utilization!
- LAN utilization: .0015
- end-end delay = Internet delay + access link delay + LAN delay
  = 2 sec + minutes + usecs



In this example we have, like the real world, the institutional network with 1 Gbps link, and the links with other routers on the internet of 1.54 Mbps.
Suppose that the HTTP request messages are negligibly small and thus create no traffic in the networks or in the access link (from institutional router to Internet router).
The total response time is the LAN delay, the access Delay and the Internet delay.

The traffic intensity on the LAN is $\frac{1.50 \ \ Mbps}{1 \ \ Gbps} = 0.0015$ (so it is milliseconds of delay)

The traffic intensity on the access link is $\frac{1.50 \ \ Mbps}{1.54 \ \ Mbps} = 0.97$ (it can reach minutes of delay).

One possible solutions is to increase the access link rate, but this solutions is very high-cost, because we have to upgrade the link of $100 \ \ Mbps$:

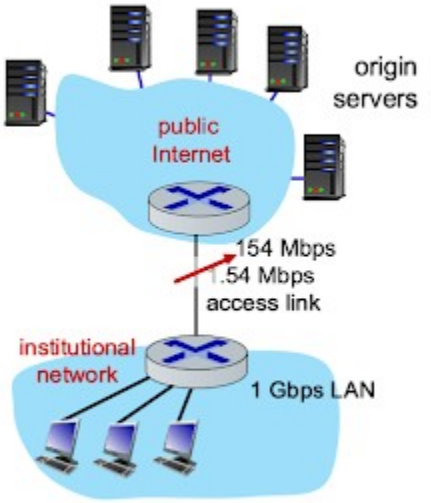
Scenario:

- access link rate: ~~1.54~~ 154 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
  - avg data rate to browsers: 1.50 Mbps

Performance:

- access link utilization = ~~.97~~ .0097
- LAN utilization: .0015
- end-end delay = Internet delay + access link delay + LAN delay
  = 2 sec + ~~minutes~~ + usecs msecs

Cost: faster access link (expensive!)



The second (and better) solution is to install a Web cache, that is cheaper and it makes the average total response time faster!

Scenario:

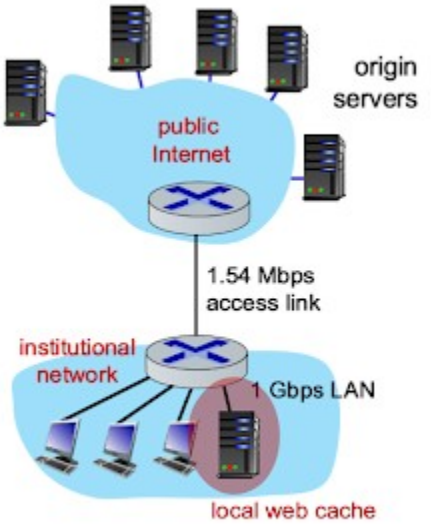
- access link rate: 1.54 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
  - avg data rate to browsers: 1.50 Mbps

Cost: web cache (cheap!)

Performance:

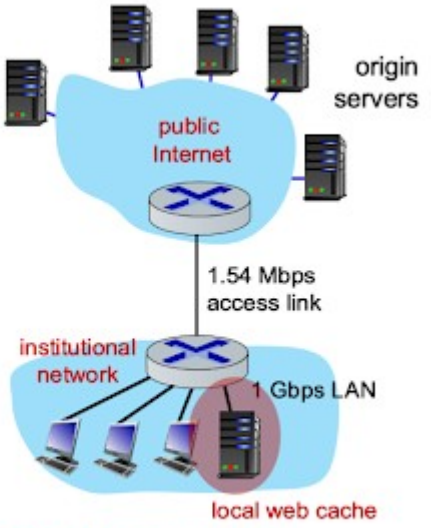
- LAN utilization: .?
- access link utilization = ?
- average end-end delay = ?

How to compute link utilization, delay?



suppose cache hit rate is 0.4:

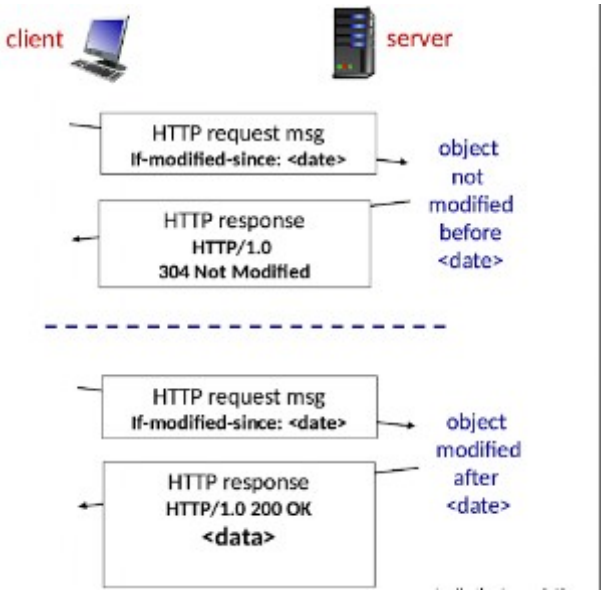
- 40% requests served by cache, with low (msec) delay
- 60% requests satisfied at origin
  - rate to browsers over access link
    = 0.6 * 1.50 Mbps = .9 Mbps
  - access link utilization = 0.9/1.54 = .58 means low (msec) queueing delay at access link
- average end-end delay:
  = 0.6 * (delay from origin servers)
  + 0.4 * (delay when satisfied at cache)
  = 0.6 (2.01) + 0.4 (~msecs) = ~ 1.2 secs



lower average end-end delay than with 154 Mbps link (and cheaper too!)

The caching introduces a new problem: the copy of an object residing in the cache may be stale (outdated).
Fortunately, HTTP has a mechanism that allows a cache to verify that its objects are up to date. This mechanism is called the **conditional GET**.
An HTTP request message is a so-called conditional GET message if the request message uses the GET method and the request message includes an If-Modified-Since: header line (this value is exactly equal to the value of the Last-Modified: header line that was sent by the server last time). This conditional GET is telling the server to send the object only if the object has been modified since the specified date.

In the response the server can send the message “304 Not Modified” if the object in the Web cache is the last version, or can send the new object like a normal GET response.

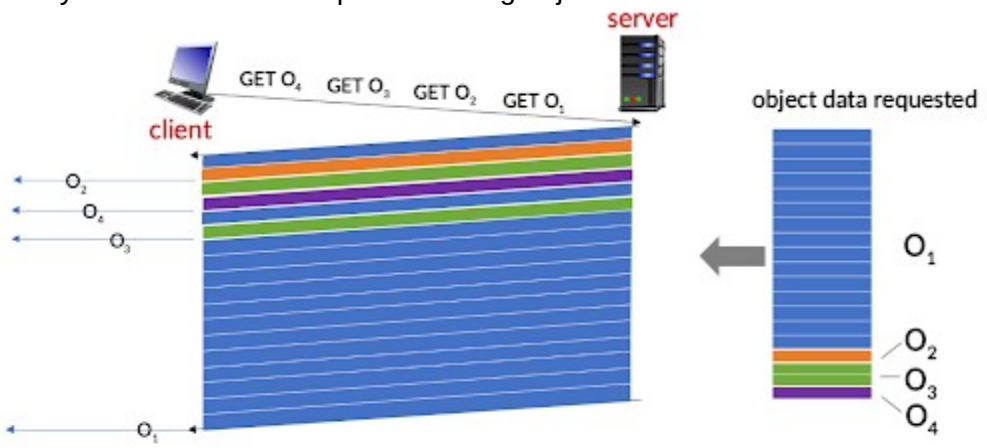


Now let's see the various versions of the HTTP protocol, with the main goal of decreasing delay of multiple-object requests.
The **HTTP1.1** introduced multiple and pipelined GETs over a single TCP connection, in this protocol the server **response in order** (FCFS, First Come First Served), but **small objects** may have to **wait** for transmission **behind large objects** (HOL blocking, Head Of Line blocking)



objects delivered in order requested: O_2, O_3, O_4 wait behind O_1

The **HTTP/2** increased flexibility at server in sending objects to the client adding client-specified object priority and with the technique of dividing objects into frames:



O_2, O_3, O_4 delivered quickly, O_1 slightly delayed

In the **HTTP/3** it was added security and object error and congestion control over UDP (to reduce stalling of TCP connections)

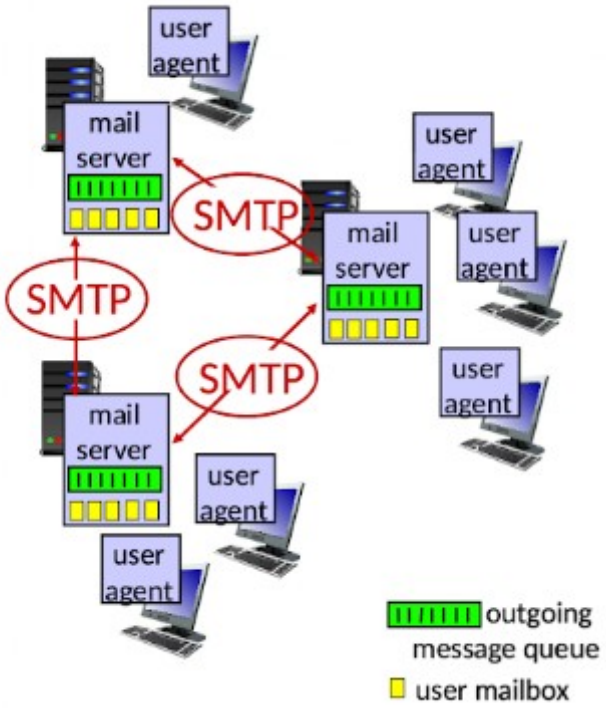
Electronic Mail in the Internet

Electronic mail has been around since the beginning of the Internet (email is older than the HTTP). It was the most popular application when the Internet was in its infancy, and has become more elaborate and powerful over the years. It remains one of the Internet's most important and utilized applications.

In the high-level view of the Internet mail system we have three major components:

- **User agents**
  They are also called “mail reader”, they compose, edit and read all messages (some examples are Outlook and Apple Mail)
- **Mail servers**
  They form the core of the email infrastructure. Each recipient has a **mailbox** on a Mail server, and it contains incoming messages for the user. Mail servers has also **outgoing message queue** (when the server cannot deliver the mail to the recipient server)
- **Simple Transfer Mail Protocol** (SMTP)
  It is an application-layer protocol for Internet electronic mail. It uses TCP

Every SMTP device (except the user agents) has two sides: both client and server



SMTP

SMTP, defined in RFC 5321, is at the heart of Internet electronic mail.
It transfers messages from senders’ mail servers to the recipients’ mail servers.

Let’s see how it works:
The user a invokes his user agent to write a mail for user b. The mail is sent by the user agent to the mail server (port 25) of user a. The client side of the server sees the message in the queue and it sends it to the mail server of user b (if it is possible). Then user b invokes his user agent to read the message when he wants.

1) Alice uses UA to compose e-mail message "to" bob@someschool.edu

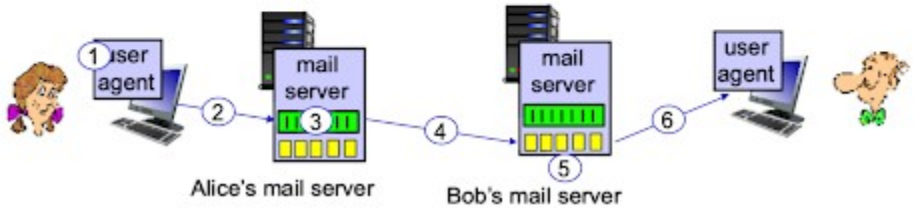
2) Alice's UA sends message to her mail server using SMTP; message placed in message queue

3) client side of SMTP at mail server opens TCP connection with Bob's mail server

4) SMTP client sends Alice's message over the TCP connection

5) Bob's mail server places the message in Bob's mailbox

6) Bob invokes his user agent to read message



The transfer of an SMTP packet is similar to HTTP:

1. SMTP handhaking (the SMTP client indicates the e-mail address of the sender and the e-mail address of the recipient)
2. SMTP transfer of message (counting on TCP)
3. SMTP closure

The command and the response interaction are equal to HTTP (commands in ASCII text and the response in status code and phrase)

Let's next take a look at an example transcript of messages exchanged between an SMTP client (C) and an SMTP server (S):

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250  Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

The client issued five commands: **HELO**, **MAIL FROM**, **RCPT TO**, **DATA** and **QUIT**.
We mention here that SMTP uses persistent connections: If the sending mail server has several messages to send to the same receiving mail server, it can send all of the messages over the

same TCP connection. For each message, the client begins the process with a new MAIL FROM: crepes.fr, designates the end of message with an isolated period, and issues QUIT only after all messages have been sent.

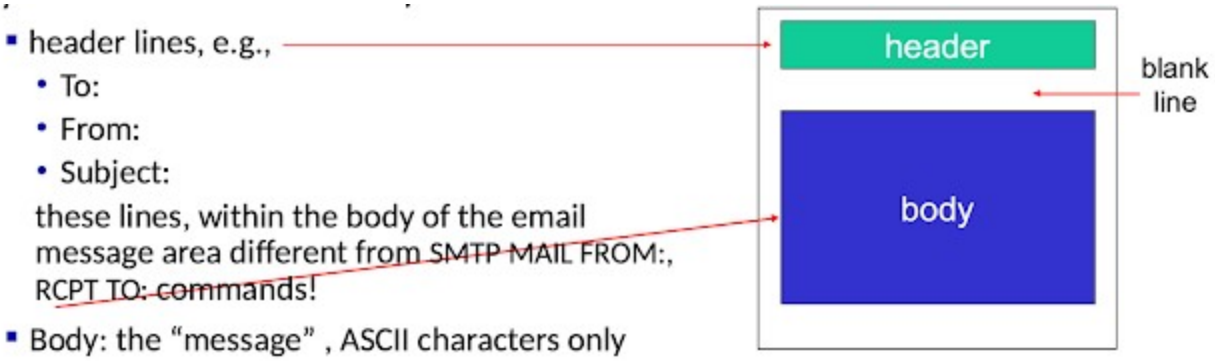
Comparison with HTTP

Let's now briefly compare SMTP with HTTP. Both protocols are used to transfer files from one host to another (object and email messages).
When transferring the files, both persistent HTTP and SMTP use persistent connections
However, there are important differences:

- **HTTP** is mainly a **pull protocol** (someone loads information to a web server and users use HTTP to pull the information from the server), in particular the TCP connection is initiated by the machine that wants to receive the file
  On the other hand, **SMTP** is primarily a **push protocol**, the sending mail server pushes the file to the receiving mail server. In particular, the TCP connection is initiated by the machine that wants to send the file
- **SMTP** requires each message, including the body of each message, to be in **7-bit ASCII** format
  HTTP does **not** impose this **restriction**
- **HTTP** encapsulates **each object** in **its own HTTP response message**.
  SMTP places **all of the message's** objects **into one message**

Mail message format

As the HTTP protocol, in the SMTP protocol the **header**, containing peripheral information, precedes the body of the message itself. As with HTTP, each header line contains readable text, consisting of a keyword followed by a colon followed by a value.
Some of the keywords are **required** and **others** are **optional**.
The header **must have** a ¿: header line and a ¿: header line.
A header **may include** a *Subject*: header line and others optional header lines
It is important to note that these header lines are **different** from the SMTP commands we studied before: The commands in that section were part of the SMTP handshaking protocol; the header lines examined in this section are part of the mail message itself.
After the last header line there is a blank line followed by the body.



Mail access protocols

Once SMTP delivers the message from Alice's mail server to Bob's mail server, the message is placed in Bob's mailbox.
Throughout this discussion we have tacitly assumed that Bob reads his mail by logging onto the server host and then executing a mail reader that runs on that host. Up until the early 1990s this was the standard way of doing things.
But today, mail access uses a client-server architecture, by executing a mail client on a local PC, users enjoy a rich set of features, including the ability to view multimedia messages and attachments

We could think that we can place a server on Bob's PC so that Alice's mail server would dialogue directly with Bob's PC. But there is some problem:

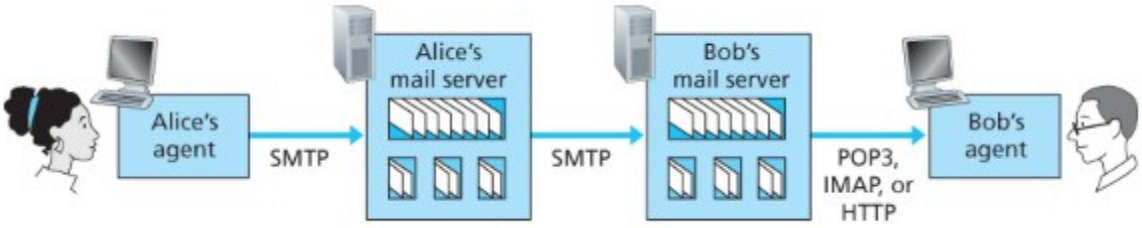
- Bob's PC would have to remain always on, and connected to the Internet
- Bob can't read the mail with other devices

Instead, a typical user runs a user agent on the local PC but accesses its mailbox stored on an always-on shared mail server.

So we have two intermediator servers, Alice's server mail (used for the outgoing message queue) and Bob's server mail (used to store the mail sent to Bob).
So we miss how Bob's user agent obtain his messages from Bob's mail server (not with SMTP because it is a push protocol)

The puzzle is completed by introducing a special mail access protocol that transfers messages from Bob's mail server to his local PC, like:

- POP3 (Post Office Protocol version 3)
- IMAP (Internet Mail Access Protocol)
- HTTP



POP3 is an extremely simple mail access protocol. Because the protocol is so simple, its functionality is rather limited
POP3 begins when the user agent (the client) opens a TCP connection to the mail server (the server) on port **110**. With the TCP connection established, POP3 progresses through three phases: authorization, transaction, and update.
During the first phase, authorization, the user agent sends a username and a password (in the clear) to authenticate the user.
During the second phase, transaction, the user agent retrieves messages; also during this phase, the user agent can mark messages for deletion, remove deletion marks, and obtain mail statistics.

The third phase, update, occurs after the client has issued the quit command, ending the POP3 session; at this time, the mail server deletes the messages that were marked for deletion.
In a POP3 transaction, the user agent issues commands, and the server responds to each command with a reply, with two possible responses: $+OK$ and $-ERR$.
The authorization phase has two principal commands: $user \ <username>$ and $pass \ <password>$
A user agent using POP3 can often be configured (by the user) to "download and delete" or to "download and keep."
In **download and delete** the mails are partitioned over the machines used by Bob to access the mail (because once a mail is read, it is removed from the mailbox and it is stored to the machine). In the **download-and-keep** mode there isn't this problem because the mails remain stored in the mailbox.
The POP3 server **does not carry state information** across POP3 sessions.

The POP3 protocol does not provide any means for a user to create remote folders and assign messages to folders. To solve this and other problems, the **IMAP** protocol was invented. It has many more features than POP3, but it is also significantly more complex.
An IMAP server will associate each message with a folder; when a message first arrives at the server, it is associated with the recipient's INBOX folder. The IMAP protocol provides commands to allow users to create folders and move messages from one folder to another.
Note that, unlike POP3, an IMAP server maintains user state information across IMAP sessions (like the names of the folders and which messages are associated with which folders).
Another important feature of IMAP is that it has commands that permit a user agent to obtain components of messages (useful when you have a low-bandwidth connection, avoiding media for example).

More and more users today are sending and accessing their email through their **Web browsers**.
With this service, the user agent is an ordinary Web browser, and the user communicates with its remote mailbox via **HTTP**.
When a sender, such as Alice, wants to send an e-mail message, the e-mail message is sent from her browser to her mail server over HTTP rather than over SMTP. Alice's mail server, however, still sends messages to, and receives messages from, other mail servers using SMTP.

DNS, The Internet's Directory Service

Just as humans can be identified in many ways (names, social security numbers, passport number, etc), so can Internet hosts too. One identifier for a host is its **hostname**.
The hostnames (for example www.facebook.com) are mnemonic and are therefore appreciated by humans. However hostnames provide little, if any, about the location within the Internet of the host (for example if the hostname ends with ".it", we know that it's probably located in Italy, but nothing more).

Furthermore, because hostnames can consist of variable-length alphanumeric characters, they would be difficult to process by routers. For these reasons, hosts are also identified by so-called **IP addresses**.
An IP address consists of four bytes and looks like $121.7.106.83$, where each period separates one of the bytes expressed in decimal notation from 0 to 255. An IP address is **hierarchical** because as we scan the address from left to right, we obtain more and more specific information about where the host is located in the Internet.

Service provided by DNS

People prefer the more mnemonic hostname identifier, while routers prefer fixed-length, hierarchically structured IP addresses. In order to reconcile these preferences, we need a directory service that translates hostnames to IP addresses. This is the main task of the Internet's **DNS** (domain name system).
The DNS is componed by:

- A **distributed database**, implemented in a hierarchy of DNS servers
- An **application-layer protocol**, that allows hosts to query the distributed database (to resolve **names**)

The DNS protocol runs over **UDP** and uses port **53**.

DNS is commonly employed by other application-layer protocols, including HTTP and SMTP, to translate user-supplied hostnames to IP addresses.
Let's see an example where the user request the URL $www.someschool.edu/index.html$:

1. The same user machine runs the client side of the DNS application.
2. The browser extracts the hostname, `www.someschool.edu`, from the URL and passes the hostname to the client side of the DNS application.
3. The DNS client sends a query containing the hostname to a DNS server.
4. The DNS client eventually receives a reply, which includes the IP address for the hostname.
5. Once the browser receives the IP address from DNS, it can initiate a TCP connection to the HTTP server process located at port 80 at that IP address.

So we can notice that DNS adds additional delay to the Internet applications that use it (fortunately the desired IP address is often cached in a "nearby" DNS server).

DNS provides this services:

- **Hostname-to-IP-address translation**
- **Host aliasing**
  A host with a complicated hostname can have one or more alias names.
  The "main" hostname is said to be a **canonical hostname** and the others are **alias hostnames**, which are typically more mnemonic
- **Mail server aliasing**
  DNS can be invoked by a mail application to obtain the canonical hostname for a supplied alias hostname as well as the IP address of the host
- **Load distribution**
  DNS is also used to perform load distribution among replicated servers, such as

replicated Web servers.
Busy sites are replicated over multiple servers, with each server running on a different end system and each having a different IP address. For replicated Web servers, a set of IP addresses is thus associated with one canonical hostname.
When clients make a DNS query for a name mapped to a set of addresses, the server responds with the entire set of IP addresses, but rotates the ordering of the addresses within each reply (the client will contact the first listed IP address, so DNS rotation distributes the traffic among the replicated servers).

The DNS is different from the applications we've already seen. Unlike these applications, the DNS is not an application with which a user directly interacts. Instead, the DNS provides a core Internet function (it is a network management application)

Overview of how DNS works

We now present a high-level overview of how DNS works. Our discussion will focus on the hostname-to-IP-address translation service.
From the perspective of the invoking application in the user's host, **DNS is a black box** providing a simple, straightforward translation service. But in fact, the black box that implements the service is complex, consisting of a large number of DNS servers distributed around the globe.

We could think about a design with a single centralized DNS server that contains all the mappings, but there would be some problems:

- **A single point of failure**
  If the DNS server crashes, so does the entire Internet!
- **Traffic volume**
  A single DNS server would have to handle all DNS queries (generated from hundreds of million of hosts)
- **Distant centralized database**
  A single DNS server cannot be "close to" all the querying clients, this can lead to significant delays
- **Maintenance**
  The single DNS server would have to keep records for all Internet hosts, and it would have to be updated frequently to account for every new host

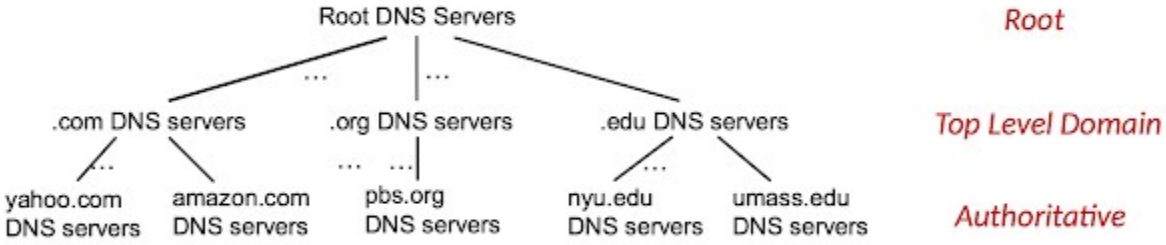
In summary, a centralized database in a single DNS server simply **doesn't scale**.
Consequently, the **DNS is distributed** by design.

In order to deal with the issue of scale, the DNS uses a large number of servers, organized in a hierarchical fashion and distributed around the world.
No single DNS server has all of the mappings for all of the hosts in the Internet. Instead, the mappings are distributed across the DNS servers.
To a first approximation, there are three classes of DNS servers:

- **Root DNS servers**
  There are over 400 root name servers scattered all over the world. These root name

servers are managed by 13 different organizations (so 13 are the server and the others are the copies of these).
Root name servers provide the IP addresses of the TLD servers

- **Top-level domain (TLD) DNS servers**
  For each of the top-level domains (such as com, org, net, edu, and gov, and also uk, fr, ca, jp, and it) there is TLD server (or server cluster).
  The company Verisign Global Registry Services maintains the TLD servers for the com top-level domain, and the company Educause maintains the TLD servers for the edu top-level domain.
- **Authoritative DNS servers**
  Every organization with publicly accessible hosts (such as Web servers and mail servers) on the Internet must provide publicly accessible DNS records that map the names of those hosts to IP addresses.
  An organization can choose to implement its own authoritative DNS server to hold these records; alternatively, the organization can pay to have these records stored in an authoritative DNS server of some service provider
  Most universities and large companies implement and maintain their own primary and secondary (backup) authoritative DNS server



Let's see an example (first approximation), suppose a DNS client wants to determine the IP address for the hostname *www.amazon.com*.

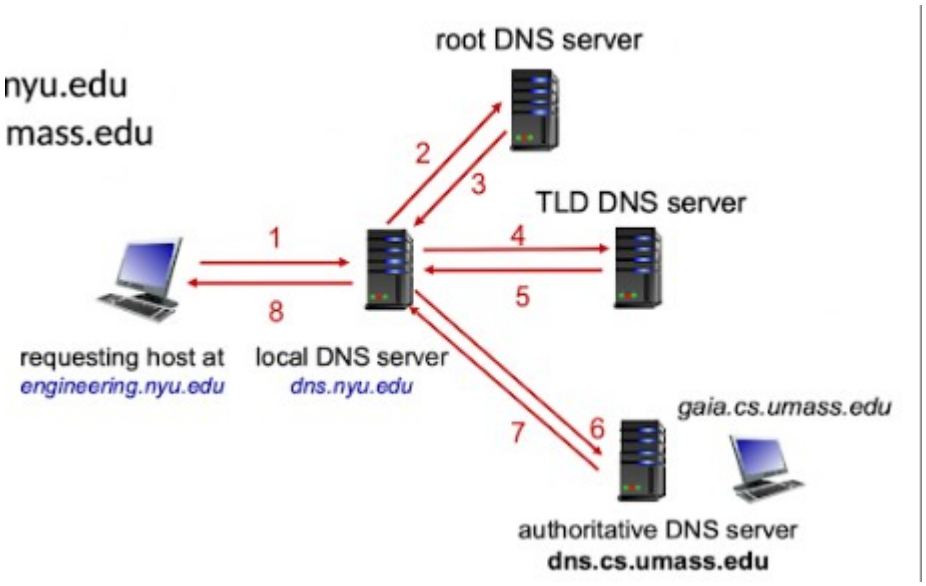
- The client first contact one of the root servers, which returns IP addresses for TLD servers for the top-level domain *com*
- The client then contacts one of these TLD servers, which returns the IP address of an authoritative server for *amazon.com*
- Finally, the client contacts one of the authoritative servers for amazon.com, which returns the IP address for the hostname *www.amazon.com*

There is another important type of DNS server called the **local DNS server**. Each ISP has a local DNS server. A host's local DNS server is typically "**close** to" the host.
When a host makes a DNS query, the query is sent to the local DNS server, which acts like a proxy, forwarding the query into the DNS server hierarchy, as we'll discuss in more detail below

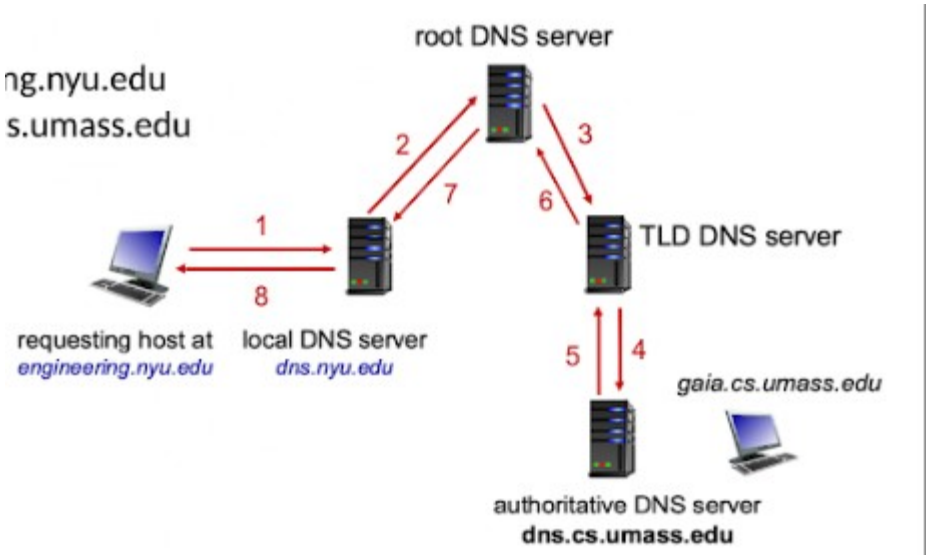
A **DNS query** can be:

- **Iterative**
  That means that the DNS server that make the request to another DNS server will receive the IP of the next DNS server to call

It's like "I don't know this name, but ask this server"



- **Recursive**
  That means that the DNS server that make the request to another DNS server will receive the IP-address-translate for the name requested, that's because the called DNS will make directly another request instead to response with the IP of the next DNS server to call



In general, the query from the requesting host to the local DNS server is recursive, and the remaining queries are iterative

Our discussion thus far has ignored **DNS caching**, a critically important feature of the DNS system. In truth, DNS extensively exploits DNS caching in order to **improve the delay performance** and to reduce the number of DNS messages ricocheting around the Internet. The idea behind DNS caching is very simple. In a query chain, when a DNS server receives a DNS reply it can cache the mapping in its local memory.

Because hosts and mappings between hostnames and IP addresses are by no means permanent, DNS servers discard cached information after a period of time TTL (often set to two days). TLD servers are typically cached in local name servers, to bypass root DNS servers.

DNS Records and Messages

The DNS servers that together implement the DNS distributed database store **resource records** (RRs), including RRs that provide hostname-to-IP address mappings
Each DNS reply message carries one or more resource records.

A resource record is a four-tuple, that contains the following fields:

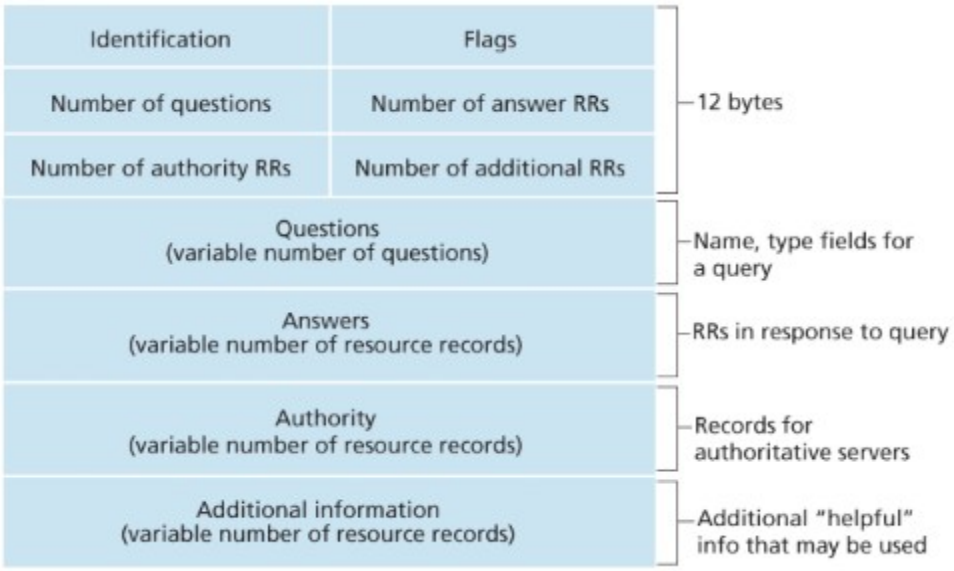
- Name
- Value
- Type
- TTL (Time To Live)

The meaning of *name* and *value* fields depend on *type* field:

- If $Type = A$, the record provides the standard hostname-to-IP address mapping
  - The *name* is a hostname
  - The *value* is the IP address for the hostname
- If $Type = NS$, the record is used to route DNS queries further along in the query chain
  - The *name* is a domain
  - The *value* is the hostname of an authoritative DNS server that knows how to obtain the IP addresses for hosts in the domain
- If $Type = CNAME$, the record can provide querying hosts the canonical name for a hostname
  - The *name* is an alias hostname
  - The *value* is the canonical hostname for the alias hostname *name*
- If $Type = MX$, the record allow the hostnames of mail servers to have simple aliases
  - The *name* an alias hostname of a mail server
  - The *value* is the canonical name of a mail server that has an alias hostname *name*

If a DNS server is authoritative for a particular hostname, then the DNS server will contain a Type A record for the hostname (even if the DNS server is not authoritative, it may contain a Type A record in its cache).
If a server is not authoritative for a hostname, then the server will contain a Type NS record for the domain that includes the hostname. It will also contain a Type A record that provides the IP address of the DNS server in the Value field of the NS record

Earlier in this section, we referred to DNS query and reply messages. These are the only two kinds of DNS messages. Furthermore, both query and reply messages have the same format:



The first 12 bytes is the **header section**. The first field is a 16-bit number that **identifies the query**. This identifier is copied into the reply message to a query, allowing the client to match received replies with sent queries.
There are a number of flags in the flag field:

- A 1-bit query/reply flag indicates whether the message is a query (0) or a reply (1)
- A 1-bit authoritative flag is set in a reply message when a DNS server is an authoritative server for a queried name
- A 1-bit recursion-desired flag is set when a client (host or DNS server) desires that the DNS server perform recursion when it doesn't have the record
- A 1-bit recursion-available field is set in a reply if the DNS server supports recursion

In the header, there are also four number-of fields. These fields indicate the number of occurrences of the four types of data sections that follow the header.

The **question section** contains information about the query that is being made (name, type of the question)
The **answer section** contains the resource records for the name that was originally queried. A reply can return multiple RRs in the answer, since a hostname can have multiple IP addresses.
The **authority section** contains records of other authoritative servers.
The **additional section** contains other helpful records

We've only seen how to retrieve the records from the database, but to be retrieved, the records must be inserted into the database.
A **registrar** is a commercial entity that verifies the uniqueness of the domain name, enters the domain name into the DNS database (as discussed below), and collects a small fee from you for its services.
Prior to 1999, a single registrar, Network Solutions, had a monopoly on domain name registration for com, net, and org domains. But now there are many registrars competing for customers, and the Internet Corporation for Assigned Names and Numbers (ICANN) accredits the various registrars.

When you register a domain name with some registrar, you also need to provide the registrar with the names and IP addresses of your primary and secondary authoritative DNS servers.
For each authoritative DNS server, the registrar would then make sure that a Type NS and a Type A record are entered into the TLD servers (for example the .com TLD server).
You'll also have to create your authoritative DNS server, and make sure that the Type A resource record for your Web server and the Type MX resource record for your mail server are entered into it.

DNS security

We have seen that DNS is a critical component of the Internet infrastructure, with many important services, including the Web and e-mail, simply incapable of functioning without it.

The first type of attack that comes to mind is a **DDoS bandwidth-flooding attack** against DNS servers.
For example, an attacker could attempt to send to each DNS root server a deluge of packets, so many that the majority of legitimate DNS queries never get answered. But today there is the traffic filtering and local DNS servers cache makes that the root servers are bypassed.

A potentially more effective **DDoS attack against** DNS would be to send a deluge of **DNS queries** to **top-level-domain servers**, for example, to all the top-level-domain servers that handle the .com domain.
It would be harder to filter DNS queries directed to DNS servers; and top-level-domain servers are not as easily bypassed as are root servers. But the severity of such an attack would be partially mitigated by caching in local DNS servers.

DNS could potentially be attacked in other ways:

- In a man-in-the-middle attack the attacker intercepts queries from hosts and returns bogus replies
- In the DNS poisoning attack, the attacker sends bogus replies to a DNS server, tricking the server into accepting bogus records into its cache

Either of these attacks could be used, for example, to redirect an unsuspecting Web user to the attacker's Web site

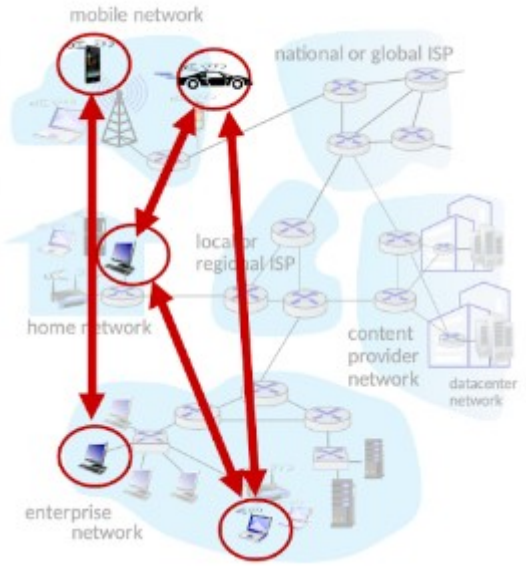
Peer-to-Peer File Distribution

The applications described in this chapter thus far, including the Web, e-mail, and DNS, all employ client-server architectures with significant reliance on **always-on** infrastructure **servers**.
In a P2P architecture there is minimal, or no, reliance on always-on infrastructure servers. Instead, pairs of intermittently connected hosts, called peers, communicate directly with each other. The peers are not owned by a service provider, but are instead desktops and laptops controlled by users.

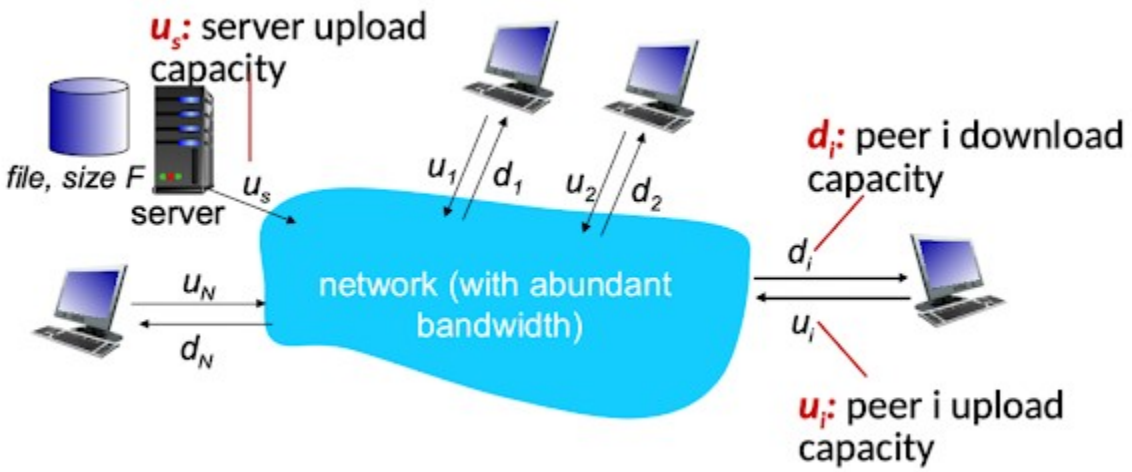
In this section we consider a very natural P2P application, namely, distributing a large file from a single server to a large number of hosts (called **peers**).

In client-server file distribution, the server must send a copy of the file to each of the peers, placing an enormous burden on the server and consuming a large amount of server bandwidth. In P2P file distribution, **each peer can redistribute** any portion of the file it has received to any other peers, thereby **assisting the server** in the distribution process. The most popular P2P file distribution protocol is BitTorrent.

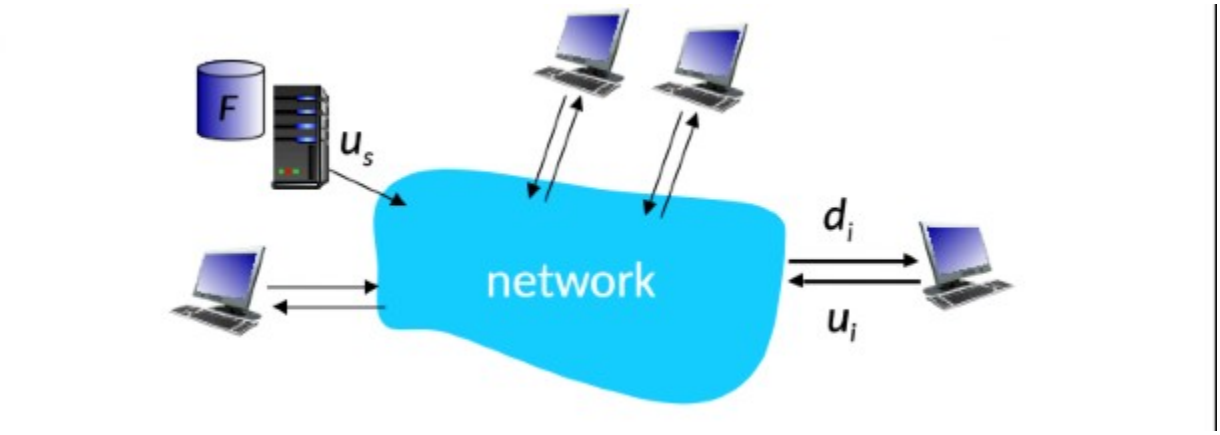
So, in these types of applications, arbitrary **end systems directly communicate with each other**. The peers request service from other peers, and provide service in return to other peers (**self scalability**). But these applications require complex management because peers should be intermittently connected, but they change IP addresses



To compare client-server architectures with peer-to-peer architectures, and illustrate the inherent self-scalability of P2P, we now consider a simple quantitative model for distributing a file to a fixed set of peers for both architecture types.
Denote the upload rate of the server's access link by u_s, the upload rate of the ith peer's access link by u_i, and the download rate of the ith peer's access link by d_i. Also denote the size of the file to be distributed (in bits) by F and the number of peers that want to obtain a copy of the file by N.



The **distribution time** is the time it takes to get a copy of the file to all N peers. Let's make the assumption that the Internet core has abundant bandwidth, implying that all of the bottlenecks are in access networks. We also suppose that the server and clients are not participating in any other network applications, so that all of their upload and download access bandwidth can be fully devoted to distributing this file.



Let's determine D_{cs}, the distribution time for the **client-server architecture**, so:

- The server must transmit one copy of the file for each of the N peers (so the server must transmit $N \cdot F$ bits).

  The time to distribute the file must be at least $\frac{N \cdot F}{u_s}$

- Let's denote d_{min} the download rate of the peer with the lowest download rate. Thus the minimum distribution time is at least $\frac{F}{d_{min}}$

Putting these two observation together, we obtain:

$$D_{cs} \geq max\left\{\frac{N \cdot F}{u_s}, \frac{F}{d_{min}}\right\}$$

We see from this equation that for N large enough, the client-server distribution time is given by $\frac{N \cdot F}{u_s}$.

Let's now go through a similar analysis for the **P2P architecture**, where each peer can assist the server in distributing the file. In particular, when a peer receives some file data, it can use its own upload capacity to redistribute the data to other peers.
Calculating the distribution time for the P2P architecture is very complicated, let's make the following observations:

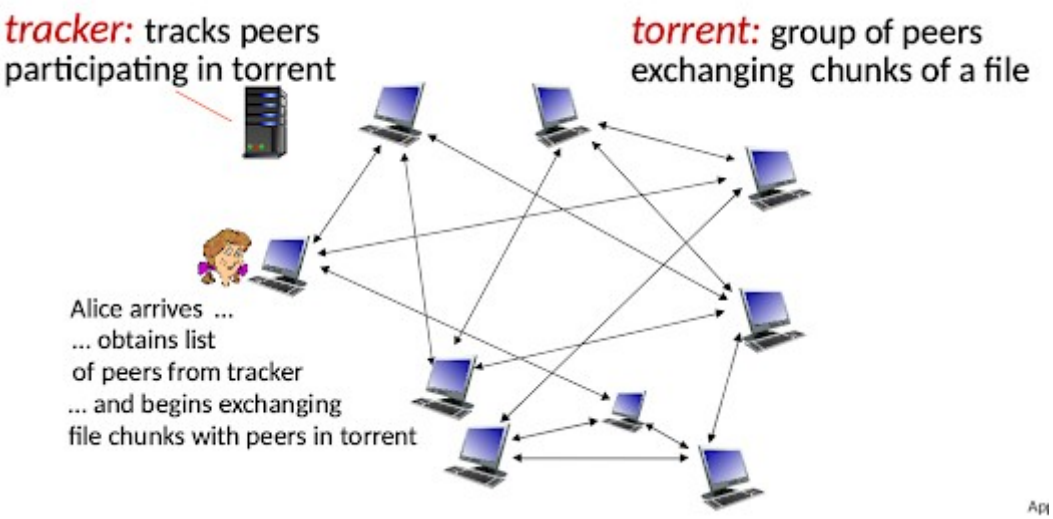
- At the beginning of the distribution, only the server has the file. To get this file into the community of peers, the server must send each bit of the file at least once into its access link. Thus, the minimum distribution time is at least $\frac{F}{u_s}$. (Unlike the client-server scheme, a bit sent once by the server may not have to be sent by the server again, as the peers may redistribute the bit among themselves)
- For the same reason of the client-server architecture, the peer with the lowest download rate cannot obtain the file in less than $\frac{F}{d_{min}}$. Thus the minimum distribution time is at least $\frac{F}{d_{min}}$
- Finally, observe that the total upload capacity of the system as a whole is equal to the upload rate of the server plus the upload rates of each of the individual peers ($u_{tot} = u_s + u_1 + \ldots + u_n$). Thus, the minimum distribution time is also at least $\frac{N \cdot F}{u_{tot}}$.

Putting these three observation together, we obtain the minimum distribution time for P2P:

$$D_{P2P} \geq max\left\{\frac{F}{u_s}, \frac{F}{d_{min}}, \frac{N \cdot F}{u_{tot}}\right\}$$

If we compare the minimum distribution time for the client-server and P2P architectures, assuming that all peers have the same upload rate u, we can observe that for the client-server architecture, the distribution time increases linearly and without bound as the number of peers increases. However, for the P2P architecture, the minimal distribution time is not only always less than the distribution time of the client-server architecture (it is also less than one hour for any number of peers).
Thus, applications with the P2P architecture can be **self-scaling**.

BitTorrent is a popular P2P protocol for file distribution. In BitTorrent lingo, the collection of all peers participating in the distribution of a particular file is called a torrent. Peers in a torrent download equal-size chunks of the file from one another, with a typical chunk size of 256 KBytes.
When a peer first joins a torrent, it has no chunks. Over time it accumulates more and more chunks.
Once a peer has acquired the entire file, it may (selfishly) leave the torrent, or (altruistically) remain in the torrent and continue to upload chunks to other peers.
Each torrent has an infrastructure node called a **tracker**. When a peer joins a torrent, it registers itself with the tracker and periodically informs the tracker that it is still in the torrent, the tracker keeps track of the peers that are participating in the torrent.

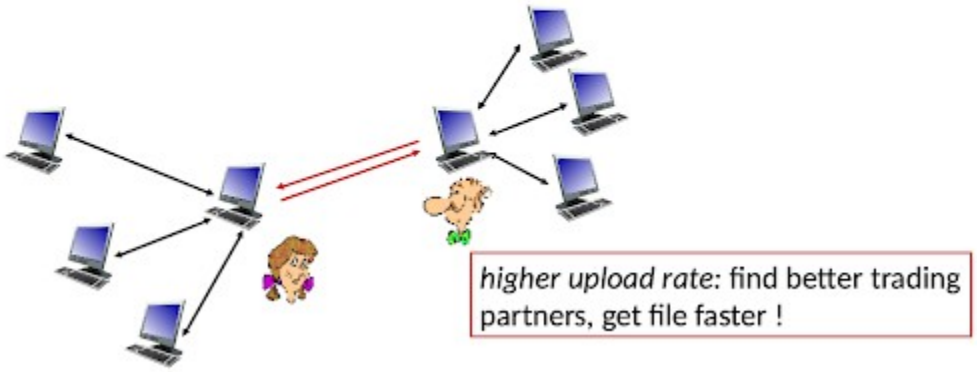


As shown in figure, when a new peer, Alice, joins the torrent, the tracker randomly selects a subset of peers, then Alice attempts to establish concurrent TCP connections with all the peers (peers in connection are called **neighbor peers**) on this list. As time evolves, some peers could leave and others join the list.

Periodically, Alice will ask each of her neighboring peers for the list of the chunks they have, with this knowledge Alice issues requests for chunks she currently doesn't have. In deciding which missing chunks to request, Alice uses a technique called **rarest first** (the chunks there are rarest among their neighbors).
In this manner, the rarest chunks get more quickly redistributed, aiming to (roughly) equalize the numbers of copies of each chunk in the torrent.

To determine which requests she responds to, the basic idea is that Alice gives priority to the neighbors that are currently supplying her data at the **highest rate**.
So Alice continually measures the rate at which she receives bits and determines the four peers that are feeding her bits at the highest rate. Every 10 seconds, she recalculates the rates and possibly modifies the set of four peers. These four peers are said to be **unchoked** (and the others are **choked**).
Importantly, every 30 seconds, she also picks one additional neighbor (said **optimistically unchoked**) at random and sends it chunks and this new peer may join the top 4. This mechanism is often referred to as tit-for-tat

(1) Alice "optimistically unchokes" Bob
(2) Alice becomes one of Bob's top-four providers; Bob reciprocates
(3) Bob becomes one of Alice's top-four providers



Video Streaming and Content Distribution Networks

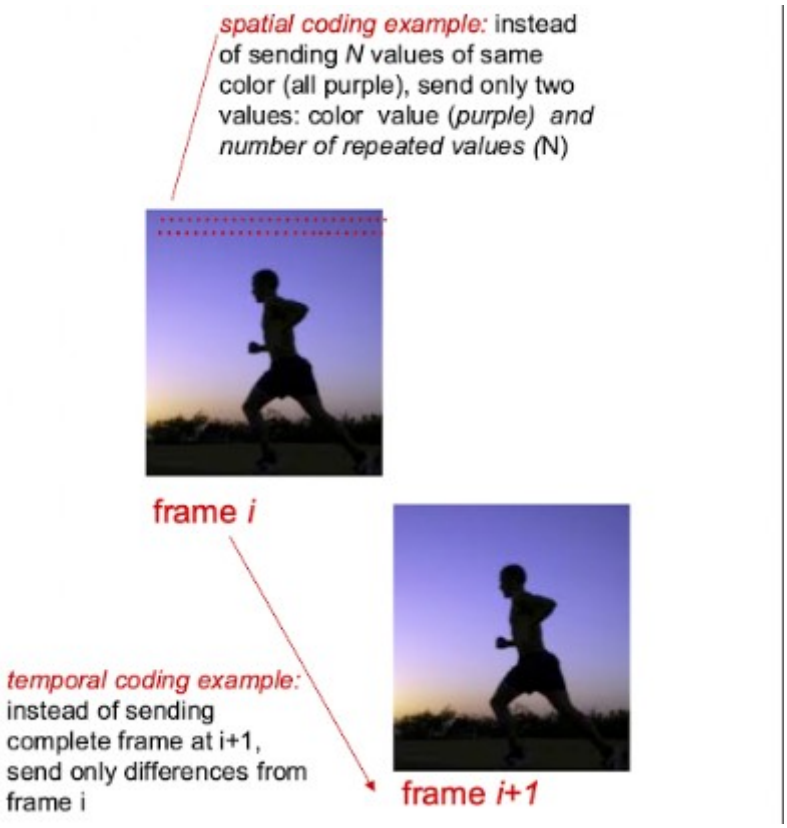
Streaming prerecorded video now accounts for the majority of the traffic in residential ISPs in North America (80%)

Internet video

In streaming stored video applications, the underlying medium is prerecorded video. These prerecorded videos are placed on servers, and users send requests to the servers to view the videos on demand.
Before talking about the video streaming, we should first get a quick feel for the video medium itself.
A **video** is a sequence of images, typically being displayed at a constant rate, for example, at 24 or 30 images per second.
An uncompressed, digitally encoded **image** consists of an array of pixels, with each **pixel** encoded into a number of bits to represent luminance and color.
An important characteristic of video is that it can be **compressed**, thereby trading off video quality with bit rate. Of course, the higher the bit rate, the better the image quality and the better the overall user viewing experience.

From a networking perspective, perhaps the most salient characteristic of video is its high bit rate (4K streaming needs a bitrate of more than 10Mbps).
In order to provide continuous playout, the network must provide an average **throughput** to the streaming application that is at least as large as the bit rate of the compressed video.
There are some technique to decrease the number of bits used to encode images:

- Spatial (within an image)
- Temporal (between more images)



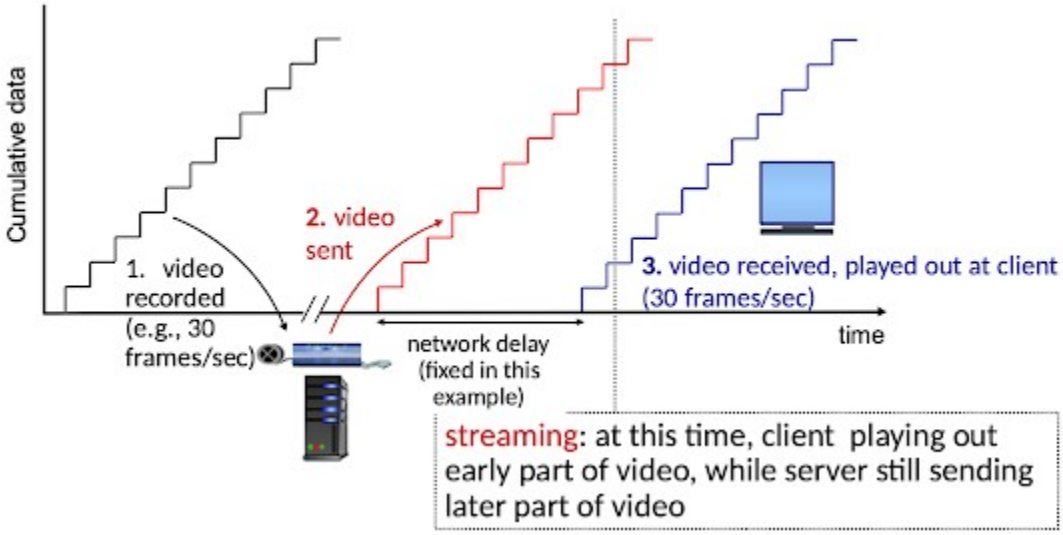
HTTP Streaming and DASH

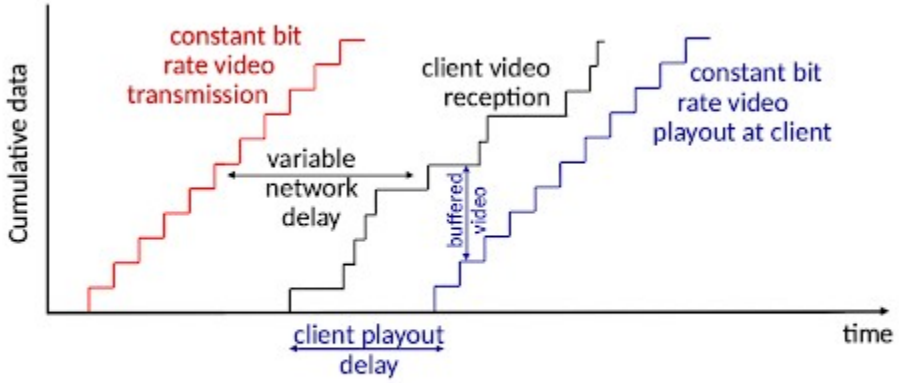
In HTTP streaming, the video is simply stored at an HTTP server as an ordinary file with a specific URL. When a user wants to see the video, the client establishes a TCP connection with the server and issues an HTTP GET request for that URL.

The server then sends the video file, within an HTTP response message, as quickly as the underlying network protocols and traffic conditions will allow. The variability of the bandwidth and the packet loss/delay will delay playout or result in poor quality.

On the client side, the bytes are collected in a client application buffer (needed to match continues playout constraint). Once the number of bytes in this buffer exceeds a predetermined threshold, the client application begins playback.

Thus, the video streaming application is displaying video as it is receiving and buffering frames corresponding to later parts of the video. Other challenges include the client interactivity (pause, fast-forward, rewind etc).





- *client-side buffering and playout delay:* compensate for network-added delay, delay jitter

But the clients receive the same encoding of the video (despite the large variations of bandwidth of different clients and also over time for the same client).
This has led to the development of a new type of HTTP-based streaming, often referred to as Dynamic Adaptive Streaming over HTTP (**DASH**). In DASH, the video is encoded into several different versions, with each version having a different bit rate and, correspondingly, a different quality level. The client selects different chunks one at a time with HTTP GET request messages.
DASH allows clients with different Internet access rates to stream in video at different encoding rates, and also allows a client to adapt to the available bandwidth if the available end-to-end bandwidth changes during the session (important for mobile users).
With DASH, each video version is stored in the HTTP server, each with a different URL. The server has a **manifest file**, which provides a URL for each version along with its bit rate.
To determine which encoding rate is better for a client in a determinate time, there are some algorithms that analyze the buffer and estimate server-to-client bandwidth.
The clients can choose different coding rates at different points in time and from different servers, so they have a form of **intelligence** and they determine:

- **When** to request a chunk
- **What** encoding rate to request
- **Where** to request chunk

Content Distribution Networks

Today, many Internet video companies are distributing on-demand multi-Mbps streams to millions of users on a daily basis. Streaming all this traffic to locations all over the world while providing continuous playout and high interactivity is clearly a challenging task.

The first option for this challenge (and the most straightforward) is to build a **single massive data center**, and stream the videos directly from the data center to clients worldwide. But there are few problems with this approach:

1. **Long, and possibly congested, path to distant clients**
   If the client is far from the data center the packets will cross many communication links and nodes, with a lot of delay (for example if the throughput of one of these links is less than the video consumption rate).
2. **Resend the same data**
   A popular video will likely be sent many times over the same communication links, with a waste of the network bandwidth and the video company will pay the provider for sending the same data into the Internet over and over again.
3. **Single point of failure**
   If the data center or its links to the Internet goes down, it would not be able to distribute any video streams

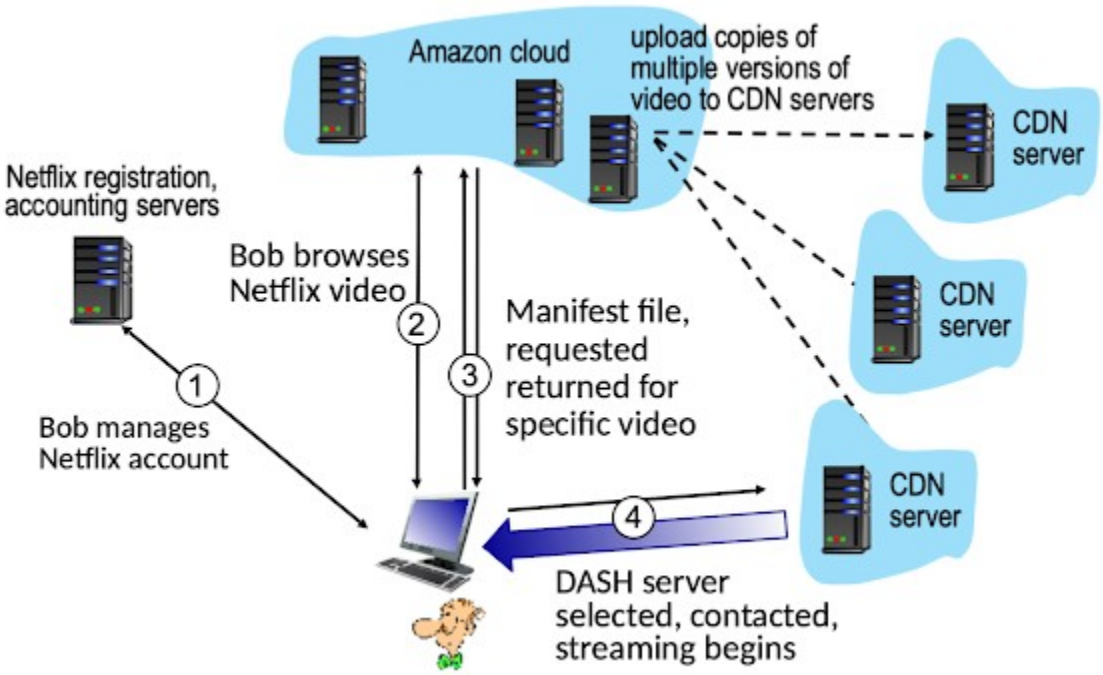
So this solution **doesn't scale** and goes against the philosophy of the Internet.

In order to meet this challenge, the video-streaming companies store and serve multiple copies of videos at multiple geographically distributed sites, creating **Content Distribution Networks** (CDNs). A CDN manages these servers, stores copies of the videos in its servers and attempts to direct each user request to a CDN location that will provide him the best user experience.
A CDN may be a **private** CDN or a **third-party** CDN (Akamai, Limelight, etc).
There are two philosophies for the servers placement:

- **Enter deep**
  The philosophy is to enter deep into the access networks, by deploying servers in access ISPs all over the world with the goal of **being closed to the user**.
  But the task of maintaining and managing the clusters becomes challenging
- **Bring home**
  The philosophy is to bring the ISPs home by building large clusters at a smaller number (for example 10) of sites. Compared with the enter deep philosophy, the bring-home design typically results in lower maintenance and management overhead (but with higher delay and lower throughput to end users)

Once its clusters are in place, the CDN replicates content across its clusters. The CDN may not want to place a copy of every video in each cluster, since some videos are rarely viewed or are only popular in some countries.

Let's see how a CDN works. When a browser in a user's host is instructed to retrieve a specific video (identified by a URL), the CDN must intercept the request so that it can determine a suitable CDN server cluster for that client at that time, and redirect the client's request to a server in that cluster. So the client will establish a TCP connection with that server and start to exchange data (for example the manifest if DASH is used)



At the core of any CDN deployment is a **cluster selection strategy**, that is, a mechanism for dynamically directing clients to a server cluster or a data center within the CDN.
The CDN learn the IP address of the client local DNS server and then select an appropriate cluster (geographically closed or current traffic condition based)

Socket programming: Creating Network Applications

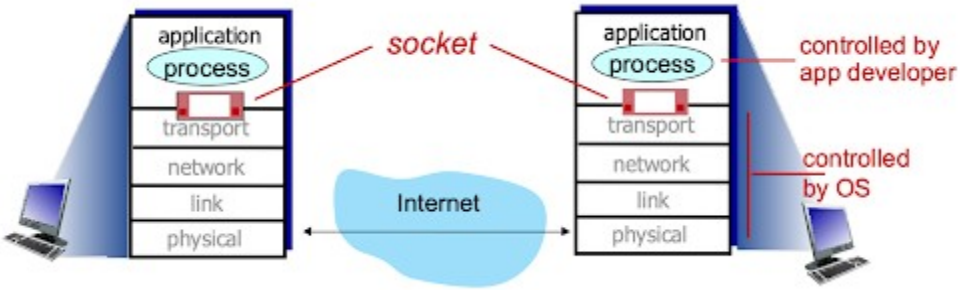
Now that we've looked at a number of important network applications, let's explore how network application programs are actually created.
Recalling from previous sections, a network application is composed of two programs (the client and the server) residing in two different systems. When these two programs are executed, a client process and a server process are created, and these processes communicate with each other by reading from, and writing to, sockets. The developer's main task is to write the code for both client and server programs.

There are two types of network applications:

- An implementation whose operation is specified in a protocol standard
  You have to follow the rules dictated in the protocol (that are known to all), for example the client (or server) side of the HTTP. It's important to notice that if you follow the rules, your client will be able to communicate with a server written by another person.

- A proprietary network application
  In this case the client and the server programs employ an application-layer protocol that has not been openly published. Other independent developers will not be able to develop code that interoperates with the application.

During the development phase of a network application, one of the first decisions the developer must make is whether the application is to run over TCP or over UDP. He also have to decide the port number (being careful to avoid using such well-known port numbers)

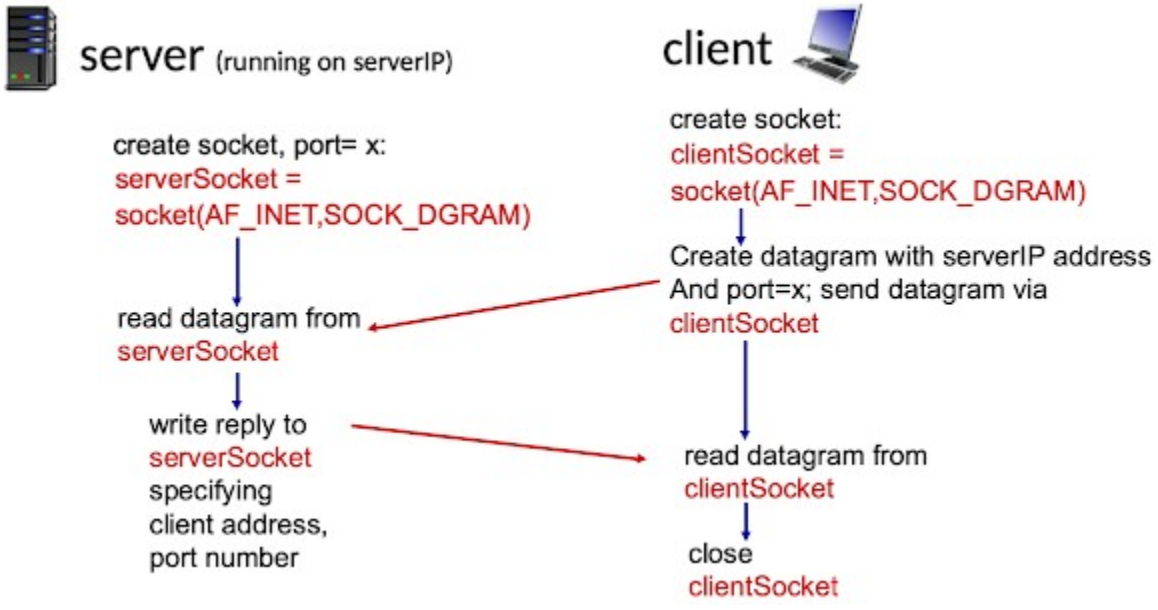


Let's make an example of a network application:

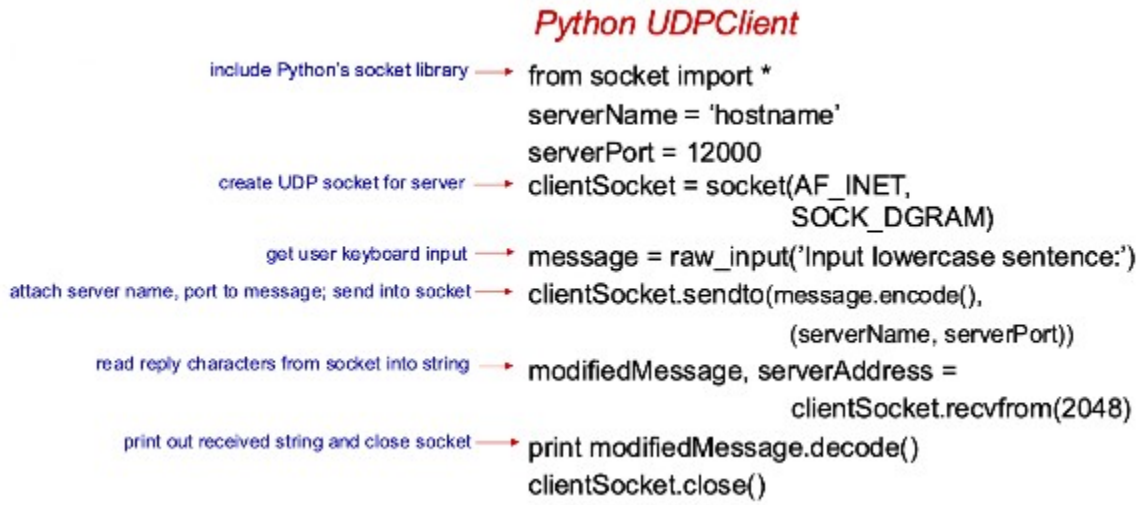
1. client reads a line of characters (data) from its keyboard and sends data to server
2. server receives the data and converts characters to uppercase
3. server sends modified data to client
4. client receives modified data and displays line on its screen

Socket programming with UDP

Before the sending process can push a packet of data out the socket door, when using UDP, it must first attach a destination address to the packet (used from the Internet to route the packet). But we've already seen that a host may run different processes, so we also have to identify the destination **socket** for that application using a **port number**.

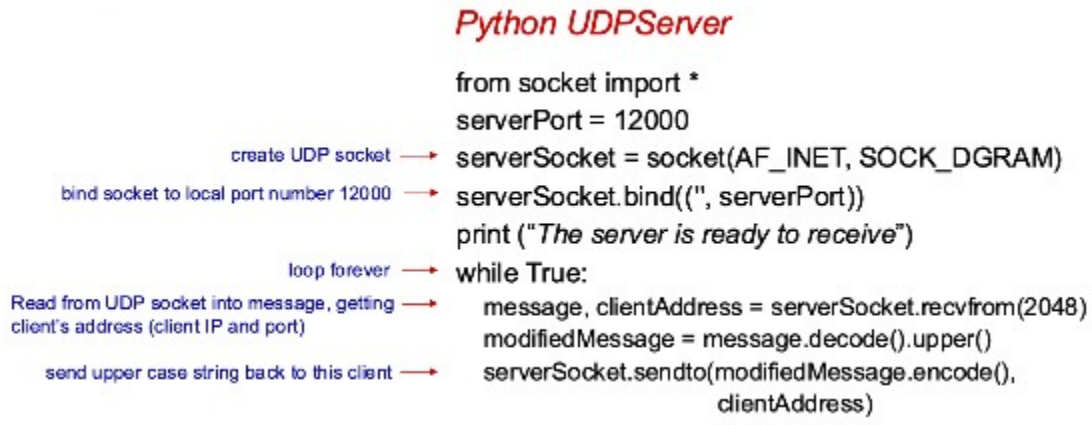


Let's see the client application for our example



The *serverName* and the *serverPort* are the information of the server to which we want to send our message. The *serverName* could be an IP address or an hostname (then a DNS will translate it into an IP address).
We create a socket called *clientSocket* which works on IPv4 (AF_INET) and it is an UDP socket (SOCK_DGRAM).
With *clientSocket*.*sendto* we send the message (converted in bytes with encode) through the socket to the destination host, passing the message and telling which destination server and which destination port to attach to the message.
When the server response arrives, the packet's data will be saved in *modifiedMessage* and the packet's source address is put into variable *serverAddress* (even it isn't required, because we already know it). 2048 is the buffer size.

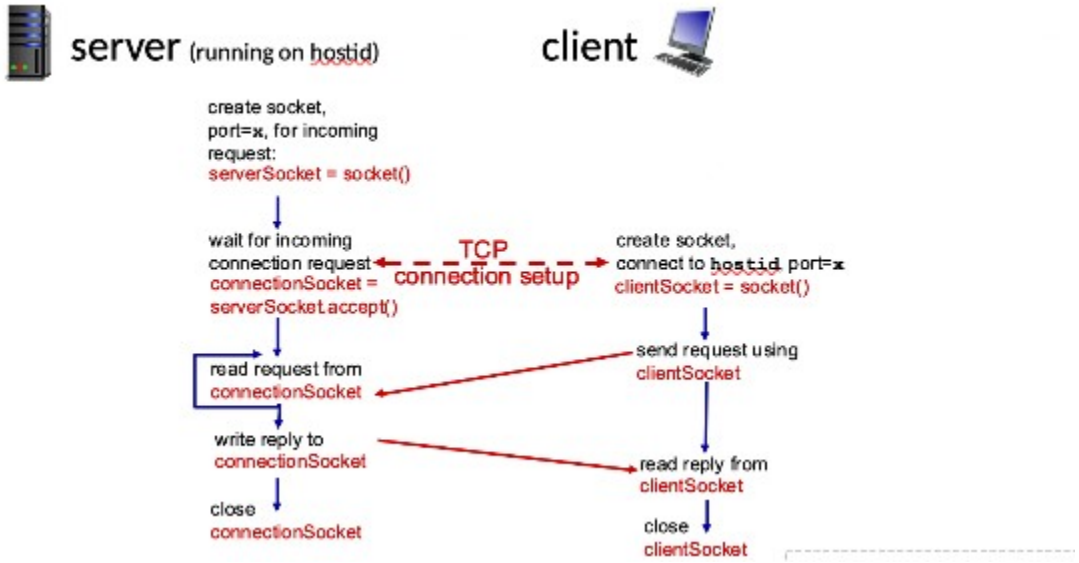
Let's see the server application:



The line *serverSocket*.*bind* assign the port 12000 to the socket we have created, then the UDP server enters in a while loop to receive and process packets.

Socket programming with TCP

Unlike UDP, TCP is a connection-oriented protocol, so it needs the handshake before starting to send data. When we create a TCP connection, we associate with it the client socket address and the server socket address (created by the server for this connection).
With the TCP connection established, when one side wants to send data to the other side, it just drops the data into the TCP connection via its socket.



The server has a special socket that welcomes some initial contact from client processes.
The server will create a socket for every TCP connection that it will establish.

Let's see the client application for our example:

Python TCPClient

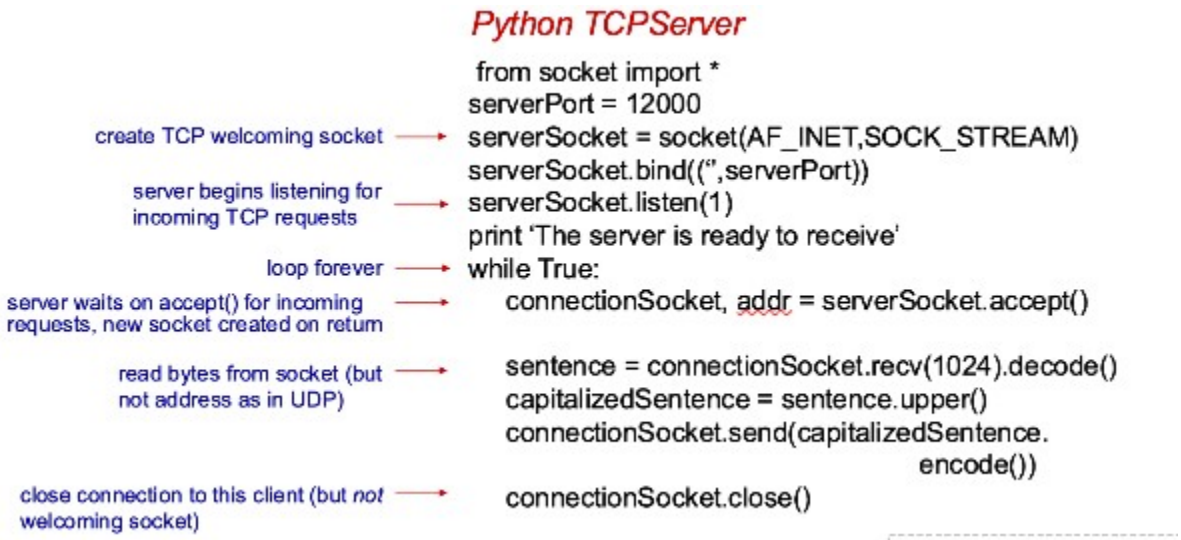
```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName,serverPort))
sentence = raw_input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print ('From Server:', modifiedSentence.decode())
clientSocket.close()
```

create TCP socket for server, remote port 12000

No need to attach server name, port

We can see that in this case we pass to the socket creation function SOCK_STREAM (that is for TCP).
With *clientSocket.connect* we establish the TCP connection with that socket server address.
Then we can send a message to that socket with *clientSocket.send*.
In this case there is no need in the message to attach the server name and the port (because the socket is associated with a single server over the TCP connection)

Let's see the server application:



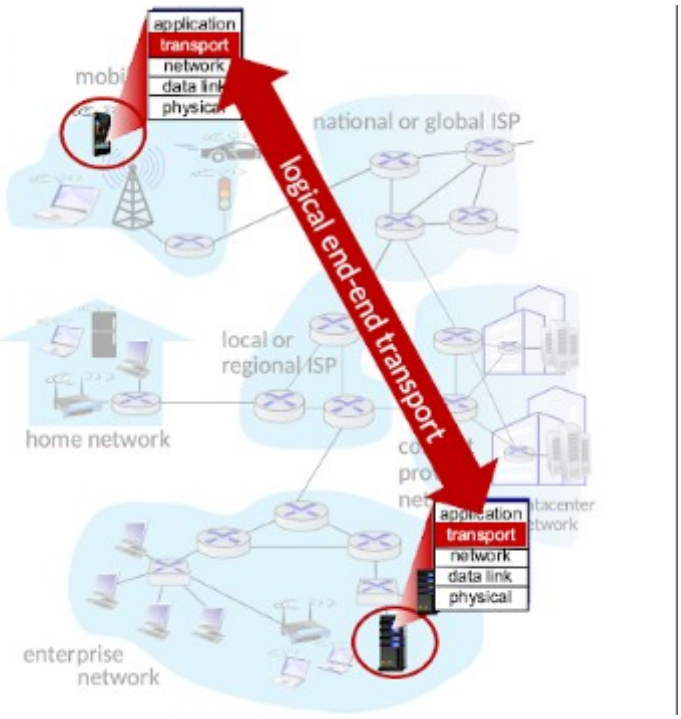
The *serverSocket*.*listen* specifies the maximum number of queued TCP connections.
When a client knocks on this door, the program invokes the *accept*() method for serverSocket, which creates a new socket in the server, called *connectionSocket*, dedicated to this particular client. Then the client and the server create the TCP connection between the client socket and this new server socket.

Transport Layer

Residing between the application and network layers, the transport layer is a central piece of the layered network architecture. It has the critical role of providing communication services directly to the application processes running on different hosts.

Introduction and Transport-Layer Services

A transport-layer protocol provides for **logical communication** between application processes running on different hosts. We call it logical because from an application perspective the hosts are directly connected, but in reality the hosts are very far from each other.
Application processes use the logical communication provided by the transport layer to send messages to each other, free from the worry of the details of the physical infrastructure used to carry these messages.



Transport-layer protocols are implemented in the end systems but not in network routers.
On the sending side, the transport layer converts the application-layer messages it receives from a sending application process into transport-layer packets (breaking the application messages into smaller chunks and adding transport-layer header), known as transport-layer **segments** in Internet terminology.
It's important to note that network routers act only on the network-layer fields of the datagram, so they don't see the transport-layer segment.

More than one transport-layer protocol may be available to network applications. For example, the Internet has two protocols: **TCP** and **UDP**. Each of these protocols provides a different set of transport-layer services to the invoking application.

Relationship Between Transport and Network Layers

Recall that the transport layer lies just above the network layer in the protocol stack.
A transport layer protocol provides **logical** connections between **processes** running on different end systems.
A network layer protocol provides **logical** connections between **hosts**.
The services that a transport protocol can provide are often constrained by the service model of the underlying network-layer protocol.
Nevertheless, certain services (like reliable data transfer or encryption) can be offered by a transport protocol even when the underlying network protocol doesn't offer the corresponding service at the network layer.

Overview of the Transport Layer in the Internet

Recall that the Internet makes two distinct transport-layer protocols available to the application layer:

- **UDP** (User Datagram Protocol)
  Which provides an unreliable, connectionless service to the invoking application
- **TCP** (Transmission Control Protocol)
  Which provides a reliable, connection-oriented service to the invoking application

When designing a network application, the application developer must specify one of these two transport protocols.
To simplify terminology, we refer to the transport-layer packet as a **segment**.

Let's say a few words about the Internet's network layer. The Internet's network layer protocol has a name: the **IP** (Internet Protocol). The IP provides logical communication between hosts. The IP service model is a **best-effort delivery service** (so makes its "best effort" to deliver segments, but it does not make guarantees like the delivery, the order and the integrity). For this reason IP is said to be an **unreliable service**.
We also mention here that every host has at least one network-layer address, a so-called **IP address**.

Let's now summarize the service models provided by UDP and TCP.
The most fundamental responsibility of UDP and TCP is to extend IP's delivery service between two end systems to a delivery service between two processes running on the end systems. Extending host-to-host delivery to **process-to-process delivery** is called transport-layer **multiplexing** and **demultiplexing**.
UDP and TCP also provide integrity checking by including **error-detection** fields in their segments' headers.
These are the only two services (process-to-process delivery and the error checking) that **UDP** provides (it is unreliable like IP).

TCP offers several additional services to applications:

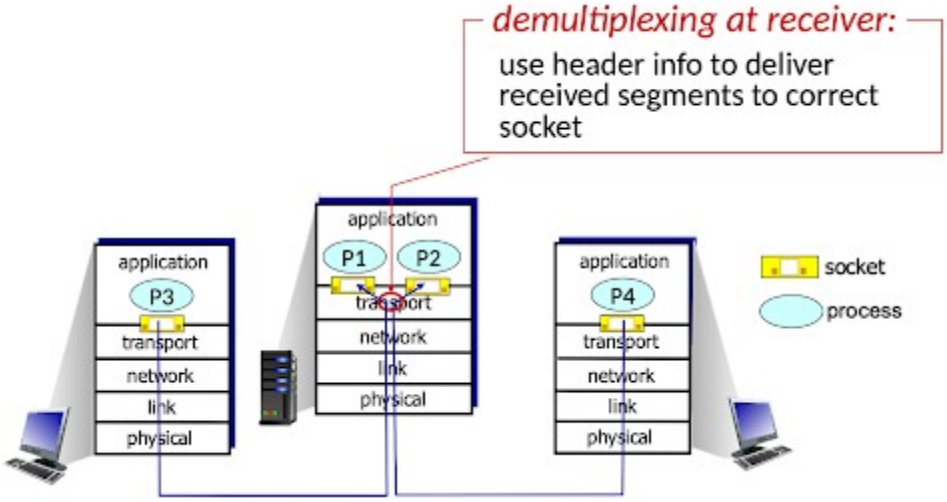
- **Reliable data transfer**
  Using flow control, sequence numbers, acknowledgments, and timers TCP ensures that data is delivered from sending process to receiving process, correctly and in order
- **Congestion control**
  Congestion control is not so much a service provided to the invoking application as it is a service for the Internet as a whole, a service for the general good
  TCP strives to give each connection traversing a congested link an equal share of the link bandwidth. This is done by regulating the rate at which the sending sides of TCP connections can send traffic into the network.

A protocol that provides reliable data transfer and congestion control is necessarily complex. We'll need several sections to cover the principles of reliable data transfer and congestion control, and additional sections to cover the TCP protocol itself

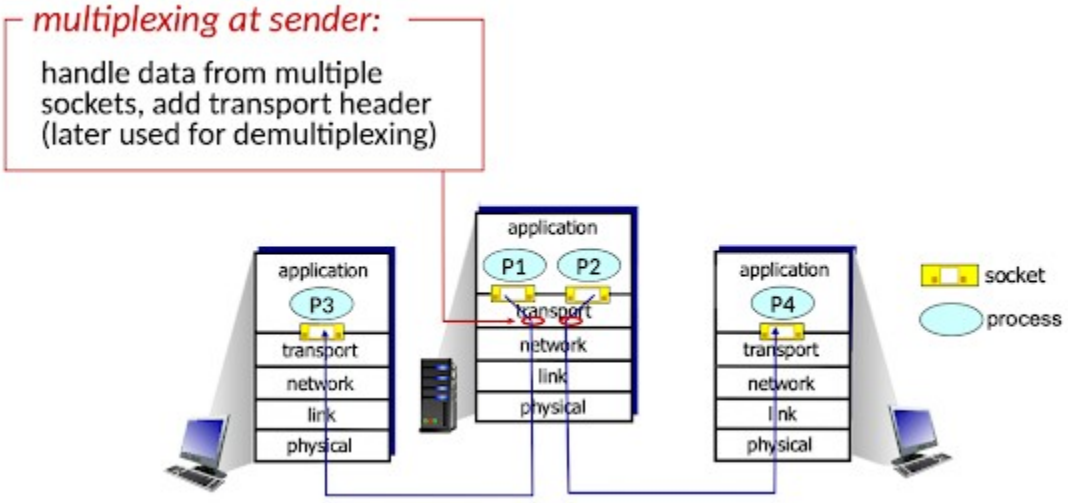
Multiplexing and Demultiplexing

In this section, we discuss transport-layer multiplexing and demultiplexing, that is, extending the host-to-host delivery service provided by the network layer to a process-to-process delivery service for applications running on the hosts.

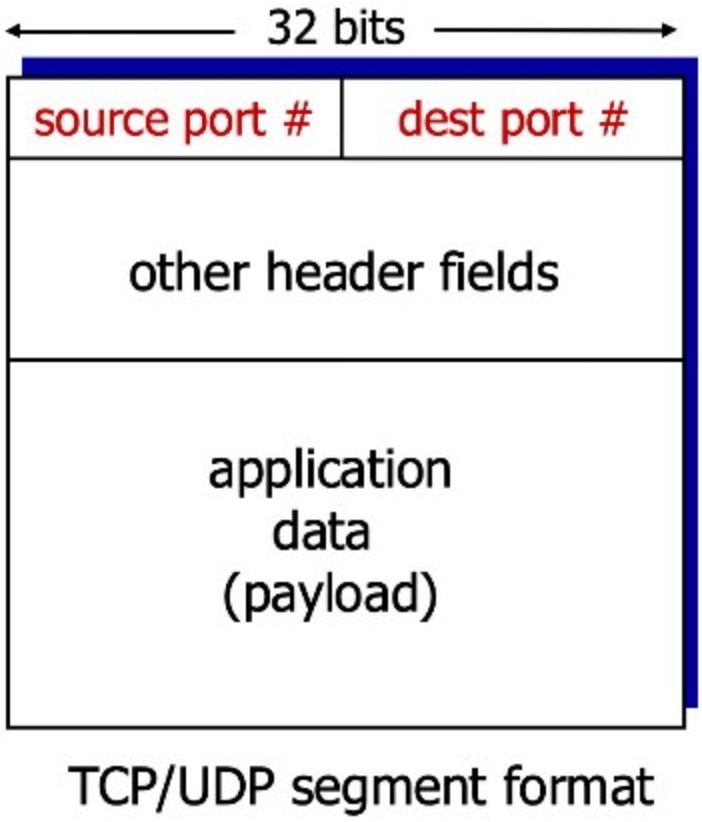
As we said before, when the transport layer in your computer receives data from the network layer below, it needs to direct the received data to one of the processes of your computer, this is done with the intermediary **sockets**, that have an identifier.
To direct an incoming transport-layer segment to the appropriate socket each transport-layer segment has a set of fields in the segment for this purpose. At the receiving end, the transport layer examines these fields to identify the receiving socket and then directs the segment to that socket.
This job of delivering the data in a transport-layer segment to the correct socket is called **demultiplexing**.



The job of gathering data chunks at the source host from different sockets, encapsulating each data chunk with header information (that will later be used in demultiplexing) to create segments, and passing the segments to the network layer is called **multiplexing**.



From the discussion above, we know that transport-layer multiplexing requires that sockets have unique identifiers, and that each segment has special fields that indicate the socket to which the segment is to be delivered.

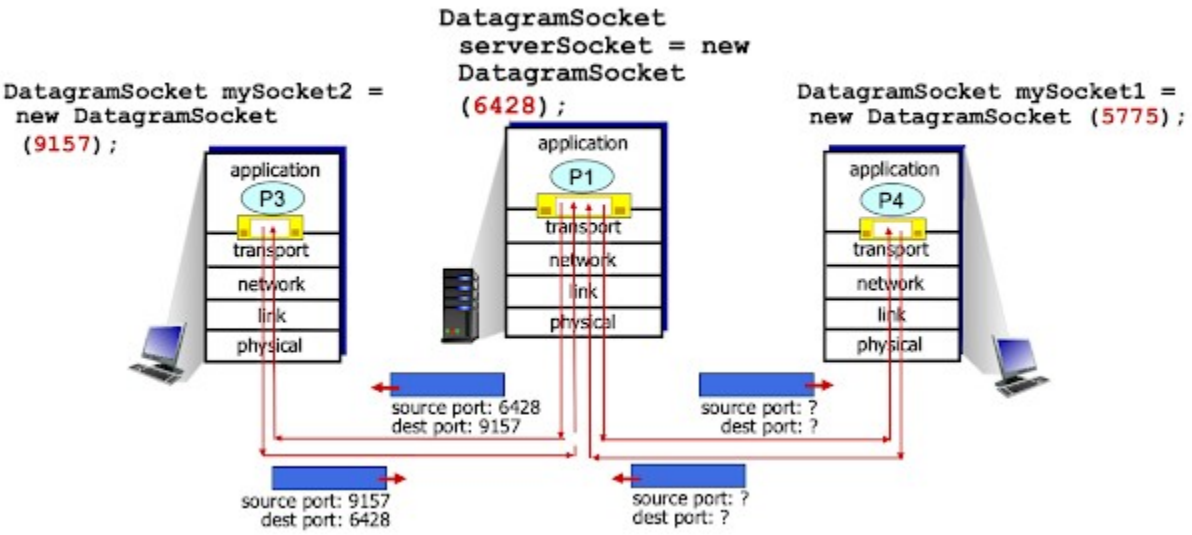


TCP/UDP segment format

These special fields, illustrated in the figure, are the **source port number field** and the **destination port number field**.
Each port number is a 16-bit number, ranging from 0 to 65535, but port numbers from 0 to 1023 are called **well-known port numbers** (reserved for well-known application protocols such as HTTP).

Let's talk about **connectionless** multiplexing and demultiplexing (UDP).
When we create a socket we must specify the **host-local port** (in the range 1024 to 65535, or the well-known if we are implementing an already-existing protocol).
Typically the client side of the application lets the transport layer automatically (and transparently) assign the port number and the server side assigns a specific port number to the receiver process.
It is important to note that a UDP socket is fully identified by a two-tuple consisting of a destination IP address and a destination port number. As a consequence, if two UDP segments have different source IP addresses and/or source port numbers, but have **the same**

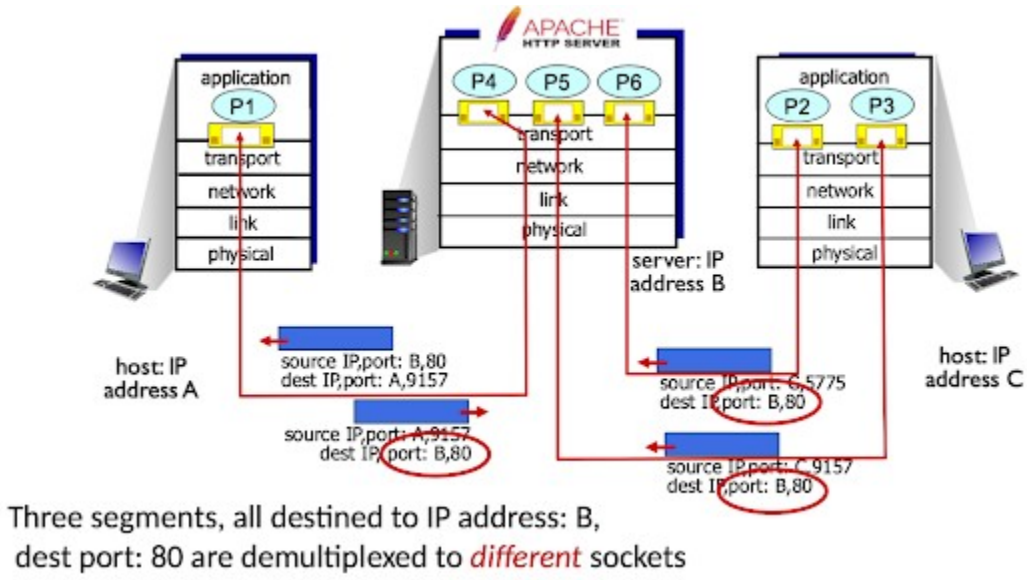
destination IP address and destination port number, then the two segments will be directed **to the same destination process via the same destination socket**.
The purpose of the source port number is like an “return address”, to send back a response segment to that process.



Let’s now talk about **connection-oriented** multiplexing and demultiplexing (TCP).
One subtle difference between a TCP socket and a UDP socket is that a TCP socket is identified by a four-tuple:

- source IP address
- source port number
- destination IP address
- destination port number

Thus, when a TCP segment arrives from the network to a host, the host uses **all four values** to direct (demultiplex) the segment to the appropriate socket.
In particular, and in contrast with UDP, two arriving TCP segments with different source IP addresses or source port numbers will (with the exception of a TCP segment carrying the original connection-establishment request) be directed to two different sockets. So the server may support simultaneous TCP sockets (each socket associated with a different client process)



Connectionless Transport: UDP

To motivate our discussion about UDP, suppose you were interested in designing a **no-frills**, **bare-bones** transport protocol.
You might first consider using a vacuous transport protocol, but you must do some actions.
In particular, on the sending side, you might consider taking the messages from the application process and passing them directly to the network layer. On the receiving side, you might consider taking the messages arriving from the network layer and passing them directly to the application process.
So a transport protocol has to provide a **multiplexing/demultiplexing** service.

UDP does just about as little as a transport protocol can do. In fact it provides the multiplexing/demultiplexing and some light error checking, it adds nothing more to IP.

UDP takes messages from the application process, attaches source and destination port number fields for the multiplexing/demultiplexing service, adds two other small fields, and passes the resulting segment to the network layer. The network layer encapsulates the transport-layer segment into an IP datagram and then makes a best-effort attempt to deliver the segment to the receiving host. If the segment arrives at the receiving host, UDP uses the destination port number to deliver the segment's data to the correct application process.

An UDP segment may be **lost** or **delivered-out-of-order** to the app.
UDP doesn't make the handshaking, in fact is said to be **connectionless** (each UDP segment is handled independently of others)

Why should an application choose UDP when you can have reliable transfer with TCP? There are some types of applications that are better suited for UDP for the following reasons:

- **Finer application-level control over what data is sent, and when**
  For example UDP has **not congestion control**
  This seems a bad thing but in some application, where the speed is more important than the reliability, this is an advantage
- **No connection establishment**
  TCP uses a three-way handshake before it starts to transfer data. UDP just blasts away without any formal preliminaries.
  Thus UDP does not introduce any **delay to establish a connection** (this delay is 1.5 RTT).
- **No connection state**
  TCP maintains connection state in the end systems. This connection state includes receive and send buffers, congestion-control parameters, and sequence and acknowledgment number parameters.
  UDP does not track any of these parameters, for this reason, a server devoted to a particular application can typically support many more active clients when the application runs over UDP rather than TCP.
- **Small packet header overhead**
  The TCP segment has 20 bytes of header overhead in every segment, whereas UDP has only 8 bytes of overhead

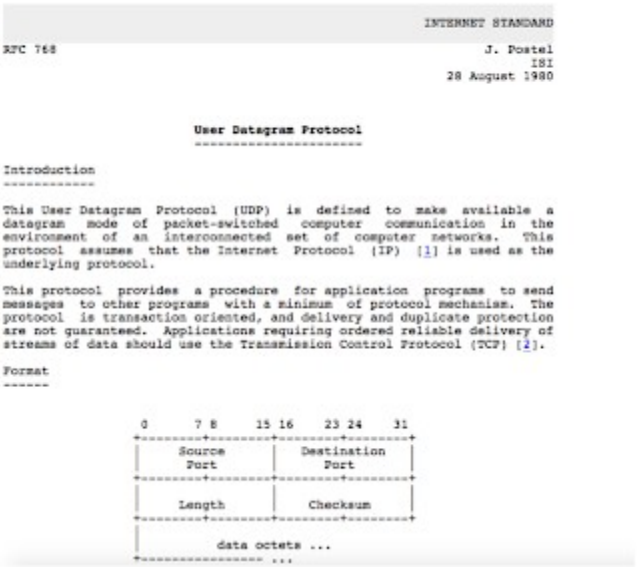
UDP protocol is used in:
- streaming multimedia apps (loss tolerant, rate sensitive)
- DNS
- SNMP (a protocol used to manage small computer networks, and it prefers UDP because it must often runs when the network is in a stressed state)
- HTTP/3 (only in some cases)

If **reliable transfer is needed also over UDP** (e.g. HTTP) you can add the needed reliability and the congestion control at the **application layer**.
Most multimedia applications uses both TCP and UDP

A small observation is that running multimedia applications over UDP is controversial, because if everyone starts streaming high-bit-rate video without using any congestion control (not provided by UDP), there would be the congestion of the network (also for TCP application).

The UDP protocol is very simple, in fact the document of the standard UDP is composed by 2 pages

UDP: User Datagram Protocol [RFC 768]



INTERNET STANDARD

RFC 768

J. Postel
ISI
28 August 1980

User Datagram Protocol

Introduction

This User Datagram Protocol (UDP) is defined to make available a datagram mode of packet-switched computer communication in the environment of an interconnected set of computer networks. This protocol assumes that the Internet Protocol (IP) [1] is used as the underlying protocol.

This protocol provides a procedure for application programs to send messages to other programs with a minimum of protocol mechanism. The protocol is transaction oriented, and delivery and duplicate protection are not guaranteed. Applications requiring ordered reliable delivery of streams of data should use the Transmission Control Protocol (TCP) [2].

Format

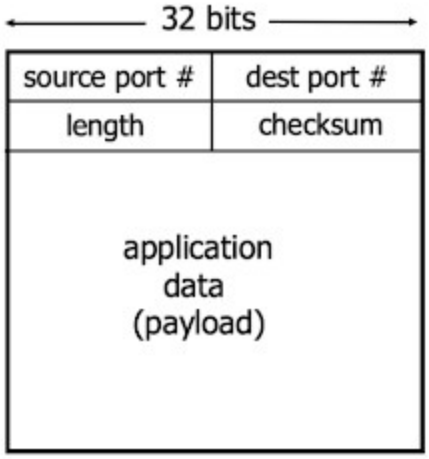
The UDP sender does this actions:

- Receive an application-layer message
- Determines UDP segment header fields
- Creates UDP segment
- Passes segment to IP

The UDP receiver does this actions:

- Receive segment from IP
- Checks UDP checksum header value
- Extracts application-layer message
- Demultiplexes message up to application via socket

UDP Segment Structure



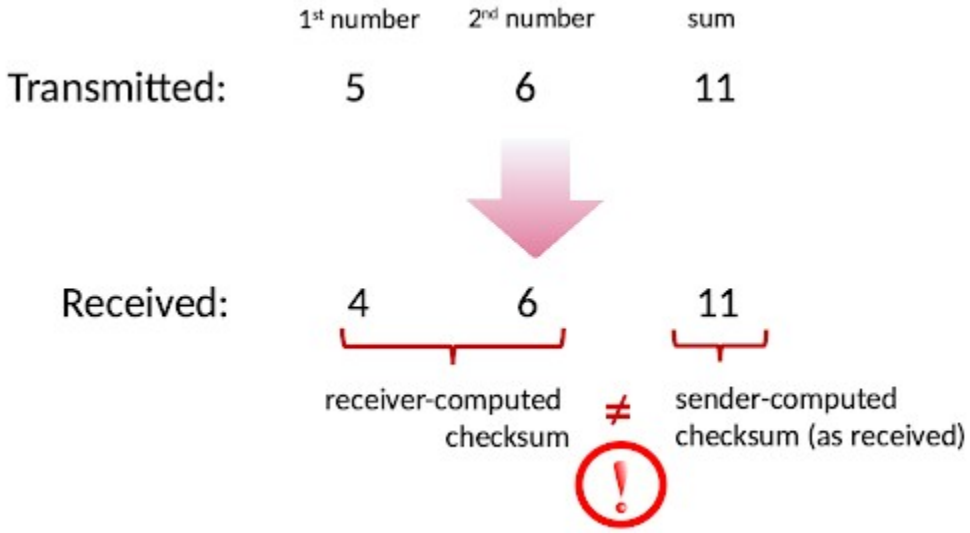
UDP segment format

The **application data** is occupied by the application message (for example an HTTP request, or a DNS response, or the audio samples in an streaming audio application etc.)
The UDP header has only **four fields** (each consisting of two bytes). We've already seen the source port number and the destination port number.
The **length** field specifies the number of bytes in the UDP segment, including the header.
The **checksum** is used by the receiving host to check whether errors have been introduced into the segment

UDP Checksum

The UDP checksum (should) provides for error detection, errors caused by the physical layer.

UDP at the sender side performs the 1s complement of the sum of all the 16-bit words in the segment, with any overflow encountered during the sum being wrapped around. This result is put in the checksum field of the UDP segment. When the receiver receives the segment, it will calculate the sum itself and compare it with the checksum, if the values are different, then there was an error during the transmission (if the values are equal there isn't 0% possibility of errors though).



example: add two 16-bit integers

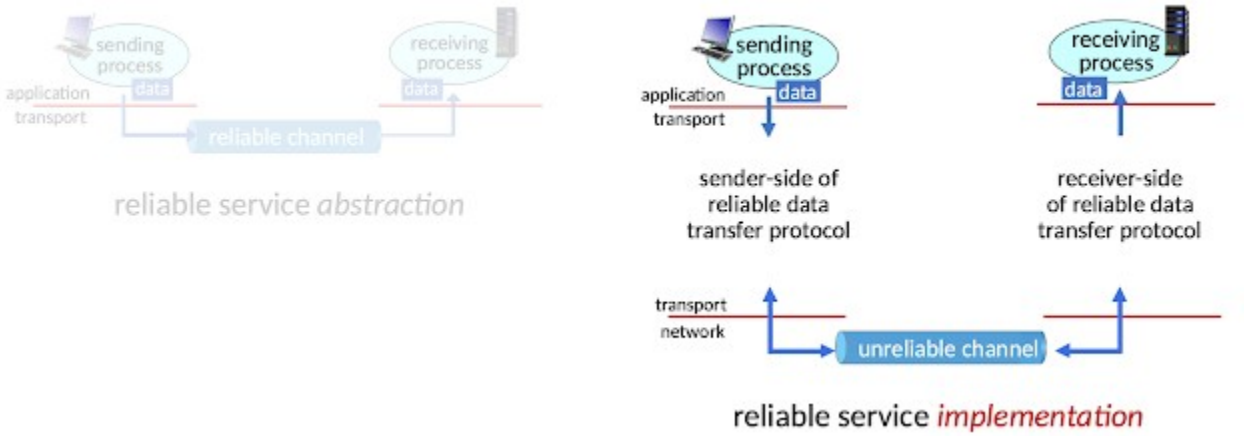
	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1
sum	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

Note: when adding numbers, a carryout from the most significant bit needs to be added to the result

You may wonder why UDP provides a checksum in the first place, as many link-layer protocols (including the popular Ethernet protocol) also provide error checking. The reason is that there is no guarantee that all the links between source and destination provide error checking; that is, one of the links may use a link-layer protocol that does not provide error checking. Furthermore, even if segments are correctly transferred across a link, it's possible that bit errors could be introduced when a segment is stored in a router's memory.

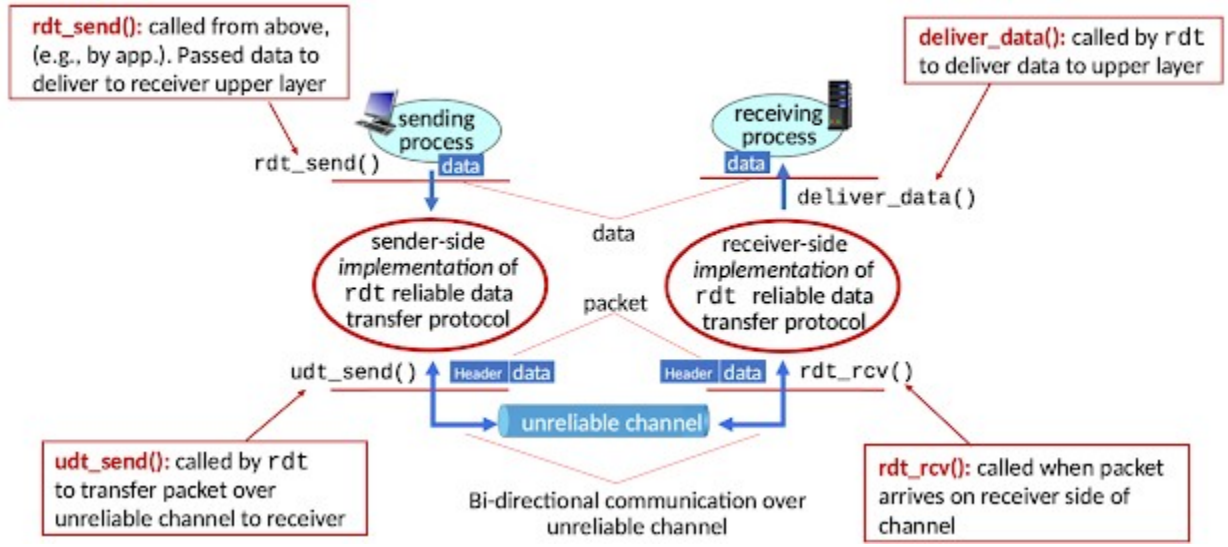
Principles of Reliable Data Transfer

In this section, we consider the problem of reliable data transfer in a general context.
The **reliable data transfer** is on top of the problems in all of networking.



The service abstraction provided to the upper-layer entities (application layer) is that of a reliable channel through which data can be transferred.
It is the responsibility of a **reliable data transfer protocol** (such as TCP) to implement this service abstraction. This task is made difficult by the fact that the layer below the reliable data transfer protocol may be unreliable. The complexity of this reliable data transfer protocol depends on the characteristics of the unreliable channel (lose, corrupt, reorder, etc).
For example, TCP is a reliable data transfer protocol that is implemented on top of an unreliable (IP) end-to-end network layer.

One assumption we'll adopt throughout our discussion of a reliable data transfer protocol here is that packets will be delivered in the order in which they were sent, with some packets possibly being lost.

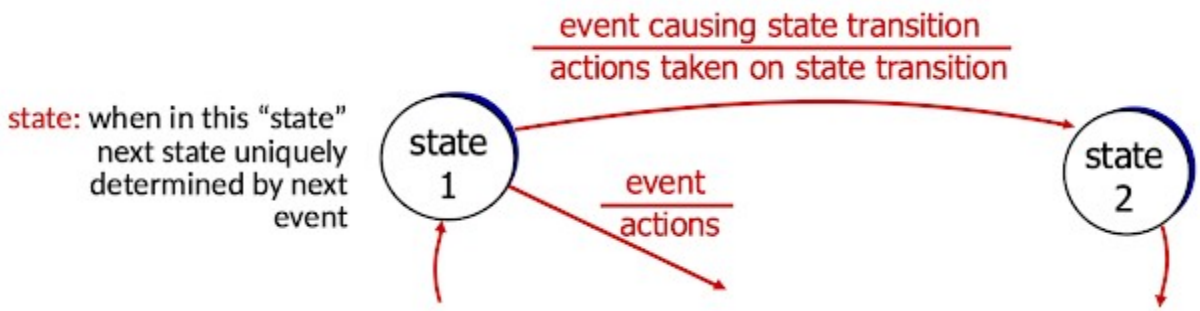


Let's see this functions (rdt stands for reliable data transfer protocol)
In the following we use the terminology "packet" rather than transport-layer "segment." Because the theory developed in this section applies to computer networks in general and not just to the Internet transport layer, the generic term "packet" is perhaps more appropriate here.

- The sending side of the data transfer protocol will be invoked from above by a call to **rdt_send()**.
  It will pass the data to be delivered to the upper layer at the receiving side.
- On the receiving side, **rdt_rcv()** will be called when a packet arrives from the receiving side of the channel
- When the rdt protocol wants to deliver data to the upper layer, it will do so by calling **deliver_data()**
- Both the send and receive sides of rdt send packets to the other side by a call to **udt_send()** (where udt stands for unreliable data transfer)
  Both because they also need to exchange control packets back and forth

Building a Reliable Data Transfer Protocol

We now step through a series of protocols, each one becoming more complex, arriving at a flawless, reliable data transfer protocol.
We will use finite state machines (FSM), to specify sender and receiver.



When **no action** is taken on an event, or no event occurs and an action is taken, we'll use the symbol Λ below or above the horizontal, respectively, to explicitly denote the lack of an action or event.
The **initial state** of the FSM is indicated by the **dashed arrow**.

rdt1.0

This protocol is for a Reliable Data Transfer over a **Perfectly Reliable Channel**
So, the underlying channel is **perfectly reliable** (means that we have no bit errors e no loss of packets).
We have **two separate FSM**, one that defines operations for the **sender** (which sends data into the underlying channel) and one that defines operations for the **receiver** (which reads data into the underlying channel).



The FSM's have just **one state**, so the only transition that we could have is from the one state back to itself.
With a perfectly reliable channel there is no need for the receiver side to provide any feedback to the sender since nothing can go wrong.
Note that we have also assumed that the receiver is able to receive data as fast as the sender happens to send data. Thus, there is no need for the receiver to ask the sender to slow down!

rdt2.0

This protocol is for a Reliable Data Transfer over a **Channel with Bit Errors**
Such bit errors typically occur in the physical components of a network as a packet is transmitted, propagates, or is buffered. We'll continue to assume for the moment that all transmitted packets are received (although their bits may be corrupted) in the order in which they were sent.
Like humans, this message-dictation protocol uses both **positive acknowledgments** ("OK") and **negative acknowledgments** ("Please repeat that.")
These control messages allow the receiver to let the sender know what has been received correctly, and what has been received in error and thus requires repeating.

To implement this protocol we have to introduce three additional capabilities to handle the presence of bit errors:

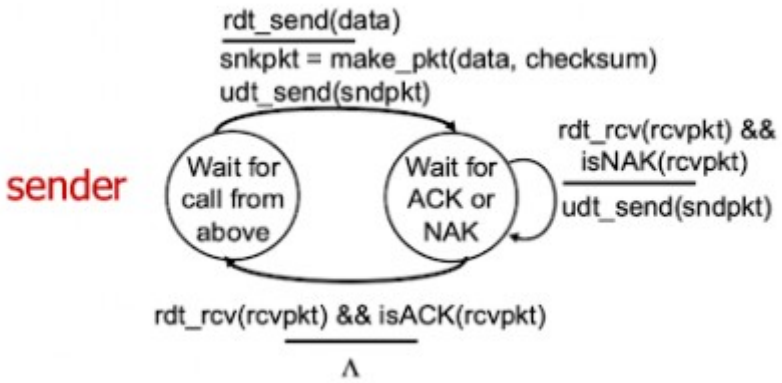
- An **error detection** mechanism
  Checksum field of UDP is for exactly this purpose (even if it isn't 100% safe), but we will examine in next sections others techniques in greater detail
- A **receiver feedback**
  Since the sender and the receiver could be separated by thousands of miles, the only way to let the sender know if a packet has arrived to the receiver is to provide explicit feedback to the sender.
  With **acknowledgments** (**ACKs**), the receiver explicitly tells to the sender that the

packet received was OK
With **negative acknowledgments** (**NAKs**), the receiver explicitly tells to the sender that the packet received had errors
This receiver feedback packet need only be one bit long (for example 0 for NAK and 1 for ACK).

- The **retransmission**
  A packet that is received in error at the receiver (so if the sender receives an NAK from the receiver) will be retransmitted by the sender.

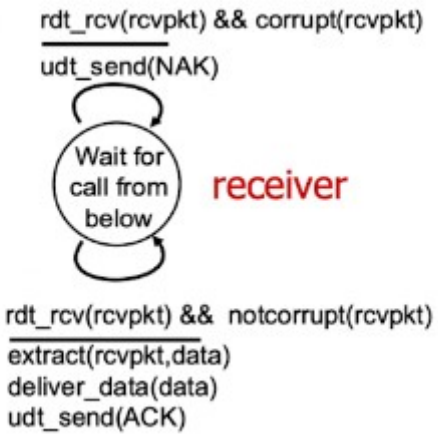
So the sender sends one packet, then waits for the receiver's response.

The sender side has two states, because we need the state of the waiting from the sender to the receiver's response.



We can see that if a NAK is received, then the sender will resend the packet and retake the state of waiting, otherwise if an ACK is received, then the sender will send the next packet.
It is important to observe that when the sender is in the wait-for-ACK-or-NAK state, it cannot get more data from the upper layer. Because of this behavior, protocols such as rdt2.0 are known as **stop-and-wait protocols**.

The receiver has only one state but has two different possible actions (send ACK or send NAK):



rdt2.1

The *rdt* 2.0 protocol otherwise has a fatal flaw, because we haven't accounted for the possibility that the ACK or NAK packet could be corrupted. We can simply and minimally add the checksum, but the more difficult problem is how the protocol should recover from errors in ACK or NAK packets.

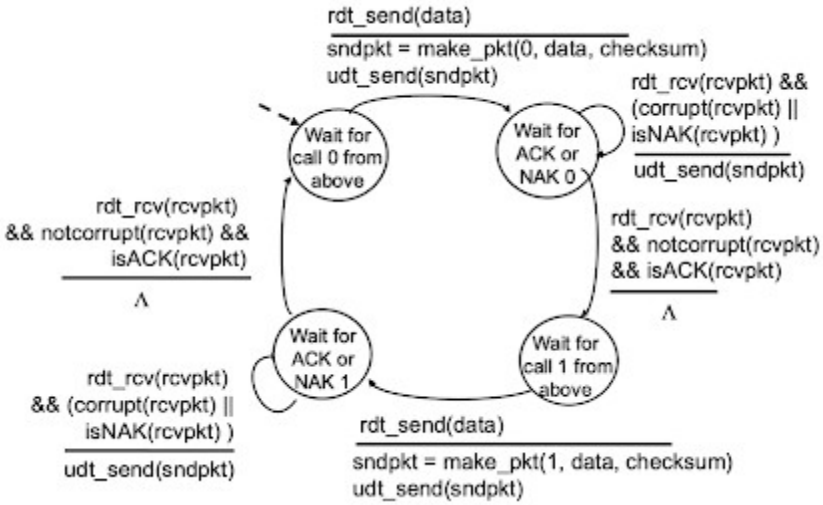
Let's consider three possibilities for handling corrupted ACK or NAK packets.

- We can create other response packets from sender to receiver that are ACK and NAK for the ACK and NAK from the receivers, but if this packet is also corrupted? We're heading down a difficult path.
- A second alternative is to add enough checksum bits to allow the sender not only to detect, but also to recover from, bit errors. This solves the immediate problem for a channel that can corrupt packets but not lose them
- A third approach is to resend the current packet when the sender receives a corrupted ACK or NAK packet.
  But this approach may introduce **duplicate packets** into the sender-to-receiver channel. The fundamental difficulty with duplicate packets is that the receiver doesn't know whether the ACK or NAK it last sent was received correctly at the sender. Thus, it cannot know a priori whether an arriving packet contains new data or is a retransmission!

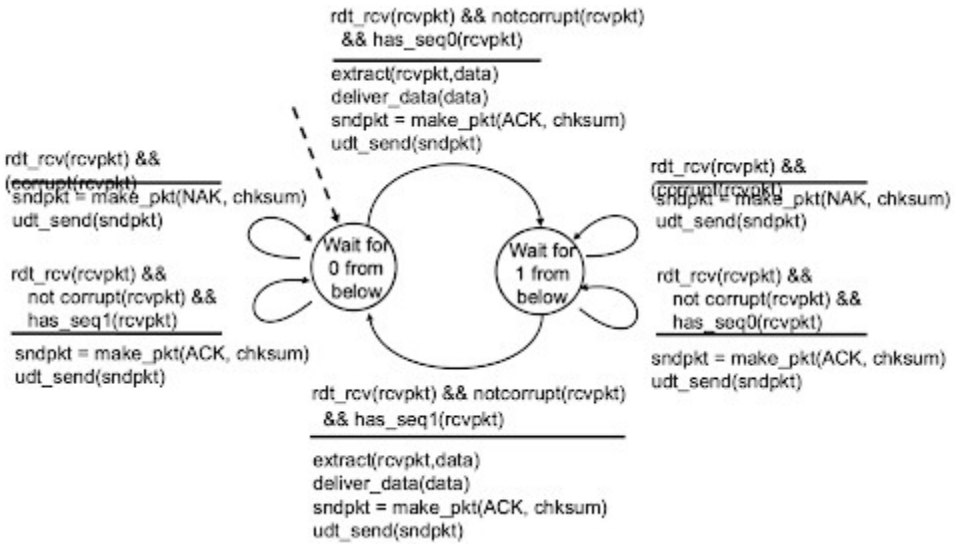
A simple solution to this new problem (of duplicate packets) is to add a new field to the data packet, the **sequence number** (that is the same for the same packet and change when the sender sends a new packet). The receiver then needs only check this sequence number to determine whether or not the received packet is a retransmission.

Since we are currently assuming a channel that does not lose packets, ACK and NAK packets do not themselves need to indicate the sequence number of the packet they are acknowledging. The sender knows that a received ACK or NAK packet (whether garbled or not) was generated in response to its most recently transmitted data packet.

The sender will have twice as many states has before and will add the sequence number to the packets and the check to if a ACK/NAK is corrupted:

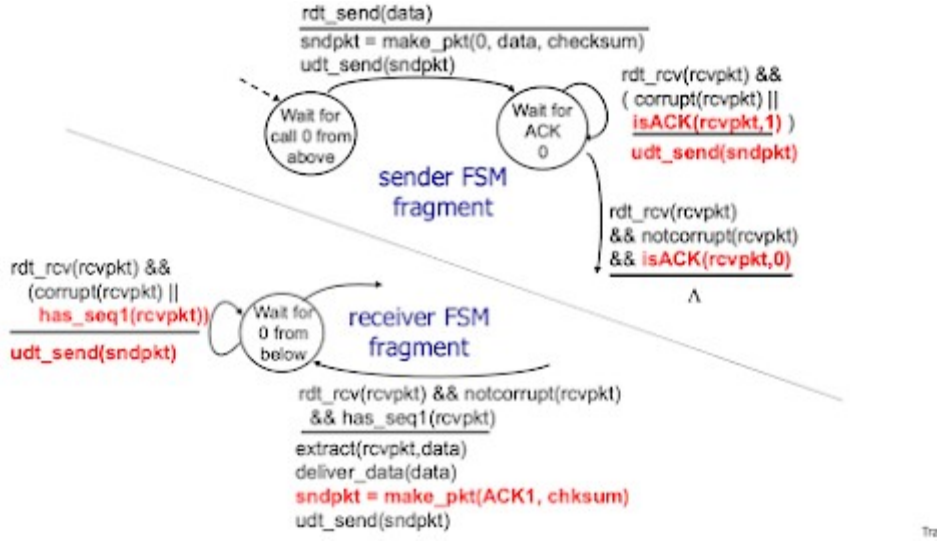


The receiver will have also twice as many states has before:



rdt2.2

We can create a protocol with the same functionality as the *rdt* 2.1 but without the use of NAK, using only ACKs.
Instead of NAK, the receiver sends ACK for the last packet received correctly (but the sender must include the sequence number of the packet being ACKed). Duplicate ACK for the same packet to the sender results in the same action as a NAK.
These protocols are called NAK-free



rdt3.0

This protocol is for Reliable Data Transfer over a Lossy Channel with Bit Errors.
Suppose now that in addition to corrupting bits, the underlying channel can lose packets as well, a not-uncommon event in today's computer networks (including the Internet).

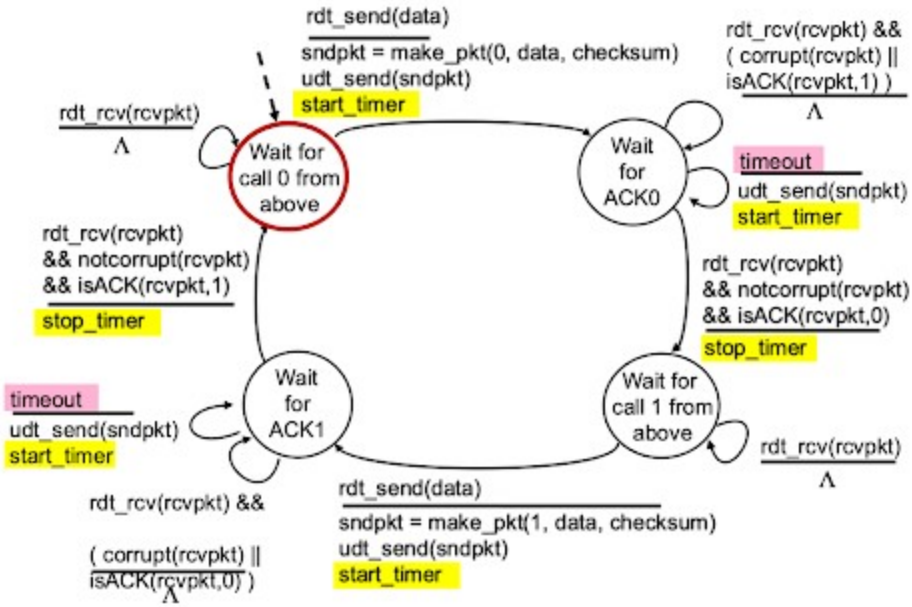
Two additional concerns must now be addressed by the protocol: **how** to detect packet loss and **what to do** when packet loss occurs.

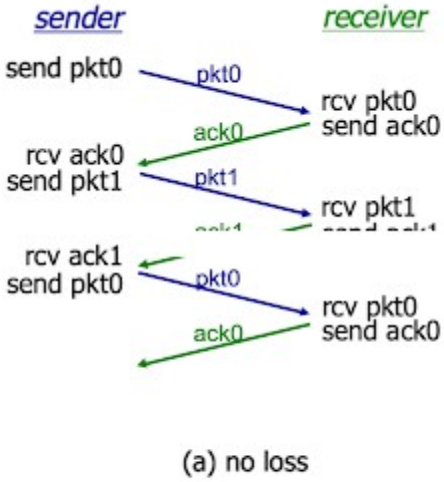
If the sender is willing to wait long enough so that it is certain that a packet has been lost, it can simply retransmit the data packet.
But how long must the sender wait to be certain that something has been lost? For example one RTT plus the amount of time is needed to process a packet at the receiver.
The approach thus adopted in practice is for the sender to judiciously choose a time value such that packet loss is likely, although not guaranteed, to have happened. In this way we introduce the possibility of duplicate data packets, but we handled them in $rdt\,2.2$.

So in all cases (packet lost, ACK response lost, packet/ACK delayed) the action is the same: **retransmit**.
Implementing a time-based retransmission mechanism requires a countdown timer that can interrupt the sender after a given amount of time has expired, so the sender has to:

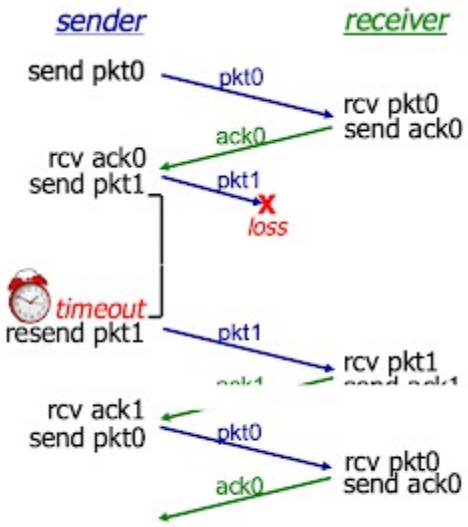
1. start the timer each time a packet (either a first-time packet or a retransmission) is sent
2. respond to a timer interrupt (taking appropriate actions),
3. stop the timer

Let's see the sender for this protocol:

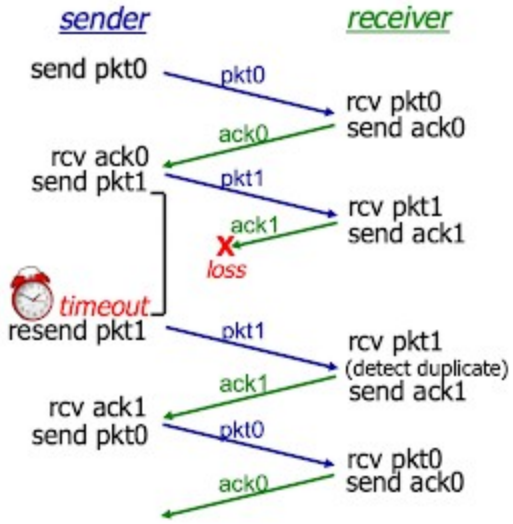




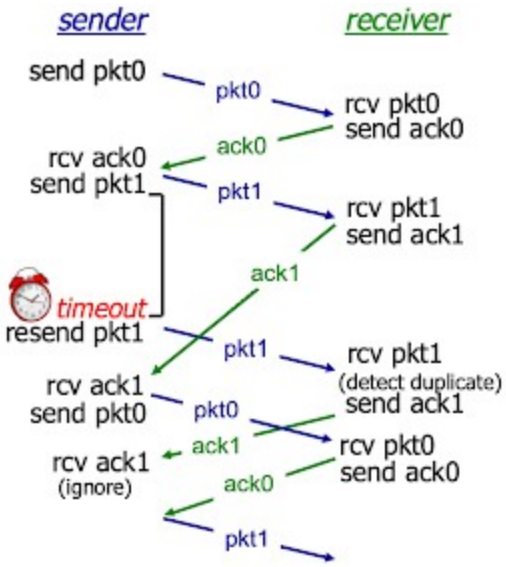
(a) no loss



(b) packet loss



(c) ACK loss

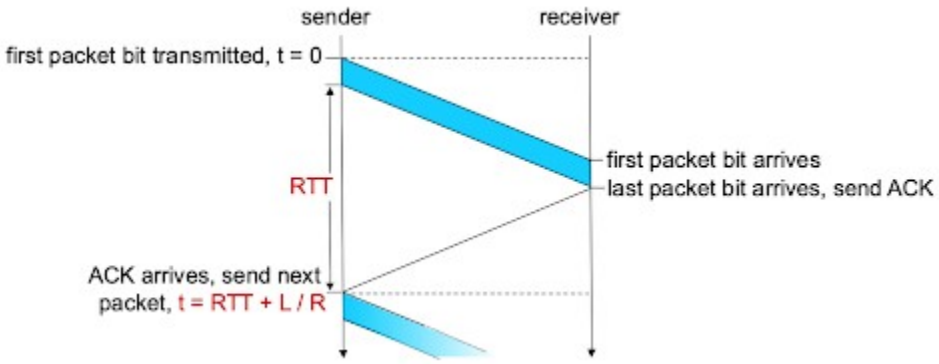


(d) premature timeout/ delayed ACK

Pipelined Reliable Data Transfer Protocol

Protocol *rdt* 3.0 is a functionally correct protocol, but it is unlikely that anyone would be happy with its performance, particularly in today's high-speed networks. At the heart of rdt3.0's performance problem is the fact that it is a stop-and-wait protocol.

Let's consider only the percentual time when the sender is busy in sending, denoted by U_{sender}, if the RTT time is higher than this time, then the sender is busy only for a small time but it has to wait for the ACK response and does nothing in the mid-time.

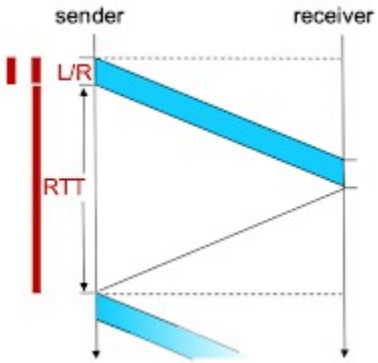


Let's see a numeric example:

- example: 1 Gbps link, 15 ms prop. delay, 8000 bit packet
  - time to transmit packet into channel:

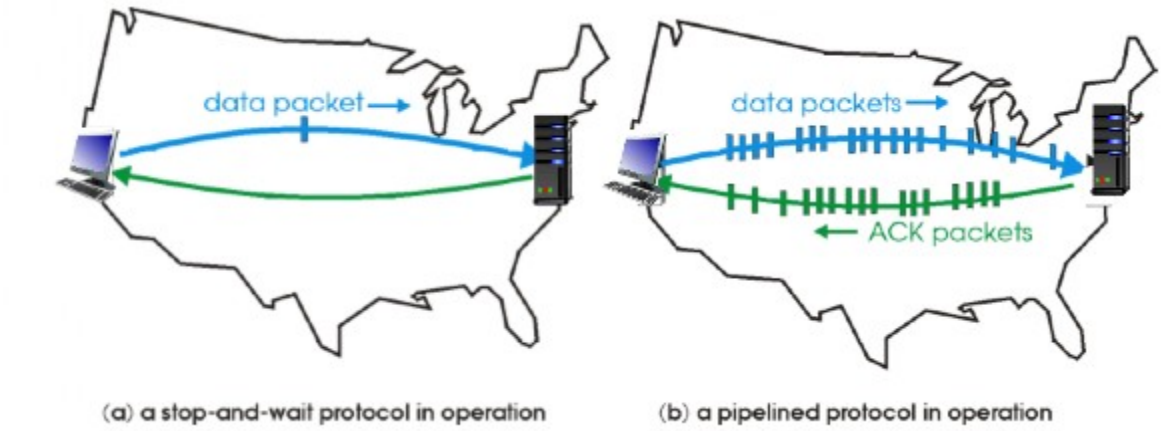
$$D_{trans} = \frac{L}{R} = \frac{8000\ bits}{10^9\ bits/sec} = 8\ microsecs$$

$$U_{sender} = \frac{L/R}{RTT + L/R} = \frac{.008}{30.008} = 0.00027$$

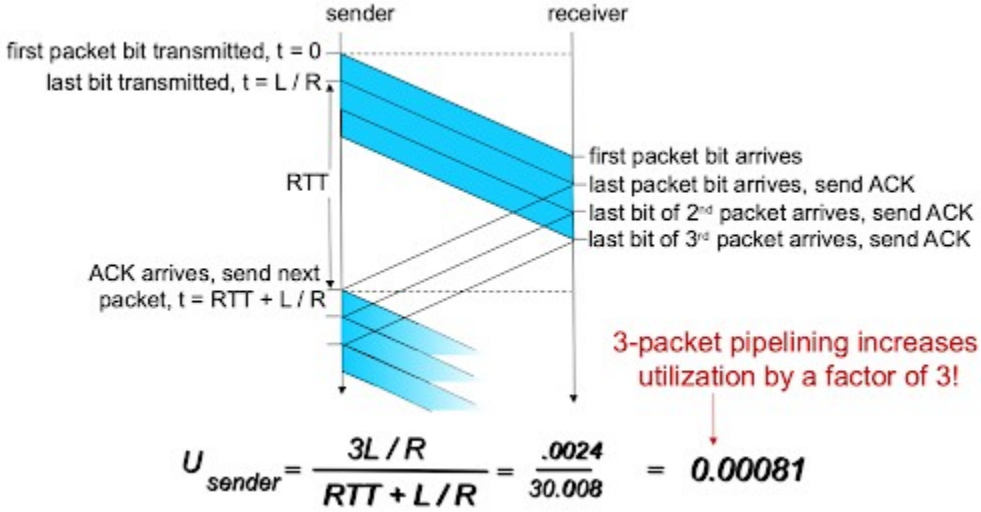


We can see that this protocol isn't good for performance and limits the performance of the underlying structure.
Viewed another way, the sender was able to send only 1,000 bytes in 30.008 milliseconds, an effective throughput of only 267 kbps, even though a 1 Gbps link was available!
//PRIMO PARZIALE FINO A QUA
The solution to this particular performance problem is simple: Rather than operate in a stop-and-wait manner, the sender is allowed to send multiple packets without waiting for acknowledgments.
Since the many in-transit sender-to-receiver packets can be visualized as filling a pipeline, this technique is known as **pipelining**, with the following consequences:

- The range of sequence numbers must be increased, since each in-transit packet (not counting retransmissions) must have a unique sequence number and there may be multiple, in-transit, unacknowledged packets
- The sender and receiver sides of the protocols may have to buffer more than one packet. Minimally, the sender will have to buffer packets that have been transmitted but not yet acknowledged. Buffering of correctly received packets may also be needed at the receiver, as discussed below.



Let's see how the pipeline can increase the performance with our example:



$$U_{sender} = \frac{3L/R}{RTT + L/R} = \frac{.0024}{30.008} = 0.00081$$

The range of sequence numbers needed and the buffering requirements will depend on the manner in which a data transfer protocol responds to lost, corrupted, and overly delayed packets. Two basic approaches toward pipelined error recovery can be identified: **Go-Back-N** and **selective repeat**

Go-Back-N (GBN)

In a **Go-Back-N (GBN) protocol**, the sender is allowed to transmit multiple packets (when available) without waiting for an acknowledgment, but is constrained to have no more than some **maximum allowable number**, N, of unacknowledged packets in the pipeline.

- k-bit seq # in pkt header



In the image we can see four intervals, if we define $send_base$ to be the sequence number of the oldest unacknowledged packet and $nextseqnum$ to be the smallest unused sequence number (the next packet to be sent) the intervals are:

- $[0, send_base - 1]$ are the packets that have already been transmitted and acknowledged
- $[send_base, nextseqnum - 1]$ are the packets that have been sent but not yet acknowledged
- $[nextseqnum, send_base + N - 1]$ are the packets that can be send immediately
- $[send_base + N, \ldots]$ are the packets that cannot be used until an unacknowledged packet currently in the pipeline (specifically, the packet with sequence number base) has been acknowledged

N is often called **window size** as is the number of packets that can be transmitted and not already acknowledged, and the GBN protocol is said to be a **sliding-window protocol**.
We will see later that N can't be unlimited because of flow control and congestion control.

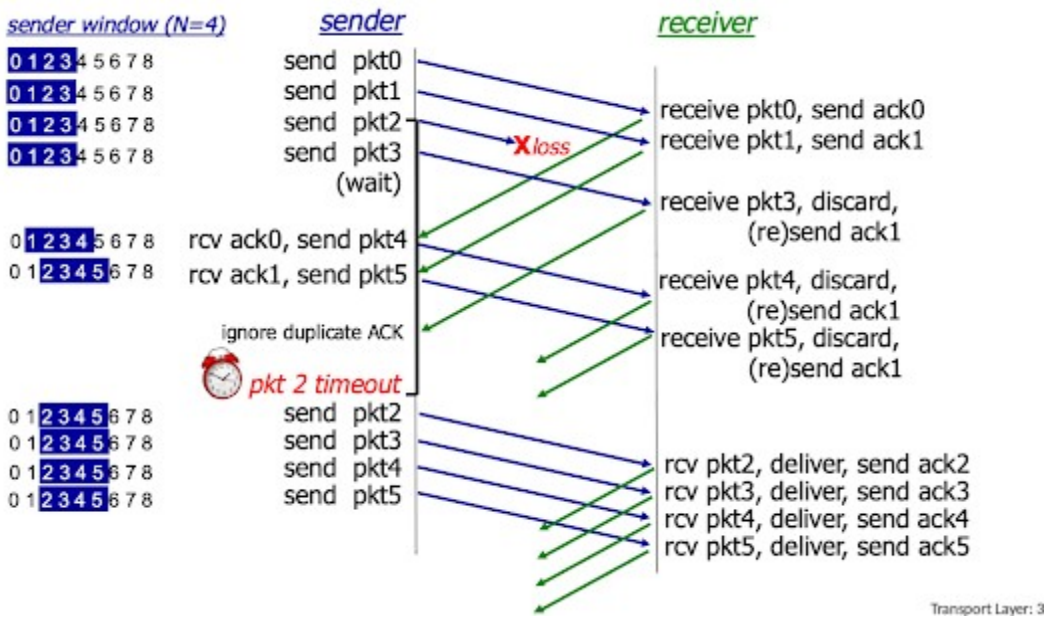
The ACK is **cumulative**, means that $ACK(n)$ that all packet up to, and include, n is ACKed.
When it is received $ACK(n)$, $send_base$ is moved to $n+1$.
There is a timer for oldest in-flight packet, when $timeout(n)$ ends, will retransmit packet n and all higher sequence number packets in the window.

The receiver's actions in GBN are also simple. If a packet with sequence number n is received correctly and is in order (that is, the data last delivered to the upper layer came from a packet with sequence number n−1), the receiver sends an ACK for packet n and delivers the data portion of the packet to the upper layer. In all other cases, the receiver discards the packet and resends an ACK for the most recently received in-order packet.
So if a packet k has been delivered to the upper layer, then we know that all the packets with a sequence number lower than k have been delivered. So the cumulative ACK is a natural choice.

The receiver will discard all the out-of-order packets, that's because even it it buffer them, for example $n+1$, the sender will resend at priory all the packets after n, so the receiver doesn't need to buffer any packets. So the only value that the receiver has to know is the sequence number of the next in-order packet rcv_base.



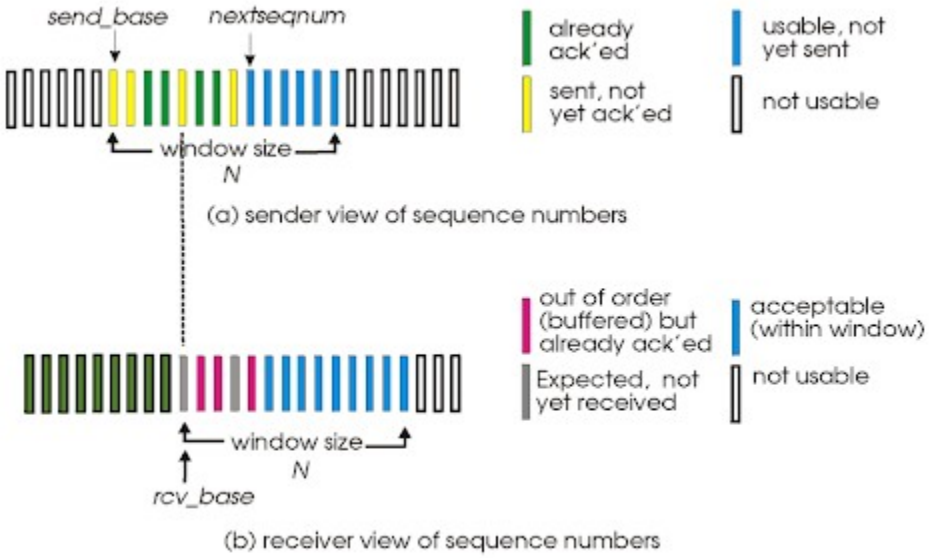
So in general a GBN protocol will work like this:



Selective Repeat (SN)

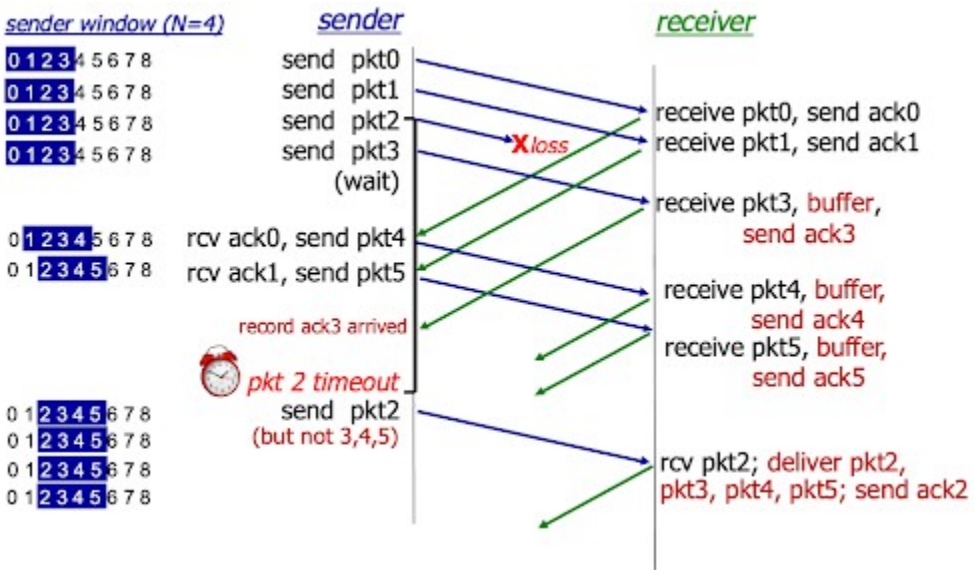
The GBN protocol allows the sender to potentially “fill the pipeline” with packets, thus avoiding the channel utilization problems we noted with stop-and-wait protocols.
There are, however, scenarios in which GBN itself suffers from performance problems. In particular, when the window size and bandwidth-delay product are both large, many packets can be in the pipeline.
A single packet error can thus cause GBN to retransmit a large number of packets, many unnecessarily.

As the name suggests, **selective-repeat protocols** avoid unnecessary retransmissions by having the sender retransmit only those packets that it suspects were received in error (that is, were lost or corrupted) at the receiver.
This individual, as-needed, retransmission will require that the receiver individually acknowledge correctly received packets
The SR receiver will acknowledge a correctly received packet whether or not it is in order. Out-of-order packets are **buffered** until any missing packets (that is, packets with lower sequence numbers) are received, at which point a batch of packets can be delivered in order to the upper layer.
So in these protocols there are timeouts, and retransmits, individually for every packet in the window



The sender receives **data from above** and checks the next available sequence packet, if the number is in the window it sends the packet.
There is a **timeout** for each packet that is not yet ACKed
When an ACK is received, the sender stops the timeout and marks the packet. If the sequence number of the packet ACKed is $send_base$, the window base is moved forward to the unacknowledged packet with the smallest sequence number.

When the receiver receive correctly a packet in $[rcv_base, rcv_base + N - 1]$, an ACK is returned to the sender.
If the packet was not previously received, it is buffered. If the packet has a sequence number equal to rcv_base, then this packet and all the consecutively numbered packets in the buffer are delivered to the upper layer, and the receive window is then moved forward by the number of packets delivered to the upper layer.
When the receiver receive correctly a packet in $[rcv_base - N, rcv_base - 1]$, an ACK is generated (maybe this packet has arrived because the previous ACK was lost or corrupted).
Otherwise (if the packet isn't in this two intervals) ignore the packet.

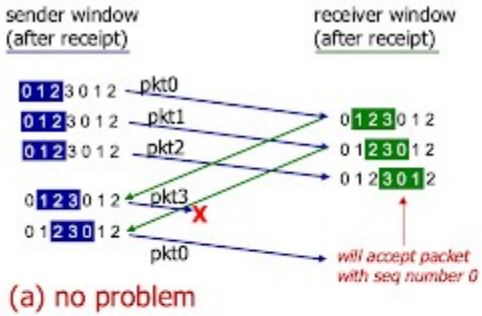


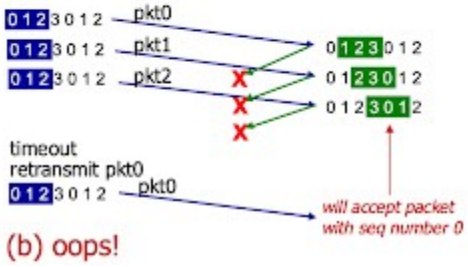
We mentioned before the problem that the window of the sender and of the receiver could be not synchronized, and this is a big problem, for example:

Selective repeat: a dilemma!

example:

- seq #s: 0, 1, 2, 3 (base 4 counting)
- window size=3





In this case the receiver can't see the sender side, and in the receiver view the packet with 0 that arrives the second time is another packet instead of the sender side.
To avoid this problem, the window size must be less than or equal to half the size of the sequence number space for SR protocols.

Connection-Oriented Transport: TCP

Now that we have covered the underlying principles of reliable data transfer, let's turn to TCP, the Internet's transport-layer, connection-oriented, reliable transport protocol.

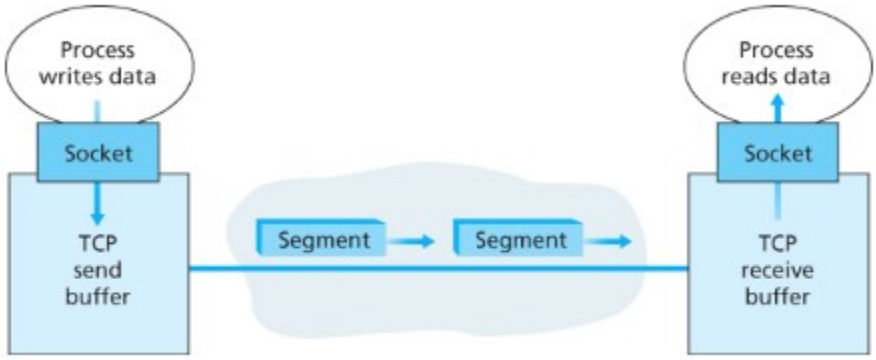
The TCP connection

TCP is said to be **connection-oriented** (by a **logical connection**) because before one application process can begin to send data to another, the two processes must first “handshake” with each other (it is a three-way handshake).
As part of TCP connection establishment, both sides of the connection will initialize many TCP state variables associated with the TCP connection

Recall that because the TCP protocol runs only in the end systems and not in the intermediate network elements, the intermediate network elements do not maintain TCP connection state. In fact, the intermediate routers are completely oblivious to TCP connections: they see datagrams, not connections.

A TCP connection provides a **full-duplex service**: If there is a TCP connection between Process A on one host and Process B on another host, then application-layer data can flow from Process A to Process B at the same time as application-layer data flows from Process B to Process A.
A TCP connection is also always **point-to-point**, that is, between a single sender and a single receiver.

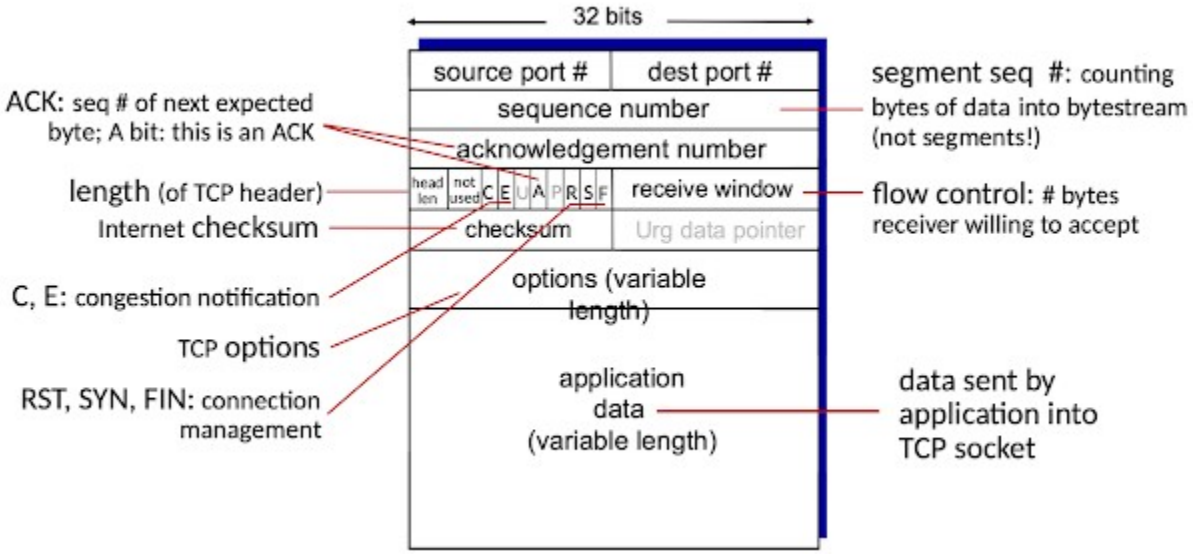
Once a TCP connection is established, the two application processes can send data to each other. The maximum amount of data that can be grabbed and placed in a segment is limited by the maximum segment size (**MSS**), referred to application layer data, without the TCP headers.



TCP works with cumulative ACKs and pipelining and it has TCP congestion control and flow control, so the sender won’t overwhelm the receiver.

TCP segment structure

Having taken a brief look at the TCP connection, let’s examine the TCP segment structure.
The TCP segment consists of header fields and a data field. The data field contains a chunk of application data. When TCP sends a large file, such as an image as part of a Web page, it typically breaks the file into chunks of size MSS.



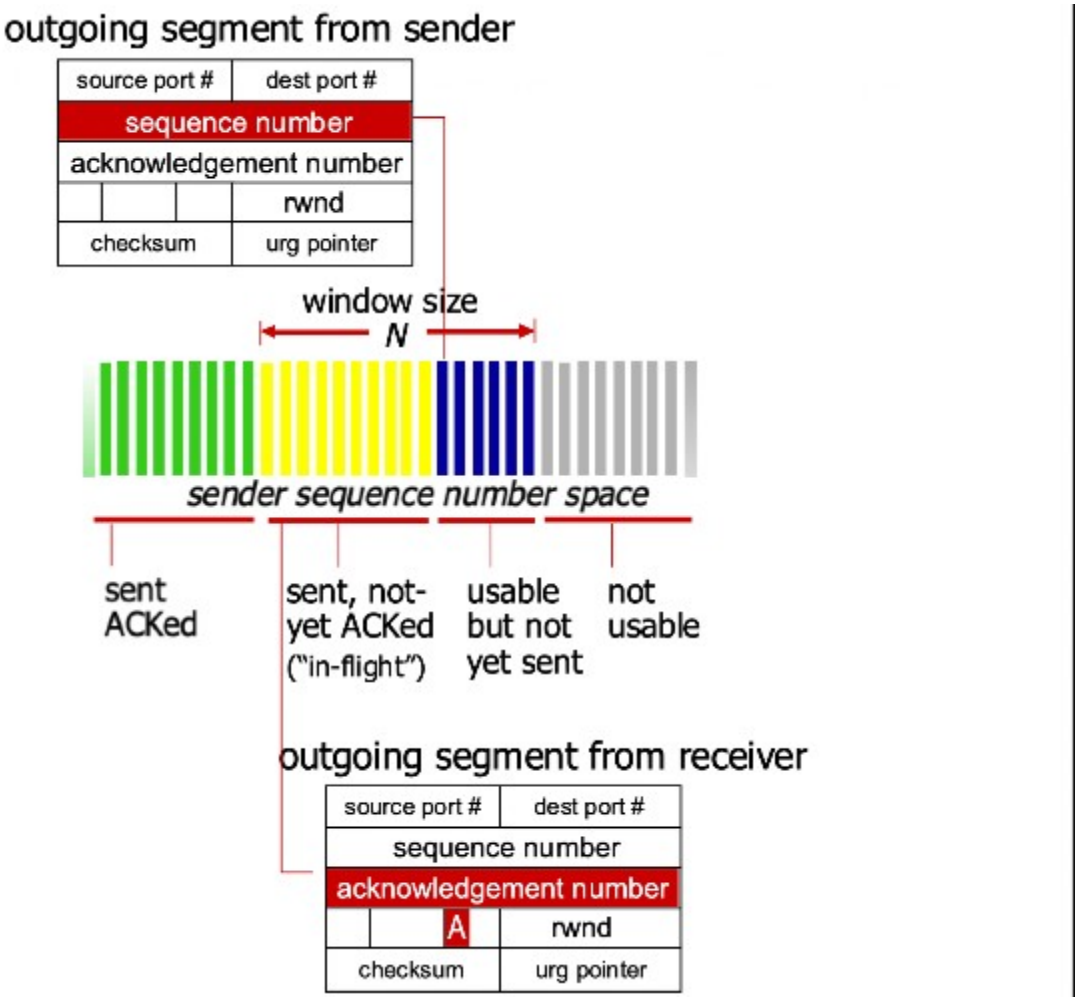
Let's have a look at the TCP segment structure.
As with UDP, the header includes **source and destination port numbers**, which are used for multiplexing/demultiplexing data from/to upper-layer applications. Also, as with UDP, the header includes a **checksum** field. A TCP segment header also contains the following fields:

- The 32-bit **sequence number** field and the 32-bit **acknowledgment number** field are used by the TCP sender and receiver in implementing a reliable data transfer service, as discussed below
- The 16-bit **receive window** field is used for flow control. We will see shortly that it is used to indicate the number of bytes that a receiver is willing to accept
- The 4-bit **header length** field specifies the length of the TCP header in 32-bit words. The TCP header can be of variable length due to the TCP options field.
- The optional and variable-length **options** field is used when a sender and receiver negotiate the maximum segment size (MSS) or as a window scaling factor for use in high-speed networks
- The **flag** field contains 6 bits
    - The **ACK** bit is used to indicate that the value carried in the acknowledgment field is valid; that is, the segment contains an acknowledgment for a segment that has been successfully received
    - The **RST**, **SYN**, and **FIN** bits are used for connection setup and teardown, as we will discuss at the end of this section
    - The **CWR** and **ECE** bits are used in explicit congestion notification
    - Setting the **PSH** bit indicates that the receiver should pass the data to the upper layer immediately
    - The **URG** bit is used to indicate that there is data in this segment that the sending-side upper-layer entity has marked as "urgent".
- The location of the last byte of this urgent data is indicated by the 16-bit **urgent data pointer** field.

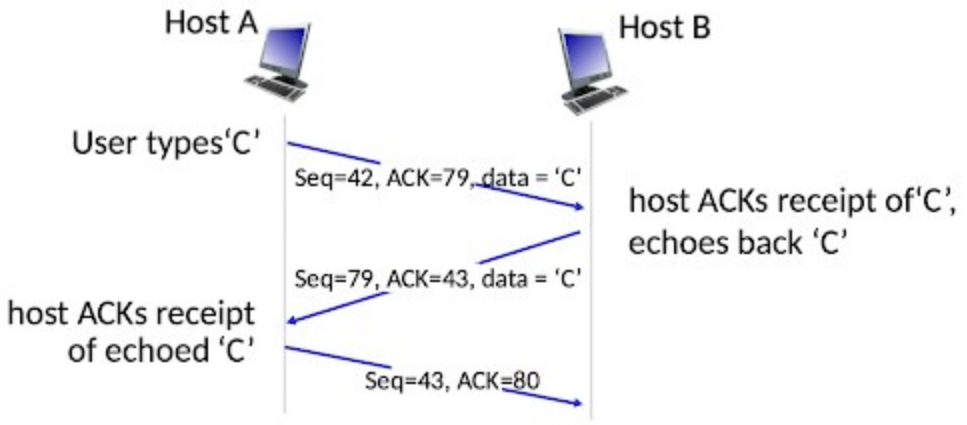
Two of the most important fields in the TCP segment header are the **sequence number** field and the **acknowledgment number** field.

TCP views data as an unstructured, but ordered, stream of bytes. TCP's use of sequence numbers reflects this view in that sequence numbers are over the stream of transmitted bytes and not over the series of transmitted segments.
The **sequence number for a segment** is therefore the byte-stream number of the first byte in the segment

Now let's consider **acknowledgement numbers**. These are a little trickier than sequence numbers.
The acknowledgment number that Host A puts in its segment is the sequence number of the next byte Host A is expecting from Host B. It is important to remember that TCP uses **cumulative acknowledgment**.
The problem of out-of-order segments isn't resolved by TCP, because it leave the decision up to the implementer, but the choices are two: discard or buffer (as we've already seen)



Let's see an example of sequence and acknowledgement number with **Telnet**, defined in RFC 854, is a popular application-layer protocol used for remote login. It runs over TCP and is designed to work between any pair of hosts.



simple telnet scenario

We can notice that the second segment is sent from the server to the client. It serves a dual purpose: an acknowledgement for the segment 42 (with 43) and echoing the letter 'C'.
Note that the acknowledgment for client-to-server data is carried in a segment carrying server-to-client data; this acknowledgment is said to be **piggybacked** on the server-to-client data segment

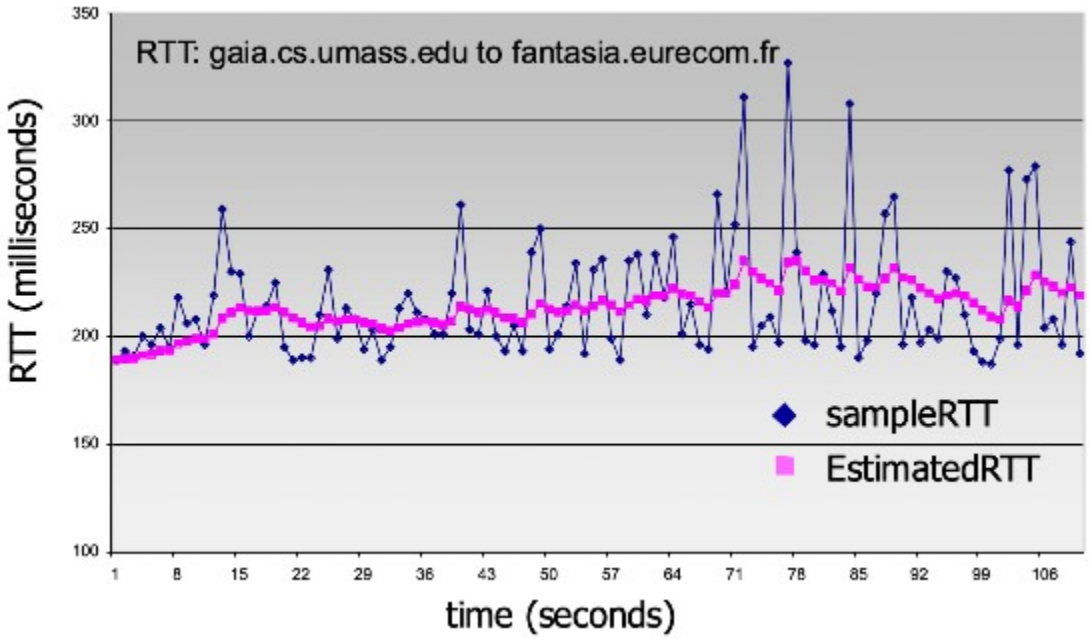
Round-Trip Time Estimation and Timeout

TCP, like our rdt protocol in Section 3.4, uses a timeout/retransmit mechanism to recover from lost segments. Although this is conceptually simple, many subtle issues arise when we implement a timeout/retransmit mechanism in an actual protocol such as TCP.
Clearly, the timeout should be larger than the connection's round-trip time (RTT), that is, the time from when a segment is sent until it is acknowledged.

Let's begin our study of TCP timer management by considering how TCP estimates the **round-trip time** between sender and receiver.
Instead of measuring a SampleRTT (the amount of time between when the segment is sent and when an acknowledgment for the segment is received) for every transmitted segment, most TCP implementations take only one SampleRTT measurement at a time.
Obviously, the SampleRTT values will fluctuate from segment to segment due to congestion in the routers and to the varying load on the end systems.
In order to estimate a typical RTT, it is therefore natural to take some sort of average of the SampleRTT values. TCP maintains an average, called EstimatedRTT, of the SampleRTT values.
TCP updates EstimatedRTT according to this formula:

$$EstimatedRTT = (1 - \alpha) \cdot EstimatedRTT + \alpha \cdot SampleRTT$$

A typical value for α is $\frac{1}{8} = 0.125$. Note that EstimatedRTT is a **weighted average**, of the SampleRTT values, that puts more weight on recent samples than on old samples.
In statistics, such an average is called an Exponential Weighted Moving Average (**EWMA**)



In addition to having an estimate of the RTT, it is also valuable to have a measure of the variability of the RTT, called DevRTT, that is an estimate of how much SampleRTT typically deviates from EstimatedRTT.

$$DevRTT = (1 - \beta) \cdot DevRTT + \beta \cdot |SampleRTT - EstimatedRTT|$$

The recommended value of β is 0.25.

The time out interval should be greater than or equal to EstimatedRTT, with a margin. But the margin should not be much high, but if the fluctuation of SampleRTT is high it should be larger. So we use the value of DevRTT:

$$TimeoutInterval = EstimatedRTT + 4 \cdot DevRTT$$

Reliable Data Transfer

Recall that the Internet's network-layer service (IP service) is unreliable.
IP does not guarantee datagram delivery, does not guarantee in-order delivery of datagrams, and does not guarantee the integrity of the data in the datagrams.

TCP creates a **reliable data transfer service** on top of IP's unreliable best-effort service.
How TCP provides a reliable data transfer involves many of the principles that we studied in Section 3.4

In our earlier development of reliable data transfer techniques, it was conceptually easiest to assume that an individual timer is associated with each transmitted but not yet acknowledged segment. Thus, the recommended TCP timer management procedures use only a **single retransmission timer**.

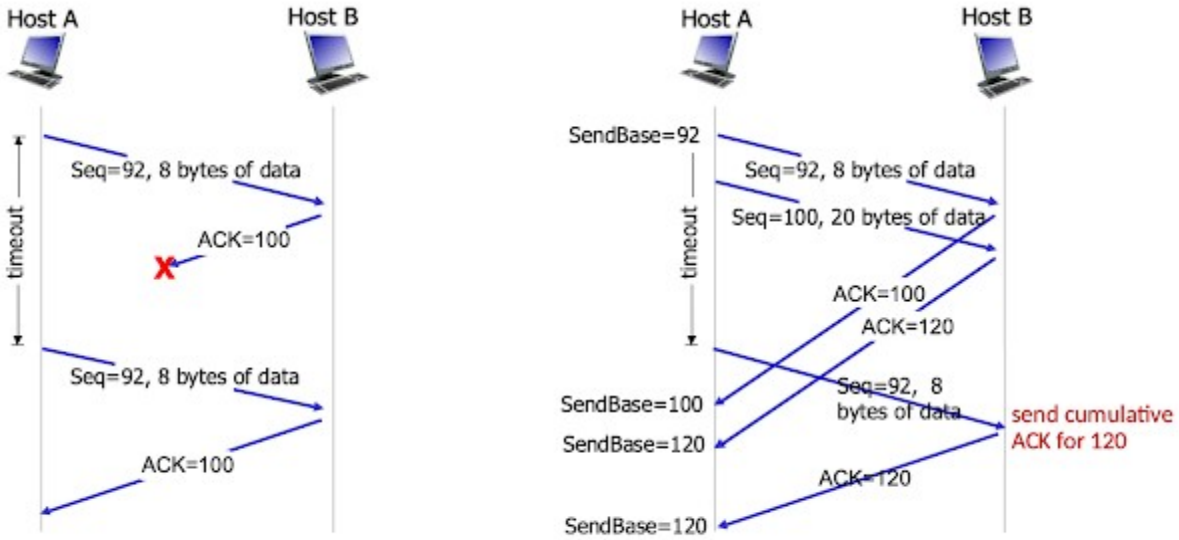
We will discuss how TCP provides reliable data transfer in two incremental steps. We first present a highly simplified description of a TCP sender that uses only timeouts to recover from

lost segments; we then present a more complete description that uses duplicate acknowledgments in addition to timeouts.

The **TCP sender** have three major events related to data transmission and retransmission:

- **Data received from application**
  - Create segment with sequence number (the byte-stream number of first data byte in segment)
  - Start the timer if not already running (the timer for the oldest unACKed segment, the expiration interval is **TimeOutInterval**)
  - Pass the segment to IP
- **Timer timeout**
  - Retransmit the segment that caused the timeout (the not-yet-acknowledged segment with smallest sequence number)
  - Restart the timer
- **ACK received**
  - If ACK acknowledges previously unACKed segments (the ACK field value is greater than Sendbase), update what is know to be ACKed (Sendbase = ACK field value) and start the timer if there are any other not-yet-acknowledged segments

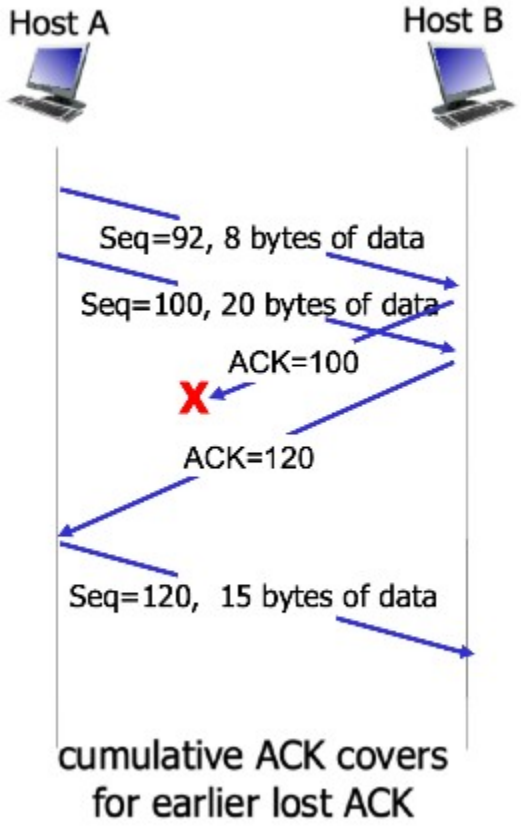
Let's see some scenarios to understand how this protocol works



In the first scenario the ACK sent by the receiver is lost, so the timeout event occurs and the sender resends the segment, the receiver will discard the bytes in the retransmitted segment because that data is already received, but it sent the ACK again.

In the second scenario, the sender sends two packets that arrive at the receiver, but neither of the acknowledgments arrives at Host A before the timeout. When the timeout event occurs, Host A resends the first segment with sequence number 92 and restarts the timer. As long as

the ACK for the second segment arrives before the new timeout, the second segment will not be retransmitted.



In the third scenario we have that the first ACK is lost, but the second (cumulative) ACK arrives, so the sender knows that all the segments up to 120 are delivered.

We now discuss a few modifications that most TCP implementations employ:

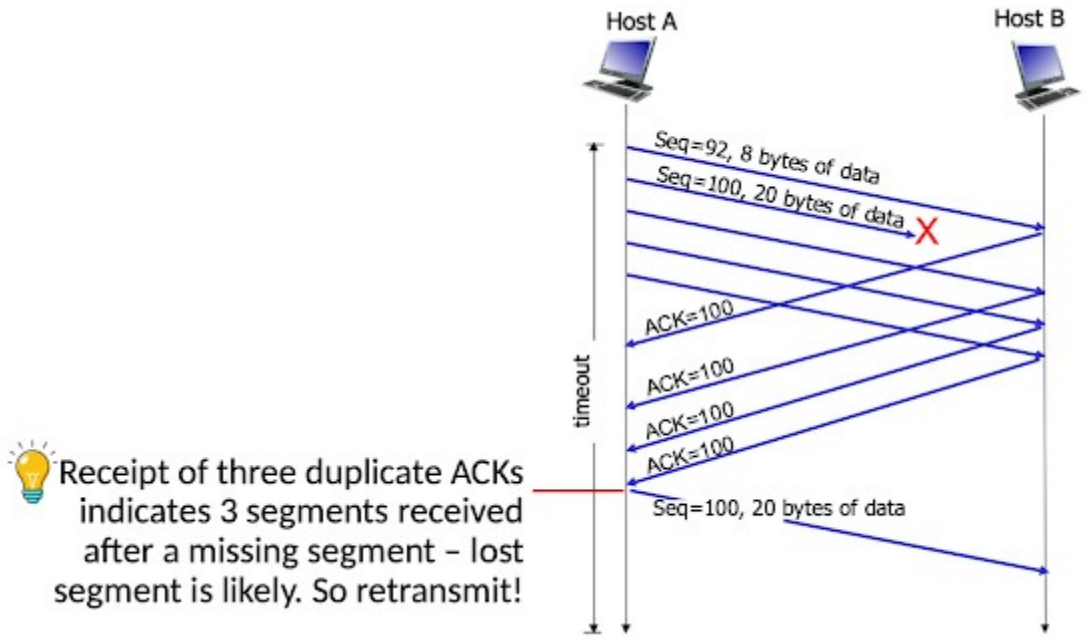
- Each time TCP retransmits (because of timeout event), it sets the next timeout interval to twice the previous value (however each time that the sender receive data from application above or it receives an ACK, it sets the timeout interval with $EstimatedRTT$ and $DevRTT$ like discussed before)
  This action provides a limited form of congestion control

With the doubling of the timeout interval, we have a delay for the resending of a lost packet. But the sender can detect packet loss well before the timeout event occurs by noting so-called **duplicate ACKs**. To understand the sender actions when it receive a duplicate ACK, let's see why the receiver sends multiple ACKs:

Event at receiver	TCP receiver action
arrival of in-order segment with expected seq #. All data up to expected seq # already ACKed	delayed ACK. Wait up to 500ms for next segment. If no next segment, send ACK
arrival of in-order segment with expected seq #. One other segment has ACK pending	immediately send single cumulative ACK, ACKing both in-order segments
arrival of out-of-order segment higher-than-expect seq. # . Gap detected	immediately send *duplicate ACK*, indicating seq. # of next expected byte
arrival of segment that partially or completely fills gap	immediate send ACK, provided that segment starts at lower end of gap

When a TCP receiver receives a segment with a sequence number that is larger than the next, expected, in-order sequence number, it detects a gap in the data stream: that is, a missing segment. This gap could be the result of lost or reordered segments within the network. Since TCP does not use negative acknowledgments, the receiver cannot send an explicit negative acknowledgment back to the sender. Instead, it simply **reacknowledges** (that is, generates a **duplicate ACK** for) the last in-order byte of data it has received.

Because a sender often sends a large number of segments back to back, if one segment is lost, there will likely be many back-to-back duplicate ACKs.
If the TCP sender receives three duplicate ACKs (why three?) for the same data, it takes this as an indication that the segment following the segment that has been ACKed three times has been lost.
The reasoning for not doing the retransmit until the third duplicate seems to be that until that point it's more likely to just be out-of-order delivery and the retransmit isn't really needed.
In the case that three duplicate ACKs are received, the TCP sender performs a **fast retransmit**, retransmitting the missing segment before that segment's timer expires.



Let us close our study of TCP's error-recovery mechanism by considering the following question: Is TCP a GBN or an SR protocol?
The TCP acknowledgments are cumulative and correctly received but out-of-order segments are not individually ACKed by the receiver.
Consequently, the TCP sender need only maintain the smallest sequence number of a transmitted but unacknowledged byte (SendBase) and the sequence number of the next byte to be sent (NextSeqNum).
So TCP looks like a GBN-style protocol, but:

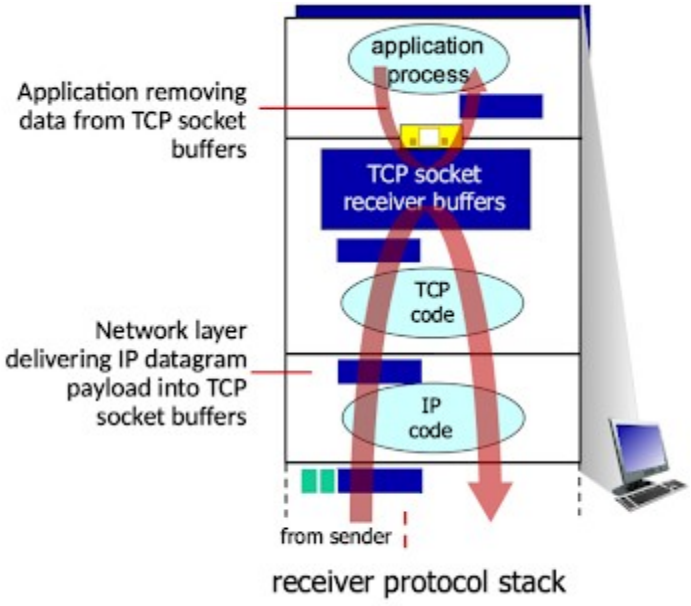
- Many TCP implementations buffer out-of-order segments
- If a packet $x < n$ is lost (or the ACK is lost), the GBN would resend all the packets after, and including, n. But TCP will resend only n, or nothing if it receives an ack for $y > x$

Flow control

Recall that the hosts on each side of a TCP connection set aside a receive buffer for the connection.
When the TCP connection receives bytes that are correct and in sequence, it places the data in the receive buffer. The associated application process will read data from this buffer, but not necessarily at the instant the data arrives. If the application is relatively slow at reading the data, the sender can very easily overflow the connection's receive buffer by sending too much data too quickly.

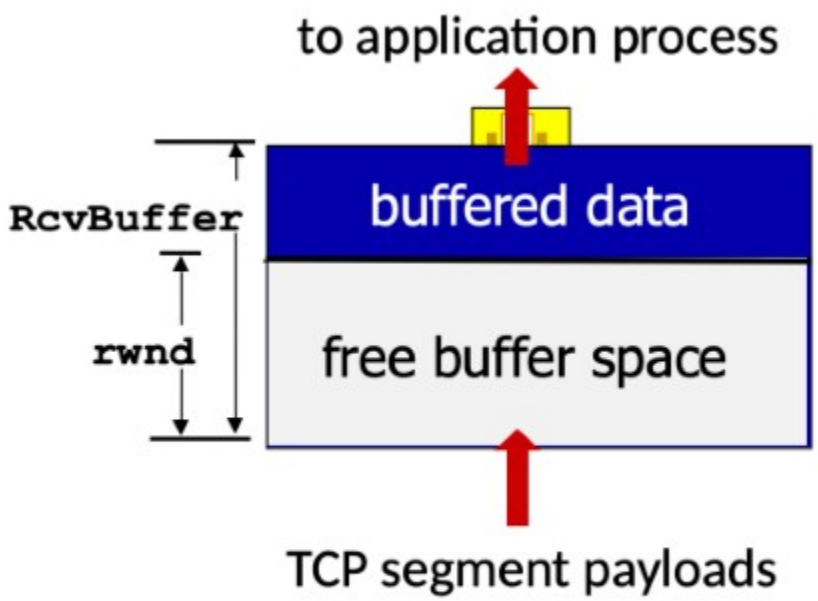
Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?





TCP provides a **flow-control service** to its applications to eliminate the possibility of the sender overflowing the receiver's buffer. Flow control is thus a speed-matching service, matching the rate at which the sender is sending against the rate at which the receiving application is reading.

TCP provides flow control by having the sender maintain a variable called the **receive window** (we have seen it in the TCP segment structure). Informally, the receive window is used to give the sender an idea of how much free buffer space is available at the receiver.
So with the **flow control**, the receiver controls the sender, so the sender won't overflow the receiver's buffer by transmitting too much data too fast.



TCP receiver-side buffering

So the receiver allocates a receive buffer to this connection, of size $RcvBuffer$. From time to time, the application process in Host B reads from the buffer.
We have other two variables:

- $LastByteRead$
  The number of the last byte in the data stream read from the buffer by the application process in B
- $LastByteRcvd$
  The number of the last byte in the data stream that has arrived from the network and has been placed in the receive buffer at B

Because TCP is not permitted to overflow this buffer, we always have:

$$LastByteRead - LastByteRcvd \leq RcvBuffer$$

The **receive window** $rwnd$, that is the amount of spare room in the buffer, is:

$$rwnd = RcvBuffer - [LastByteRcvd - LastByteRead]$$

Obviously, $rwnd$ is variable over the time.

So Host B tells Host A how much spare room it has in the connection buffer by placing its current value of $rwnd$ in the receive window field of every segment it sends to A. Initially, Host B sets $rwnd = RcvBuffer$.

The sender will not overflow the receiver by keeping the amount of unacknowledged data less than the value of $rwnd$

There is one minor technical problem with this scheme. When $rwnd = 0$, the sender stops the sending of segments, but in this manner it won't have any response/advice of when the $rwnd$ is different from zero. So Host A continue to send segments with one data byte when B's receive window is zero, only to know the new values of $rwnd$

Having described TCP's flow-control service, we briefly mention here that UDP does not provide flow control and consequently, segments may be lost at the receiver due to buffer overflow

TCP connection management

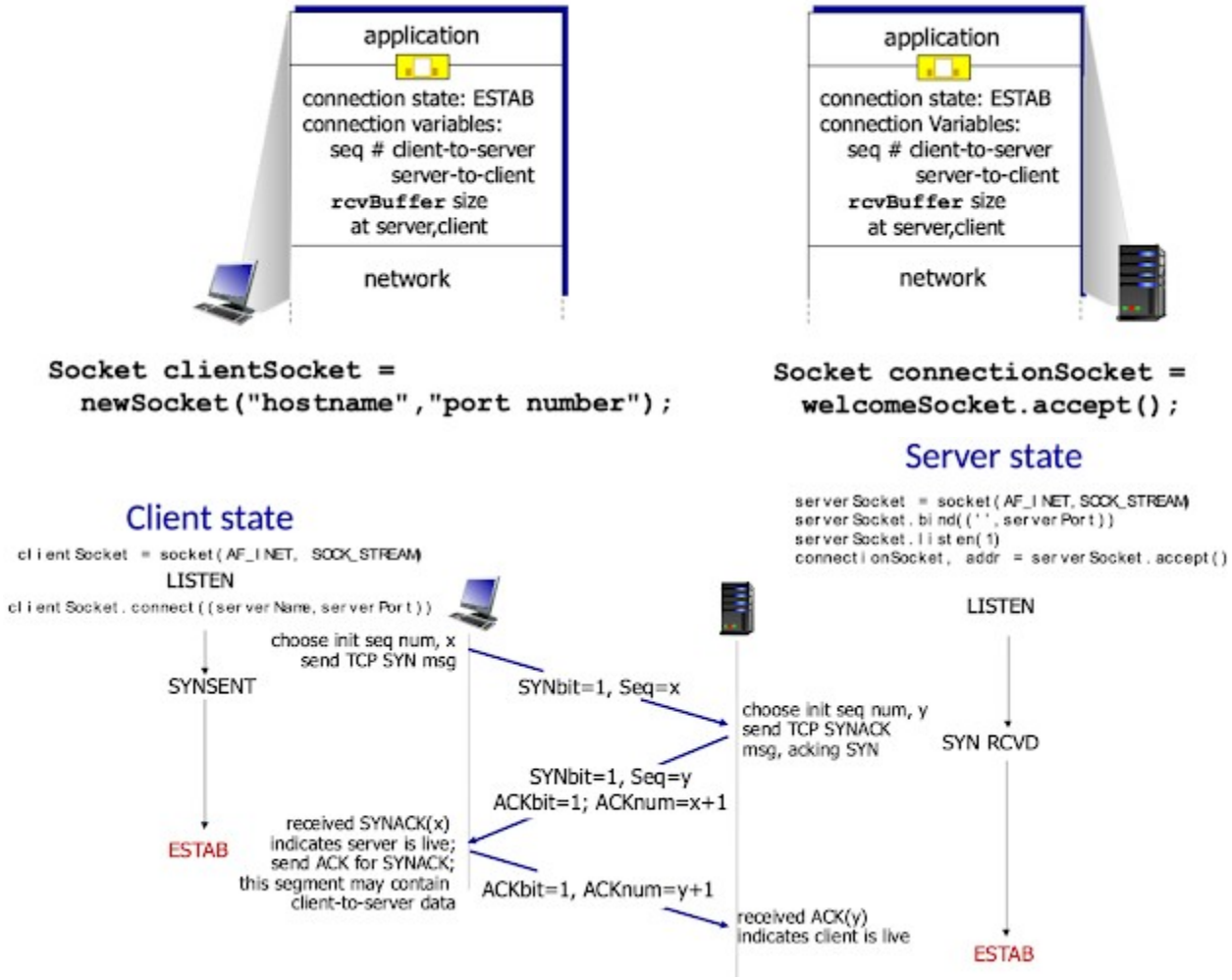
In this subsection we take a closer look at how a TCP connection is established and torn down. Although this topic may not seem particularly thrilling, it is important because TCP connection establishment can significantly add to perceived delays (for example, when surfing the Web).

Let's first take a look at how a TCP connection is established: suppose that a client application process informs the client TCP that it wants to establish a connection to a process in the server. The connection is established in the following manner:

1. The client-side TCP first sends a special TCP segment to the server-side TCP. This special segment contains no application-layer data. But one of the flag bits in the segment's header, the **SYN** bit, is set to 1. For this reason this segment is called **SYN segment**
   In addition, the client randomly chooses an initial sequence number ($client_isn$) and puts this number in the sequence number field of the initial TCP SYN segment.
2. Once the IP datagram containing the TCP SYN segment arrives at the server host (assuming it does arrive!), the server extracts the TCP SYN segment from the datagram, allocates the TCP buffers and variables to the connection, and sends a connection-granted segment to the client TCP.
   This connection-granted segment also contains no application-layer data. However, it does contain three important pieces of information in the segment header:
   - the SYN bit is set to 1
   - the ACK bit is set to 1
   - the acknowledgment field of the TCP segment header is set to $client_isn + 1$
   - the server chooses its own initial sequence number ($server_isn$) and puts this value in the sequence number field

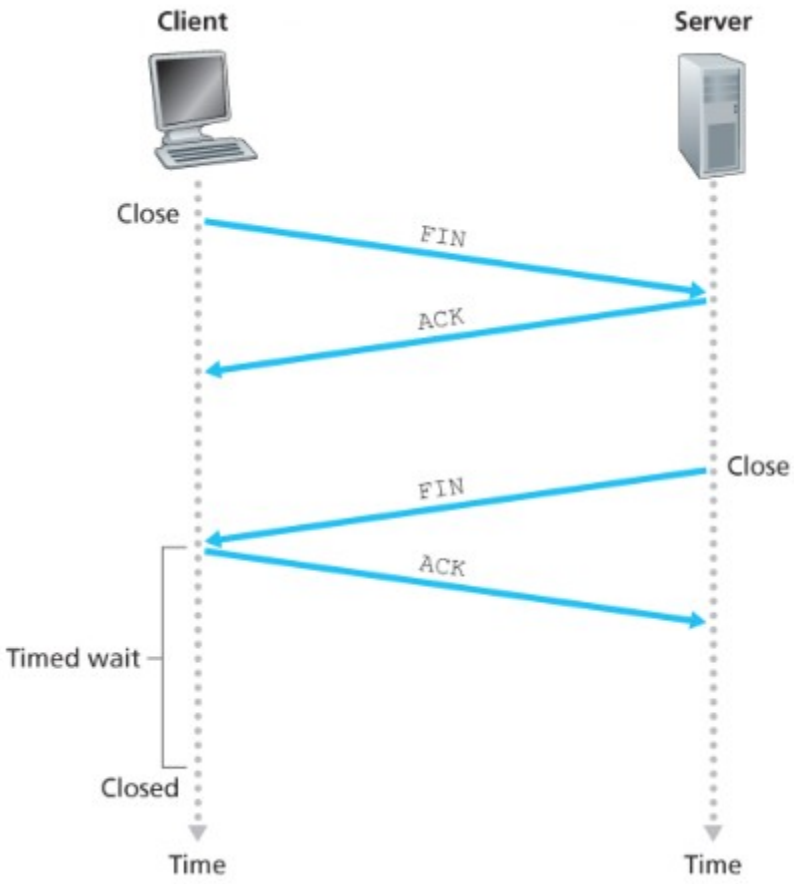
   This connection-granted segment is saying, in effect, "I received your SYN packet to start a connection with your initial sequence number, $client_isn$. I agree to establish this connection. My own initial sequence number is $server_isn$."
   The connection-granted segment is referred to as a **SYNACK segment**
3. Upon receiving the SYNACK segment, the client also allocates buffers and variables to the connection.
   The client host then sends the server yet another segment; this last segment acknowledges the server's connection-granted segment. The SYN bit is set to zero, since the connection is established. This third stage of the three-way handshake may carry client-to-server data in the segment payload.

Once these three steps have been completed, the client and server hosts can send segments containing data to each other. For this reason, this connection-establishment procedure is often referred to as a **three-way handshake**.

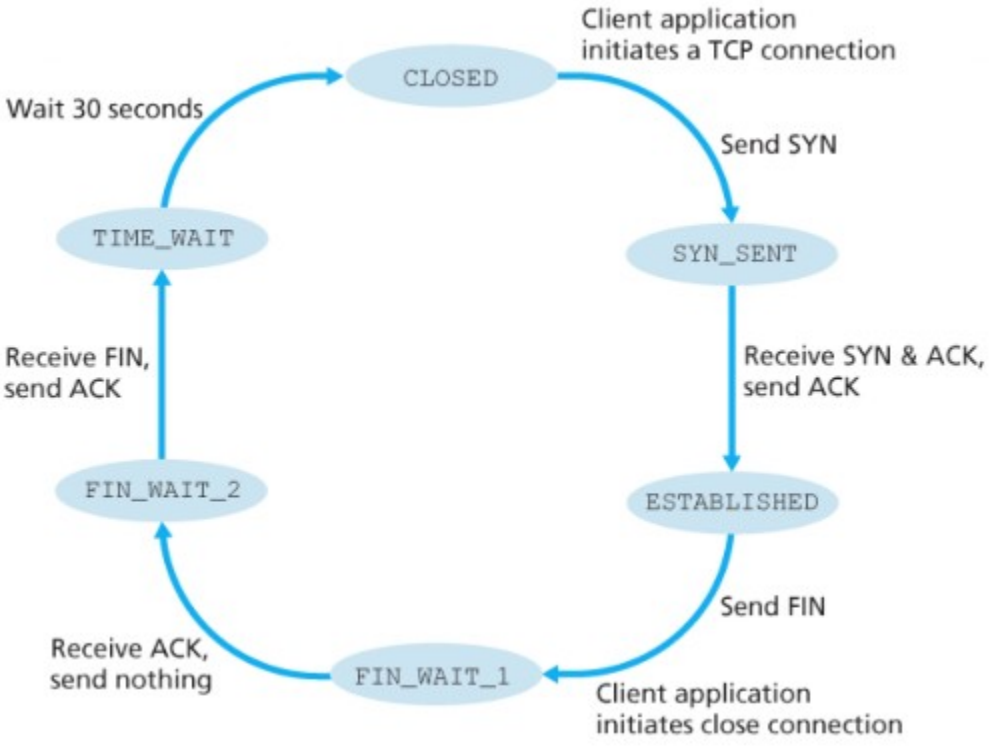


All good things must come to an end, and the same is true with a TCP connection. Either of the two processes participating in a TCP connection can end the connection. When a connection ends, the “resources” in the hosts must be deallocated

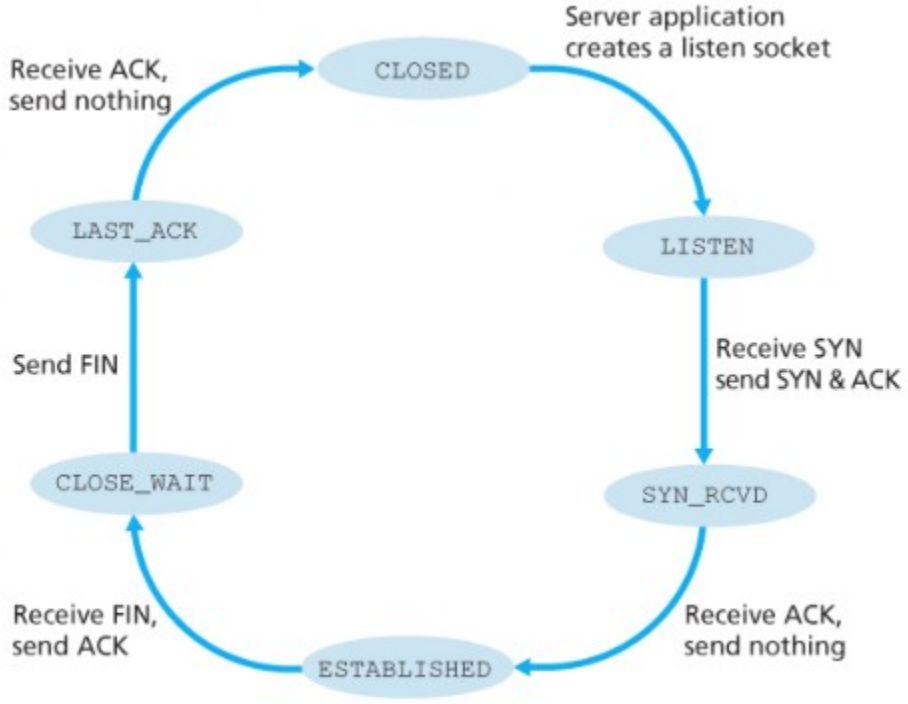
When the client wants to close the connection, it sends a special TCP segment to the server process. This special segment has a flag bit in the segment’s header, the **FIN bit** , set to 1. When the server receives this segment, it sends the client an acknowledgment segment in return. The server then sends its own shutdown segment, which has the FIN bit set to 1. Finally, the client acknowledges the server’s shutdown segment. At this point, all the resources in the two hosts are now deallocated.



Let's see the various TCP states in a client:

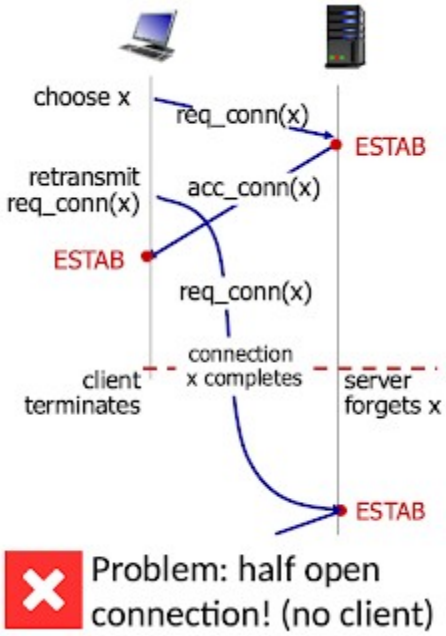


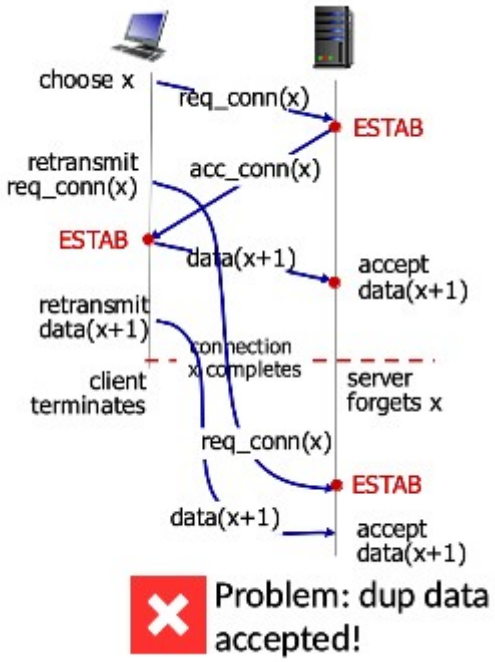
And the server TCP states:



A two-way handshake doesn't work in network because:

- There is variable delays
- We have to retransmit message lost
- We have to reorder message
- We can't "see" the other side





When a client tries to establish a connection on a host port without any server running on it, then the host will send a special reset segment to the source. This TCP segment has the RST flag bit set to 1

Principles of congestion control

In the previous sections, we examined both the general principles and specific TCP mechanisms used to provide for a reliable data transfer service in the face of packet loss. We mentioned earlier that, in practice, such loss typically results from the overflowing of router buffers as the network becomes congested.

The congestion of the network is caused when too many sources send too much data too fast for the network to handle. The congestion control is different from the flow control because the second is based on one sender and one receiver, the first is based on many senders.

The Causes and the Costs of Congestion

Let's begin our general study of congestion control by examining three increasingly complex scenarios in which congestion occurs.

In the first scenario we have **two senders** and **a router with infinite buffers**:

- λ_{in} is the average rate in $bytes / sec$ in which Host A is sending data into the connection (for example, passing data to the transport-level protocol via a socket)
  These data are original in the sense that each unit of data is sent into the socket only

once.

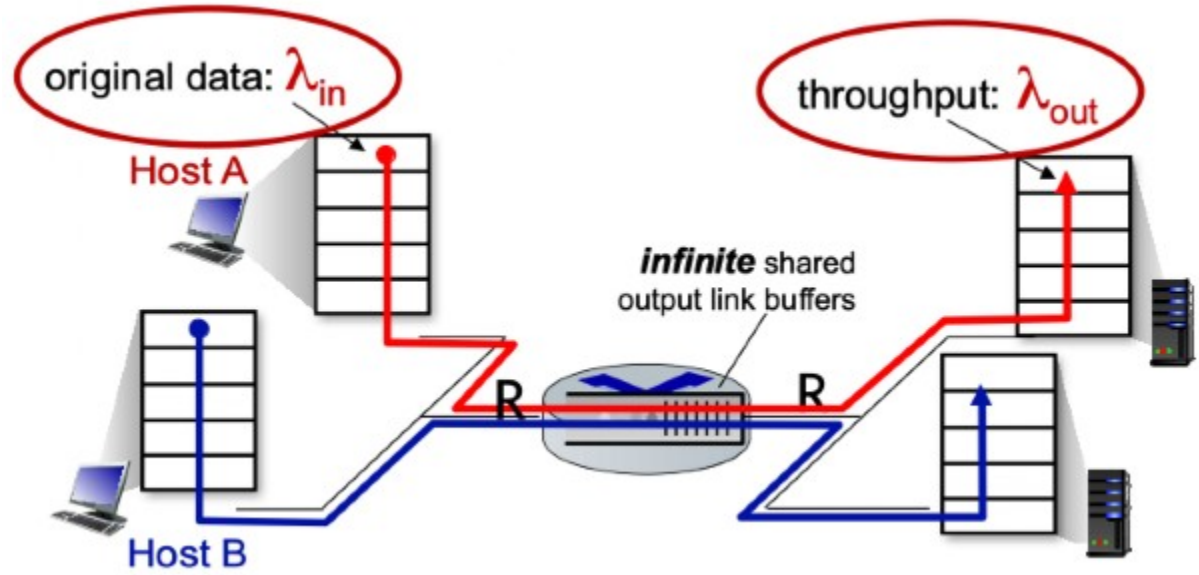
The underlying transport- level protocol is a simple one (no error recovery like retransmission, no flow control and no congestion control)

Ignoring the additional overhead due to adding transport- and lower-layer header information, the rate at which Host A offers traffic to the router in this first scenario is thus λ_{in} $bytes / sec$.

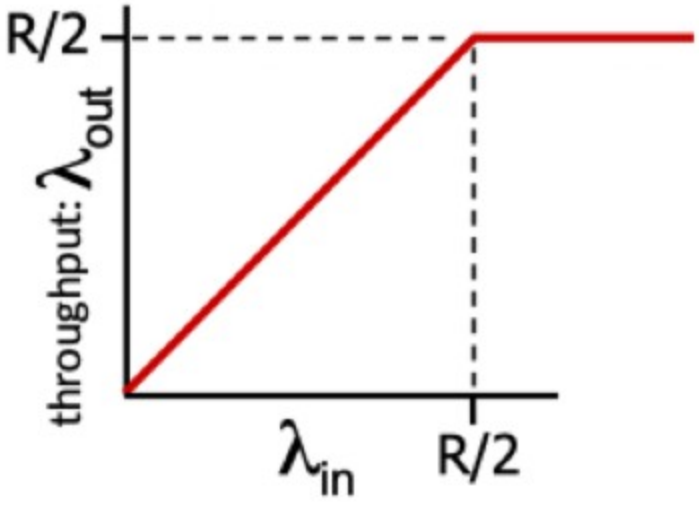
Host B operates in a similar manner, and we assume for simplicity that it too is sending at a rate of λ_{in} $bytes / sec$

- λ_{out} is the throughput at which the Hosts C and D receive the data
- R is the capacity of the shared links
- We have 2 flows, one from Host A to Host C, and one from Host B to Host D

The router has buffers that allow it to store incoming packets when the packet-arrival rate exceeds the outgoing link's capacity. In this first scenario, we assume that the router has an infinite amount of buffer space.

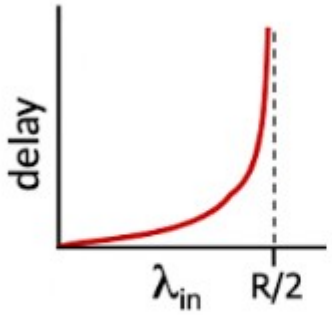


For a sending rate between 0 and R/2, the throughput at the receiver equals the sender's sending rate, everything sent by the sender is received at the receiver with a finite delay. When the sending rate is above R/2, however, the throughput is only R/2. This upper limit on throughput is a consequence of the sharing of link capacity between two connections. The link simply cannot deliver packets to a receiver at a steady-state rate that exceeds R/2. No matter how high Hosts A and B set their sending rates, they will each never see a throughput higher than R/2



maximum per-connection throughput: R/2

Having a throughput of R/2 might actually appear to be a good thing, because the link is fully utilized in delivering packets to their destinations. But the consequence of operating near link capacity, shown in the next image, is that the average delay becomes larger and larger.



large delays as arrival rate λ_{in} approaches capacity

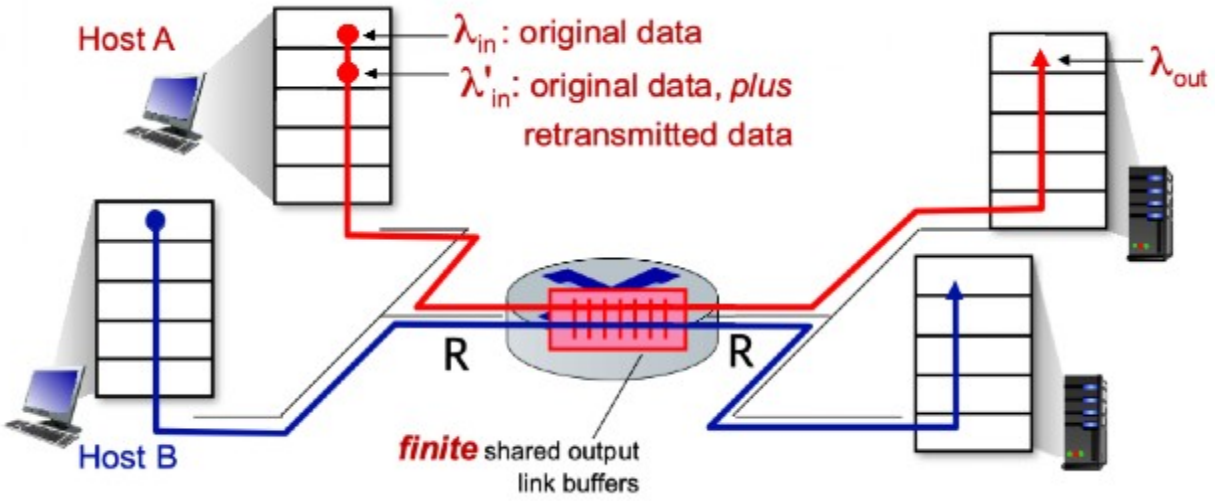
When the sending rate exceeds R/2, the average number of queued packets in the router is unbounded, and the average delay between source and destination becomes infinite. Thus, while operating at an aggregate throughput of near R may be ideal from a throughput standpoint, it is far from ideal from a delay standpoint.

Even in this (extremely) idealized scenario, we've already found one cost of a congested network, **large queuing delays** are experienced as the packet-arrival rate nears the link capacity.

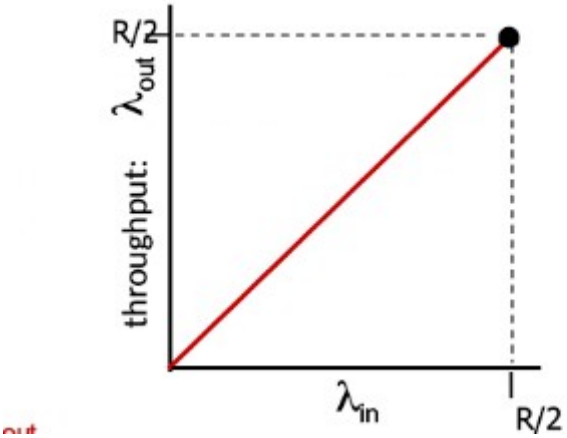
In the second scenario we have **two senders** and **a router with finite buffers**.
The first difference with the previous scenario is that the amount of router buffering is assumed to be finite. A consequence of this real-world assumption is that packets will be dropped when arriving at an already-full buffer.
We assume that each connection is reliable (so retransmission if a packet is dropped).

Because packets can be retransmitted, we must now be more careful with our use of the term sending rate. The rate at which the transport layer sends segments is higher than λ_{in}, because we have retransmission, so the new rate is λ'_{in} $bytes/sec$ with $\lambda'_{in} > \lambda_{in}$. λ'_{in} is called **offered load** to the network.

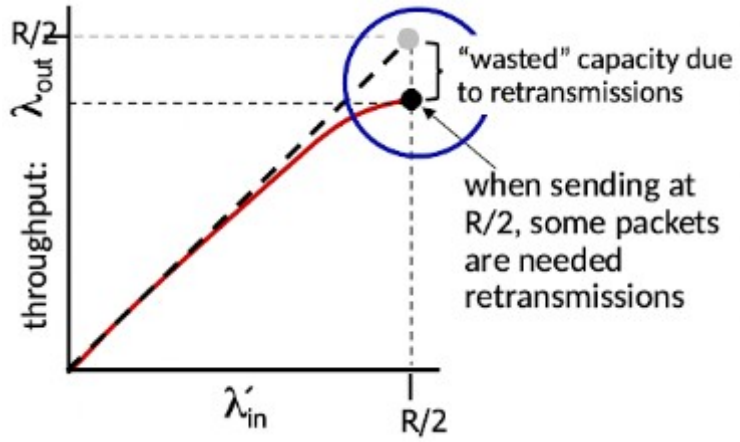
The performance realized under scenario 2 will now depend strongly on how retransmission is performed.
First, consider the unrealistic case that Host A is able to somehow (magically!) determine whether or not a buffer is free in the router and thus sends a packet only when a buffer is free. In this case, no loss would occur, λ_{in} would be equal to λ'_{in}, and the throughput of the connection would be equal to λ_{in}.



Note that the average host sending rate cannot exceed R/2 under this scenario, since packet loss is assumed never to occur.



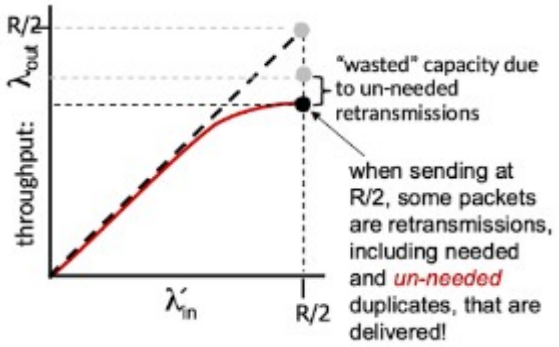
Consider next the slightly more realistic case that the sender retransmits only when a packet is known for certain to be lost (so in this case can be dropped at the router due to full buffers).



We see here another cost of a congested network, the sender must perform **retransmissions** in order to compensate for dropped (lost) packets due to buffer overflow.

Finally, let us consider the case that the sender may time out prematurely and retransmit a packet that has been delayed in the queue but not yet lost, that is the realistic scenario.

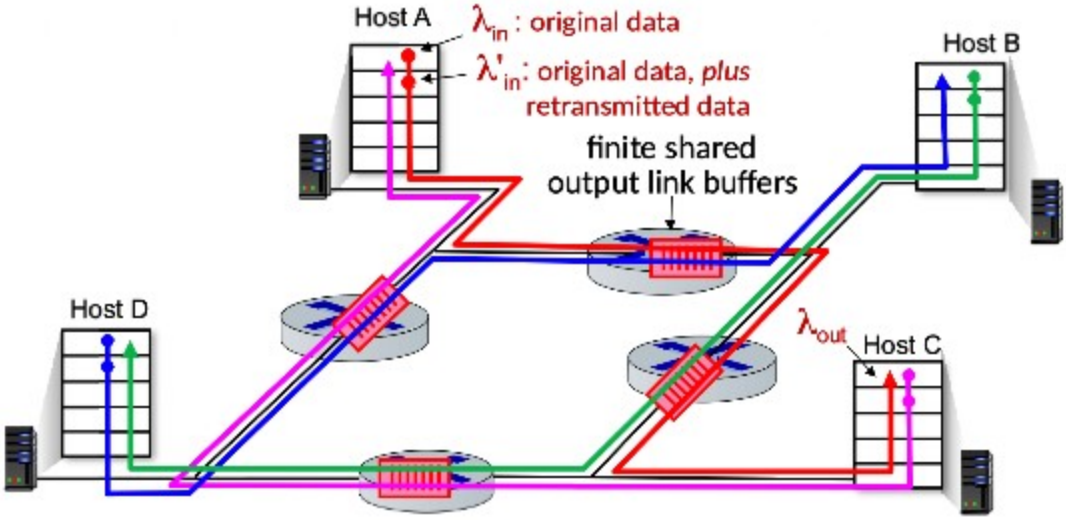
In this case, both the original data packet and the retransmission may reach the receiver. Of course, the receiver needs but one copy of this packet and will discard the retransmission. In this case, the work done by the router in forwarding the retransmitted copy of the original packet was wasted, as the receiver will have already received the original copy of this packet.



Here then is yet another cost of a congested network, **unneeded retransmissions** by the sender in the face of large delays may cause a router to use its link bandwidth to forward unneeded copies of a packet

In the second scenario we have **four senders**, **a router with finite buffers** and **multihop paths**.
In our final congestion scenario, four hosts transmit packets, each over overlapping two-hop paths.

We again assume that each host uses a timeout/retransmission mechanism to implement a reliable data transfer service, that all hosts have the same value of λ_{in}, and that all router links have capacity R $bytes/sec$.



Let's consider the connection from Host A to Host C, passing through routers R1 and R2. The A–C connection shares router R1 with the D–B connection and shares router R2 with the B–D connection.

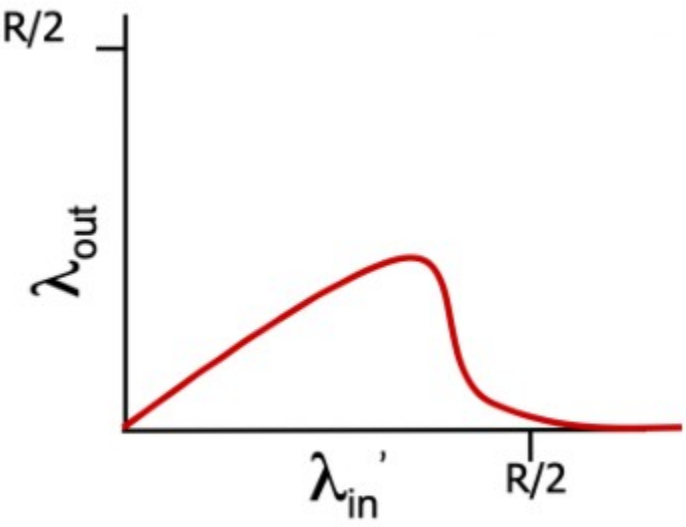
For extremely small values of λ_{in}, buffer overflows are rare (as in congestion scenarios 1 and 2), and the throughput approximately equals the offered load. Thus, for small values of λ_{in}, an increase in λ_{in} results in an increase in λ_{out}.

Having considered the case of extremely low traffic, let's next examine the case that λ_{in} (and hence λ'_{in}) is extremely large.

Consider router R2. The A–C traffic arriving to router R2 (which arrives at R2 after being forwarded from R1) can have an arrival rate at R2 that is at most R, the capacity of the link from R1 to R2, regardless of the value of λ_{in}.

Because the A–C and B–D traffic must compete at router R2 for the limited amount of buffer space, the amount of A–C traffic that successfully gets through R2 (that is, is not lost due to buffer overflow) becomes smaller and smaller as the offered load from B–D gets larger and larger.

In the limit, as the offered load approaches infinity, an empty buffer at R2 is immediately filled by a B–D packet, and the throughput of the A–C connection at R2 goes to zero. This, in turn, implies that the A–C end-to-end throughput goes to zero in the limit of heavy traffic.

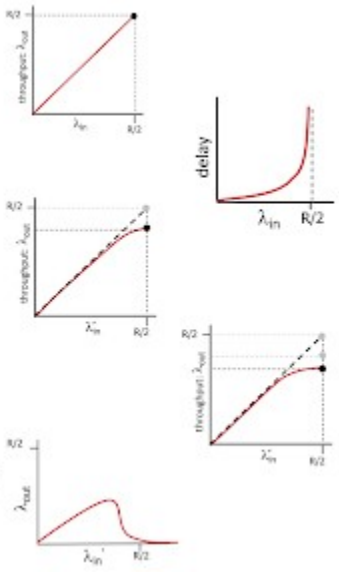


The reason for the eventual decrease in throughput with increasing offered load is evident when one considers the amount of wasted work (if a packet is dropped in the second router, the work of the first is wasted) done by the network.

So here we see yet another cost of dropping a packet due to congestion, when a packet is dropped along a path, **the transmission capacity** that was used at each of the upstream links to forward that packet to the point at which it is dropped **ends up having been wasted**

Let's summarise all the concepts we've seen:

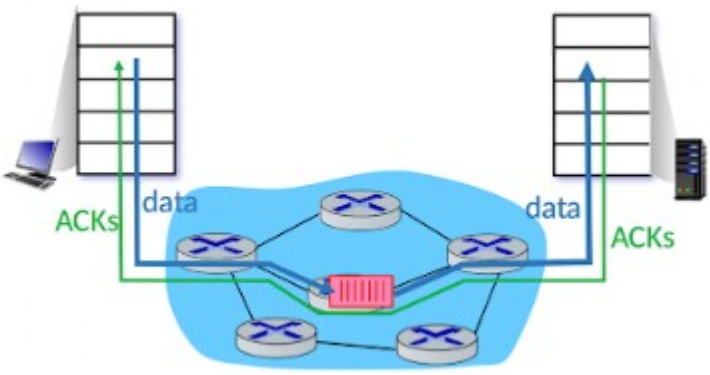
- throughput can never exceed capacity
- delay increases as capacity approached
- loss/retransmission decreases effective throughput
- un-needed duplicates further decreases effective throughput
- upstream transmission capacity / buffering wasted for packets lost downstream



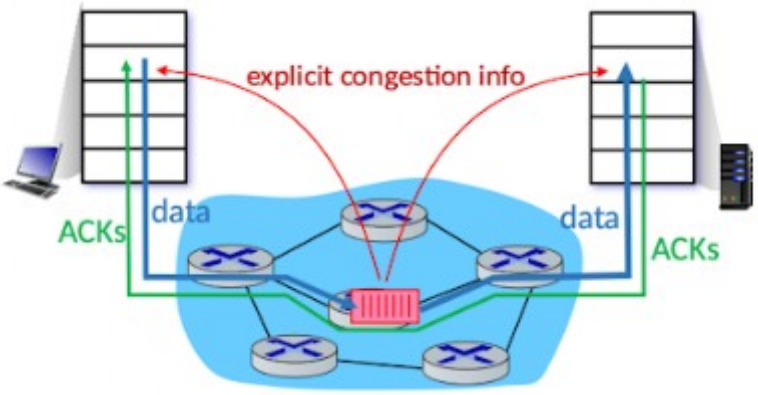
Approaches to Congestion Control

At the highest level, we can distinguish among congestion-control approaches by whether the network layer provides explicit assistance to the transport layer for congestion-control purposes:

- **End-to-end congestion control**
  In an end-to-end approach to congestion control, the network layer provides no explicit support to the transport layer for congestion-control purposes.
  Even the presence of network congestion must be inferred by the end systems based only on observed network behavior (for example, packet loss and delay)
  TCP takes this end-to-end approach toward congestion control, since the IP layer is not required to provide feedback to hosts regarding network congestion. TCP segment loss (as indicated by a timeout or the receipt of three duplicate acknowledgments) is taken as an indication of network congestion, and TCP decreases its window size accordingly.



- **Network-assisted congestion control**
  With network-assisted congestion control, routers provide explicit feedback to the sender and/or receiver regarding the congestion state of the network.
  This feedback may be as simple as a single bit indicating congestion at a link. More sophisticated feedback is also possible (router informs the sender of the maximum host sending rate it (the router) can support on an outgoing link)



For network-assisted congestion control, congestion information is typically fed back from the network to the sender in one of two ways:

- Direct feedback may be sent from a network router to the sender
- Router marks/updates a field in a packet flowing from sender to receiver to indicate congestion

TCP congestion control

In this section we return to our study of TCP. As we learned in Section 3.5, TCP provides a reliable transport service between two processes running on different hosts
Another key component of TCP is its congestion-control mechanism. As indicated in the previous section, TCP must use end-to-end congestion control rather than network-assisted congestion control, since the IP layer provides no explicit feedback to the end systems regarding network congestion.

The approach taken by TCP is to have each sender limit the rate at which it sends traffic into its connection as a function of perceived network congestion (so it increase the sending rate if it perceives little congestion and it decrease the sending rate if it perceives high congestion).
There are three question about this approach:

- How does a TCP sender limit the rate at which it sends traffic into its connection?
- How does a TCP sender perceive that there is congestion on the path between itself and the destination?
- What algorithm should the sender use to change its send rate as a function of perceived end-to-end congestion?

The TCP congestion-control mechanism operating at the sender keeps track of an additional variable, the **congestion window**. The congestion window, denoted $cwnd$, imposes a constraint on the rate at which a TCP sender can send traffic into the network.
So, mixed with our precedence knowledge, the amount of unacknowledged data at a sender may not exceed the minimum of $cwnd$ and $rwnd$, that is:

$$LastByteSent - LastByteAcked \leq min\{cwnd,\ rwnd\}$$

In order to focus on congestion control (as opposed to flow control), let us henceforth assume that the TCP receive buffer is so large that the receive-window constraint $rwnd$ can be ignored; thus, the amount of unacknowledged data at the sender is solely limited by $cwnd$. We will also assume that the sender always has data to send, i.e., that all segments in the congestion window are sent. So, by adjusting the value of $cwnd$, the sender can therefore adjust the rate at which it sends data into its connection (that is $cwnd / RTT$).

Let's next consider how a TCP sender perceives that there is congestion on the path between itself and the destination. Let us define a "loss event" at a TCP sender as the occurrence of either a timeout or the receipt of three duplicate ACKs from the receiver, this is an indication of congestion in the network (because routers overflow and lose packets).

Having considered how congestion is detected, let's next consider the more optimistic case when the network is congestion-free, that is, when a loss event doesn't occur.
In this case the sender will use acknowledgments to increase its congestion window size (and hence its transmission rate).

Because TCP uses acknowledgments to trigger (or clock) its increase in congestion window size, TCP is said to be **self-clocking**.

Given the mechanism of adjusting the value of cwnd to control the sending rate, the critical question remains: How should a TCP sender determine the rate at which it should send?
If TCP senders collectively send too fast, they can congest the network; however, if TCP senders are too cautious and send too slowly, they could under utilize the bandwidth in the network.
TCP answers these questions using the following guiding principles:

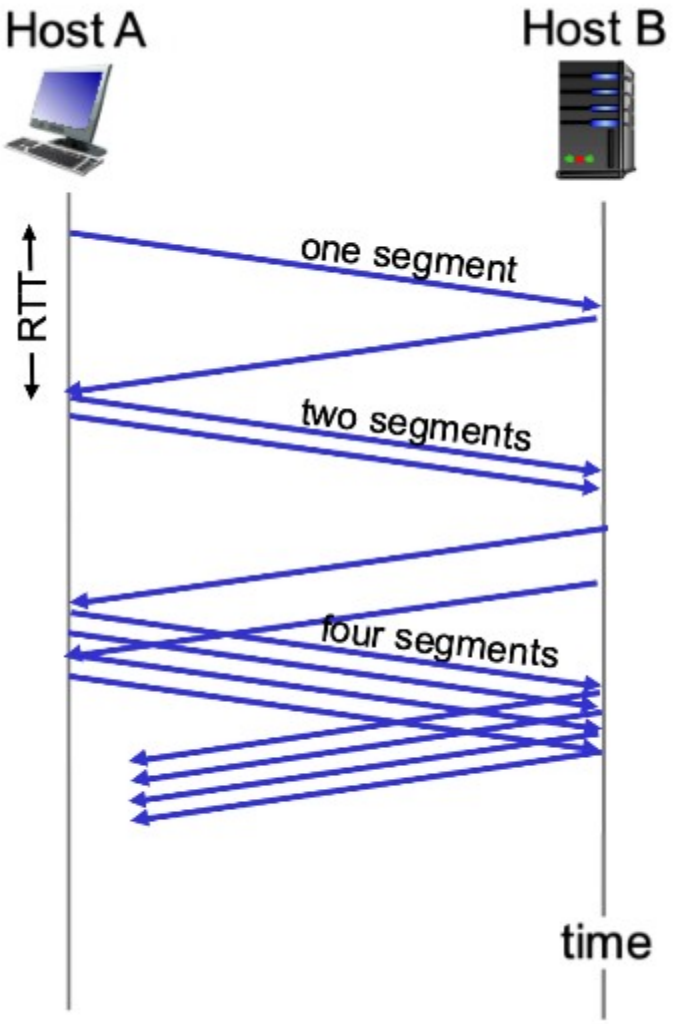
- **A lost segment implies congestion, and hence, the TCP sender's rate should be decreased when a segment is lost**
  From a congestion-control standpoint, the question is how the TCP sender should decrease its congestion window size, and hence its sending rate, in response to this inferred loss event.
- **An acknowledged segment indicates that the network is delivering the sender's segments to the receiver, and hence, the sender's rate can be increased when an ACK arrives for a previously unacknowledged segment**
  The congestion window size can thus be increased
- **Bandwidth probing**
  Note that there is no explicit signaling of congestion state by the network, ACKs and loss events serve as implicit signals, and that each TCP sender acts on local information asynchronously from other TCP senders

Given this overview of TCP congestion control, we're now in a position to consider the details of the celebrated **TCP congestion-control algorithm**, which hase three major components:

1. Slow start
2. Congestion avoidance
3. Fast recovery

Slow start and congestion avoidance are mandatory components of TCP, differing in how they increase the size of cwnd in response to received ACKs

When a TCP connection begins, the value of $cwnd$ is typically initialized to a small value of 1 MSS, resulting in an initial sending rate of roughly MSS / RTT. This initial rate is very low and the TCP sender would like to find the amount of available bandwidth quickly.
Thus, in the **slow-start state**, the value of $cwnd$ begins at 1 MSS and increases by 1 MSS every time a transmitted segment is first acknowledged, that is doubling the sending rate every RTT.



So, the initial rate is slow, but it rumps up exponentially fast.
But when should this exponential growth end? Slow start provides several answers to this question:

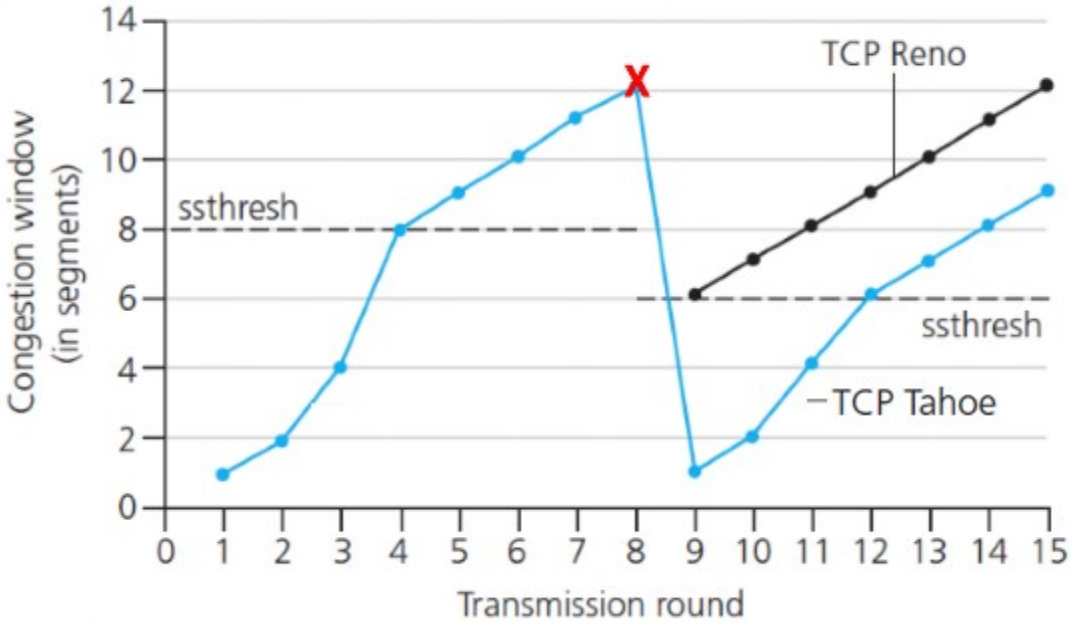
- If there is a loss event (i.e., congestion) indicated by a timeout, the TCP sender sets the value of *cwnd* to 1 and begins the slow start process anew
  It also sets the value of a second state variable, *ssthresh* (shorthand for “slow start threshold”) to *cwnd* / 2 (half of the value of the congestion window value when congestion was detected)
- Since *ssthresh* is half the value of *cwnd* when congestion was last detected, it might be a bit reckless to keep doubling *cwnd* when it reaches or surpasses the value of *ssthresh*. Thus, when the value of *cwnd* equals *ssthresh*, slow start ends and TCP transitions into congestion avoidance mode.
- The final way in which slow start can end is if three duplicate ACKs are detected, in which case TCP performs a fast retransmit and enters the fast recovery state (we will se this state later).

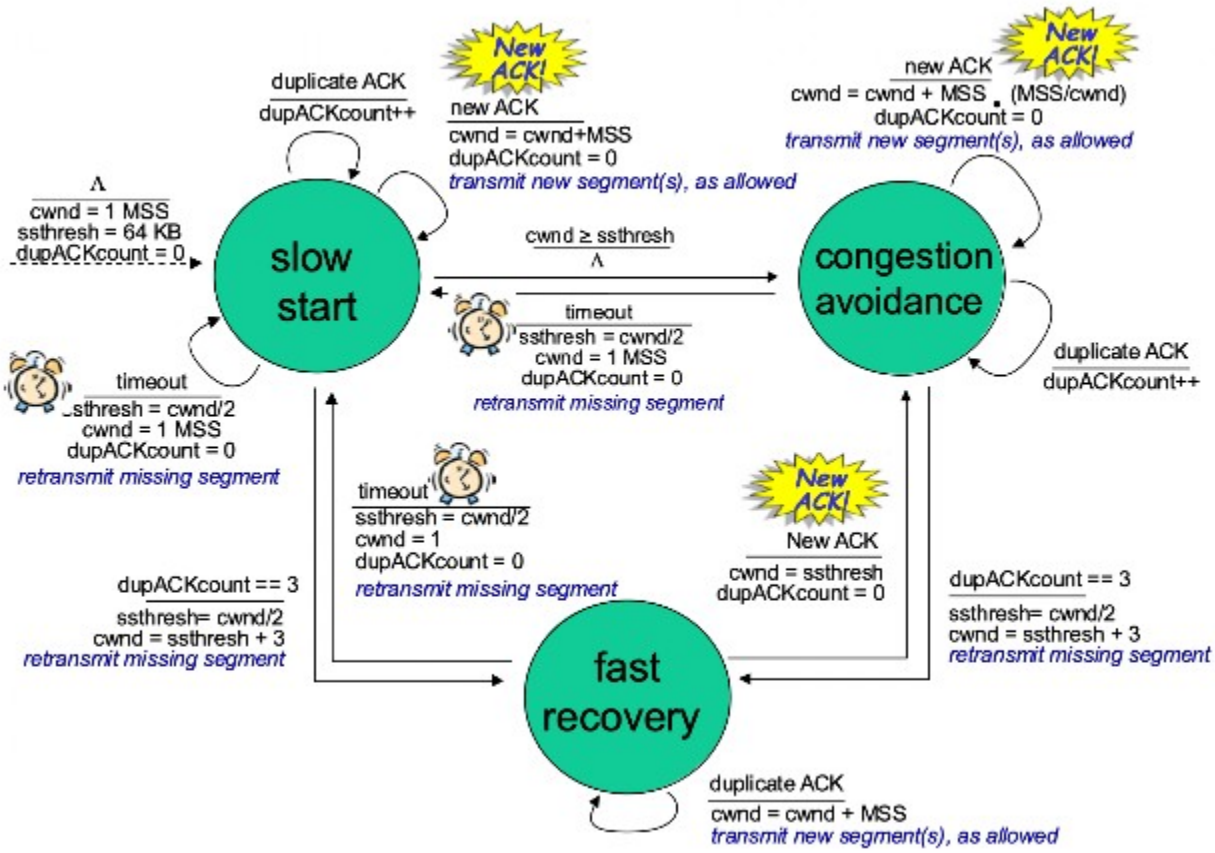
On entry to the **congestion-avoidance state**, the value of cwnd is approximately half its value when congestion was last encountered (congestion could be just around the corner).
Thus, rather than doubling the value of $cwnd$ every RTT, TCP adopts a more conservative approach and increases the value of $cwnd$ by just a single MSS every RTT.
This can be accomplished in several ways. A common approach is for the TCP sender to increase $cwnd$ by MSS bytes ($\frac{MSS}{cwnd}$) whenever a new acknowledgment arrives. For example, if MSS is 1,460 bytes and $cwnd$ is 14,600 bytes, then 10 segments are being sent within an RTT. Each arriving ACK (assuming one ACK per segment) increases the congestion window size by $\frac{1}{10}$ MSS, and thus, the value of the congestion window will have increased by one MSS after ACKs when all 10 segments have been received.
But when should congestion avoidance's linear increase (of 1 MSS per RTT) end? TCP's congestion-avoidance algorithm behaves the same when a timeout occurs. As in the case of slow start: The value of $cwnd$ is set to 1 MSS, and the value of $ssthresh$ is updated to half the value of $cwnd$ when the loss event occurred.

Recall, however, that a loss event also can be triggered by a triple duplicate ACK event.
So TCP's behavior to this type of loss event should be less drastic than with a timeout-indicated loss: TCP halves the value of $cwnd$ (adding in 3 MSS for good measure to account for the triple duplicate ACKs received) and records the value of $ssthresh$ to be half the value of $cwnd$ when the triple duplicate ACKs were received. The fast-recovery state is then entered.

In **fast recovery**, the value of $cwnd$ is increased by 1 MSS for every duplicate ACK received for the missing segment that caused TCP to enter the fast-recovery state. Eventually, when an ACK arrives for the missing segment, TCP enters the congestion-avoidance state after deflating $cwnd$. If a timeout event occurs, fast recovery transitions to the slow-start state after performing the same actions as in slow start and congestion avoidance: The value of $cwnd$ is set to 1 MSS, and the value of $ssthresh$ is set to half the value of $cwnd$ when the loss event occurred.

An early version of TCP, called TCP Tahoe, doesn't implement the fast recovery. The newer version, called TCP Reno, implements it.

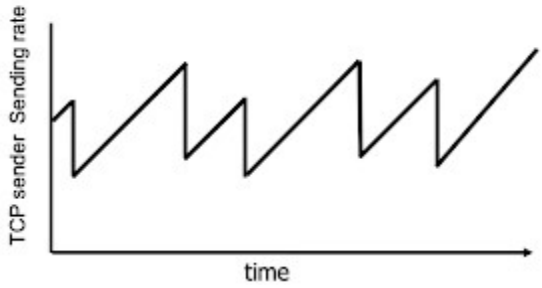




TCP congestion control is often referred to as an **additive-increase**, **multiplicative-decrease** (**AIMD**) form of congestion control (ignoring the slowstart state and the fast recovery).

Additive Increase
increase sending rate by 1 maximum segment size every RTT until loss detected

Multiplicative Decrease
cut sending rate in half at each loss event



AIMD sawtooth behavior: *probing* for bandwidth

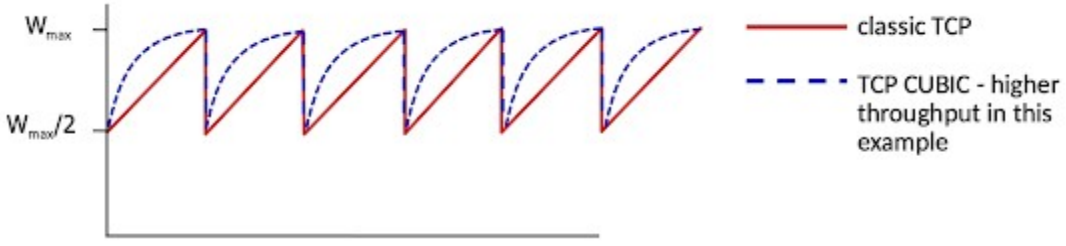
As we've already said, when the window size is w bytes and the current round-trip time is RTT seconds, then TCP's transmission rate is roughly w / RTT.

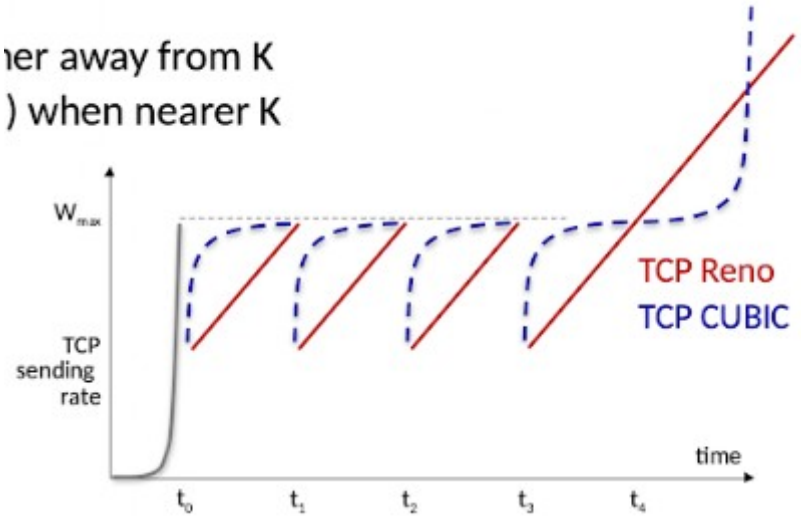
Denote by W_{max} the value of w when a loss event occurs. So the TCP transmission rate ranges from $\frac{W_{max}}{2 \cdot RTT}$ to $\frac{W_{max}}{RTT}$.

Because TCP's throughput (that is, rate) increases linearly between the two extreme values, we have

$$average\ \ throughput\ \ of\ \ a\ \ connection = 0.75 \cdot \frac{W}{RTT}$$

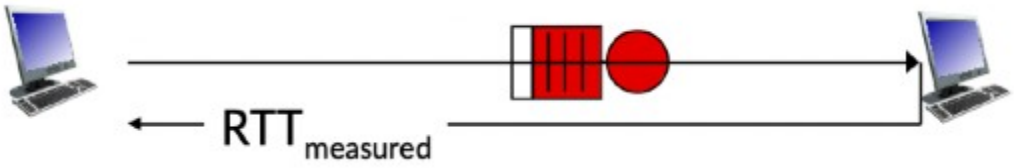
There are other TCP implementations, like the **TCP cubic**, that is, if W_{max} is the sending rate at which congestion loss was detected, after cutting rate/window in half on loss, initially ramp to W_{max} faster, but then approach W_{max} more slowly.





Let's focus on the delay (so the RTT) of the packets. A packet loss means that there is a **bottleneck link** that is congested.
If we increase the TCP sending rate, we will not increase the end-to-end throughput with a congested bottleneck link, but we will only increase the RTT.
So the goal is to keep the end-to-end pipe just full, but not fuller. In this manner we keep the bottleneck link busy transmitting, and we avoid high delays/buffering.

So let's denote the minimum observed RTT (the RTT with an uncongested path) with RTT_{min}.
So the uncongested throughput with congestion window $cwnd$ is $\frac{cwnd}{RTT_{min}}$.



Considering the measured throughput that is $\frac{cwnd}{RTT_{measured}}$ we have two cases:

- If the measured throughput is "very close" to the uncongested throughput: increase $cwnd$ linearly
- If the measured throughput is "far below" to the uncongested throughput: decrease $cwnd$ linearly

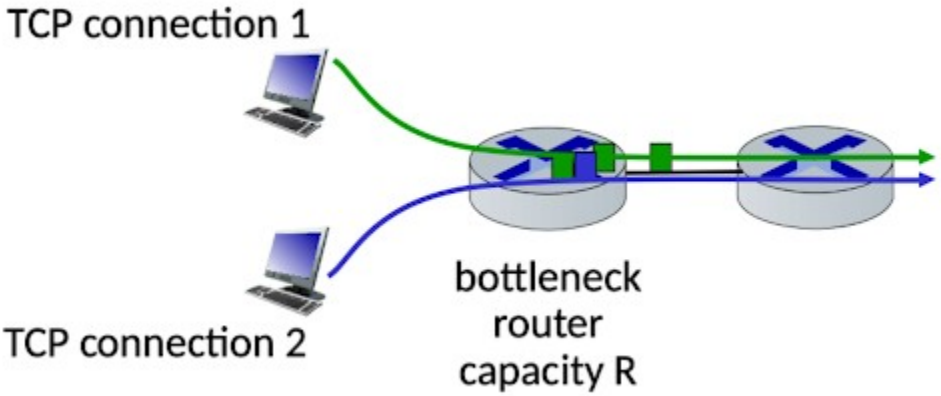
The advantage of this congestion control is that it delete the inducing/forcing of **loss**. So this approach maximizes the throughput while keeping the delay low.

Fairness

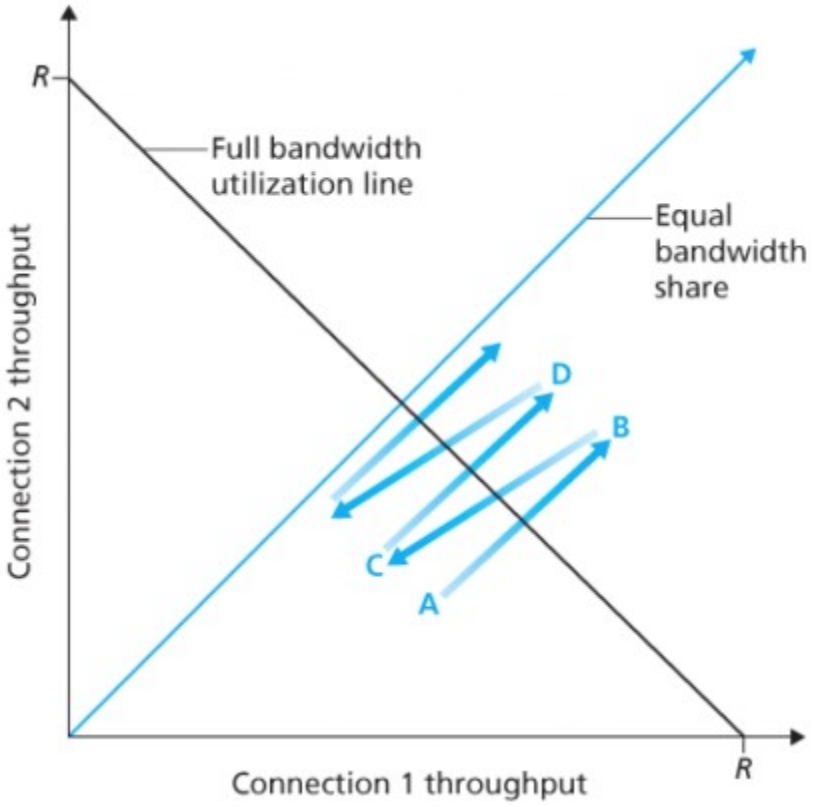
Consider K TCP connections, each with a different end-to-end path, but all passing through a bottleneck link with transmission rate R bps.

A congestion-control mechanism is said to be **fair** if the average transmission rate of each connection is approximately R / K; that is, each connection gets an **equal share** of the link bandwidth.

Is TCP's AIMD algorithm fair, particularly given that different TCP connections may start at different times and thus may have different window sizes at a given point in time?
Let's consider the simple case of two TCP connections sharing a single link with transmission rate R



Assume that the two connections have the same MSS and RTT (so same congestion window size, so same throughput) that they have a large amount of data to send, and that no other TCP connections or UDP datagrams traverse this shared link. Also, ignore the slow-start phase of TCP and assume the TCP connections are operating in CA mode (AIMD) at all times.



To be fair, we want the achieved throughputs to fall somewhere near the intersection of the equal bandwidth share line and the full bandwidth utilization line.
Let's suppose that at a given point in time the throughputs of the two connections are the point A.
The sum of the two throughput is lower than the full bandwidth utilization, so both connections will increase their windows by 1 MSS per RTT as a result of TCP's congestion-avoidance algorithm. Thus, the joint throughput of the two connections proceeds along a 45-degree line, reaching point B.
Now the sum of the two throughput is higher than the full bandwidth utilization, so both connections will decrease their windows by a factor of 2, the sum of the throughput will reach point C.
Iterating these procedures we can notice that at some point the sum of the throughput will fluctuate along the equal bandwidth share line, so the fairness is respected (only under idealized assumptions like same RTT and fixed number of sessions only in congestion avoidance status, so in the real world TCP isn't very fair).

Although a number of idealized assumptions lie behind this scenario, it still provides an intuitive feel for why TCP results in an equal sharing of bandwidth among connections.
Many multimedia applications, such as Internet phone and video conferencing, often do not run over TCP for this very reason, they do not want their transmission rate throttled, even if the network is very congested.
Instead, these applications prefer to run over UDP, which does not have built-in congestion control (preferring the losing of the packet but having a fixed rate).
Because TCP congestion control will decrease its transmission rate in the face of increasing congestion (loss), while UDP sources need not, it is possible for UDP sources to crowd out TCP traffic.

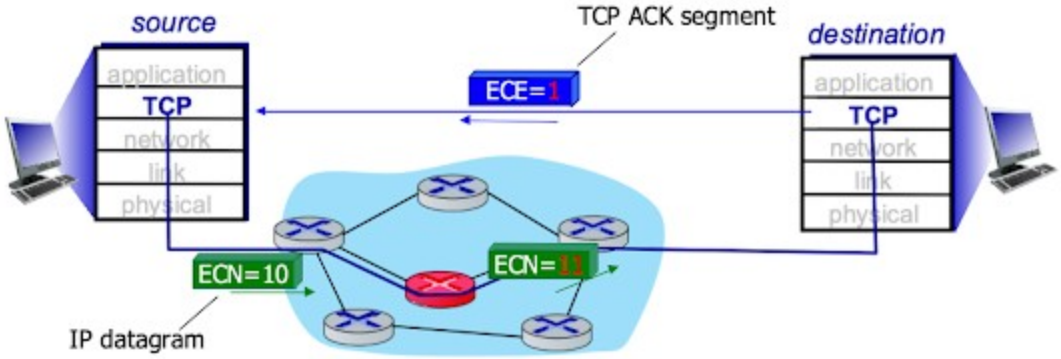
But even if we could force UDP traffic to behave fairly, the fairness problem would still not be completely solved. This is because there is nothing to stop a TCP-based application from using multiple parallel connections. When an application uses multiple parallel connections, it gets a larger fraction of the bandwidth in a congested link

Explicit Congestion Notification (ECN): Network-assisted Congestion Control

Since the initial standardization of slow start and congestion avoidance in the late 1980's, TCP has implemented the form of end-end congestion control that we studied in Section 3.7.1: a TCP sender receives no explicit congestion indications from the network layer, and instead infers congestion through observed packet loss.

More recently, extensions to both IP and TCP have been proposed, implemented, and deployed that allow the network to **explicitly signal congestion** to a TCP sender and receiver. This form of network-assisted congestion control is known as **Explicit Congestion Notification** (ECN).

At the network layer, **two bits** (with four possible values, overall) in the **Type of Service field** of the IP datagram header are used for ECN.
One setting of the ECN bits is used by a router to indicate that it (the router) is experiencing congestion.
A second setting of the ECN bits is used by the sending host to inform routers that the sender and receiver are ECN-capable, and thus capable of taking action in response to ECN-indicated network congestion.



As shown in the image, when the TCP in the receiving host receives an ECN congestion indication via a received datagram, the TCP in the receiving host informs the TCP in the sending host of the congestion indication by setting the ECE (**Explicit Congestion notification Echo**) bit in a receiver-to-sender TCP ACK segment.
Other transport-layer protocols besides TCP may also make use of network-layer-signaled ECN.

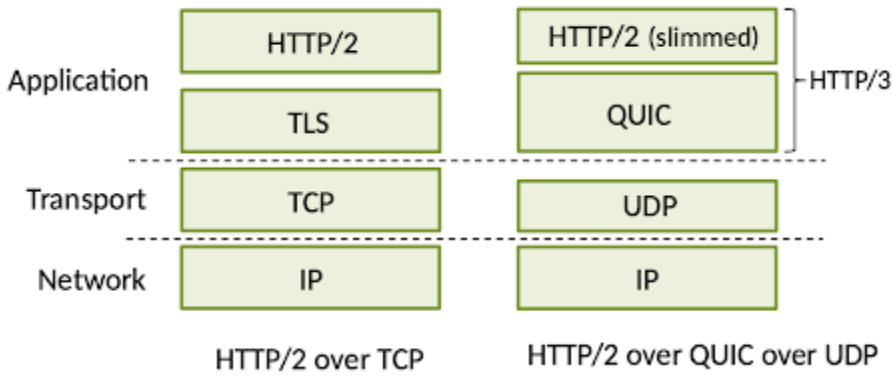
Evolution of transport-layer functionality

TCP and UDP were the principal transport protocols for over 40 years.
In the years different “flavors” of TCP protocol were developed in order to satisfy different scenarios, here some examples:

Scenario	Challenges
Long, fat pipes (large data transfers)	Many packets “in flight”; loss shuts down pipeline
Wireless networks	Loss due to noisy wireless links, mobility; TCP treat this as congestion loss
Long-delay links	Extremely long RTTs
Data center networks	Latency sensitive
Background traffic flows	Low priority, “background” TCP flows

There is the possibility of moving transport-layer functions to the application layer (using UDP as transport protocol). An example of this practice is QUIC (HTTP/3).

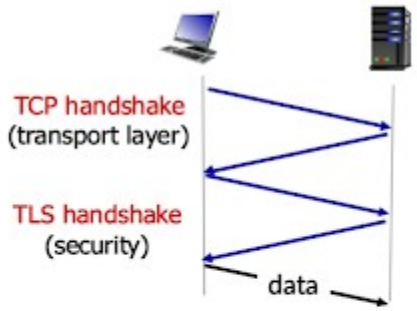
As we said **QUIC** is an application layer protocol that stands on top of UDP. Quic increases the performance of HTTP and it is deployed on many Google servers and apps (Chrome, YouTube app, etc).



QUIC adopts the approaches that we have studied in this chapter for:

- Connection establishment
- Error control
- Congestion control

So the algorithms are almost the same and the connection established guarantees reliability, congestion control, authentication, encryption and the state is established in one RTT.



TCP (reliability, congestion control state) + TLS (authentication, crypto state)

- 2 serial handshakes



QUIC: reliability, congestion control, authentication, crypto state

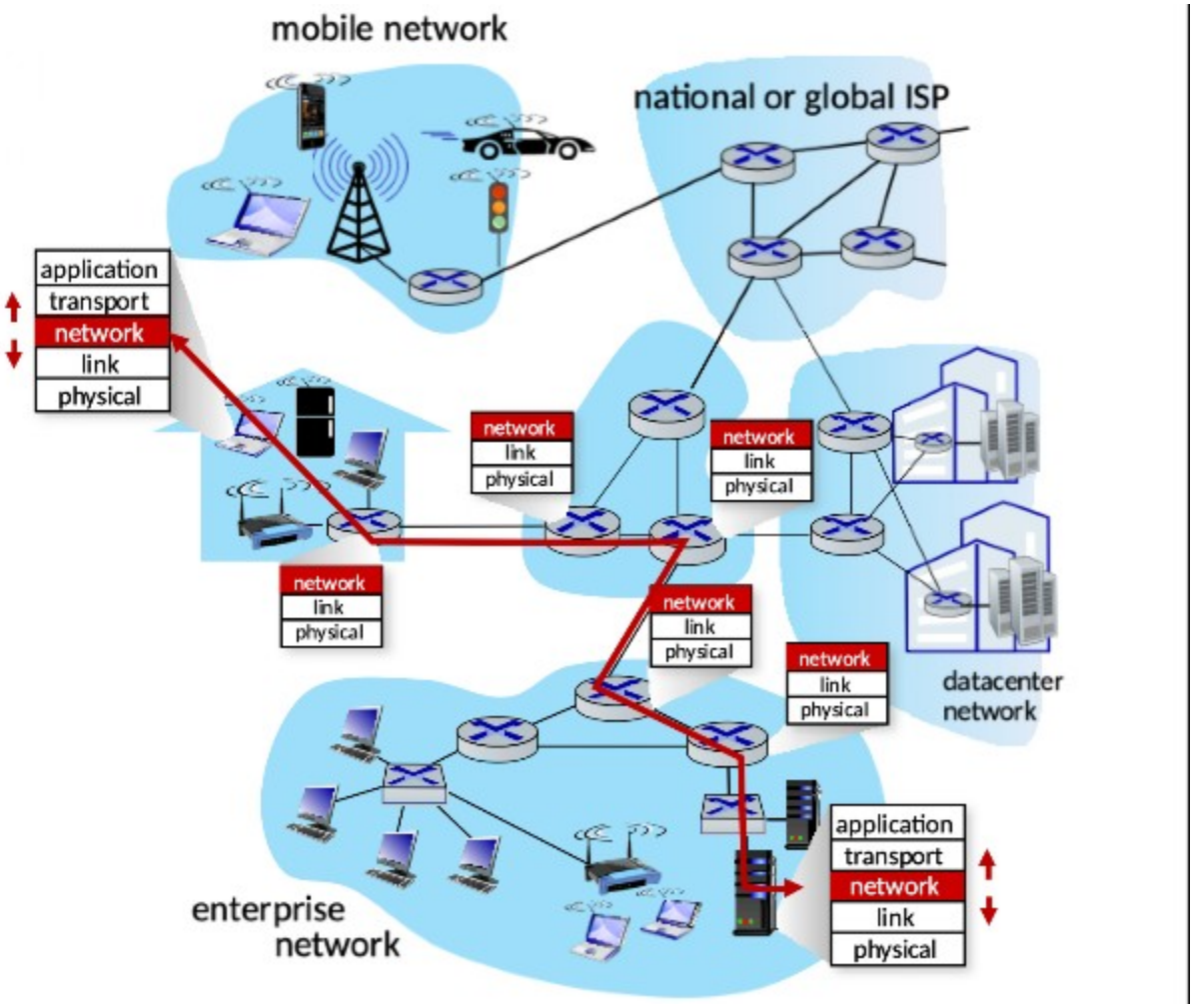
- 1 handshake

With QUIC we can have multiple application-level “streams” multiplexed over a single QUIC connection (obviously with a separate reliable data transfer, but with a common congestion control)

In QUIC we have parallelism, and we don’t suffer of HOL blocking
Another advantage of QUIC is that it is closer to the programmer, this allows it to have more flexibility.

The Network Layer: Data Plane

Overview of Network Layer



The image shows a simple network with two hosts, H1 and H2, and several routers on the path between H1 and H2.

Let's suppose that H1 is sending information to H2, and consider the role of the network layer in these hosts and in the intervening routers. The network layer in H1 takes segments from the transport layer in H1, encapsulates each segment into a datagram, and then sends the datagrams to its nearby router, R1. At the receiving host, H2, the network layer receives the datagrams from its nearby router R2, extracts the transport-layer segments, and delivers the segments up to the transport layer at H2.

The primary **data-plane** role of each router is to forward datagrams from its input links to its output links; the primary role of the **network control plane** is to coordinate these local, per-router forwarding actions so that datagrams are ultimately transferred end-to-end, along paths of routers between source and destination hosts.

Note that the routers in the figure are shown with a truncated protocol stack, that is, with no upper layers above the network layer, because routers do not run application and transport-

layer protocols. We can notice that **every Internet device** has the network layer (and its protocols) and, obviously, all the layers below it.

The network layer provides a point to point connection between transmitter and receiver.

Forwarding and Routing: The Data and Control Planes

The primary role of the network layer is deceptively simple, to move packets from a sending host to a receiving host. To do so, two important network-layer functions can be identified:

- **Forwarding** (a router-local action)
  When a packet arrives at a router's input link, the router must move the packet to the appropriate output link (to a specific other router).
  As we will see, forwarding is but one function (è solo una delle funzioni) implemented in the **data plane**
  In the more general case the packet might also be blocked or duplicated.
  Forwarding takes place at very short timescales (typically a few nanoseconds), and thus is typically implemented in hardware



- **Routing** (a network-wide process)
  The network layer must determine the route or path taken by packets as they flow from a sender to a receiver.
  The algorithms that calculate these paths are referred to as **routing algorithms**.
  Routing is implemented in the **control plane** of the network layer.
  Routing takes place on much longer timescales (typically seconds), and as we will see is often implemented in software.

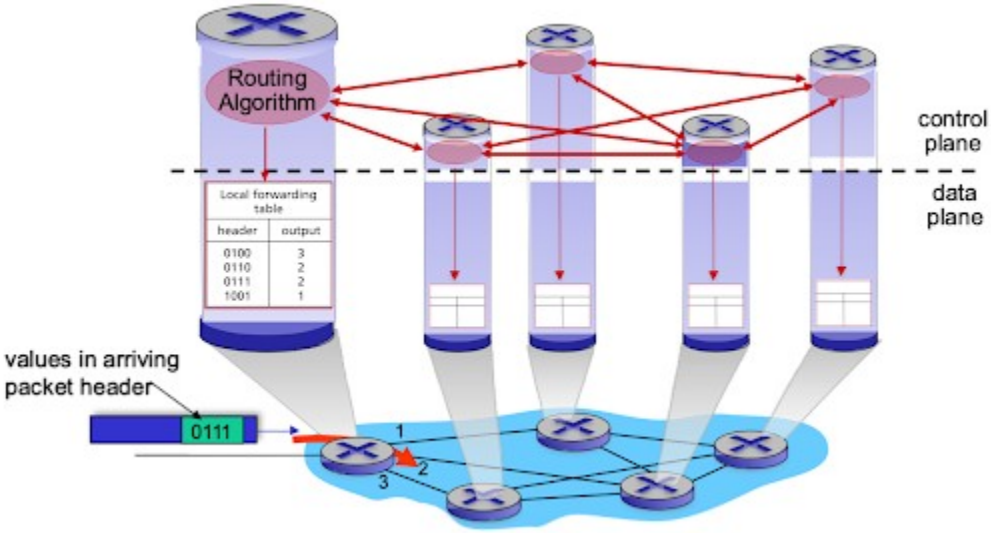
A key element in every network router is its **forwarding table**. A router forwards a packet by examining the value of one or more fields in the arriving packet's header, and then using these header values to index into its forwarding table.
We'll look inside a router and examine the forwarding function in much greater detail.
Forwarding is the key function performed by the data-plane functionality of the network layer.

We will see two control-plane approaches:

- **The traditional approach**
- **The Software-Defined Networking approach**

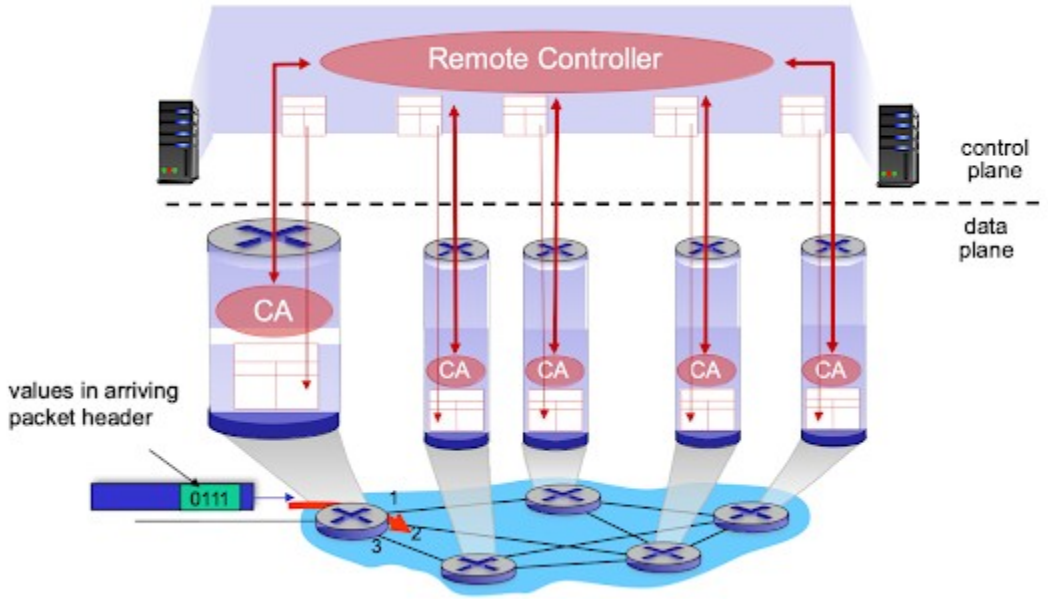
But now you are undoubtedly wondering how a router's forwarding tables are configured in the first place. This is a crucial issue, one that exposes the important interplay between forwarding (in data plane) and routing (in control plane).
The routing algorithm determines the contents of the routers' forwarding tables.

In the **traditional approach**, a routing algorithm runs in every router, so both forwarding and routing functions are contained within a router
As we'll see in later sections, the routing algorithm function in one router communicates with the routing algorithm function in other routers to compute the values for its forwarding table. How is this communication performed? By exchanging routing messages containing routing information according to a routing protocol!



The traditional approach to implementing routing functionality with each router having a routing component that communicates with the routing component of other routers, has been the traditional approach adopted by routing vendors in their products, at least until recently.

In the **SDN approach**, there is a physically separate, remote controller that computes and distributes the forwarding tables to be used by each and every router.
The components are identical as the traditional approach, but the control-plane routing functionality is separated from the physical router (the routing device performs forwarding only, while the remote controller computes and distributes forwarding tables).
How might the routers and the remote controller communicate? By exchanging messages containing forwarding tables and other pieces of routing information.



The SDN are highly improved because we concentrate the “computing” in the specialized servers

Network Service Model

Before delving into the network layer’s data plane, let’s wrap up our introduction by taking the broader view and consider the different types of service that might be offered by the network layer.
The **network service model** defines the characteristics of end-to-end delivery of packets between sending and receiving hosts.

Let’s now consider some possible services that the network layer could provide. These services could include:

- **Guaranteed delivery**
  This service guarantees that a packet sent by a source host will eventually arrive at the destination host
- **Guaranteed delivery with bounded delay**
  This service not only guarantees delivery of the packet, but delivery within a specified host-to-host delay bound (for example, within 100 msec)
- **In-order packet delivery**
  This service guarantees that packets arrive at the destination in the order that they were sent
- **Guaranteed minimal bandwidth**
  This network-layer service emulates the behavior of a transmission link of a specified bit rate between sending and receiving hosts
  As long as the sending host transmits bits at a rate below the specified bit rate, then all packets are eventually delivered to the destination host

- **Security**
  The network layer could encrypt all datagrams at the source and decrypt them at the destination, thereby providing confidentiality to all transport-layer segments
- Others

The Internet's network layer provides a single service, known as **best-effort service** (and therefore an easy implementation), so none of these services are implemented. It seems controversial to call it best-effort even if it doesn't provide these services, but the best-effort is for the speed.

Network Architecture	Service Model	Quality of Service (QoS) Guarantees ?			
		Bandwidth	Loss	Order	Timing
Internet	best effort	none	no	no	no
ATM	Constant Bit Rate	Constant rate	yes	yes	yes
ATM	Available Bit Rate	Guaranteed min	no	yes	no
Internet	Intserv Guaranteed (RFC 1633)	yes	yes	yes	yes
Internet	Diffserv (RFC 2475)	possible	possibly	possibly	no

Other network layers, like ATM (Asynchronous Transfer Mode) provide some or all these services.

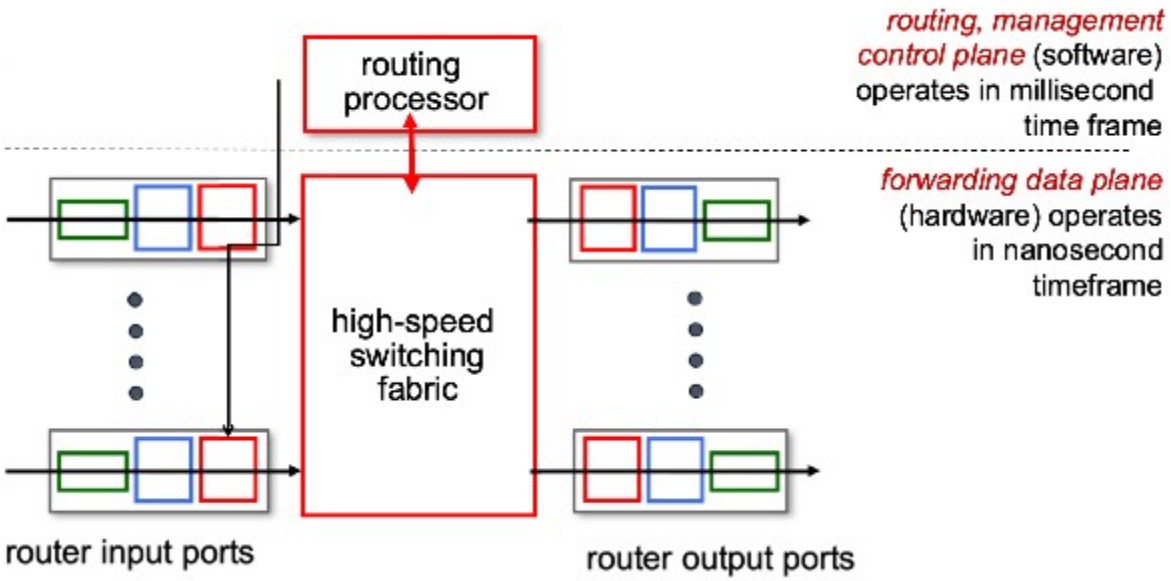
Let's make some reflection about Internet's network layer:

- The **simplicity of the mechanism** has allowed Internet to be widely deployed adopted
- The sufficient **provisioning of bandwidth** allows performance of real-time applications to be "good enough" for "most of the time"
- **Replicated, application-layer distributed services** (datacenters, content distribution networks) connecting close to clients' networks, allow services to be provided from multiple locations
- **Congestion control** of "elastic" services helps

What's Inside a Router?

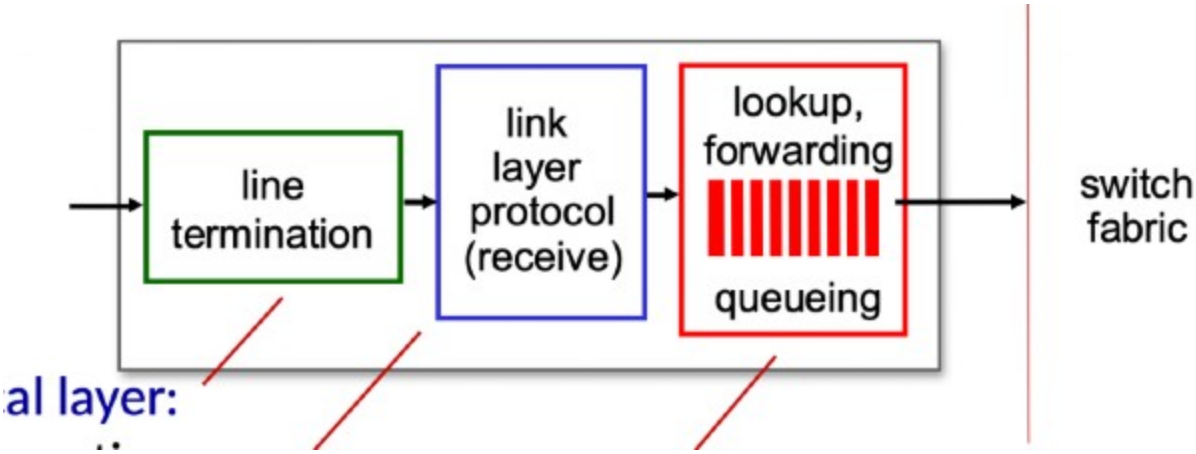
Now that we've overviewed the data and control planes within the network layer, the important distinction between forwarding and routing, and the services and functions of the network layer, let's turn our attention to its **forwarding function**, **the actual transfer** of packets **from** a router's **incoming links to** the appropriate **outgoing links** at that router.

A high-level view of a generic router architecture is this:



We can identify **four components**:

- **Input ports**
  An **input port** performs several key functions.



  It performs the physical layer function of terminating an incoming physical link at a router, a bit-level reception (**physical layer**, in green)
  An input port also performs link-layer functions needed to interoperate with the link layer at the other side of the incoming link, for example Ethernet (**link layer**, in blue)
  Perhaps most crucially, a lookup function is also performed at the input port (**decentralized switching**, in red). So using header field values, it lookup output port using forwarding table in input port memory ("match plus action")
- **Switching fabric**
  The switching fabric connects the router's input ports to its output ports. This switching fabric is completely contained within the router, a network inside of a network router!
  So this component transfers packet from input link to appropriate output link.
- **Output ports**
  An **output port** stores packets received from the switching fabric and transmits these

packets on the outgoing link by performing the necessary link-layer and physical-layer functions.

- **Routing processor**
  The routing processor performs **control-plane functions**.
  In **traditional routers**, it executes the routing protocols, maintains routing tables and attached link state information, and computes the forwarding table for the router.
  In **SDN routers**, the routing processor is responsible for communicating with the remote controller in order to (among other activities) receive forwarding table entries computed by the remote controller, and install these entries in the router's input ports.
  The routing processor also performs the network management functions that we'll study later

A router's **input ports**, **output ports**, and **switching fabric** are almost always implemented in **hardware** because the datagram-processing pipeline must operate too fast for software implementation.

The **control plane functions** (executing the routing protocols, responding to attached links that go up or down, communicating with the remote controller in the SDN case and performing management functions) are thus usually implemented in **software** and executed on the **routing processor** (typically a traditional CPU).

The **forwarding** can be the traditional **destination-based** (based only on destination IP address) or can be **generalized** (based on any set of header field values).

For concreteness and simplicity, we'll initially assume in this section that forwarding decisions are based only on the packet's destination address, rather than on a generalized set of packet header fields.

Input Port Processing and Destination-Based Forwarding

The **lookup** performed in the input port **is central** to the router's operation, it is here that the router uses the forwarding table to look up the output port to which an arriving packet will be forwarded via the switching fabric.
The forwarding table is either computed and updated by the routing processor (using a routing protocol to interact with the routing processors in other network routers) or is received from a remote SDN controller.

The forwarding table is copied from the routing processor to the line cards over a separate bus (e.g., a PCI bus). With such a shadow copy at each line card, forwarding decisions can be made locally, at each input port, without invoking the centralized routing processor on a per-packet basis and thus avoiding a centralized processing bottleneck.

A brute force implementation of the forwarding table (that means that would have one entry for every possible destination address) is out of question because with 32-bit IP addresses, we should have 4 billion possible addresses.

As an example of how this issue of scale can be handled, let's suppose that our router has four links, numbered 0 through 3, we could have the following forwarding table with just four entries:

Destination Address Range	Link interface
11001000 00010111 00010*** ********	0
11001000 00010111 00011000 ********	1
11001000 00010111 00011*** ********	2
otherwise	3

With this style of forwarding table, the router matches a **prefix** of the packet's destination address with the entries in the table; if there's a match, the router forwards the packet to a link associated with the match.

If a prefix doesn't match any of the first three entries, then the router forwards the packet to the default interface 3.

You may have noticed that it is possible for a destination address to match more than one entry (for example 1 and 2).
When there are **multiple matches**, the router uses the **longest prefix matching** rule; that is, it finds the longest matching entry in the table and forwards the packet to the link interface associated with the longest prefix match.
We'll see exactly why this longest prefix-matching rule is used when we study Internet addressing in more detail.

Given the existence of a forwarding table, lookup is conceptually simple, hardware logic just searches through the forwarding table looking for the longest prefix match. But the lookup must be performed in nanoseconds.
Thus, not only must lookup be performed in hardware, but techniques beyond a simple linear search through a large table are needed.
The lookup is implemented often using Ternary Content Addressable Memories (TCAMs), with a TCAM, a 32-bit IP address is presented to the memory, which returns the content of the forwarding table entry for that address in essentially **constant time**.

Once a packet's output port has been determined via the lookup, the packet can be sent into the switching fabric. In some designs, a packet may be temporarily blocked from entering the switching fabric if packets from other input ports are currently using the fabric.

Although "lookup" is arguably the most important action in input port processing, many other actions must be taken:

1. **physical- and link-layer processing** must occur, as discussed previously
2. the packet's version number, checksum and time-to-live field **must be checked** and the latter two fields rewritten

3. counters used for **network management** (such as the number of IP datagrams received) must be updated.

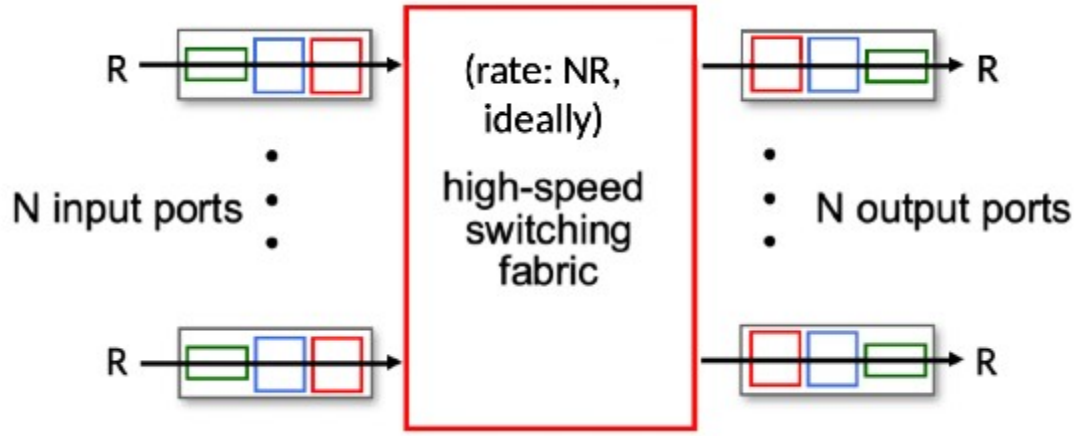
Let's close our discussion of input port processing by noting that the input port steps of looking up a destination IP address ("match") and then sending the packet into the switching fabric to the specified output port ("action") is a specific case of a more general "match plus action" abstraction that is performed in many networked devices, not just routers:

- In link-layer switches, link-layer destination addresses are looked up and several actions may be taken in addition to sending the frame into the switching fabric towards the output port
- In firewalls, devices that filter out selected incoming packets, an incoming packet whose header matches a given criteria (e.g., a combination of source/destination IP addresses and transport-layer port numbers) may be dropped (action)
- In a network address translator (NAT), an incoming packet whose transport-layer port number matches a given value will have its port number rewritten before forwarding (action).

Indeed, the "match plus action" abstraction is both powerful and prevalent in network devices today, and is central to the notion of generalized forwarding that we'll study

Switching

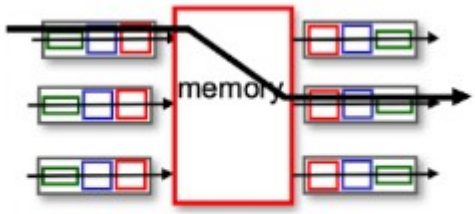
The **switching rate** is the rate at which packets can be transferred from inputs to outputs, it is often measured as multiple of input/output line rate. So for N inputs, the switching rate is N times line rate desirable



The switching fabric is at the very heart of a router, as it is through this fabric that the packets are actually switched (that is, forwarded) from an input port to an output port. Switching can be accomplished in a number of ways:

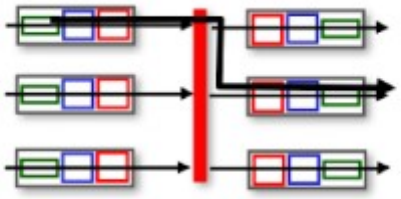
- **Switching via memory**
  The simplest, earliest routers were traditional computers, with switching between input and output ports being done under direct control of the CPU (routing processor). Input and output ports functioned as traditional I/O devices in a traditional operating system. An input port with an arriving packet first signaled the routing processor via an interrupt.

The packet was then copied from the input port into processor memory. The routing processor then extracted the destination address from the header, looked up the appropriate output port in the forwarding table, and copied the packet to the output port's buffers.
The **speed is limited** by the memory bandwidth (2 bus crossing per datagram).
Note also that two packets cannot be forwarded at the same time, even if they have different destination ports, since only one memory read/write can be done at a time over the shared system bus.



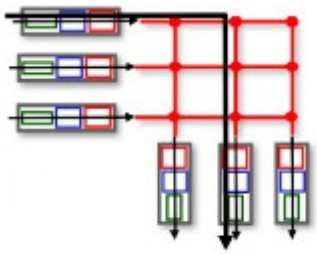
memory

- **Switching via bus**
  In this approach, an input port transfers a packet directly to the output port over a shared bus, without intervention by the routing processor.
  This is typically done by having the input port pre-pend a switch-internal label (header) to the packet indicating the local output port to which this packet is being transferred and transmitting the packet onto the bus. All output ports receive the packet, but only the port that matches the label will keep the packet. The label is then removed at the output port, as this label is only used within the switch to cross the bus.
  Because every packet must cross the single bus, the switching speed of the router is limited to the bus speed (the bus bandwidth)



bus interc

- **Switching via an interconnection network**
  One way to overcome the bandwidth limitation of a single, shared bus is to use a more sophisticated interconnection network, such as those that have been used in the past to interconnect processors in a multiprocessor computer architecture.
  A crossbar switch is an interconnection network consisting of $2N$ buses that connect N input ports to N output ports.
  Each vertical bus intersects each horizontal bus at a crosspoint, which can be opened or closed at any time by the switch fabric controller (whose logic is part of the switching fabric itself).



When a packet arrives from port A and needs to be forwarded to port Y, the switch controller closes the crosspoint at the intersection of busses A and Y, and port A then sends the packet onto its bus, which is picked up (only) by bus Y.
Note that a packet from port B can be forwarded to port X at the same time, since the A-to-Y and B-to-X packets use different input and output busses.

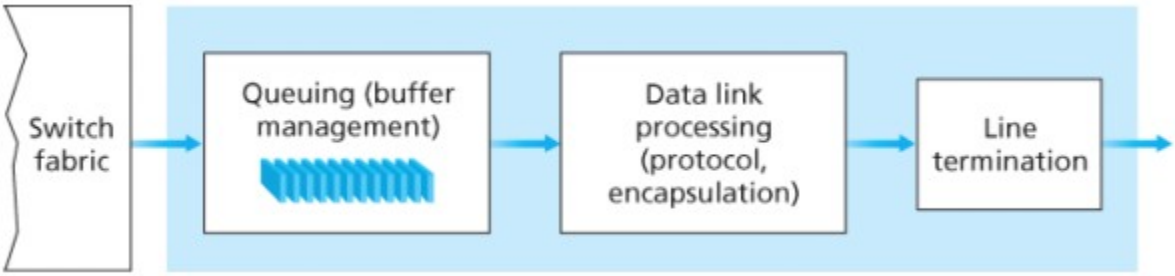
A crossbar switch is **non-blocking**, a packet being forwarded to an output port will not be blocked from reaching that output port as long as no other packet is currently being forwarded to that output port. However, if two packets from two different input ports are destined to that same output port, then one will have to wait at the input, since only one packet can be sent over any given bus at a time.

More sophisticated interconnection networks use multiple stages of switching elements to allow packets from different input ports to proceed towards the same output port at the same time through the multi-stage switching fabric.

A router's switching capacity can also be scaled by running multiple switching fabrics in parallel. In this approach, input ports and output ports are connected to N switching fabrics that operate in parallel. An input port breaks a packet into K smaller chunks, and sends ("sprays") the chunks through K of these N switching fabrics to the selected output port, which reassembles the K chunks back into the original packet.

Output Port Processing

Output port processing takes packets that have been stored in the output port's memory and transmits them over the output link. This includes selecting and de-queueing packets for transmission, and performing the needed link-layer and physical-layer transmission functions.



Where Does Queueing Occur?

If we consider input and output port functionality and the configurations of the switching fabric, it's clear that packet **queues** may form at both the input ports and the output ports.
The location and extent of queueing (either at the input port queues or the output port queues) will depend on the traffic load, the relative speed of the switching fabric, and the line speed.

Let's now consider these queues in a bit more detail, since as these queues grow large, the router's memory can eventually be exhausted and **packet loss** will occur when no memory is available to store arriving packets (now we can understand exactly where occurs the packet loss)

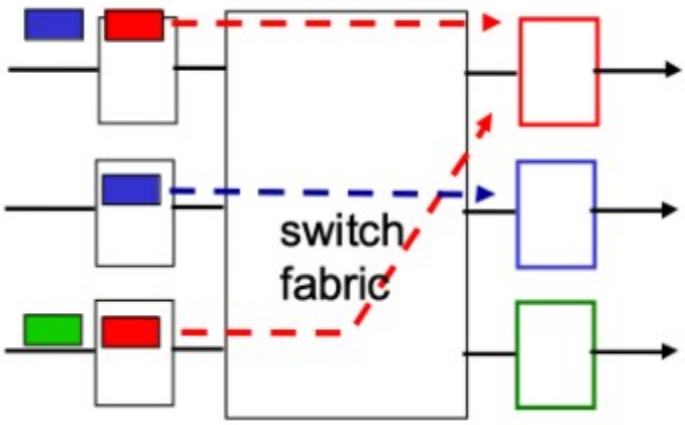
Suppose that the input and output line speeds (transmission rates) all have an identical transmission rate of R_{line} packets per second, and that there are N input ports and N output ports. To further simplify the discussion, let's assume that all packets have the same fixed length, and that packets arrive to input ports in a synchronous manner. That is, the time to send a packet on any link is equal to the time to receive a packet on any link, and during such an interval of time, either zero or one packets can arrive on an input link. Define the switching fabric transfer rate R_{switch} as the rate at which packets can be moved from input port to output port. If R_{switch} is N times faster than R_{line}, then only negligible queuing will occur at the input ports. This is because even in the worst case, where all N input lines are receiving packets, and all packets are to be forwarded to the same output port, each batch of N packets (one packet per input port) can be cleared through the switch fabric before the next batch arrives.

But what happens if **the switch fabric is not fast enough** (relative to the input line speeds) to transfer all arriving packets through the fabric without delay? In this case, packet queuing can also occur at the input ports, as packets must join input port queues to wait their turn to be transferred through the switching fabric to the output port.

To illustrate an important consequence of this queuing, consider a crossbar switching fabric and suppose that:

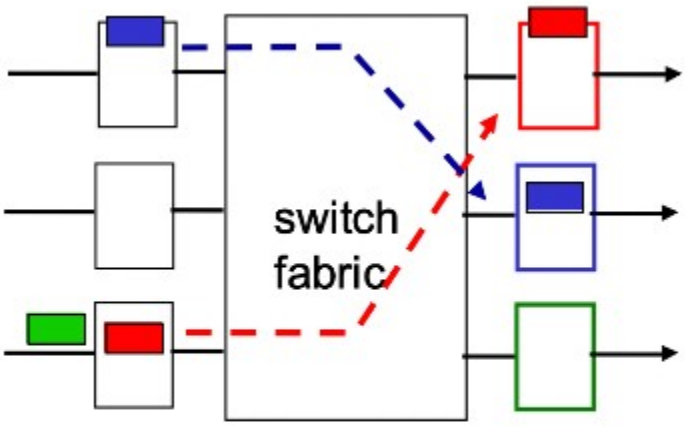
1. all link speeds are identical
2. that one packet can be transferred from any one input port to a given output port in the same amount of time it takes for a packet to be received on an input link
3. packets are moved from a given input queue to their desired output queue in an FCFS manner

Multiple packets can be transferred in parallel, as long as their output ports are different. However, if two packets at the front of two input queues are destined for the same output queue, then one of the packets will be blocked and must wait at the input queue, the switching fabric can transfer only one packet to a given output port at a time.



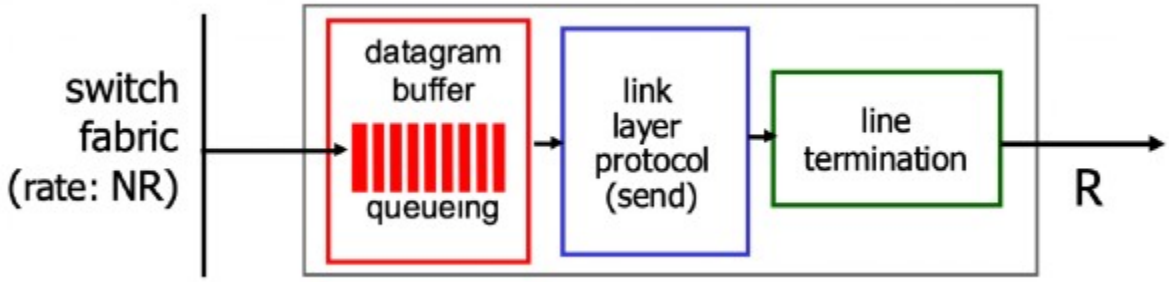
output port contention: only one red datagram can be transferred. lower red packet is *blocked*

Suppose that the switch fabric chooses to transfer the packet from the front of the upper-left queue. In this case, the red packet in the lower-left queue must wait. But not only must this red packet wait, so too must the green packet that is queued behind that packet in the lower-left queue, even though there is no contention for the green output port (the destination for the green packet). This phenomenon is known as head-of-the-line (**HOL**) **blocking** in an input-queued switch, a queued packet in an input queue must wait for transfer through the fabric (even though its output port is free) because it is blocked by another packet at the head of the line.



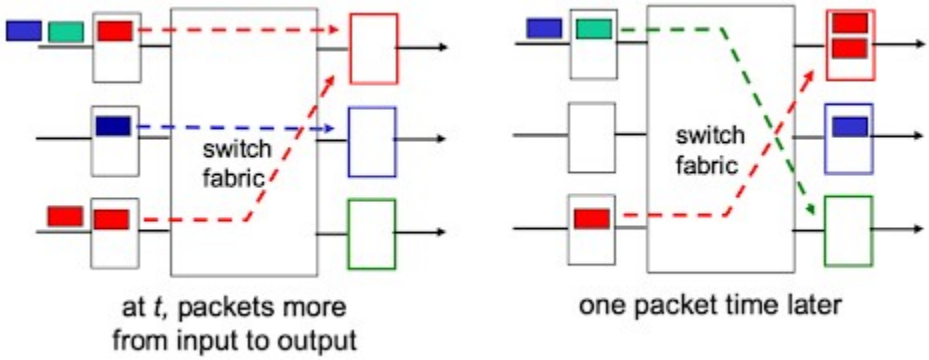
one packet time later: green packet experiences HOL blocking

Let's next consider whether queueing can occur at a switch's output ports. Suppose that R_{switch} is again N times faster than R_{line} and that packets arriving at each of the N input ports are destined to the same output port.
In this case, in the time it takes to send a single packet onto the outgoing link, N new packets will arrive at this output port (one from each of the N input ports). So, since the output port can transmit only a single packet in a unit of time (the packet transmission time), the N arriving packets will have to queue (wait) for transmission over the outgoing link. Then N more packets can possibly arrive in the time it takes to transmit just one of the N packets that had just previously been queued.
So, packet queues can form at the output port even when the switching fabric is faster enough (N time faster).



We can notice that this queue can grow very fast, and the memory will be filled.

When there is not enough memory to buffer an incoming packet, a decision must be made to either **drop the arriving packet** (a policy known as **drop-tail**) or **remove** one or more **already-queued packets** (**priority drop**) to make room for the newly arrived packet (in some cases could be removed a packet before the buffer is full in order to provide a congestion signal).
The buffer management has also to decide which packet **marks** in order to send signal congestion (ECN, RED)



At time t, a packet has arrived at two (of three) of the incoming input ports, each destined for the uppermost outgoing port. Assuming identical line speeds and a switch operating at three times the line speed, one time unit later (that is, in the time needed to receive or send a packet), the two original packets have been transferred to the outgoing port and are queued awaiting transmission.

In the next time unit, one of these two packets will have been transmitted over the outgoing link. In our example, two new packets have arrived at the incoming side of the switch; one of these packets is destined for this uppermost output port. A consequence of such queuing is that a **packet scheduler** at the output port must choose one packet, among those queued, for transmission (a topic we'll cover in the following section).

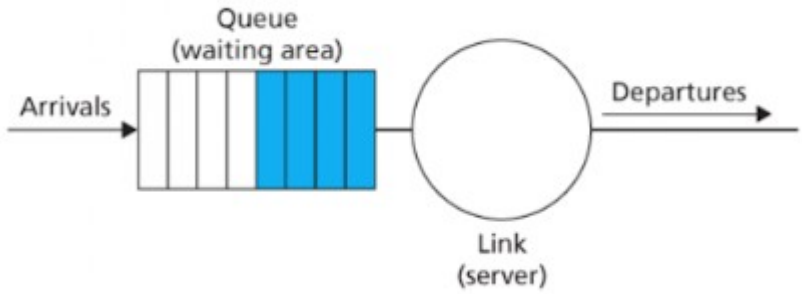
Given that router buffers are needed to absorb the fluctuations in traffic load, a natural question to ask is **how much buffering is required**.
Too much buffer can increase delay, so the goal is to find the right size.

- For many years, the rule of thumb [RFC 3439] for buffer sizing was that the amount of buffering (B) should be equal to an average round-trip time times the link capacity (C), so with a link capacity of $10\ \ Gbps$ and an RTT of $250\ \ msec$ the buffer will be of $2.5\ \ Gbit$.
- More recent theoretical and experimental efforts, however, suggest that when there are a large number of TCP flows (N) passing through a link, the amount of buffering needed is $\frac{RTT \cdot C}{\sqrt{\square}}$ (with a significant decrease of the buffer size).

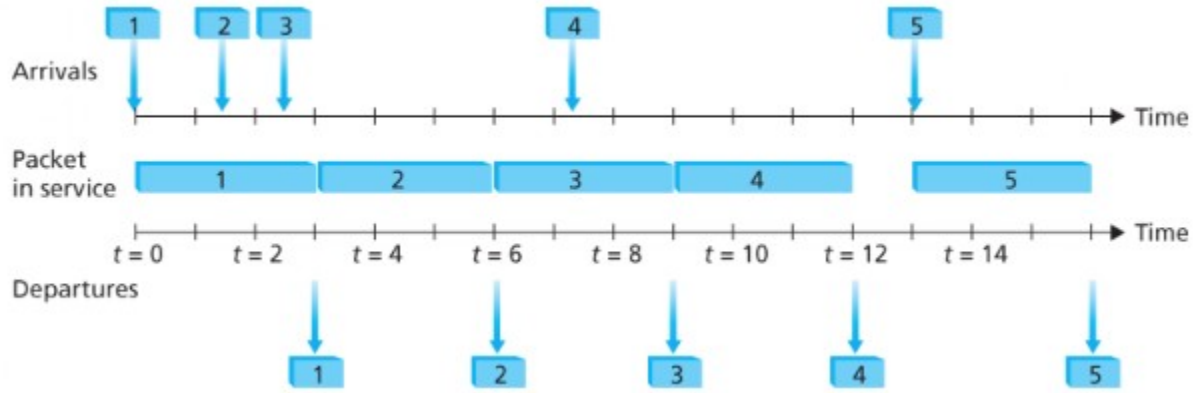
Packet scheduling

Let's now return to the question of **determining the order** in which queued packets are transmitted over an outgoing link.

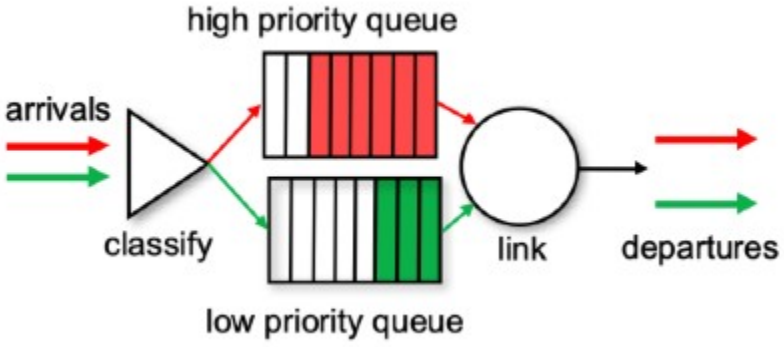
The first (and easiest) link-scheduling discipline is the First-Come-First-Served (**FCFS**), that is equal to the FIFO.



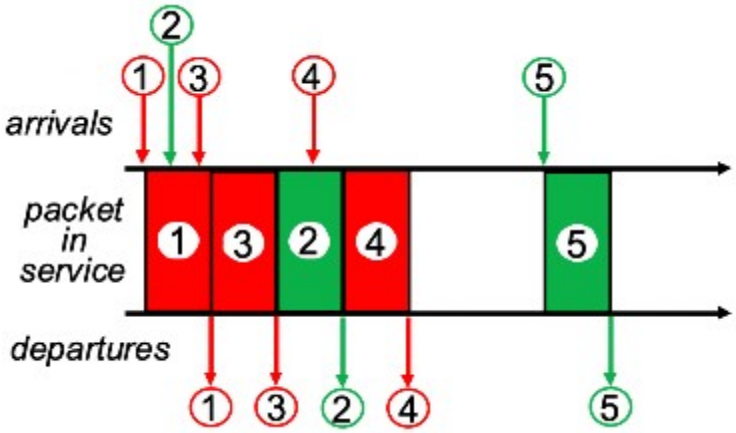
This scheduling discipline selects packets for link transmission in the same order in which they arrived at the output link queue.
Here's an example of a FCFS queue in operation with the timeline, with a number indicating the order in which packets arrive, and considering that each packet takes three units of time to be transmitted.



The second link-scheduling discipline is the **priority queueing**, under this discipline packets arriving at the output link are **classified** into priority classes upon arrival at the queue.

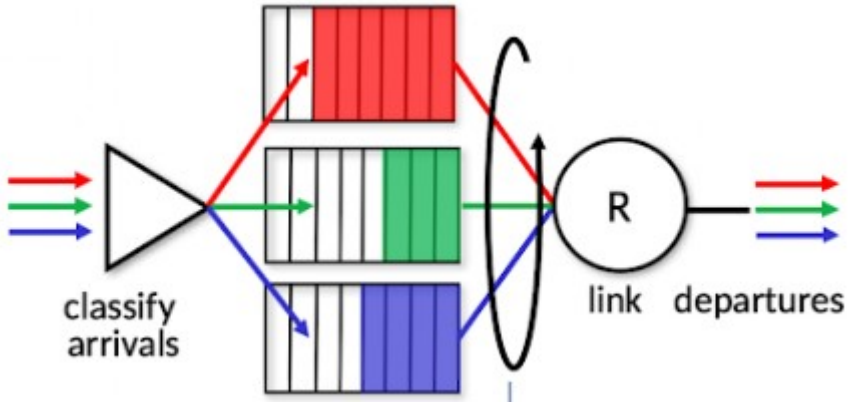


For example this is useful when we want to give priority to packets carrying network information over user traffic (decision made on header fields).
Each priority class typically has his own queue. When choosing a packet to transmit, the priority queuing discipline will transmit a packet from the highest priority class that has a nonempty queue (that is, has packets waiting for transmission). The choice among packets in the same priority class is typically done in a FCFS manner.
Here's an example of a priority queue in operation:

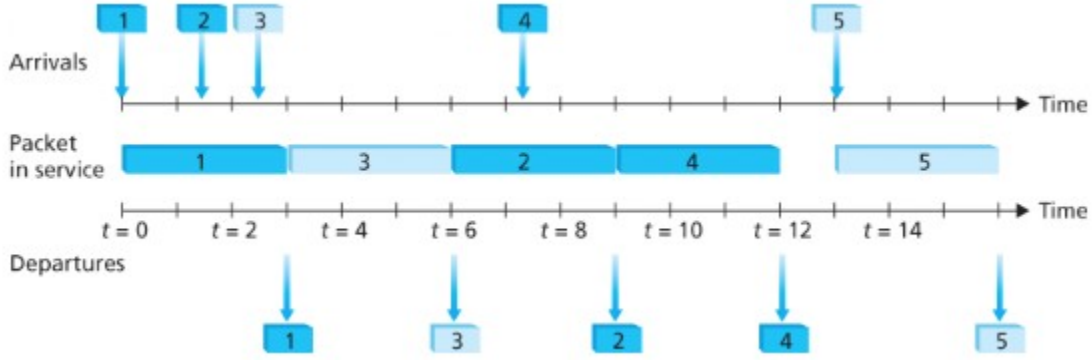


Under a **non-preemptive priority queuing discipline**, the transmission of a packet is not interrupted once it has begun. In this case, packet 4 queues for transmission and begins being transmitted after the transmission of packet 2 is completed

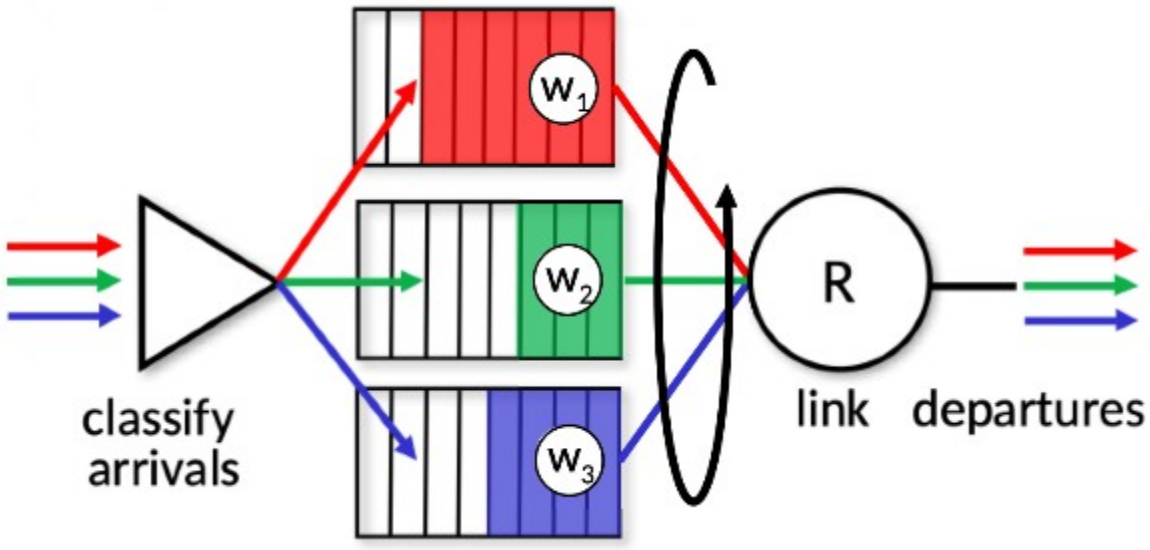
Another link-scheduling discipline is the **Round Robin** (RR), where the packets are sorted into classes as with priority queuing.
However, rather than there being a strict service priority among classes, a round robin scheduler **alternates service among the classes**.
In the simplest form of round robin scheduling, a class 1 packet is transmitted, followed by a class 2 packet, followed by a class 3 packet, followed by a class 1 packet, and so on.



A so-called **work-conserving queuing discipline** will never allow the link to remain idle whenever there are packets (of any class) queued for transmission. A work-conserving round robin discipline that looks for a packet of a given class but finds none will immediately check the next class in the round robin sequence.
Here's an example with two classes:



A generalized form of round robin queuing that has been widely implemented in routers is the so-called **weighted fair queuing (WFQ) discipline**.



Here, arriving packets are classified and queued in the appropriate per-class waiting area. As in round robin scheduling, a WFQ scheduler will serve classes in a circular manner and it is work-conserving.

WFQ differs from round robin in that each class may receive a **differential amount of service** in any interval of time.
Specifically, each class, i, is assigned a weight, w_i. Under WFQ, during any interval of time during which there are class i packets to send, class i will then be guaranteed to receive a fraction of service equal to $\frac{w_i}{\sum_j w_j}$, where the sum in the denominator is taken over all classes that also have packets queued for transmission.
So for each class we have a minimum bandwidth guarantee (that is for a class i, the minimum is always a fraction $\frac{w_i}{\sum_j w_j}$ of the bandwidth)

Network Neutrality

The network neutrality is a set of concepts:

- **Technical** (hardware and software)
  For example how an ISP should share/allocation its resources
  The mechanism are packet scheduling and buffer management
- **Social and economic**
  The protection of the free speech
  With the goal of encouraging innovation and competition
- **Legal**
  Respect the legal rules and policies

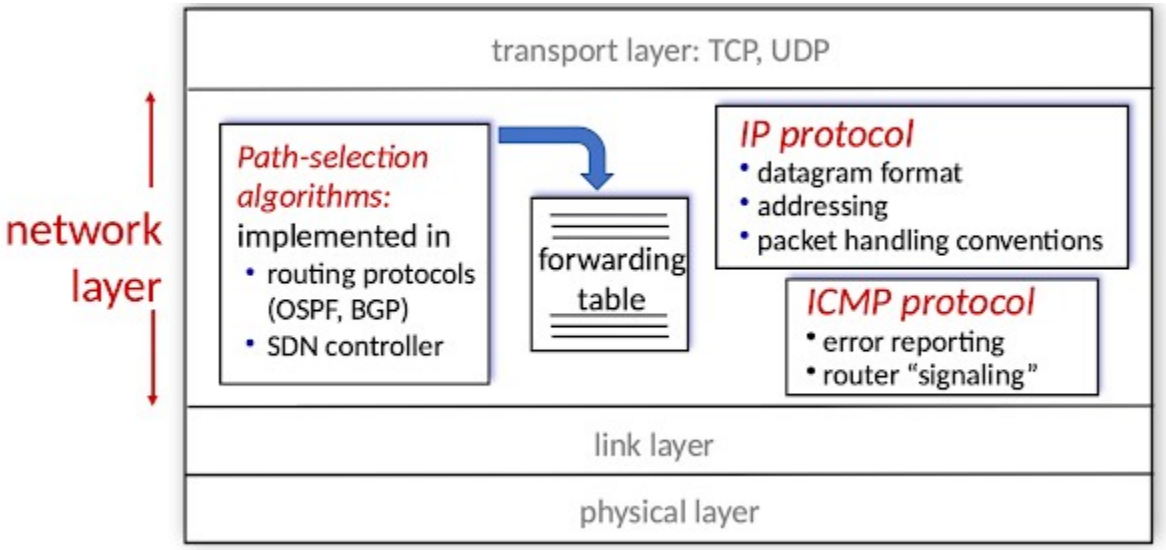
Each country has different “takes” on network neutrality

The 2015 US FCC Order on Protecting and Promoting an Open Internet has three “clear, bright line” rules:

- **No blocking**
  Shall not block lawful content, applications, services, or non-harmful devices, subject to reasonable network management
- **No throttling**
  Shall not impair or degrade lawful Internet traffic on the basis of Internet content, application, or service, or use of a non-harmful device, subject to reasonable network management
- **No paid prioritization**
  Shall not engage in paid prioritization

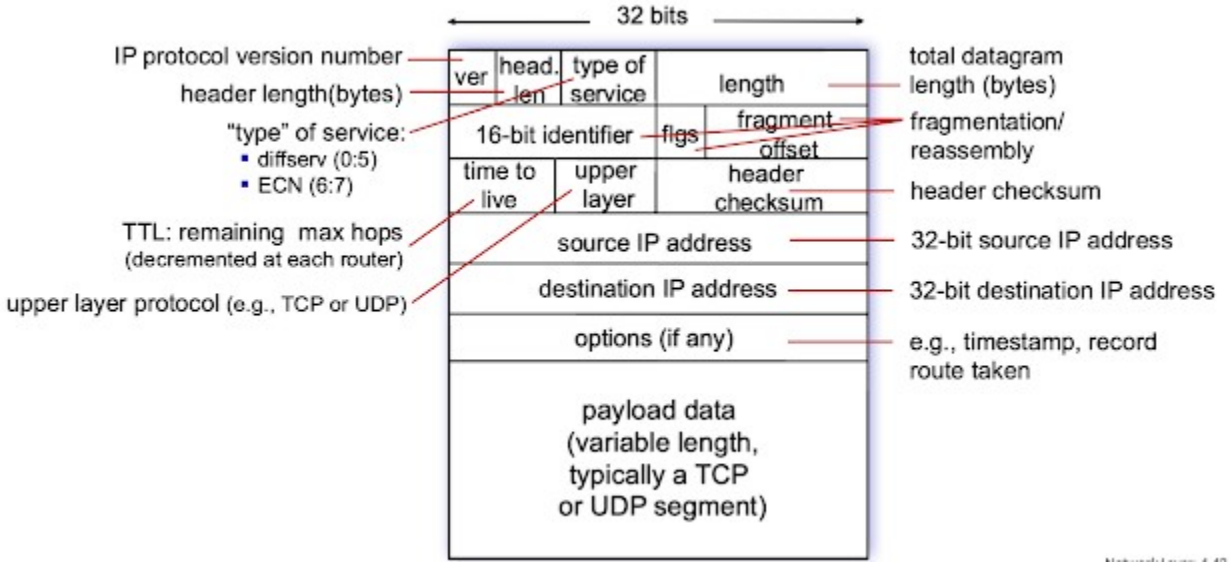
The Internet Protocol (IP): IPv4, Addressing, IPv6, and More

Our study of the network layer thus far has often been without reference to any specific computer network architecture or protocol. In this section we’ll focus on key aspects of the network layer on today’s Internet and the celebrated **Internet Protocol** (IP).



IPv4 Datagram Format

Recall that the Internet’s network-layer packet is referred to as a datagram. We begin our study of IP with an overview of the syntax and semantics of the IPv4 datagram.



The key fields in the IPv4 datagram are the following:

- **Version number**
  These 4 bits specify the IP protocol version of the datagram. By looking at the version number, the router can determine how to interpret the remainder of the IP datagram. Different versions of IP use different datagram formats.
- **Header length**
  Because an IPv4 datagram can contain a variable number of options, these 4 bits are needed to determine where in the IP datagram the payload (the data like the transport layer segment) actually begins.
  Most IP datagrams do not contain options, so the typical IP datagram has a 20-byte header.
- **Type of service**
  The type of service (TOS) bits were included in the IPv4 header to allow different types of IP datagrams to be distinguished from each other (for example to distinguish between real-time datagrams from non real-time traffic).
  We also learned that two of the TOS bits are used for Explicit Congestion Notification.
- **Datagram length**
  This is the total length of the IP datagram (header plus data), measured in bytes.
  Since this field is 16 bits long, the theoretical maximum size of the IP datagram is 65,535 bytes.
  However, datagrams are rarely larger than 1,500 bytes, which allows an IP datagram to fit in the payload field of a maximally sized Ethernet frame.
- **Identifier, flags, fragmentation offset**
  These three fields have to do with so-called IP fragmentation, a topic we will consider shortly.
- **Time-to-live**
  The time-to-live (TTL) field is included to ensure that datagrams do not circulate forever (due to, for example, a long-lived routing loop) in the network. This field is decremented by one each time the datagram is processed by a router. If the TTL field reaches 0, a router must drop that datagram.

- **Protocol**
  This field is typically used only when an IP datagram reaches its final destination. The value of this field indicates the specific transport-layer protocol to which the data portion of this IP datagram should be passed (TCP, UDP or others).
  Note that the protocol number in the IP datagram has a role that is analogous to the role of the port number field in the transport-layer segment. The protocol number is the glue that binds the network and transport layers together, whereas the port number is the glue that binds the transport and application layers together.
- **Header checksum**
  The header checksum aids a router in detecting bit errors in a received IP datagram. The header checksum is computed by treating each 2 bytes in the header as a number and summing these numbers using 1s complement arithmetic.
  A router computes the header checksum for each received IP datagram and detects an error condition if the checksum carried in the datagram header does not equal the computed checksum.
  Routers typically discard datagrams for which an error has been detected. Note that the checksum must be recomputed and stored again at each router, since the TTL field, and possibly the options field as well, will change.

  A question often asked at this point is, why does TCP/IP perform error checking at both the transport and network layers?
  1. Note that only the IP header is checksummed at the IP layer, while the TCP/UDP checksum is computed over the entire TCP/UDP segment.
  2. TCP/UDP and IP do not necessarily both have to belong to the same protocol stack. TCP can, in principle, run over a different network-layer protocol and IP can carry data that will not be passed to TCP/UDP.
- **Source and destination IP addresses**
  When a source creates a datagram, it inserts its IP address into the source IP address field and inserts the address of the ultimate destination into the destination IP address field. Often the source host determines the destination address via a DNS lookup
- **Options**
  The options fields allow an IP header to be extended. Header options were meant to be used rarely, hence the decision to save overhead by not including the information in options fields in every datagram header.
  Since datagram headers can be of variable length (due to this options), one cannot determine a priori where the data field will start and the amount of time needed to process an IP datagram at a router can vary greatly
- **Data (payload)**
  Finally, we come to the last and most important field.
  In most circumstances, the data field of the IP datagram contains the transport-layer segment (TCP or UDP) to be delivered to the destination. However, the data field can carry other types of data, such as ICMP messages

Note that an IP datagram has a total of 20 bytes of header (assuming no options). If the datagram carries a TCP segment, then each (non-fragmented) datagram carries a total of 40

bytes of header (20 bytes of IP header plus 20 bytes of TCP header) along with the application-layer message.

IPv4 Datagram Fragmentation

We'll see in Chapter 6 that not all link-layer protocols can carry network-layer packets of the same size. Some protocols can carry big datagrams, whereas other protocols can carry only little datagrams. (for example Ethernet 1,500 bytes of data, some wide-area links 576 bytes of data).
The maximum amount of data that a link-layer frame can carry is called the **Maximum Transmission Unit** (MTU). Because each IP datagram is encapsulated within the link-layer frame for transport from one router to the next router, the MTU of the link-layer protocol places a hard limit on the length of an IP datagram.

Having a hard limit on the size of an IP datagram is not much of a problem. What is a problem is that each of the links along the route between sender and destination can use different link-layer protocols, and each of these protocols can have different MTUs.

So when a router determine the outgoing link for a IP datagram, and discover that that outgoing link has an MTU that is smaller than the length of the IP datagram, the solution is to fragment the payload in the IP datagram into two or more smaller IP datagrams, encapsulate each of these smaller IP datagrams in a separate link-layer frame; and send these frames over the outgoing link.
Each of these smaller datagrams is referred to as a **fragment**.

Fragments need to be reassembled before they reach the transport layer at the destination. Indeed, both TCP and UDP are expecting to receive complete, unfragmented segments from the network layer.
Sticking to the principle of keeping the network core simple, the designers of IPv4 decided to put the job of datagram **reassembly in the end systems** rather than in network routers.

When a destination host receives a series of datagrams from the same source, it needs to determine whether any of these datagrams are fragments of some original, larger datagram. If some datagrams are fragments, it must further determine when it has received the last fragment and how the fragments it has received should be pieced back together to form the original datagram. To allow the destination host to perform these reassembly tasks, the designers of IP (version 4) put **identification**, **flag**, and **fragmentation offset** fields in the IP datagram header.

When a datagram is created, the sending host stamps the datagram with an identification number as well as source and destination addresses. Typically, the sending host increments the identification number for each datagram it sends. When a router needs to fragment a datagram, each resulting datagram (that is, fragment) is stamped with the source address, destination address, and identification number of the original datagram. When the destination receives a series of datagrams from the same sending host, it can examine the identification numbers of

the datagrams to determine which of the datagrams are actually fragments of the same larger datagram.

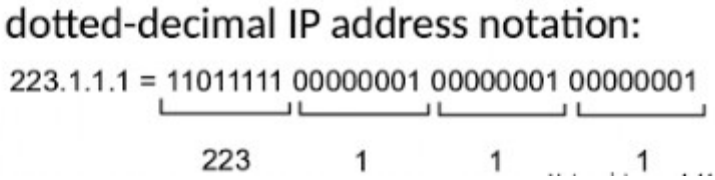
Because IP is an unreliable service, one or more of the fragments may never arrive at the destination. For this reason, in order for the destination host to be absolutely sure it has received the last fragment of the original datagram, the last fragment has a flag bit set to 0, whereas all the other fragments have this flag bit set to 1.

Also, in order for the destination host to determine whether a fragment is missing (and also to be able to reassemble the fragments in their proper order), the offset field is used to specify where the fragment fits within the original IP datagram.

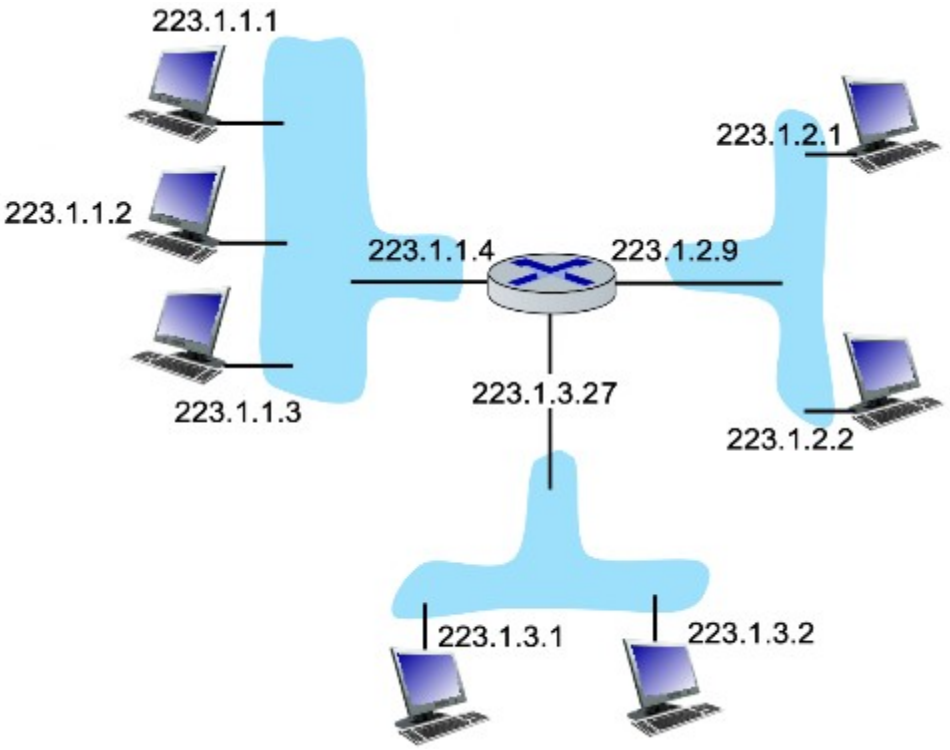
IPv4 Addressing

We now turn our attention to IPv4 addressing.
Before discussing IP addressing, however, we'll need to say a few words about how hosts and routers are connected into the Internet.
A host typically has only a single link into the network; when IP in the host wants to send a datagram, it does so over this link. The boundary between the host and the physical link is called an **interface**. (A host has typically one or two interfaces, typically wired Ethernet and Wireless 802.11)
Now consider a router and its interfaces. Because a router's job is to receive a datagram on one link and forward the datagram on some other link, a router necessarily has two or more links to which it is connected.
A router thus has multiple interfaces, one for each of its links. Because every host and router is capable of sending and receiving IP datagrams, IP requires each host and router interface to have its own IP address.
Thus, **an IP address is technically associated with an interface, rather than with the host or router containing that interface**.

Each IP address is 32 bits long (equivalently, 4 bytes), and there are thus a total of 2^{32} (or approximately 4 billion) possible IP addresses. These addresses are typically written in so-called **dotted-decimal notation**, in which each byte of the address is written in its decimal form and is separated by a period (dot) from other bytes in the address.

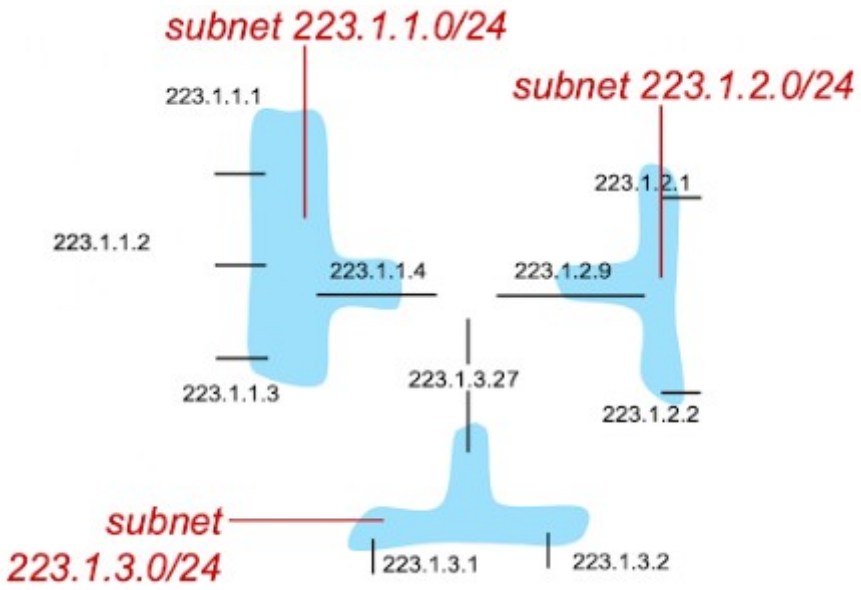


Each interface on every host and router in the global Internet must have an IP address that is globally **unique** (except for interfaces behind NATs, as discussed later).
A portion of an interface's IP address will be determined by the **subnet** to which it is connected.



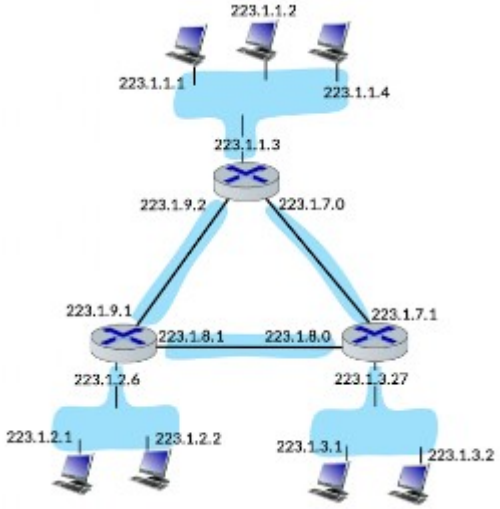
In this figure, one router (with three interfaces) is used to interconnect seven hosts.
Take a close look at the IP addresses assigned to the host and router interfaces, as there are several things to notice.
The three hosts in the upper-left portion, and the router interface to which they are connected, all have an IP address of the form 223.1.1.xxx. That is, they all have the same leftmost 24 bits in their IP address.
These four interfaces are also interconnected to each other by a network that contains no routers (this network could be interconnected by an Ethernet LAN, in which case the interfaces would be interconnected by an Ethernet switch, or by a wireless access point).
We'll represent this routerless network connecting these hosts as a cloud for now, and dive into the internals of such networks in next sections.

In IP terms, this network interconnecting three host interfaces and one router interface forms a **subnet**.
IP addressing assigns an address to this subnet: 223.1.1.0/24, where the /24 ("slash-24") notation, sometimes known as a subnet mask, indicates that the leftmost 24 bits of the 32-bit quantity define the **subnet address**.
So every IP Address has two parts: the subnet address and the host part.
We can identify two other subnets in the figure: 223.1.2.0/24 and 223.1.3.0/24



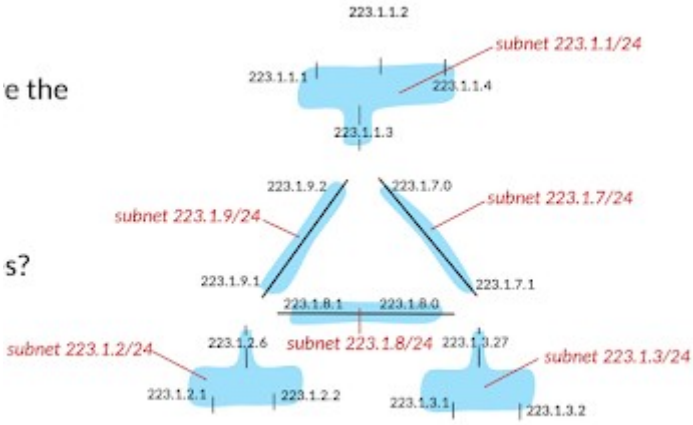
A **subnet** is a set of device interfaces that can physically reach each other **without passing through an intervening router**.

But the subnets aren't only between host and routers, but also between routers and other routers, like showed in this example:



e the

s?



For a general interconnected system of routers and hosts, we can use the following recipe to define the subnets in the system:
To determine the subnets, detach each interface from its host or router, creating islands of isolated networks, with interfaces terminating the end points of the isolated networks. Each of these isolated networks is called a subnet.

In principle, the different subnets in an organization could have quite different subnet addresses. In practice, however, their subnet addresses often have much in common. To understand why, let's next turn our attention to how addressing is handled in the global Internet.

The Internet's address assignment strategy is known as **Classless InterDomain Routing** (CIDR).
CIDR generalizes the notion of subnet addressing. As with subnet addressing, the 32-bit IP address is divided into two parts and again has the dotted-decimal form a.b.c.d/x, where x indicates the number of bits in the first part of the address.
The x most significant bits of an address of the form a.b.c.d/x constitute the network portion of the IP address, and are often referred to as the **prefix** (or network prefix) of the address. An organization is typically assigned a block of contiguous addresses, that is, a range of addresses with a common prefix.

When we cover the Internet's BGP routing protocol we'll see that only these x leading prefix bits are considered by routers outside the organization's network.
This considerably reduces the size of the forwarding table in these routers, since a single entry of the form a.b.c.d/x will be sufficient to forward packets to any destination within the organization.



The remaining 32-x bits of an address can be thought of as distinguishing among the devices **within** the organization, all of which have the same network prefix. These are the bits that will be considered when forwarding packets at routers within the organization (these bits may have an additional subnetting structure).

Before CIDR was adopted, there was an addressing scheme known as **classful addressing**. The name came from the fact that there was three classes of subnets:

- A (8 bit network portion)
- B (16 bit network portion)
- C (24 bit network portion)

The requirement that the subnet portion of an IP address be exactly 1, 2, or 3 bytes long turned out to be problematic for supporting the rapidly growing number of organizations with small and medium-sized subnets (a class C support 254 host, a class B support 65,634 host, without a middle ground).

We would be remiss if we did not mention yet another type of IP address, the **IP broadcast address** 255.255.255.255. When a host sends a datagram with destination address 255.255.255.255, the message is delivered to all hosts on the same subnet. Routers optionally forward the message into neighboring subnets as well (although they usually don't).

Having now studied IP addressing in detail, we need to know how hosts and subnets get their addresses in the first place. Let's begin by looking at how an organization gets a block of

addresses for its devices, and then look at how a device (such as a host) is assigned an address from within the organization's block of addresses.

In order to obtain a **block of IP addresses** for use within an organization's subnet, a network administrator might first contact its ISP, which would provide addresses from a larger block of addresses that had already been allocated to the ISP.
To understand better, let's make an example, considering an ISP allocated at address block 200.23.16.0/20, if there are 8 organizations under that ISP, it could divide its address block into 8 equal-size and continuos address blocks:

ISP's block	11001000 00010111 00010000 00000000	200.23.16.0/20

ISP can then allocate out its address space in 8 blocks:

Organization 0	11001000 00010111 00010000 00000000	200.23.16.0/23
Organization 1	11001000 00010111 00010010 00000000	200.23.18.0/23
Organization 2	11001000 00010111 00010100 00000000	200.23.20.0/23
...	....	....
Organization 7	11001000 00010111 00011110 00000000	200.23.30.0/23

Obtaining a set of addresses from an ISP is not the only way for an organization to get a block of addresses.

Clearly, there must also be a way for the ISP itself to get a block of addresses.
There is a global authority that has ultimate responsibility for managing the IP address space and allocating address blocks to ISPs and other organizations, the **Internet Corporation for Assigned Names and Numbers** (ICANN):

- Allocates IP addresses
- Manage DNS root servers
- Assigning domain names and resolving domain name disputes
- Allocates addresses to regional Internet registries (that are 5)
- Handle the allocation/management of addresses within their regions

Once an organization has obtained a block of addresses, it can assign **individual IP addresses** to the host and router interfaces in its organization.
A system administrator will typically manually configure the IP addresses into the router (often remotely, with a network management tool).
An host address can be configured:

- **Manually**
- **Dynamically** (with DHCP)

The **Dynamic Host Configuration Protocol** (DHCP) allows a host to obtain (be allocated) an IP address automatically:

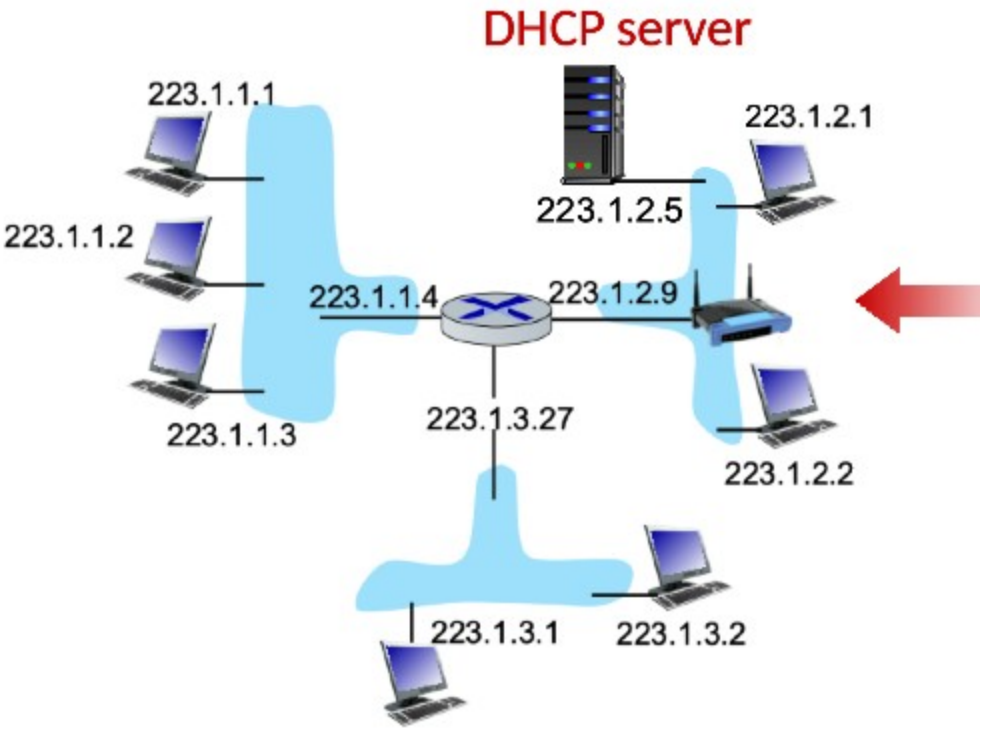
- Can be configured to:
    - Give **the same IP address** to a given host each time it connects to the network
    - Assign a **temporary ID address** that will be different each time the host connects to the network

- Allows a host to learn additional information:
    - its subnet mask
    - the address of its first-hop router (default gateway)
    - the address of its local DNS server

Because of DHCP's ability to automate the network-related aspects of connecting a host into a network, it is often referred to as a **plug-and-play** or **zeroconf** (zero-configuration) **protocol**.

DHCP is very useful because it simplify the work of the network administrator (that don't have to configure all this settings manually) and where host join and leave the network frequently it is very important (otherwise the network administrator should reconfigure continuously and the people using these devices don't have the expertise to configure their laptops manually).

DHCP is a **client-server protocol**. A client is typically a newly arriving host wanting to obtain network configuration information, including an IP address for itself.
A host can renew its lease on the address that it use. Obviously DHCP allows the reuse of addresses (only hold address while connected/on)
In the simplest case each subnet has a DHCP server, but it could be not in this way.
If no server is present in a subnet, then a **DHCP relay agent** (typically a router) that knows the address of a DHCP server for that network is needed.
Typically the DHCP server will be co-located in router, serving all subnets to which router is attached.

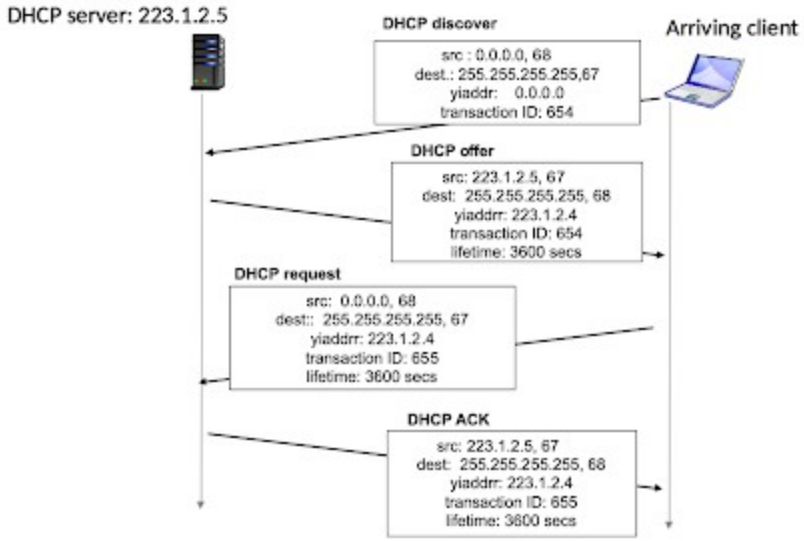


For a newly arriving host (that want, in our example, to the router pointed by the red arrow), the DHCP protocol is a four-step process:

- **DHCP server discovery** (is there a DHCP server out there?)
  The first task of a newly arriving host is to find a DHCP server with which to interact. This is done using a **DHCP discover message**, which a client sends within a UDP packet to port 67.
  The host doesn't know anything about the network, so it doesn't know the IP of the DHCP server, so the DHCP discover message will have the broadcast destination IP address (255.255.255.255) and a source address 0.0.0.0.
- **DHCP server offer(s)** (I'm a DHCP server, here's an IP address you can use)
  A DHCP server receiving a DHCP discover message responds to the client with a **DHCP offer message** that is broadcast to all nodes on the subnet, again using the IP broadcast address of 255.255.255.255.
  Since several DHCP servers can be present on the subnet, the client may find itself in the enviable position of being able to choose from among several offers.

  Each server offer message contains the transaction ID of the received discover message, the proposed IP address for the client, the network mask, and an IP address lease time (the amount of time for which the IP address will be valid).
- **DHCP request** (OK, I would like to use this IP address)
  The newly arriving client will choose from among one or more server offers and respond to its selected offer with a **DHCP request message**, echoing back the configuration parameters.
- **DHCP ACK** (OK, you've got that IP address)
  The server responds to the DHCP request message with a **DHCP ACK message**, confirming the requested parameters.

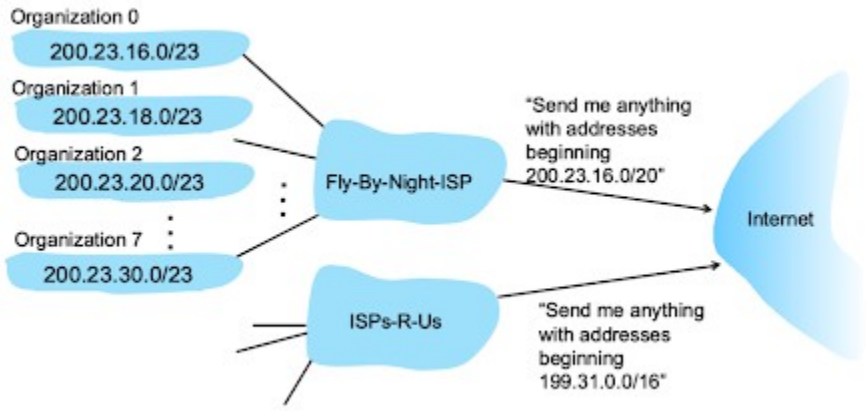
Let's see an example of the four-step process to get an IP address with DHCP protocol, where *yiaddr* is "your internet address".



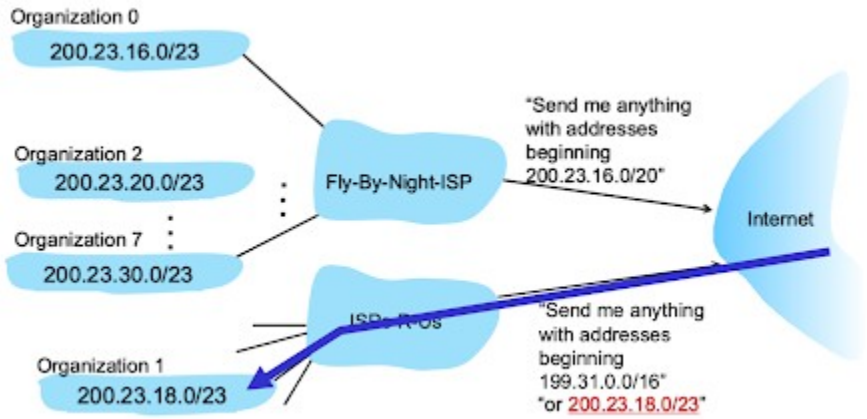
Once the client receives the DHCP ACK, the interaction is complete and the client can use the DHCP-allocated IP address for the lease duration. Since a client may want to use its address

beyond the lease's expiration, DHCP also provides a mechanism that allows a client to **renew** its lease on an IP address.
Since a new IP address is obtained from DHCP each time a node connects to a new subnet, a TCP connection to a remote application cannot be maintained as a mobile node moves between subnets. We will examine mobile IP, an extension to the IP infrastructure that allows a mobile node to use a single permanent address as it moves between subnets.

The **hierarchical addressing** allows efficient advertisement of routing information, because we can use the network address to understand to which network forward a packet.
Here's an example:



If Organization 1 move from Fly-By-Night-ISP to ISPs-r-Us, then ISPs-r-Us advertises a more specific route to Organization 1 (with the mask this isn't a problem)



One very important questions is if 32-bit long IP addresses are enough for the Internet. ICANN allocated last chunk of IPv4 addresses to RRs in 2011 (so the answer is no).
But NAT (next paragraph) helps the IPv4 address space exhaustion.

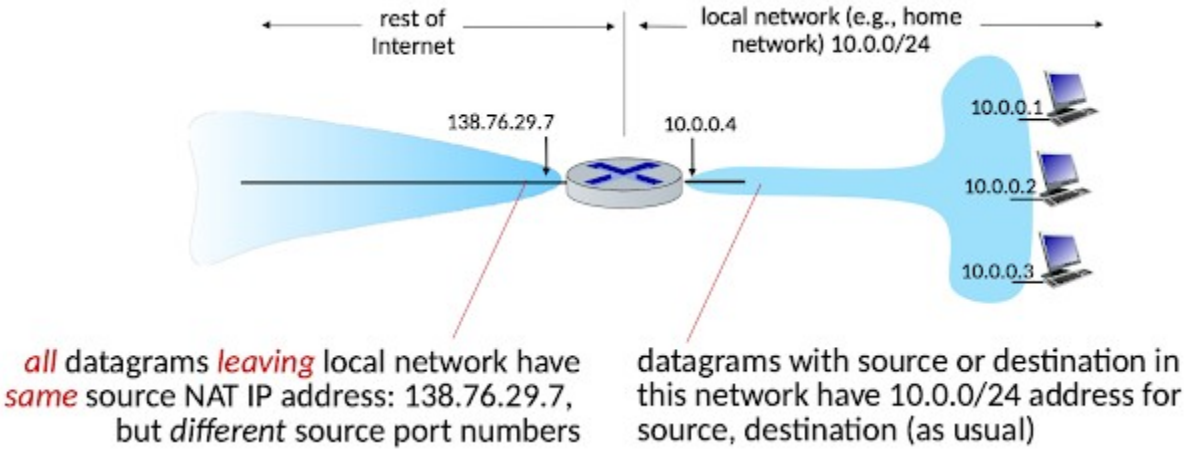
Network Address Translation (NAT)

Given our discussion about Internet addresses and the IPv4 datagram format, we're now well aware that every IP-capable device needs an IP address.
Let's make some considerations:

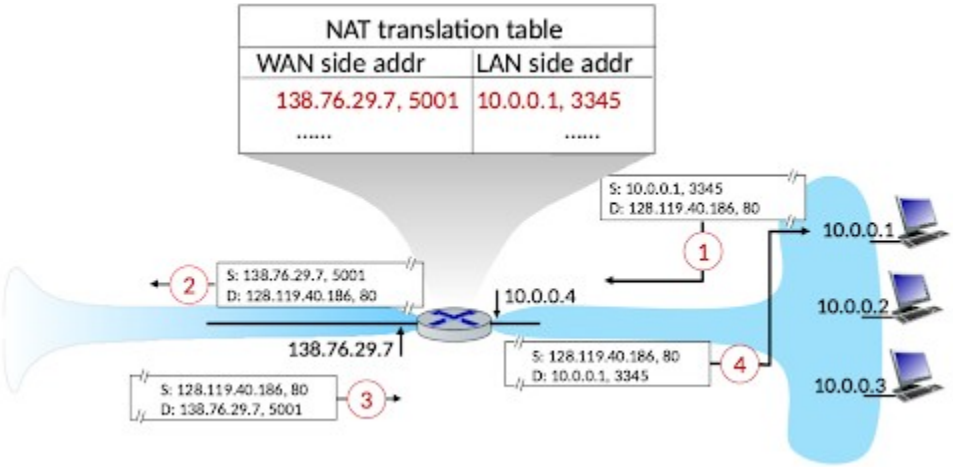
- In SOHO (small office, home office) subnets, a range of addresses would need to be allocated by the ISP to cover all of the SOHO's IP devices
- If the subnet grew bigger, a larger block of addresses would have to be allocated (but maybe the ISP has already allocated the contiguous portion of the SOHO network's current address range)
- What typical homeowner wants (or should need) to know how to manage IP addresses in the first place?

Fortunately, there is a simpler approach to address allocation that has found increasingly widespread use in such scenarios: the **Network Address Translation** (NAT).

With **NAT**, all the devices in local network share just **one** IPv4 address as far as outside world is concerned.



Let's see an example of how NAT works:



The **NAT-enabled router**, residing in the home, has an interface that is part of the home network on the right of the figure.
Addressing within the home network is exactly as we have seen above (the address space 10.0.0.0/8 is one of three portions of the IP address space that is reserved in [RFC 1918] for a private network or a realm with private addresses).
The three portions of IP address that can only be used in local networks are:

- 10/8
- 172.16/12
- 192.168/16

Devices within a given home network can send packets to each other using 10.0.0.0/24 addressing. However, packets forwarded beyond the home network into the larger global Internet clearly cannot use these addresses (as either a source or a destination address) because there are hundreds of thousands of networks using this block of addresses.
That is, the 10.0.0.0/24 addresses can only have meaning within the given home network. But if private addresses only have meaning within a given network, how is addressing handled when packets are sent to or received from the global Internet, where addresses are necessarily unique? The answer lies in understanding **NAT**.

The NAT-enabled router does not look like a router to the outside world. Instead the **NAT router** behaves to the outside world as a **single device** with a single IP address.
In essence, the NAT-enabled router is hiding the details of the home network from the outside world.

If all datagrams arriving at the NAT router from the WAN have the same destination IP address, then how does the router know the internal host to which it should forward a given datagram? The trick is to use a **NAT translation table** at the NAT router, and to include **port numbers** as well as IP addresses in the table entries.
So for every datagram that arrives from inside the network to the NAT router, it generates a new port number (that aren't currently in the NAT translation table), it **replace** the original source port with this new just generated and the IP address with its IP address, and then add an entry to its NAT translation table.

When the response datagram arrives at the NAT router, the router indexes the NAT translation table using the destination IP address and destination port number to obtain the appropriate IP address and destination port number for the browser in the home network.

So the implementation of a NAT router must (transparently):

- **Replace outgoing datagrams**
  Replace $(source\ \ IP\ \ address, port\ \ \#)$ of every outgoing datagram to $(NAT\ \ IP\ \ address, new\ \ port\ \ \#)$
  The remote clients/servers will respond using $(NAT\ \ IP\ \ address, new\ \ port\ \ \#)$ as destination address
- **Remember in the NAT translation table**
  Remember every $(source\ \ IP\ \ address, port\ \ \#)$ to $(NAT\ \ IP\ \ address, new\ \ port\ \ \#)$ translation pair
- **Replace ingoing datagrams**
  Replace $(NAT\ \ IP\ \ address, new\ \ port\ \ \#)$ in destination fields of every incoming packet datagram with $(source\ \ IP\ \ address, port\ \ \#)$ stored in NAT table

The advantages are:

- Just **one** IP address needed from provider ISP for **all** devices in the local network
- We can change the address of a host in the local network without the need of notifying the outside world
- We can change ISP without changing addresses of devices in local networks
- The security (the devices inside the local network aren't directly addressable/visibile by outside the world)

But NAT has also been controversial:

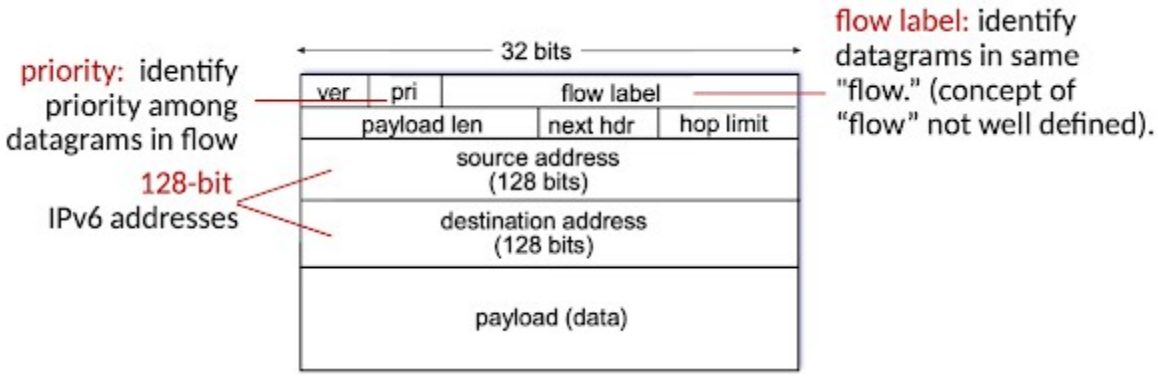
- port numbers are meant to be used for addressing processes, not for addressing hosts
- routers "should" only process up to layer 3
- violates end-to-end argument (port # manipulation by network-layer device)
- problems for servers running on home network (solution with NAT Traversal)

IPv6

In the early 1990s, the Internet Engineering Task Force began an effort to develop a successor to the IPv4 protocol. A prime motivation for this effort was the realization that the 32-bit IPv4 address space was beginning to be used up, with new subnets and IP nodes being attached to the Internet (and being allocated unique IP addresses) at a breathtaking rate.

To respond to this need for a large IP address space, a new IP protocol, IPv6, was developed. The designers of IPv6 also took this opportunity to tweak and augment other aspects of IPv4, based on the accumulated operational experience with IPv4.
Other motivations are the speed of processing/forwarding in IPv4, that is longer due to the variabile length of the header.
IPv6 also enable different network-layer treatment of "flows".

The format of the IPv6 datagram is this:



The most important changes introduced in IPv6 are evident in the datagram format:

- **Expanded addressed capabilities**
  IPv6 increases the size of the IP address from 32 to 128 bits (every grain of sand in the planet can be IP-addressable).
  In addition to unicast and multicast addresses, IPv6 has introduced a new type of

address, called an **anycast** address, that allows a datagram to be delivered to any one of a group of hosts.

- **A streamlined 40-byte header**
  The resulting 40-byte fixed-length header allows for faster processing of the IP datagram by a router. A new encoding of options allows for more flexible options processing.
- **Flow labeling**
  Identify datagrams in the same flow
  IPv6 has an elusive definition of a **flow** (audio transmission could be a flow, traffic carried by a high-priority user could be a flow).
  What is clear, however, is that the designers of IPv6 foresaw the eventual need to be able to differentiate among the flows, even if the exact meaning of a flow had yet to be determined.

The following fields are defined in IPv6:

- **Version**
  This 4-bit field identifies the IP version number. Not surprisingly, IPv6 carries a value of 6 in this field. Note that putting a 4 in this field does not create a valid IPv4 datagram.
- **Traffic class**
  The 8-bit traffic class field, like the TOS field in IPv4, can be used to give priority to certain datagrams within a flow, or it can be used to give priority to datagrams from certain applications (for example, voice-over-IP) over datagrams from other applications (for example, SMTP e-mail).
- **Flow label**
  As discussed above, this 20-bit field is used to identify a flow of datagrams.
- **Payload length**
  This 16-bit value is treated as an unsigned integer giving the number of bytes in the IPv6 datagram following the fixed-length, 40-byte datagram header
- **Next header**
  This field identifies the protocol to which the contents (data field) of this datagram will be delivered (for example, to TCP or UDP). The field uses the same values as the protocol field in the IPv4 header.
- **Hop limit**
  The contents of this field are decremented by one by each router that forwards the datagram. If the hop limit count reaches zero, the datagram is discarded.
- **Source and destination addresses**
  The various formats of the IPv6 128-bit address are described in RFC 4291.
- **Data**
  This is the payload portion of the IPv6 datagram. When the datagram reaches its destination, the payload will be removed from the IP datagram and passed on to the protocol specified in the next header field.

We notice that several fields appearing in the IPv4 datagram are no longer present (for speed and complexity reasons) in the IPv6 datagram:

- **Fragmentation/reassembly**
  IPv6 does not allow for fragmentation and reassembly at intermediate routers; these

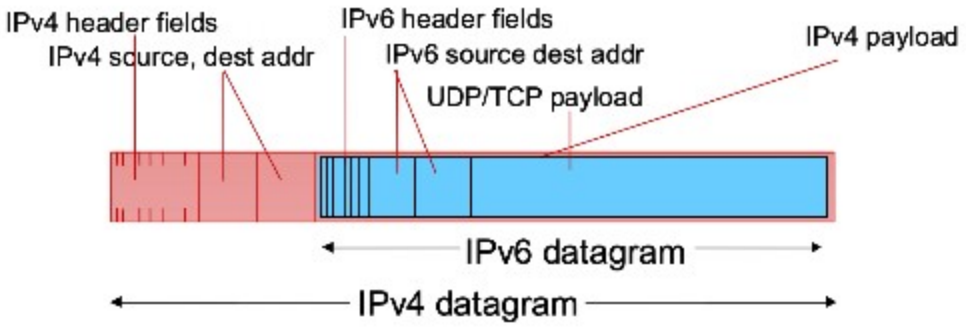
operations can be performed only by the source and destination.
If an IPv6 datagram received by a router is too large to be forwarded over the outgoing link, the router simply drops the datagram and sends a “Packet Too Big” ICMP error message back to the sender.

- **Header checksum**
  Because the transport-layer (for example, TCP and UDP) and link-layer (for example, Ethernet) protocols in the Internet layers perform checksumming, the designers of IP probably felt that this functionality was sufficiently redundant in the network layer that it could be removed. (It should be recalculated at every router due to TTL change)
- **Options**
  An options field is no longer a part of the standard IP header. However, it has not gone away. Instead, the options field is one of the possible next headers pointed to from within the IPv6 header. That is, just as TCP or UDP protocol headers can be the next header within an IP packet, so too can an options field. The removal of the options field results in a fixed-length, 40-byte IP header.

Now that we have seen the technical details of IPv6, let us consider a very practical matter: How will the public Internet, which is **based on IPv4, be transitioned to IPv6**? The problem is that while new IPv6- capable systems can be made backward-compatible, that is, can send, route, and receive IPv4 datagrams, already deployed IPv4-capable systems are not capable of handling IPv6 datagrams.

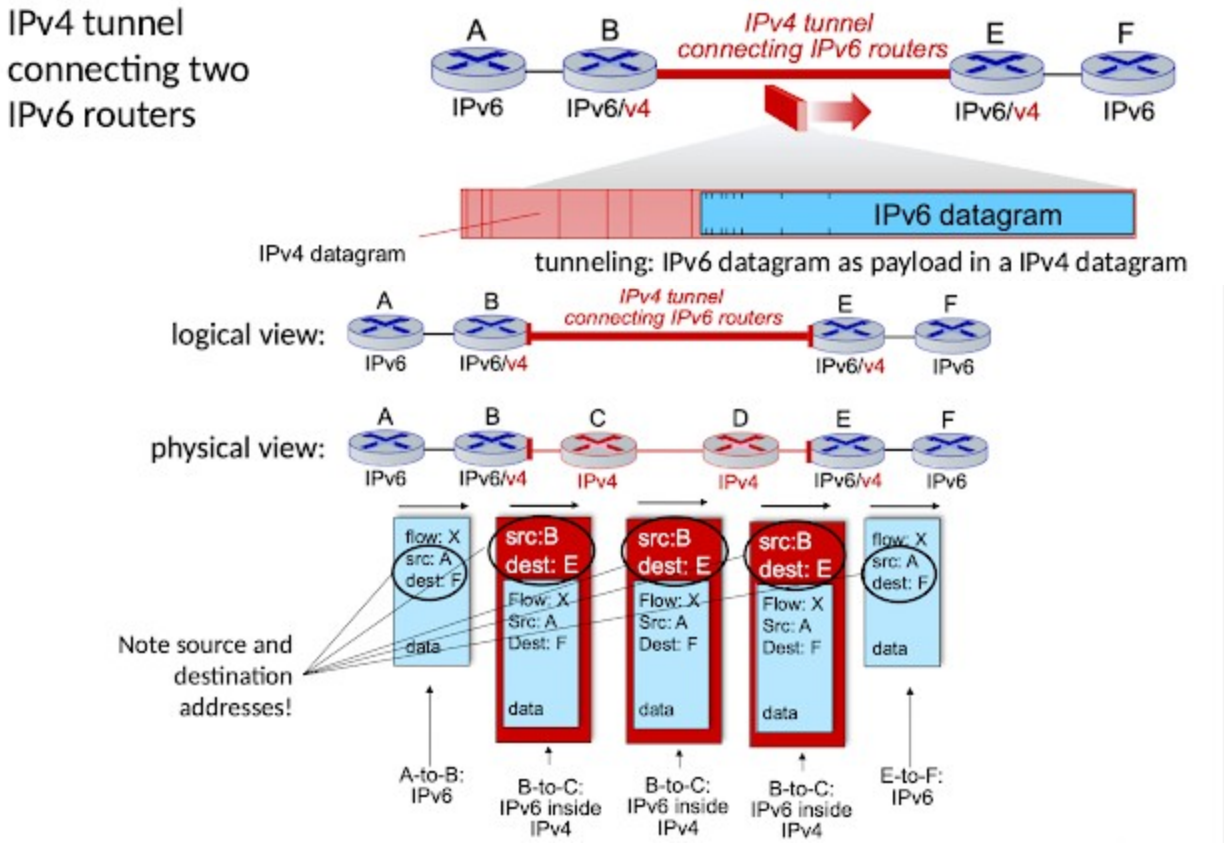
One option would be to declare a **flag day**: a given time and date when all Internet machines would be turned off and upgraded from IPv4 to IPv6. But a flag day involving billions of devices is even more unthinkable today.

The approach to IPv4-to-IPv6 transition that has been most widely adopted in practice involves **tunneling**.



The basic idea behind tunneling is the following: Suppose two IPv6 nodes want to interoperate using IPv6 datagrams but are connected to each other by intervening IPv4 routers.
We refer to the intervening set of IPv4 routers between two IPv6 routers as a **tunnel**.
With tunnelling, the IPv6 node on the sending side of the tunnel takes the entire IPv6 datagram and puts it in the data (payload) field of an IPv4 datagram. This IPv4 datagram is addressed to the IPv6 node on the receiving side of the tunnel. Then this datagram is routed along the IPv4 routers (like a normal datagram) till the IPv6 router.

The IPv6 node on the receiving side of the tunnel eventually receives the IPv4 datagram (it is the destination of the IPv4 datagram!), determines that the IPv4 datagram contains an IPv6 datagram (by observing that the protocol number field in the IPv4 datagram is 41 [RFC 4213], indicating that the IPv4 payload is a IPv6 datagram), extracts the IPv6 datagram, and then routes the IPv6 datagram exactly as it would if it had received the IPv6 datagram from a directly connected IPv6 neighbor.



We end this section by noting that while the adoption of IPv6 was initially slow to take off, momentum has been building.
NIST reports that more than a third of US government second-level domains are IPv6-enabled. On the client side, Google reports that only about 30 percent of the clients accessing Google services do so via IPv6 and Europe's Third Generation Partnership Program [3GPP 2016] has specified IPv6 as the standard addressing scheme for mobile multimedia.

One important lesson that we can learn from the IPv6 experience is that it is enormously difficult to change network-layer protocols.
In summary, in the future we can certainly expect to see changes in the Internet's network layer, but these changes will likely occur on a time scale that is much slower than the changes that will occur at the application layer.

Generalized Forwarding and SDN

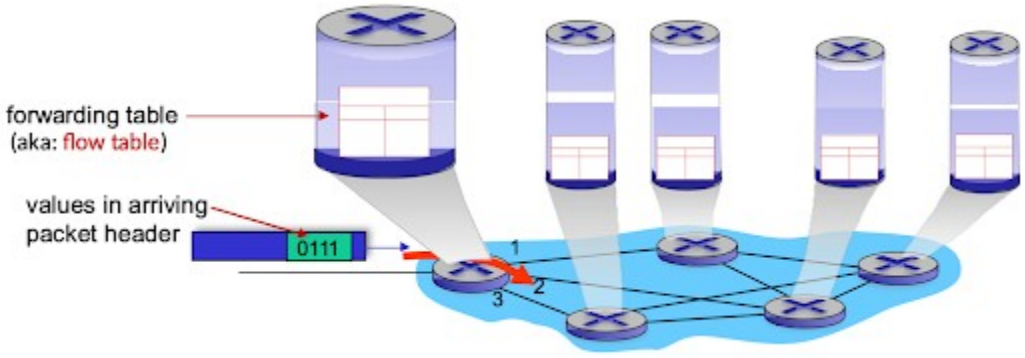
At the start of the chapter we noted that an Internet router's forwarding decision has traditionally been based solely on a packet's destination address.
In the previous section, however, we've also seen that there has been a proliferation of middleboxes that perform **many layer-3 functions** (NAT, firewalls, load-balancers).

This proliferation of middleboxes, layer-2 switches, and layer-3 routers (each with its own specialized hardware, software and management interfaces) has undoubtedly resulted in costly headaches for many network operators. However, recent advances in software-defined networking have promised, and are now delivering, a unified approach towards providing many of these network-layer functions, and certain link-layer functions as well, in a modern, elegant, and integrated manner.

Recall that characterized **destination-based forwarding** as the two steps of looking up a destination IP address ("match"), then sending the packet into the switching fabric to the specified output port ("action").

In generalized forwarding, a match-plus-action table generalizes the notion of the destination-based forwarding table that we encountered.
The **match** can be made over multiple header fields associated with different protocols at different layers in the protocol stack.
The **action** can include forwarding to one or more output ports, load balancing packets, rewriting header values, blocking/dropping a packet, sending a packet to a special server for further processing and action and more.

Because forwarding decisions may be made using network-layer and/or link-layer source and destination addresses, the forwarding devices of this type of forwarding are more accurately described as "**packet switches**" (rather than layer 3 "routers" or layer 2 "switches").

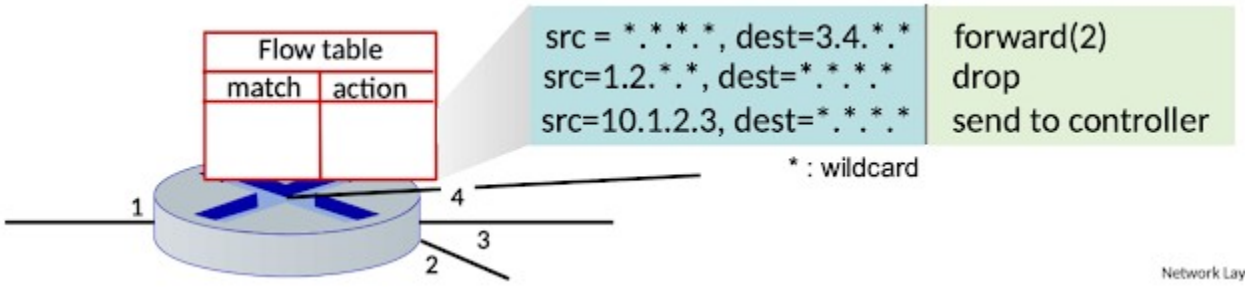


Each packet switch contains a match-plus-action table that is computed and distributed by a remote controller.

Our following discussion of generalized forwarding will be based on **OpenFlow**, a highly visible and successful standard that has pioneered the notion of the match-plus-action forwarding abstraction and controllers, as well as the SDN revolution more generally.

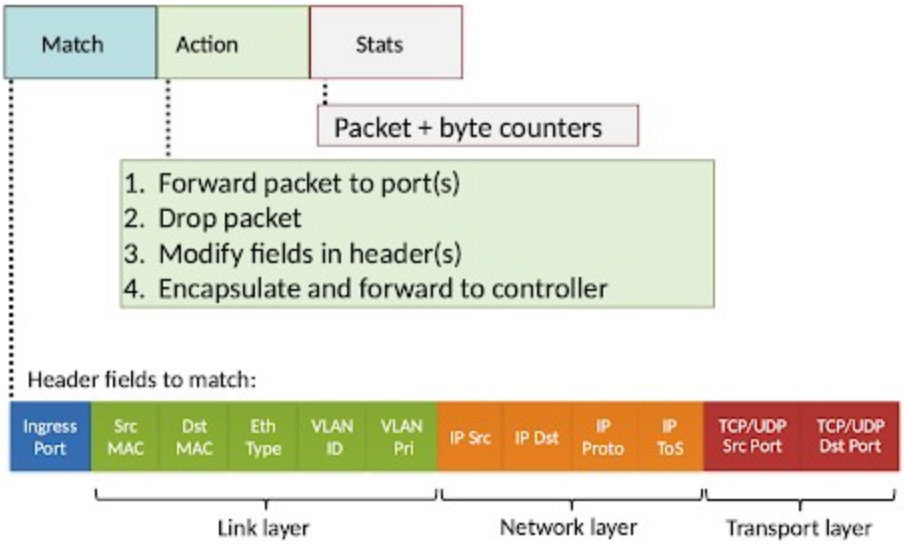
Let's see a **flow table abstraction**.
The "flow" is defined by the header field values (in link-, network-, transport-layer fields).
The generalized forwarding has simple packet-handling rules:

- **Match**: pattern values in packet header fields
- **Actions**: for matched packet: drop, forward, modify matched packet o send matched packet to controller
- **Priority**: disambiguate overlapping patterns
- **Counters**: number of bytes and number of packets



Each entry in the match-plus-action forwarding table, known as a **flow table** in OpenFlow, includes:

- A **set of header field values** to which an incoming packet will be matched.
  A packet that matches no flow table entry can be dropped or sent to the remote controller for more processing.
  In practice, a flow table may be implemented by multiple flow tables for performance or cost reasons, but we'll focus here on the abstraction of a single flow table.
- A **set of counters** that are updated as packets are matched to flow table entries.
  These counters might include the number of packets that have been matched by that table entry, and the time since the table entry was last updated.
- A **set of actions** to be taken when a packet matches a flow table entry.
  These actions might be to forward the packet to a given output port, to drop the packet, makes copies of the packet and sent them to multiple output ports, and/or to rewrite selected header fields.



Match

The figure shows the **eleven packet-header fields** and the incoming port ID that can be matched in an OpenFlow 1.0 match-plus-action rule.
The first observation we make is that OpenFlow's match abstraction allows for a match to be made on selected fields from three layers of protocol headers.

Since we've not yet covered the link layer, suffice it to say that the **source and destination MAC addresses** are the link-layer addresses associated with the frame's sending and receiving interfaces. (So a OpenFlow-enabled device can equally perform as a router forwarding datagrams as well as a switch forwarding frames).
The Ethernet type field corresponds to the upper layer protocol (e.g., IP) to which the frame's payload will be de-multiplexed, and the VLAN fields are concerned with so-called virtual local area networks.

The **ingress port** refers to the input port at the packet switch on which a packet is received. The packet's IP source address, IP destination address, IP protocol field, and IP type of service fields were discussed earlier in this chapter. The transport-layer source and destination port number fields can also be matched.

Flow table entries may also have **wildcards**. For example, an IP address of 128.119.*.* in a flow table will match the corresponding address field of any datagram that has 128.119 as the first 16 bits of its address.
Each flow table entry also has an associated **priority**. If a packet matches multiple flow table entries, the selected match and corresponding action will be that of the highest priority entry with which the packet matches.

Lastly, we observe that not all fields in an IP header can be matched. For example OpenFlow does not allow matching on the basis of TTL field or datagram length field. Why are some fields

allowed for matching, while others are not? Undoubtedly, the answer has to do with the tradeoff between functionality and complexity.

Action

Each flow table entry has a list of zero or more actions that determine the processing that is to be applied to a packet that matches a flow table entry. If there are multiple actions, they are performed in the order specified in the list.

Among the most important possible actions are:

- **Forwarding**
  An incoming packet may be forwarded to a particular physical output port, broadcast over all ports (except the port on which it arrived) or multicast over a selected set of ports. The packet may be encapsulated and sent to the remote controller for this device. That controller then may (or may not) take some action on that packet, including installing new flow table entries, and may return the packet to the device for forwarding under the updated set of flow table rules.
- **Dropping**
  A flow table entry with no action indicates that a matched packet should be dropped
- **Modify-fields**
  The values in ten packet header fields (all layer 2, 3, and 4 fields, except the IP Protocol field) may be re-written before the packet is forwarded to the chosen output port.

Let's see some examples of OpenFlow services:

Destination-based forwarding:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	*	*	*	*	*	51.6.0.8	*	*	*	*	port6

IP datagrams destined to IP address 51.6.0.8 should be forwarded to router output port 6

Firewall:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	*	*	*	*	*	*	*	*	*	22	drop

Block (do not forward) all datagrams destined to TCP port 22 (ssh port #)

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	*	*	*	*	128.119.1.1	*	*	*	*	*	drop

Block (do not forward) all datagrams sent by host 128.119.1.1

Layer 2 destination-based forwarding:

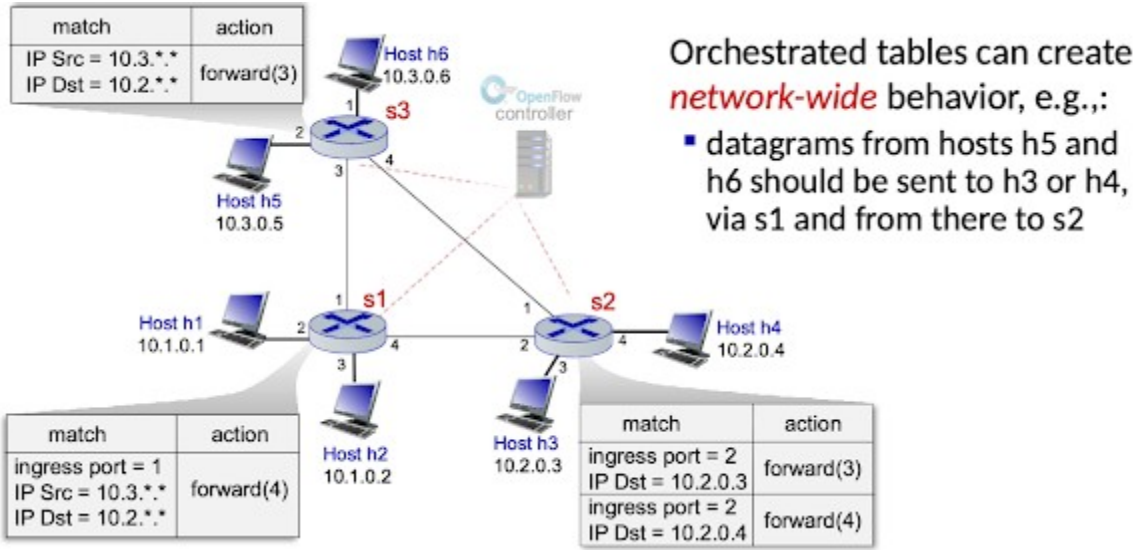
Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	22:A7:23:11:E1:02	*	*	*	*	*	*	*	*	*	port3

layer 2 frames with destination MAC address 22:A7:23:11:E1:02 should be forwarded to output port 3

We can notice that the match+action abstraction unifies different kinds of devices:

- Router
  Match: longest destination IP address
  Action: forward out a link
- Switch
  Match: destination MAC address
  Action: forward or flood
- Firewall
  Match: IP addresses and TCP/UDP number ports
  Action: permit or deny
- NAT
  Match: IP address and port
  Action: rewrite IP address and port

Let's see an example of OpenFlow:



The Link Layer and LANs

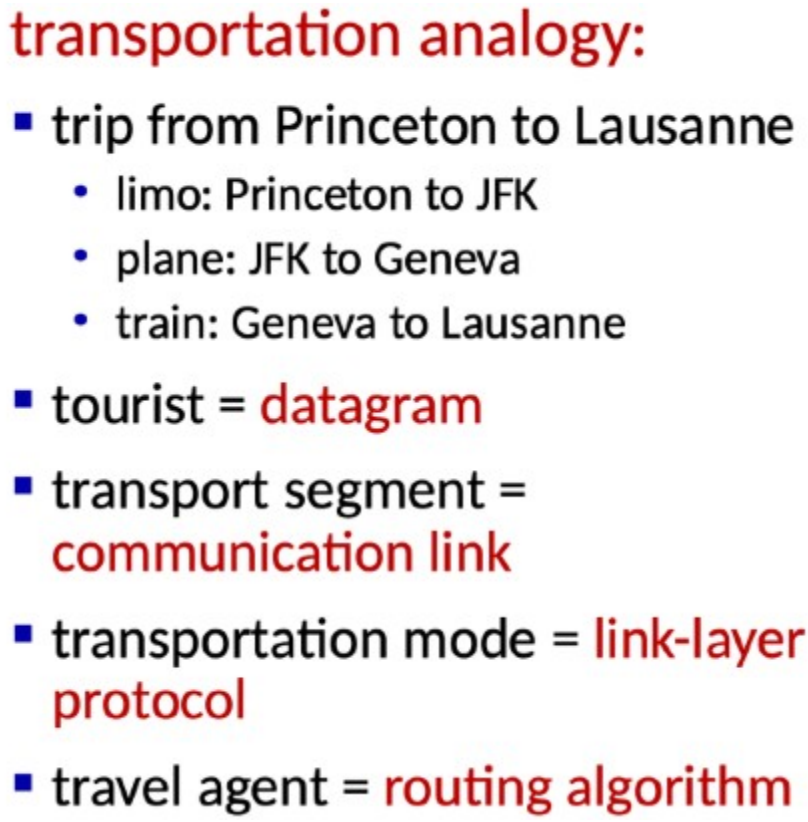
Introduction to the Link Layer

The **link layer** has the responsibility of **transferring datagram from one node to physically adjacent node** over a link

Let's begin with some important terminology.
We'll find it convenient in this chapter to refer to any device that runs a link-layer (i.e., layer 2) protocol as a **node** (for example hosts, routers, switches, WiFi access points).

We will also refer to the communication channels that connect adjacent nodes along the communication path as **links**. In order for a datagram to be transferred from source host to destination host, it must be moved over each of the individual links in the end-to-end path. Over a given link, a transmitting node encapsulates the datagram in a **link-layer frame** and transmits the frame into the link.

The datagram is transferred by different link protocols over different links (WiFi on first link, then Ethernet, …) and each link protocol provides different services (for example reliability).



The Services Provided by the Link Layer

Although the basic service of any link layer is to move a datagram from one node to an adjacent node over a single communication link, the details of the provided service can vary from one link-layer protocol to the next. Possible services that can be offered by a link-layer protocol include:

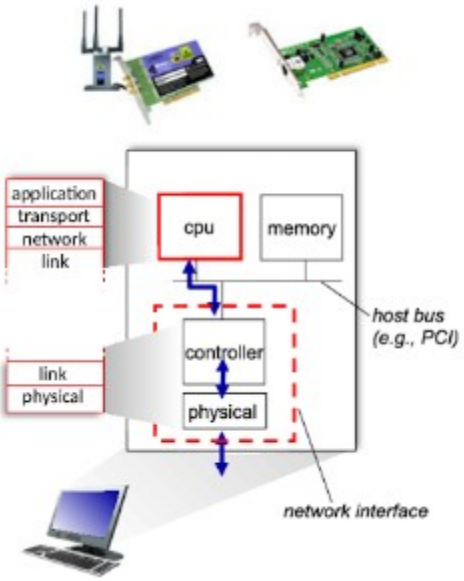
- **Framing**
  Almost all link-layer protocols encapsulate each network-layer datagram within a link-layer frame before transmission over the link. A frame consists of a data field, in which the network-layer datagram is inserted, and a number of header fields. The structure of the frame is specified by the link-layer protocol. We'll see several different frame formats when we examine specific link-layer protocols in the second half of this chapter.
- **Link access**
  A medium access control (MAC) protocol specifies the rules by which a frame is transmitted onto the link.

For point-to-point links that have a single sender at one end of the link and a single receiver at the other end of the link, the MAC protocol is simple (or nonexistent): the sender can send a frame whenever the link is idle.
The more interesting case is when multiple nodes share a single broadcast link: the so-called multiple access problem. Here, the MAC protocol serves to coordinate the frame transmissions of the many nodes.

- **Reliable delivery**
  When a link-layer protocol provides reliable delivery service, it guarantees to move each network-layer datagram across the link without error.
  A link-layer reliable delivery service is often used for links that are prone to high error rates, such as a wireless link, with the goal of correcting an error locally, on the link where the error occurs, rather than forcing an end-to-end retransmission of the data by a transport- or application-layer protocol.
  However, link-layer reliable delivery can be considered an unnecessary overhead for low bit-error links, including fiber, coax, and many twisted-pair copper links. For this reason, many wired link-layer protocols do not provide a reliable delivery service.
- **Error detection and correction**
  The link-layer hardware in a receiving node can incorrectly decide that a bit in a frame is zero when it was transmitted as a one, and vice versa. Such bit errors are introduced by signal attenuation and electromagnetic noise. Because there is no need to forward a datagram that has an error, many link-layer protocols provide a mechanism to detect such bit errors.
  This is done by having the transmitting node include error-detection bits in the frame, and having the receiving node perform an error check. Error detection in the link layer is usually more sophisticated and is implemented in hardware.
  Error correction is similar to error detection, except that a receiver not only detects when bit errors have occurred in the frame but also determines exactly where in the frame the errors have occurred (and then corrects these errors).
- **Flow control**
  Pacing between adjacent sending and receiving nodes
- **Half-duplex** and **full-duplex**
  With half duplex, nodes at both ends of the link can transmit, but not at the same time.

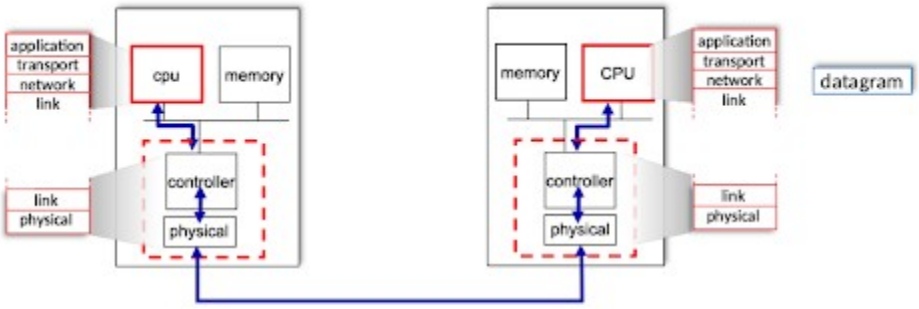
Where Is the Link Layer Implemented?

Before diving into our detailed study of the link layer, let’s conclude this introduction by considering the question of where the link layer is implemented. We’ll focus here on an end system, since we learned that the link layer is implemented in a router’s line card.



For the most part, the link layer is implemented in a network adapter, also sometimes known as a **network interface card** (NIC), for example Ethernet or Wifi card or chip, that is attached to the host's system bus. It is a combination of hardware, software and firmware.
Thus, the link layer is a combination of hardware and software: the place in the protocol stack where software meets hardware.
At the heart of the network adapter is the **link-layer controller**, usually a single, special-purpose chip that implements many of the link-layer services (framing, link access, error detection, and so on). Thus, much of a link-layer controller's functionality is **implemented in hardware**.
Until the late 1990s, most network adapters were physically separate cards but increasingly, network adapters are being integrated onto the host's motherboard, a so-called LAN-on-motherboard configuration.

On the sending side, the controller takes a datagram that has been created and stored in host memory by the higher layers of the protocol stack, encapsulates the datagram in a link-layer frame (filling in the frame's various fields), and then transmits the frame into the communication link, following the link-access protocol.
On the receiving side, a controller receives the entire frame, and extracts the network- layer datagram. If the link layer performs error detection, then it is the sending controller that sets the error-detection bits in the frame header and it is the receiving controller that performs error detection.

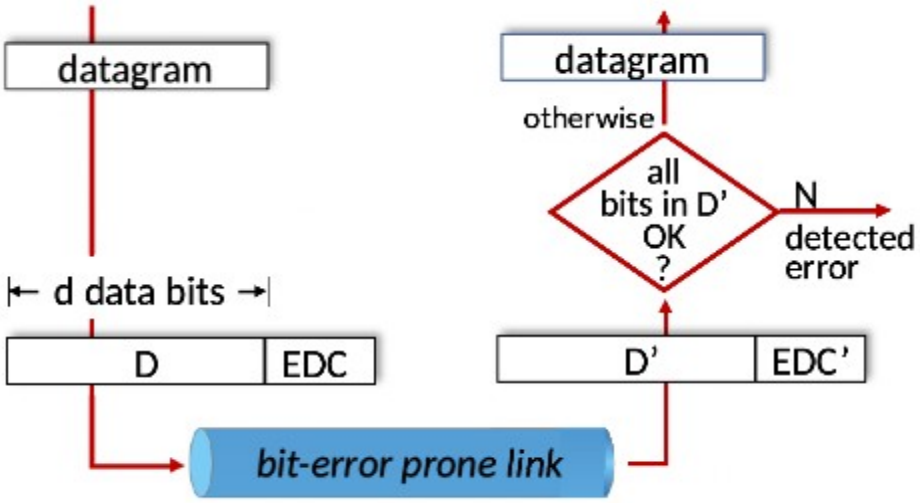


sending side:
- encapsulates datagram in frame
- adds error checking bits, reliable data transfer, flow control, etc.

receiving side:
- looks for errors, reliable data transfer, flow control, etc.
- extracts datagram, passes to upper layer at receiving side

Error-Detection and -Correction Techniques

In the previous section, we noted that **bit-level error detection and correction**, detecting and correcting the corruption of bits in a link-layer frame sent from one node to another physically connected neighboring node, are two services often provided by the link layer.
In this section, we'll examine a few of the simplest techniques that can be used to detect and, in some cases, correct such bit errors.



At the sending node, data, D, to be protected against bit errors is augmented with **error-detection and -correction bits** (EDC).
Typically, the data to be protected includes not only the datagram passed down from the network layer for transmission across the link, but also link-level addressing information, sequence numbers, and other fields in the link frame header, so D includes the header.

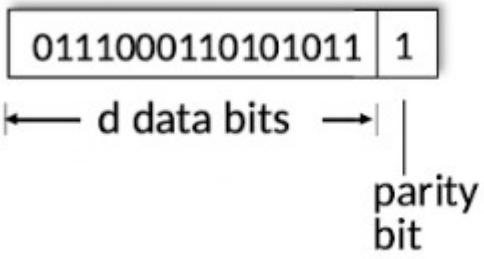
Both D and EDC are sent to the receiving node in a link-level frame. At the receiving node, a sequence of bits, D′ and EDC′ is received. Note that D′ and EDC′ may differ from the original D and EDC as a result of in-transit bit flips.

The receiver's challenge is to determine whether or not D′ is the same as the original D, given that it has only received D′ and EDC′.
Error-detection and -correction techniques allow the receiver to sometimes, **but not always**, detect that bit errors have occurred. Even with the use of error-detection bits there still may be **undetected bit errors**; that is, the receiver may be unaware that the received information contains bit errors. So we want to choose an error-detection scheme that keeps the probability of undetected bit errors small.
More sophisticated error-detection and-correction techniques include a larger EDC field, but incur a larger overhead.

Parity Checks

Perhaps the simplest form of error detection is the use of a **single parity bit**.
Suppose that the information to be sent, D, has d bits:

- In an **even parity scheme**, the sender simply includes one additional bit and chooses its value such that the total number of 1s in the $d+1$ bits (the original information plus a parity bit) is **even**.

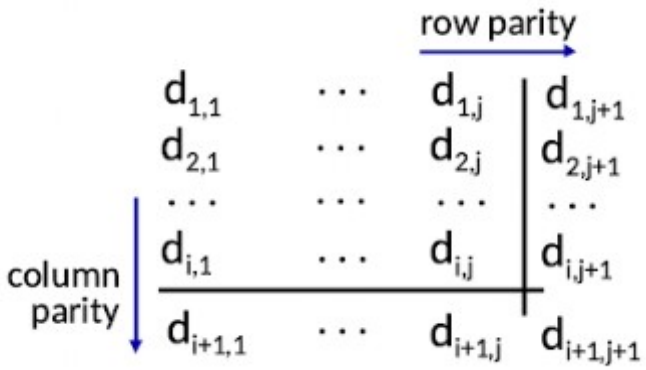


- For **odd parity schemes**, the parity bit value is chosen such that there is an **odd** number of 1s.

Receiver operation is also simple with a single parity bit. The **receiver need only count** the number of 1s in the received $d+1$ bits. If an odd number of 1-valued bits are found with an even parity scheme, the receiver knows that at least one bit error has occurred. More precisely, it knows that some odd number of bit errors have occurred.

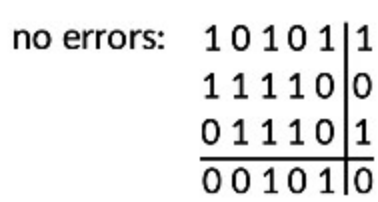
But what happens if an even number of bit errors occur? You should convince yourself that this would result in an undetected error. If the probability of bit errors is small and errors can be assumed to occur independently from one bit to the next, the probability of multiple bit errors in a packet would be extremely small.
However, measurements have shown that, rather than occurring independently, errors are often clustered together in "bursts." Under burst error conditions, the probability of undetected errors in a frame protected by single-bit parity can approach 50 percent.

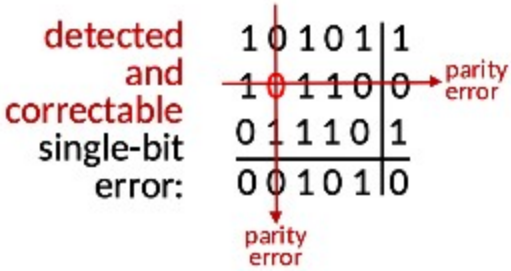
Let's consider a simple generalization of one-bit parity that will provide us with insight into error-correction techniques.



The figure shows a two-dimensional generalization of the single-bit parity scheme. Here, the d bits in D are divided into i rows and j columns. A parity value is computed for each row and for each column. The resulting $i + j + 1$ parity bits comprise the link-layer frame's error-detection bits.

Suppose now that a single bit error occurs in the original d bits of information. With this two-dimensional parity scheme, the parity of both the column and the row containing the flipped bit will be in error. The receiver can thus not only detect the fact that a single bit error has occurred, but can use the column and row indices of the column and row with parity errors to actually **identify the bit that was corrupted and correct that error**!





more

The ability of the receiver to both detect and correct errors is known as **forward error correction** (FEC). FEC techniques are valuable because they can decrease the number of sender retransmissions required.

This avoids having to wait for the round-trip propagation delay needed for the sender to receive a NAK packet and for the retransmitted packet to propagate back to the receiver: a potentially important advantage for real-time network applications or links (such as deep-space links) with long propagation delays.

Checksumming methods

In checksumming techniques, the d bits of data are treated as a sequence of $k\text{-}bit$ integers. One simple checksumming method is to simply sum these k-bit integers and use the resulting sum as the error-detection bits.

The **Internet checksum** is based on this approach: bytes of data are treated as 16-bit integers and summed. The 1s complement of this sum then forms the Internet checksum that is carried in the segment header.

The receiver checks the checksum by taking the 1s complement of the sum of the received data (including the checksum) and checking whether the result is all 1 bits. If any of the bits are 0, an error is indicated.
The checksum can be computed over all fields (TCP and UDP) or over only the header (IP).

Checksumming methods require relatively little packet overhead (in TCP and UDP only 16 bits). However, they provide relatively **weak protection** against errors as compared with cyclic redundancy check, which is discussed below and which is often used in the link layer.
Why is checksumming used at the transport layer and cyclic redundancy check used at the link layer? Recall that the transport layer is typically implemented in software in a host as part of the host's operating system. Because transport-layer error detection is implemented in software, it is important to have a simple and fast error-detection scheme such as checksumming. On the other hand, error detection at the link layer is implemented in dedicated hardware in adapters, which can rapidly perform the more complex CRC operations.

Cyclic Redundancy Check (CRC)

An error-detection technique used widely in today's computer networks is based on **cyclic redundancy check (CRC) codes**. CRC codes are also known as **polynomial codes**, since it is possible to view the bit string to be sent as a polynomial whose coefficients are the 0 and 1 values in the bit string, with operations on the bit string interpreted as polynomial arithmetic.

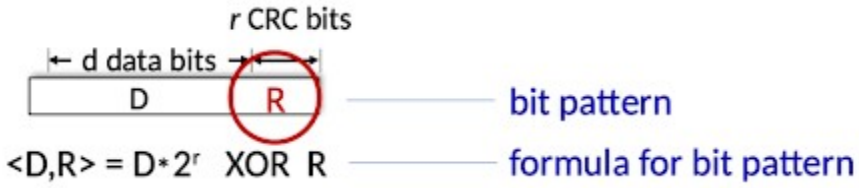
Consider the $d\text{-}bit$ piece of data, D, that the sending node wants to send to the receiving node. The sender and receiver must first agree on an $r+1$ bit pattern, known as a generator, which we will denote as G. We will require that the most significant (leftmost) bit of G be a 1
For a given piece of data, D, the sender will choose r additional bits, R, and append them to D such that the resulting $d+r$ bit pattern (interpreted as a binary number) is exactly divisible by G (i.e., has no remainder) using modulo-2 arithmetic.
The process of error checking with CRCs is thus simple: The receiver divides the $d+r$ received bits by G. If the remainder is nonzero, the receiver knows that an error has occurred; otherwise the data is accepted as being correct.

All CRC calculations are done in modulo-2 arithmetic without carries in addition or borrows in subtraction. This means that addition and subtraction are identical, and both are equivalent to the bitwise exclusive-or (XOR) of the operands, for example:

$$1011 \ \ XOR \ \ 0101 \ = \ 1110$$
$$1011 \ - \ 0101 \ = \ 1110$$
$$1011 \ + \ 0101 \ = \ 1110$$

Multiplication and division are the same as in base-2 arithmetic, except that any required addition or subtraction is done without carries or borrows.

Thus, given D and R, the quantity $\left(D \cdot 2^r\right) \ \ XOR \ \ R$ yields the $d+r$ bit pattern shown in the figure below. We'll use this algebraic characterization of the $d+r$ bit pattern from the figure in our discussion below.



Let us now turn to the crucial question of how the sender computes R.
If $<D,R>$ is divisible by G, then it exists an n such that:

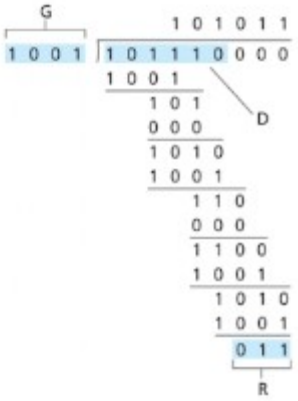
$$\left(D \cdot 2^r\right) \ XOR \ R = nG$$

Let's now XOR (that is, add modulo-2, without carry) R to both sides:

$$\left(D \cdot 2^r\right) = nG \ XOR \ R$$

That is that R is the remainder of $\frac{D \cdot 2^r}{G}$

Let's see an example with $D = 101110$, so $d = 6$, and $G = 1001$, so $r = 3$



In this case the 9 bits transmitted are 101110011.

Each of the CRC standards can detect burst errors of fewer than $r+1$ bits. (This means that all consecutive bit errors of r bits or fewer will be detected.)
This mechanism is widely used in practice (Ethernet, 802.11 WiFi)

Multiple Access Links and Protocols

In the introduction to this chapter, we noted that there are two types of network links:

- **Point-to-point link**
  Consists of a **single sender** at one end of the link and a **single receiver** at the other end of the link.
  (point-to-point protocol PPP, high-level data link control HDLC)
- **Broadcast link**
  Can have multiple sending and receiving nodes all connected to the same, single, **shared broadcast channel**. The term **broadcast** is used here because when any one node transmits a frame, the channel broadcasts the frame and each of the other nodes receives a copy.

(Ethernet, Wireless LANs)



In this section we'll take a step back from specific link-layer protocols and first examine a problem of central importance to the link layer: how to coordinate the access of multiple sending and receiving nodes to a shared broadcast channel: the **multiple access problem**.
Broadcast channels are often used in LANs, networks that are geographically concentrated in a single building.

We are all familiar with the notion of broadcasting: television has been using it since its invention. But traditional television is a one-way broadcast (that is, one fixed node transmitting to many receiving nodes), while nodes on a computer network broadcast channel can both send and receive.
Another analogies are a cocktail party (the air providing the broadcast medium) where people talk and listen and a classroom where teacher(s) and student(s) similarly share the same, single, broadcast medium. A central problem in both scenarios is that of determining who gets to talk (that is, transmit into the channel) and when.

Computer networks similarly have protocols, so-called **multiple access protocols**, by which nodes regulate their transmission into the shared broadcast channel.
Although technically each node accesses the broadcast channel through its adapter, in this section we will refer to the node as the sending and receiving device.

Because all nodes are capable of transmitting frames, more than two nodes can transmit frames at the same time. When this happens, all of the nodes receive multiple frames at the same time; that is, the transmitted frames **collide** at all of the receivers: all the frames are lost and the broadcast channel is wasted during the collision interval.

In order to ensure that the broadcast channel performs useful work when multiple nodes are active, it is necessary to somehow coordinate the transmissions of the active nodes. This coordination job is the responsibility of the multiple access protocol.
The multiple access protocols are distributed algorithms that determine how nodes share a channel (i.e. when a node can transmit). The communication about channel sharing must use channel itself (no out-of-band channel for coordination)
Over the years, dozens of multiple access protocols have been implemented in a variety of link-layer technologies. Nevertheless, we can classify just about any multiple access protocol as belonging to one of three categories:

- **Channel partitioning** protocols
  Divide the channel into smaller “pieces” (time slots, frequency, code)
  Allocate piece to node for exclusive use
- **Random access** protocols
  Channel not divided, allow collisions
  “recover” from collisions
- **Taking-turns** protocols
  Nodes takes turns, but nodes with more to send can take longer turns

Let’s see what characteristics should have an ideal multiple access protocol, for a broadcast channel of rate $R \ \ bps$:

1. When only one node has data to send, that node has a throughput of $R \ \ bps$
2. When M nodes have data to send, each of these nodes has a throughput of $\frac{R}{M} \ \ bps$.
   This need not necessarily imply that each of the M nodes always has an instantaneous rate of $\frac{R}{M}$, but rather that each node should have an average transmission rate of $\frac{R}{M}$ over some suitably defined interval of time
3. The protocol is fully decentralized; that is, there is no master node to coordinate transmissions (that represents a single point of failure for the network) and there is no synchronization of clocks/slots
4. The protocol is simple, so that it is inexpensive to implement

Channel Partitioning Protocols

Recall from our early discussion that time-division multiplexing (TDM) and frequency-division multiplexing (FDM) are two techniques that can be used to partition a broadcast channel’s bandwidth among all nodes sharing that channel.
As an example, suppose the channel supports N nodes and that the transmission rate of the channel is $R \ \ bps$.

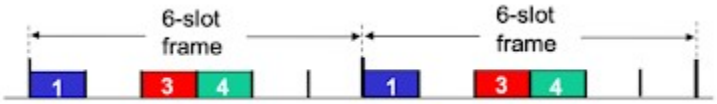
TDM divides time into **time frames** and further divides each time frame into N **time slots** (The TDM time frame should not be confused with the link-layer unit of data exchanged between sending and receiving adapters, which is also called a frame. In order to reduce confusion, in this subsection we’ll refer to the link-layer unit of data exchanged as a packet).
Each time slot is then assigned to one of the N nodes. Whenever a node has a packet to send, it transmits the packet’s bits during its assigned time slot in the revolving TDM frame. Typically, slot sizes are chosen so that a single packet can be transmitted during a slot time.
A TDM-regulated cocktail party would allow one partygoer to speak for a fixed period of time, then allow another partygoer to speak for the same amount of time, and so on. Once everyone had had a chance to talk, the pattern would repeat.

TDM is appealing because it eliminates collisions and is perfectly fair (each nodes get an average rate of $\frac{R}{N}$ bps).

However, it has two major drawbacks:

- A node is limited to an average rate of $\frac{R}{N}$ bps even when it is the only node with packets to send
- A node must always wait for its turn in the transmission sequence (again, even when it is the only node with a frame to send)

▪ example: 6-station LAN, 1,3,4 have packets to send, slots 2,5,6 idle

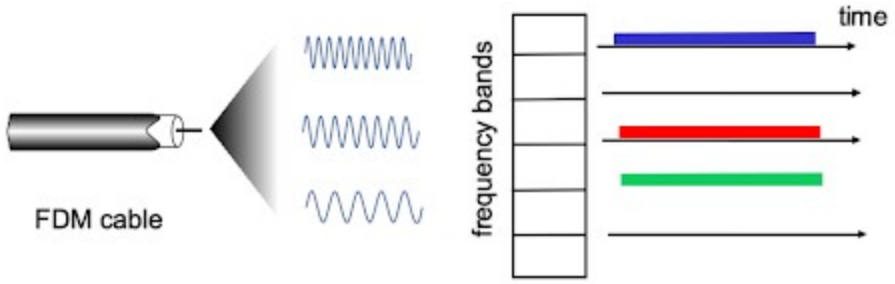


While TDM shares the broadcast channel in time, FDM divides the R bps channel into different frequencies (each with a bandwidth of $\frac{R}{N}$) and assigns each frequency to one of the N nodes.

FDM thus creates N smaller channels of $\frac{R}{N}$ bps out of the single, larger R bps channel.

FDM shares both the advantages and drawbacks of TDM. It avoids collisions and divides the bandwidth fairly among the N nodes, but it has the same problem of TDM (a node is limited to a bandwidth of $\frac{R}{N}$, even when it is the only node with packets to send)

▪ example: 6-station LAN, 1,3,4 have packet to send, frequency bands 2,5,6 idle



A third channel partitioning protocol is **code division multiple access** (CDMA). While TDM and FDM assign time slots and frequencies, respectively, to the nodes, CDMA assigns a different code to each node. Each node then uses its unique code to encode the data bits it sends. If the codes are chosen carefully, CDMA networks have the wonderful property that different nodes can transmit simultaneously and yet have their respective receivers correctly receive a sender's encoded data bits (assuming the receiver knows the sender's code) in spite of interfering transmissions by other nodes.

For now, it will suffice to know that CDMA codes, like time slots in TDM and frequencies in FDM, can be allocated to the multiple access channel users.

Random Access Protocols

In a **random access protocol**, a transmitting node always transmits at the full rate of the channel, namely, $R \ \ bps$.
When there is a **collision**, each node involved in the collision repeatedly retransmits its frame (that is, packet) until its frame gets through without a collision. But when a node experiences a collision, it doesn't necessarily retransmit the frame right away. Instead it waits a **random delay** before retransmitting the frame.
Because the random delays are independently chosen, it is possible that one of the nodes will pick a delay that is sufficiently less than the delays of the other colliding nodes and will therefore be able to sneak its frame into the channel without a collision.

Let's begin our study of random access protocols with one of the simplest random access protocols, the **slotted ALOHA protocol**. In our description of slotted ALOHA, we assume the following:

- All frames consist of exactly L bits
- Time is divided into slots of size $\frac{L}{R}$ seconds (that is, a slot equals the time to transmit one frame)
- Nodes start to transmit frames only at the beginnings of slots
- The nodes are synchronized so that each node knows when the slots begin
- If two or more frames collide in a slot, then all the nodes detect the collision event before the slot ends

Let p be a probability, that is, a number between 0 and 1. The operation of slotted ALOHA in each node is simple:

- When the node has a fresh frame to send, it waits until the beginning of the next slot and transmits the entire frame in the slot.
- If there isn't a collision, the node has successfully transmitted its frame and thus need not consider retransmitting the frame. (The node can prepare a new frame for transmission, if it has one)
- If there is a collision, the node detects the collision before the end of the slot. The node retransmits its frame in each subsequent slot with probability p (randomized) until the frame is transmitted without a collision

Slotted ALOHA would appear to have many advantages. Unlike channel partitioning, slotted ALOHA allows a node to transmit continuously at the full rate, R, when that node is the only active node. Slotted ALOHA is also highly decentralized, because each node detects collisions and independently decides when to retransmit (however require the slots to be synchronized in the slots), and it is also an extremely simple protocol.

Let's see an example:

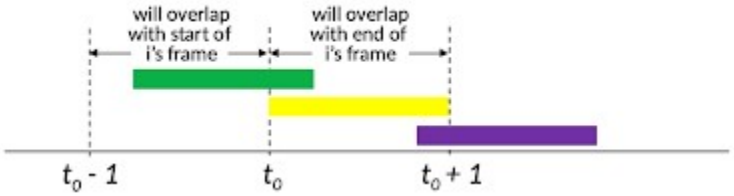


Slotted ALOHA is very efficient when there is only one node active, but, as we see in the figure, there are some problems when there are multiple active nodes.
First, when there are multiple active nodes, a certain fraction of slots will have collisions and will therefore be **wasted**.
The second concern is that another fraction of the slots will be **empty** because all active nodes refrain from transmitting as a result of the probabilistic transmission policy.
The only "unwasted" slots will be those in which exactly one node transmits. A slot in which exactly one node transmits is said to be a **successful slot**. The **efficiency** of a slotted multiple access protocol is defined to be the long-run fraction of successful slots in the case when there are a large number of active nodes, each always having a large number of frames to send (note that if no form of access control were used, and each node were to immediately retransmit after each collision, the efficiency would be zero).

We now proceed to outline the derivation of the maximum efficiency of slotted ALOHA. To keep this derivation simple, let's modify the protocol a little and assume that each node attempts to transmit a frame in each slot with probability p (each node always has a frame to send and the node transmit with a probability of p both for a new frame and for a collisioned one).
Suppose there are N nodes.
The probability a given node has a success is $p \cdot (1-p)^{N-1}$ (one transmit and the other $N-1$ don't)
The probability that any one of the N nodes has a success is $N \cdot p \cdot (1-p)^{N-1}$
Thus, when there are N active nodes, the efficiency of slotted ALOHA is $N \cdot p \cdot (1-p)^{N-1}$.
To obtain the maximum efficiency for N active nodes, we have to find the p^* that maximizes this expression. And to obtain the maximum efficiency for a large number of active nodes, we take the limit of $N \cdot p^* \cdot (1-p^*)^{N-1}$ as N approaches infinity.
After performing these calculations, we'll find that the maximum efficiency of the protocol is given by $\frac{1}{e} = 0.37$. That is, when a large number of nodes have many frames to transmit, then (at best) only 37 percent of the slots do useful work. Thus the effective transmission rate of the channel is not R bps but only $0.37 \cdot R$ bps!
A similar analysis also shows that 37 percent of the slots go empty and 26 percent of slots have collisions.

The slotted ALOHA protocol required that all nodes synchronize their transmissions to start at the beginning of a slot. The first ALOHA protocol was actually an unslotted, fully decentralized protocol. In **pure ALOHA**, when a frame first arrives (that is, a network-layer datagram is

passed down from the network layer at the sending node), the node immediately transmits the frame in its entirety into the broadcast channel.
If a transmitted frame experiences a collision with one or more other transmissions, the node will then immediately (after completely transmitting its collided frame) retransmit the frame with probability p. Otherwise, the node waits for a frame transmission time. After this wait, it then transmits the frame with probability p, or waits (remaining idle) for another frame time with probability $1 - p$.
To determine the maximum efficiency of pure ALOHA, we focus on an individual node. We'll make the same assumptions as in our slotted ALOHA analysis and take the frame transmission time to be the unit of time.
At any given time, the probability that a node is transmitting a frame is p. Suppose this frame begins transmission at time t_0.
In order for this frame to be successfully transmitted, no other nodes can begin their transmission in the interval of time $[t_0 - 1, t_0]$ (such a transmission would overlap with the beginning of the transmission of node i's frame). The probability that all other nodes do not begin a transmission in this interval is $(1 - p)^{N-1}$.
Similarly, no other node can begin a transmission while node i is transmitting, as such a transmission would overlap with the latter part of node i's transmission. The probability that all other nodes do not begin a transmission in this interval is also $(1 - p)^{N-1}$.



Thus, the probability that a given node has a successful transmission is $p \cdot (1 - p)^{2(N-1)}$.
By taking limits as in the slotted ALOHA case, we find that the maximum efficiency of the pure ALOHA protocol is only 1/(2e): exactly half that of slotted ALOHA, so 18%. This then is the price to be paid for a fully decentralized ALOHA protocol.

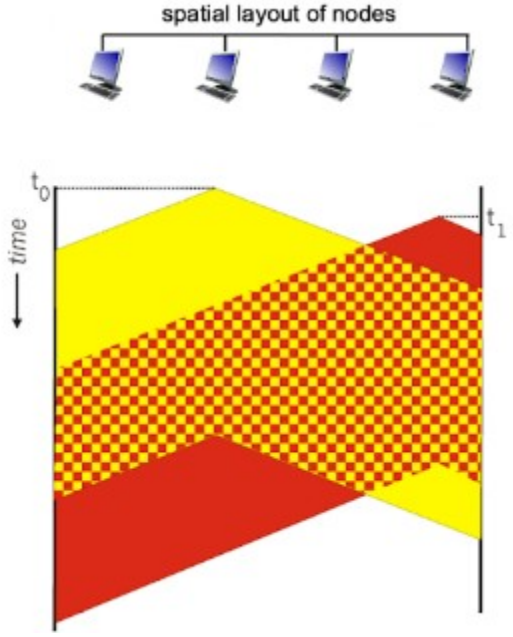
In both slotted and pure ALOHA, a node's decision to transmit is made independently of the activity of the other nodes attached to the broadcast channel. In particular, a node neither pays attention to whether another node happens to be transmitting when it begins to transmit, nor stops transmitting if another node begins to interfere with its transmission.
In our cocktail analogy, it's like that a person continues to chatter regardless of whether other people are talking. Specifically, there are two important rules for polite human conversation:

- **Listen before speaking**
  If someone else is speaking, wait until they are finished.
  In the networking world, this is called **carrier sensing**: a node listens to the channel before transmitting.
  If a frame from another node is currently being transmitted into the channel, a node then waits until it detects no transmissions for a short amount of time and then begins transmission.

- **If someone else begins talking at the same time, stop talking**
  In the networking world, this is called **collision detection**: a transmitting node listens to the channel while it is transmitting.
  If it detects that another node is transmitting an interfering frame, it stops transmitting and waits a random amount of time before repeating the sense-and-transmit-when-idle cycle.

These two rules are embodied in the family of **carrier sense multiple access** (CSMA) and **CSMA with collision detection** (CSMA/CD) protocols.

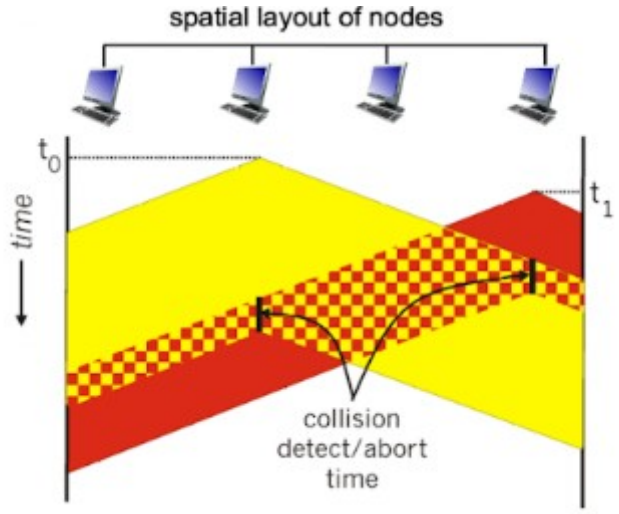
The first question that you might ask about CSMA is why, if all nodes perform carrier sensing, do collisions occur in the first place? (after all, a node will refrain from transmitting whenever it senses that another node is transmitting).
The answer to the question can best be illustrated using space-time diagrams. The figure below shows a space-time diagram of four nodes (A, B, C, D) attached to a linear broadcast bus. The horizontal axis shows the position of each node in space; the vertical axis represents time.



At time t_0, node B senses the channel is idle, as no other nodes are currently transmitting. Node B thus begins transmitting, with its bits propagating in both directions along the broadcast medium.
The downward propagation of B's bits in the figure with increasing time indicates that a nonzero amount of **time is needed** for B's bits actually **to propagate** (albeit at near the speed of light) along the broadcast medium.
At time t_1 (with $t_1 > t_0$), node D has a frame to send. Although node B is currently transmitting at time t_1, the bits being transmitted by B have yet to reach D, and thus D senses the channel idle at t_1.
In accordance with the CSMA protocol, D thus begins transmitting its frame. A short time later, B's transmission begins to interfere with D's transmission at D.

From the figure, it is evident that the end-to-end channel propagation delay of a broadcast channel (the time it takes for a signal to propagate from one of the nodes to another) will play a crucial role in determining its performance. The longer this propagation delay, the larger the chance that a carrier-sensing node is not yet able to sense a transmission that has already begun at another node in the network.

In CSMA, nodes do not perform collision detection; both B and D continue to transmit their frames in their entirety even though a collision has occurred.
In CSMA/CD, the two nodes each abort their transmission a short time after detecting a collision.
Clearly, adding collision detection to a multiple access protocol will help protocol performance by not transmitting a useless, damaged (by interference with a frame from another node) frame in its entirety.



Before analyzing the CSMA/CD protocol, let us now summarize its operation from the perspective of an adapter (in a node) attached to a broadcast channel:

- The adapter obtains a datagram from the network layer, prepares a link-layer frame, and puts the frame adapter buffer.
- If the adapter senses that the channel is idle (that is, there is no signal energy entering the adapter from the channel), it starts to transmit the frame. If, on the other hand, the adapter senses that the channel is busy, it waits until it senses no signal energy and then starts to transmit the frame.
- While transmitting, the adapter monitors for the presence of signal energy coming from other adapters using the broadcast channel.
- If the adapter transmits the entire frame without detecting signal energy from other adapters, the adapter is finished with the frame. If, on the other hand, the adapter detects signal energy from other adapters while transmitting, it aborts the transmission (that is, it stops transmitting its frame).
- After aborting, the adapter waits a random amount of time and then returns to step 2.

The need to wait a random (rather than fixed) amount of time is hopefully clear: if two nodes transmitted frames at the same time and then both waited the same fixed amount of time, they'd continue colliding forever.

But what is a good interval of time from which to choose the random backoff time? If the interval is large and the number of colliding nodes is small, nodes are likely to wait a large amount of time (with the channel remaining idle) before repeating the sense-and-transmit-when-idle step. On the other hand, if the interval is small and the number of colliding nodes is large, it's likely that the chosen random values will be nearly the same, and transmitting nodes will again collide. What we'd like is an interval that is short when the number of colliding nodes is small, and long when the number of colliding nodes is large.
The **binary exponential backoff algorithm**, used in Ethernet as well as in DOCSIS cable network multiple access protocols, elegantly solves this problem.
Specifically, when transmitting a frame that has already experienced n collisions, a node chooses the value of K at random from $\{0,1,2,\ldots,2^{n-1}\}$ (thus, the more collisions experienced by a frame, the larger the interval from which K is chosen).
For Ethernet, the actual amount of time a node waits is $K \cdot 512 \ \ bit \ \times$¿ (i.e., K times the amount of time needed to send 512 bits into the Ethernet) and the maximum value that n can take is capped at 10.
Thus, the size of the sets from which K is chosen grows exponentially with the number of collisions; for this reason this algorithm is referred to as binary exponential backoff.

We also note here that each time a node prepares a new frame for transmission, it runs the CSMA/CD algorithm, not taking into account any collisions that may have occurred in the recent past. So it is possible that a node with a new frame will immediately be able to sneak in a successful transmission while several other nodes are in the exponential backoff state.

When only one node has a frame to send, the node can transmit at the full channel rate. However, if many nodes have frames to transmit, the effective transmission rate of the channel can be much less. We define the **efficiency of CSMA/CD** to be the long-run fraction of time during which frames are being transmitted on the channel without collisions when there is a large number of active nodes, with each node having a large number of frames to send.
In order to present a closed-form approximation of the efficiency of Ethernet, let d_{prop} denote the maximum time it takes signal energy to propagate between any two adapters. Let d_{trans} be the time to transmit a maximum-size frame (approximately 1.2 msecs for a 10 Mbps Ethernet).
The approximation of the efficiency is:

$$efficency = \frac{1}{1 + 5\left(\frac{d_{\text{prop}}}{d_{trans}}\right)}$$

The efficiency goes to 1 when:

- d_{prop} goes to 0 (if the propagation delay is zero, colliding nodes will abort immediately without wasting the channel)
- d_{trans} goes to infinity (when a frame grabs the channel, it will hold on to the channel for a very long time; thus, the channel will be doing productive work most of the time)

Taking-Turns Protocols

Recall that two desirable properties of a multiple access protocol are (1) when only one node is active, the active node has a throughput of R bps, and (2) when M nodes are active, then each active node has a throughput of nearly $\frac{R}{M}$ bps.

The channel partitioning protocols have only the second property, and the ALOHA and CSMA protocols have only the first property. This has motivated researchers to create another class of protocols: the **taking-turns protocols**.

We'll discuss two of the more important taking-turns protocols here:

- polling protocol
- token-passing protocol

The **polling protocol** requires one of the nodes to be designated as a master node. The master node **polls** each of the nodes in a round-robin fashion.

In particular, the master node first sends a message to node 1, saying that it (node 1) can transmit up to some maximum number of frames. After node 1 transmits some frames, the master node tells node 2 it (node 2) can transmit up to the maximum number of frames. (The master node can determine when a node has finished sending its frames by observing the lack of a signal on the channel.) The procedure continues in this manner, with the master node polling each of the nodes in a cyclic manner.

The polling protocol eliminates the collisions and empty slots that plague random access protocols. This allows polling to achieve a much higher efficiency. But it also has a few drawbacks:

- The protocol introduces a **polling delay**: the amount of time required to notify a node that it can transmit.
  If, for example, only one node is active, then the node will transmit at a rate less than R bps, as the master node must poll each of the inactive nodes in turn each time the active node has sent its maximum number of frames.
- If the master node fails, the entire channel becomes inoperative (single point of failure)

In the **token-passing protocol** there is no master node. A small, special-purpose frame known as a token is exchanged among the nodes in some fixed order (for example, node 1 might always send the token to node 2, node 2 might always send the token to node 3, and node N might always send the token to node 1).

When a node receives a token, it holds onto the token only if it has some frames to transmit; otherwise, it immediately forwards the token to the next node. If a node does have frames to transmit when it receives the token, it sends up to a maximum number of frames and then forwards the token to the next node.

Token passing is decentralized and highly efficient. But it has its problems as well:

- The failure of one node can crash the entire channel
- If a node accidentally neglects to release the token, then some recovery procedure must be invoked to get the token back in circulation
- The token overhead and the latency

DOCSIS: The Link-Layer Protocol for Cable Internet Access

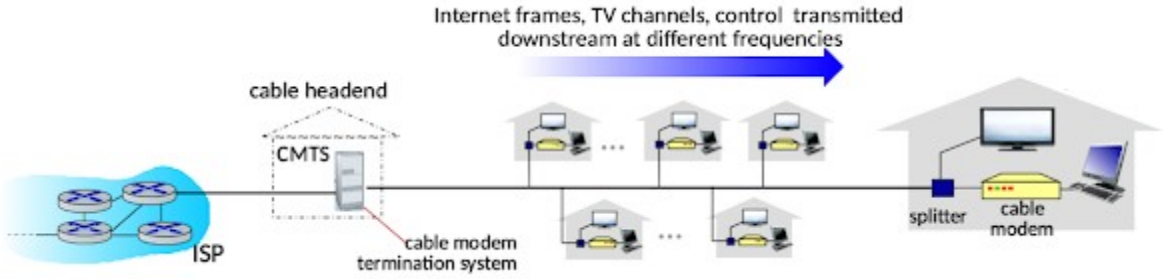
In the previous three subsections, we've learned about three broad classes of multiple access protocols: channel partitioning protocols, random access protocols, and taking turns protocols.

- channel partitioning, by time, frequency or code
  - Time Division, Frequency Division
- random access (dynamic),
  - ALOHA, S-ALOHA, CSMA, CSMA/CD
  - carrier sensing: easy in some technologies (wire), hard in others (wireless)
  - CSMA/CD used in Ethernet
  - CSMA/CA used in 802.11
- taking turns
  - polling from central site, token passing
  - Bluetooth, FDDI, token ring

A cable access network will make for an excellent case study here, as we'll find aspects of each of these three classes of multiple access protocols with the cable access network!

Recall from the first Section that a cable access network typically connects several thousand residential cable modems to a cable modem termination system (CMTS) at the cable network headend.

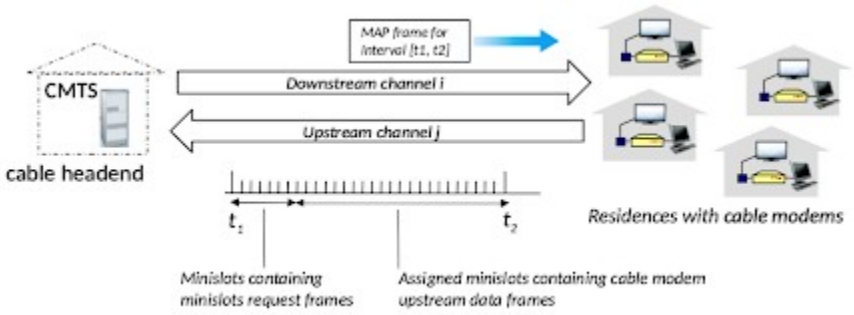
The **Data-Over-Cable Service Interface Specifications** (DOCSIS) specifies the cable data network architecture and its protocols. DOCSIS uses FDM to divide the downstream (CMTS to modem) and upstream (modem to CMTS) network segments into multiple frequency channels. Each downstream channel is 6 MHz wide, with a maximum throughput of approximately 1.6 Gbps per channel (although this data rate is seldom seen at a cable modem in practice); each upstream channel has a maximum channel width of 6.4 MHz, and a maximum upstream throughput of approximately 1 Gbps.



Each upstream and downstream channel is a broadcast channel.

Frames transmitted on the downstream channel by the CMTS are received by all cable modems receiving that channel; since there is just a single CMTS transmitting into the downstream channel, however, there is no multiple access problem.

The upstream direction, however, is more interesting and technically challenging, since multiple cable modems share the same upstream channel (frequency) to the CMTS, and thus collisions can potentially occur.



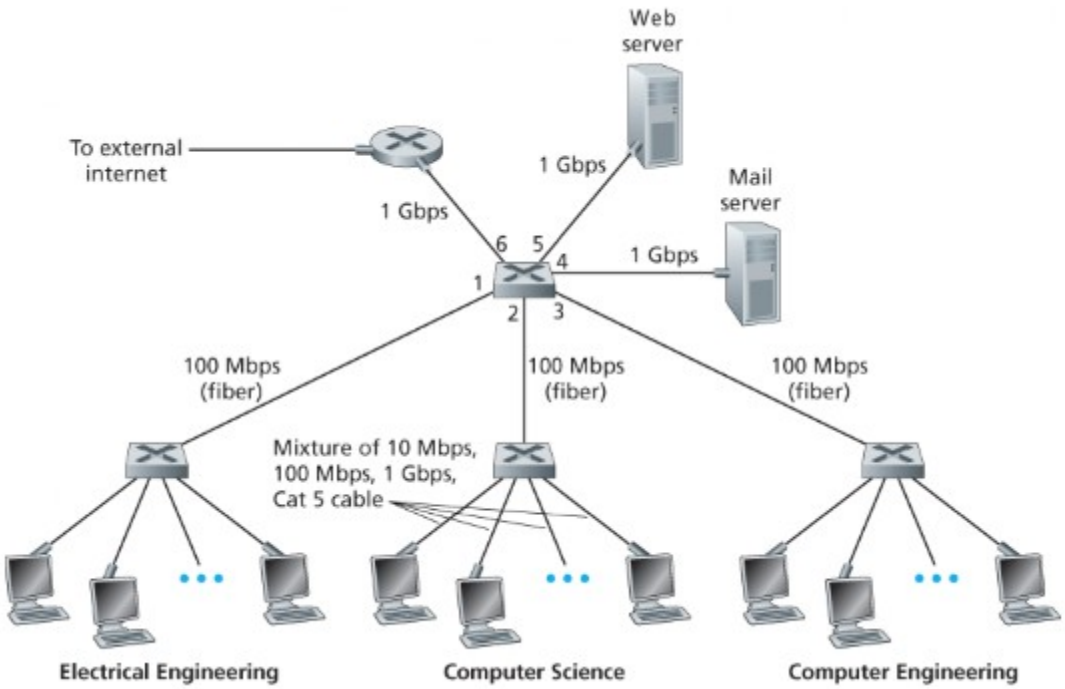
As illustrated in the figure, each upstream channel is divided into intervals of time (TDM-like), each containing a sequence of mini-slots during which cable modems can transmit to the CMTS. The CMTS explicitly grants permission to individual cable modems to transmit during specific mini-slots. The CMTS accomplishes this by sending a control message known as a MAP message on a downstream channel to specify which cable modem (with data to send) can transmit during which mini-slot for the interval of time specified in the control message. Since mini-slots are explicitly allocated to cable modems, the CMTS can ensure there are no colliding transmissions during a mini-slot.

But how does the CMTS know which cable modems have data to send in the first place? This is accomplished by having cable modems send mini-slot-request frames to the CMTS during a **special set of interval mini-slots** that are dedicated for this purpose, as shown in the figure. These mini-slot-request frames are transmitted in a random access manner and so may collide with each other.
A cable modem can neither sense whether the upstream channel is busy nor detect collisions. Instead, the cable modem infers that its mini-slot-request frame experienced a collision if it does not receive a response to the requested allocation in the next downstream control message. When a collision is inferred, a cable modem uses binary exponential backoff to defer the retransmission of its mini-slot-request frame to a future time slot. When there is little traffic on the upstream channel, a cable modem may actually transmit data frames during slots nominally assigned for mini-slot-request frames (and thus avoid having to wait for a mini-slot assignment).

A cable access network thus serves as a terrific example of multiple access protocols in action: FDM, TDM, random access, and centrally allocated time slots all within one network!

Switched Local Area Networks

Having covered broadcast networks and multiple access protocols in the previous section, let’s turn our attention next to switched local networks.



The figure shows a switched local network connecting three departments, two servers and a router with four switches. Because these switches operate at the link layer, they switch link-layer frames (rather than network-layer datagrams), don't recognize network-layer addresses, and don't use routing algorithms to determine paths through the network of layer-2 switches.

Instead of using IP addresses, we will soon see that they use link-layer addresses to forward link-layer frames through the network of switches.

Link-Layer Addressing and ARP

Hosts and routers have **link-layer addresses**. Now you might find this surprising, recalling that hosts and routers have network-layer addresses as well. You might be asking, why in the world do we need to have addresses at both the network and link layers? In addition to describing the syntax and function of the link-layer addresses, in this section we hope to shed some light on why the two layers of addresses are useful and, in fact, indispensable. We'll also cover the Address Resolution Protocol (ARP), which provides a mechanism to translate IP addresses to link-layer addresses.

In truth, it is not hosts and routers that have link-layer addresses but rather their **adapters** (that is, network interfaces) that have link-layer addresses.
A host or router with multiple network interfaces will thus have multiple link-layer addresses associated with it, just as it would also have multiple IP addresses associated with it.
It's important to note, however, that **link-layer switches do not have link-layer addresses** associated with their interfaces that connect to hosts and routers. This is because the job of the link-layer switch is to carry datagrams between hosts and routers; a switch does this job transparently, that is, without the host or router having to explicitly address the frame to the intervening switch.

A link-layer address is variously called a LAN address, a physical address, or a MAC address. Because **MAC address** seems to be the most popular term, we'll henceforth refer to link-layer addresses as MAC addresses.
For most LANs (including Ethernet and 802.11 wireless LANs), the MAC address is 6 bytes long, giving 2^{48} possible MAC addresses. These 6-byte addresses are typically expressed in hexadecimal notation, with each byte of the address expressed as a pair of hexadecimal numbers.
Although MAC addresses were designed to be permanent, it is now possible to change an adapter's MAC address via software. For the rest of this section, however, we'll assume that an adapter's MAC address is fixed.

One interesting property of MAC addresses is that no two adapters have the same address. This might seem surprising given that adapters are manufactured in many countries by many companies. How does a company manufacturing adapters in Taiwan make sure that it is using different addresses from a company manufacturing adapters in Belgium? The answer is that the IEEE manages the MAC address space.
In particular, when a company wants to manufacture adapters, it purchases a chunk of the address space consisting of 2^{24} addresses for a nominal fee. IEEE allocates the chunk of 2^{24} addresses by fixing the first 24 bits of a MAC address and letting the company create unique combinations of the last 24 bits for each adapter.

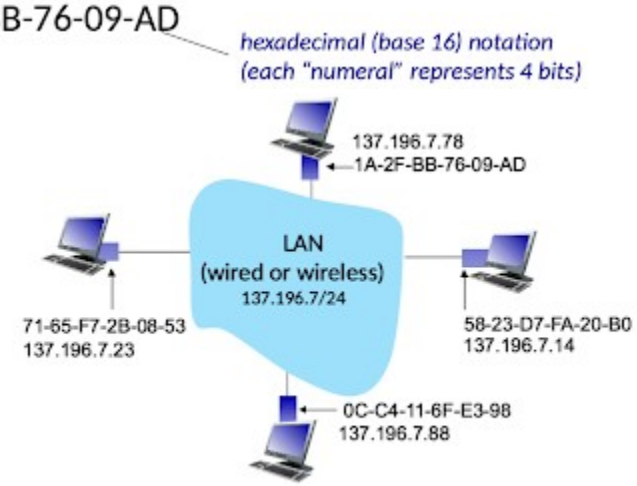
An adapter's MAC address has a flat structure (as opposed to a hierarchical structure) and doesn't change no matter where the adapter goes. A laptop with an Ethernet interface always has the same MAC address, no matter where the computer goes. A smartphone with an 802.11 interface always has the same MAC address, no matter where the smartphone goes.
Recall that, in contrast, IP addresses have a hierarchical structure (that is, a network part and a host part), and a host's IP addresses needs to be changed when the host moves, i.e., changes the network to which it is attached.
An adapter's MAC address is analogous to a person's social security number, which also has a flat addressing structure and which doesn't change no matter where the person goes. An IP address is analogous to a person's postal address, which is hierarchical and which must be changed whenever a person moves.
Just as a person may find it useful to have both a postal address and a social security number, it is useful for a host and router interfaces to have both a network-layer address and a MAC address.

When an adapter wants to send a frame to some destination adapter, the sending adapter inserts the destination adapter's MAC address into the frame and then sends the frame into the LAN. As we will soon see, a switch occasionally broadcasts an incoming frame onto all of its interfaces.
Thus, an adapter may receive a frame that isn't addressed to it. Thus, when an adapter receives a frame, it will check to see whether the destination MAC address in the frame matches its own MAC address. If there is a match, the adapter extracts the enclosed datagram and passes the

datagram up the protocol stack. If there isn't a match, the adapter discards the frame, without passing the network-layer datagram up. Thus, the destination only will be interrupted when the frame is received.

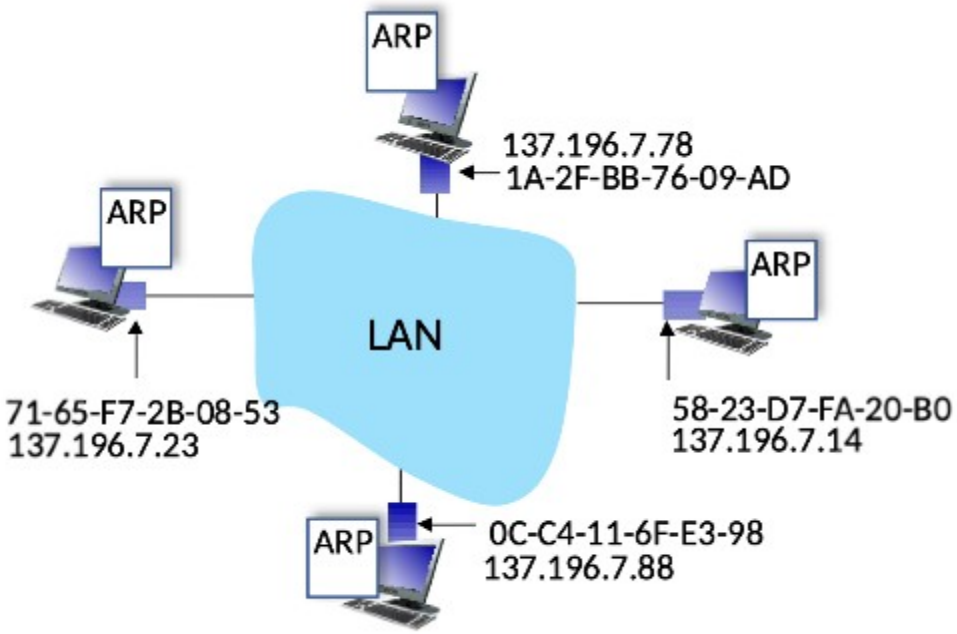
However, sometimes a sending adapter does want all the other adapters on the LAN to receive and process the frame it is about to send. In this case, the sending adapter inserts a special **MAC broadcast address** into the destination address field of the frame. For LANs that use 6-byte addresses (such as Ethernet and 802.11), the broadcast address is a string of 48 consecutive 1s (that is, FF-FF-FF-FF-FF- FF in hexadecimal notation).

- 32-bit IP address:
  - *network-layer* address for interface
  - used for layer 3 (network layer) forwarding
  - e.g.: 128.119.40.136
- MAC (or LAN or physical or Ethernet) address:
  - function: used "locally" to get frame from one interface to another physically-connected interface (same subnet, in IP-addressing sense)
  - 48-bit MAC address (for most LANs) burned in NIC ROM, also sometimes software settable
  - e.g.: 1A-2F-BB-76-09-AD



Because there are both network-layer addresses (for example, Internet IP addresses) and link-layer addresses (that is, MAC addresses), there is a need to translate between them. For the Internet, this is the job of the **Address Resolution Protocol** (ARP)

To understand the need for a protocol such as ARP, consider the network shown in the figure below. In this simple example, each host and router has a single IP address and single MAC address. As usual, IP addresses are shown in dotted-decimal notation and MAC addresses are shown in hexadecimal notation.



For the purposes of this discussion, we will assume in this section that the switch broadcasts all frames; that is, whenever a switch receives a frame on one interface, it forwards the frame on all of its other interfaces. In the next section, we will provide a more accurate explanation of how switches operate.
Now suppose that the host with IP address 137.196.7.23 wants to send an IP datagram to host 137.196.7.14. In this example, both the source and destination are in the same subnet, in the addressing sense that we've already seen. To send a datagram, the source must give its adapter not only the IP datagram but also the MAC address for destination 137.196.7.14. The sending adapter will then construct a link-layer frame containing the destination's MAC address and send the frame into the LAN.

The important question addressed in this section is, How does the sending host determine the MAC address for the destination host with IP address 137.196.7.14? As you might have guessed, it uses ARP. An ARP module in the sending host takes any IP address on the same LAN as input, and returns the corresponding MAC address. In the example at hand, sending host 137.196.7.23 provides its ARP module the IP address 137.196.7.14, and the ARP module returns the corresponding MAC address 58-23-D7-FA-20-B0.

So we see that ARP resolves an IP address to a MAC address. In many ways it is analogous to DNS, which resolves host names to IP addresses. However, one important difference between the two resolvers is that DNS resolves host names for hosts anywhere in the Internet, whereas ARP resolves IP addresses only for hosts and router interfaces on the same subnet.

Each host and router has an **ARP table** in its memory, which contains mappings of IP addresses to MAC addresses. The ARP table also contains a time-to-live (TTL) value, which indicates when each mapping will be deleted from the table.
Note that a table does not necessarily contain an entry for every host and router on the subnet; some may have never been entered into the table, and others may have expired. A typical expiration time for an entry is 20 minutes from when an entry is placed in an ARP table.

Now suppose that host 137.196.7.23 wants to send a datagram that is IP-addressed, to 137.196.7.14 and suppose that the sender's ARP table doesn't currently have an entry for the destination.
In this case, the sender uses the ARP protocol to resolve the address. First, the sender constructs a special packet called an **ARP packet**. An ARP packet has several fields, including the sending and receiving IP and MAC addresses. Both ARP query and response packets have the same format. The purpose of the ARP query packet is to query all the other hosts and routers on the subnet to determine the MAC address corresponding to the IP address that is being resolved.

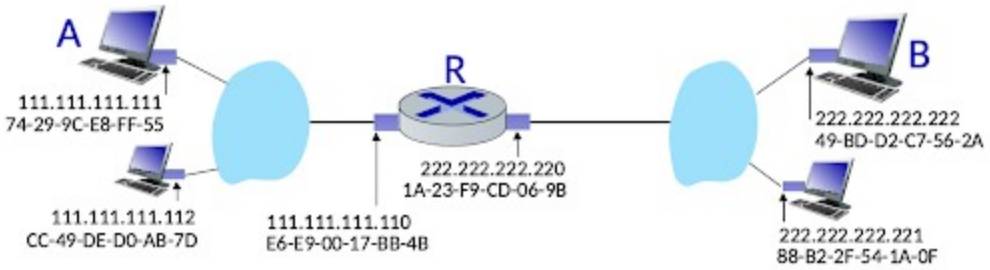
Returning to our example, 137.196.7.23 passes an ARP query packet to the adapter along with an indication that the adapter should send the packet to the MAC broadcast address, namely, FF-FF-FF-FF-FF-FF.
The adapter encapsulates the ARP packet in a link-layer frame, uses the broadcast address for the frame's destination address, and transmits the frame into the subnet.
The frame containing the ARP query is received by all the other adapters on the subnet, and (because of the broadcast address) each adapter passes the ARP packet within the frame up to its ARP module.
Each of these ARP modules checks to see if its IP address matches the destination IP address in the ARP packet. The one with a match sends back to the querying host a response ARP packet with the desired mapping. The querying host 137.196.7.23 can then update its ARP table and send its IP datagram, encapsulated in a link-layer frame whose destination MAC is that of the host or router responding to the earlier ARP query.

There are a couple of interesting things to note about the ARP protocol:

- The query ARP message is sent within a broadcast frame, whereas the response ARP message is sent within a standard frame
- ARP is plug-and-play; that is, an ARP table gets built automatically: it doesn't have to be configured by a system administrator

Students often wonder if ARP is a link-layer protocol or a network-layer protocol. As we've seen, an ARP packet is encapsulated within a link-layer frame and thus lies architecturally above the link layer. However, an ARP packet has fields containing link-layer addresses and thus is arguably a link-layer protocol, but it also contains network-layer addresses and thus is also arguably a network-layer protocol. In the end, ARP is probably best considered a protocol that straddles the boundary between the link and network layers: not fitting neatly into the simple layered protocol stack we studied. Such are the complexities of real-world protocols!

It should now be clear how ARP operates when a host wants to send a datagram to another host on the same subnet. But now let's look at the more complicated situation when a host on a subnet wants to send a network-layer datagram to a host **off the subnet**



Let's discuss this issue in the context of the figure, which shows a simple network consisting of two subnets interconnected by a router.
There are several interesting things to note:

- Each host has exactly one IP address and one adapter
- A router has an IP address for each of its interfaces
- For each router interface there is also an ARP module (in the router) and an adapter
- Subnet 1 has the network address 111.111.111/24 and that Subnet 2 has the network address 222.222.222/24.

Assume that:

- A knows B's IP address
- A knows IP address of first hop router, R
- A knows R's MAC address

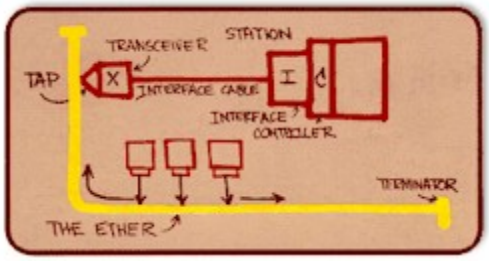
Now let's examine how a host on Subnet 1 would send a datagram to a host on Subnet 2. Specifically, suppose that host 111.111.111.111 wants to send an IP datagram to a host 222.222.222.222. The sending host passes the datagram to its adapter, as usual. But the sending host must also indicate to its adapter an appropriate destination MAC address. What MAC address should the adapter use? One might be tempted to guess that the appropriate MAC address is that of the adapter for host 222.222.222.222, namely, 49-BD-D2-C7-56-2A. This guess, however, would be wrong! If the sending adapter were to use that MAC address, then none of the adapters on Subnet 1 would bother to pass the IP datagram up to its network layer, since the frame's destination address would not match the MAC address of any adapter on Subnet 1. The datagram would just die and go to datagram heaven.

If we look carefully at the figure, we see that in order for a datagram to go from 111.111.111.111 to a host on Subnet 2, the datagram must first be sent to the router interface 111.111.111.110, which is the IP address of the first-hop router on the path to the final destination.
Thus, the appropriate MAC address for the frame is the address of the adapter for router interface 111.111.111.110, namely, E6-E9-00-17- BB-4B. How does the sending host acquire the MAC address for 111.111.111.110? By using ARP, of course!
Once the sending adapter has this MAC address, it creates a frame (containing the datagram addressed to 222.222.222.222) and sends the frame into Subnet 1. The router adapter on

Subnet 1 sees that the link-layer frame is addressed to it, and therefore passes the frame to the network layer of the router.
Hooray—the IP datagram has successfully been moved from source host to the router! But we are not finished. We still have to move the datagram from the router to the destination. The router now has to determine the correct interface on which the datagram is to be forwarded. As discussed before, this is done by consulting a forwarding table in the router. The forwarding table tells the router that the datagram is to be forwarded via router interface 222.222.222.220. This interface then passes the datagram to its adapter, which encapsulates the datagram in a new frame and sends the frame into Subnet 2. This time, the destination MAC address of the frame is indeed the MAC address of the ultimate destination. And how does the router obtain this destination MAC address? From ARP, of course!

Ethernet

Ethernet has pretty much taken over the wired LAN market. Today, Ethernet is by far the most prevalent wired LAN technology, and it is likely to remain so for the foreseeable future. One might say that Ethernet has been to local area networking what the Internet has been to global networking.



Metcalfe's Ethernet sketch

There are many reasons for Ethernet's success:

- Ethernet was the first widely deployed high-speed LAN
- Token ring, FDDI, and ATM were more complex and expensive than Ethernet, which further discouraged network administrators from switching over
- The most compelling reason to switch to another LAN technology (such as FDDI or ATM) was usually the higher data rate of the new technology; however, Ethernet always fought back, producing versions that operated at equal data rates or higher
- Because Ethernet has been so popular, Ethernet hardware (in particular, adapters and switches) has become a commodity and is remarkably cheap

The original Ethernet LAN was invented in the mid-1970s by Bob Metcalfe and David Boggs. The original Ethernet LAN used a coaxial bus to interconnect the nodes. Bus topologies for Ethernet actually persisted throughout the 1980s and into the mid-1990s. Ethernet with a bus topology is a broadcast LAN: all transmitted frames travel to and are processed by all adapters connected to the bus. All the nodes are in the same collision domain

By the late 1990s, most companies and universities had replaced their LANs with Ethernet installations using a hub-based star topology. In such an installation the hosts (and routers) are directly connected to a hub with twisted-pair copper wire. A **hub** is a physical-layer device that acts on individual bits rather than frames.

In the early 2000s Ethernet experienced yet another major evolutionary change. Ethernet installations continued to use a star topology, but the hub at the center was replaced with a **switch**. We'll be examining switched Ethernet in depth later in this chapter. For now, we only mention that a switch is not only "collision-less" but is also a bona-fide store-and-forward packet switch; but unlike routers, which operate up through layer 3, a switch operates only up through layer 2.



We can learn a lot about Ethernet by examining the Ethernet frame:



To give this discussion about Ethernet frames a tangible context, let's consider sending an IP datagram from one host to another host, with both hosts on the same Ethernet LAN (although the payload of our Ethernet frame is an IP datagram, we note that an Ethernet frame can carry other network-layer packets as well)

Let the sending adapter, adapter A, have the MAC address AA-AA-AA-AA-AA-AA and the receiving adapter, adapter B, have the MAC address BB-BB-BB-BB-BB-BB. The sending adapter encapsulates the IP datagram within an Ethernet frame and passes the frame to the physical layer. The receiving adapter receives the frame from the physical layer, extracts the IP datagram, and passes the IP datagram to the network layer.
In this context, let's now examine the six fields of the Ethernet frame:

- **Data field** (46 to 1,500 bytes)
  This field carries the IP datagram
  If the IP datagram exceeds 1,500 bytes, then the host has to fragment the datagram
  If the IP datagram is less than 46 bytes, the data field has to be "stuffed" to fill it out to 46 bytes
- **Destination address** (6 bytes)
  This field contains the MAC address of the destination adapter, BB-BB-BB-BB-BB-BB. When adapter B receives an Ethernet frame whose destination address is either BB-BB-BB-BB-BB-BB or the MAC broadcast address, it passes the contents of the frame's data field to the network layer; if it receives a frame with any other MAC address, it discards the frame.
- **Source address** (6 bytes)
  This field contains the MAC address of the adapter that transmits the frame onto the LAN, in this example, AA-AA-AA-AA-AA-AA.

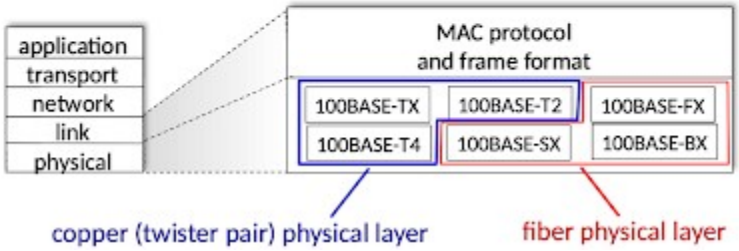
- **Type field** (2 bytes)
  The type field permits Ethernet to multiplex network-layer protocols. To understand this, we need to keep in mind that hosts can use other network-layer protocols besides IP. In fact, a given host may support multiple network-layer protocols using different protocols for different applications.
  For this reason, when the Ethernet frame arrives at adapter B, adapter B needs to know to which network-layer protocol it should pass (that is, demultiplex) the contents of the data field.
  Note that the type field is analogous to the protocol field in the network-layer datagram and the port-number fields in the transport-layer segment; all of these fields serve to glue a protocol at one layer to a protocol at the layer above.
- **Cyclic redundancy check** (CRC) (4 bytes)
  As discussed before, the purpose of the CRC field is to allow the receiving adapter, adapter B, to detect bit errors in the frame. If there are errors, the frame is dropped
- **Preamble** (8 bytes)
  The Ethernet frame begins with an 8-byte preamble field. Each of the first 7 bytes of the preamble has a value of 10101010; the last byte is 10101011.
  The first 7 bytes of the preamble serve to “wake up” the receiving adapters and to synchronize their clocks to that of the sender’s clock. Why should the clocks be out of synchronization?
  Keep in mind that adapter A aims to transmit the frame at 10 Mbps, 100 Mbps, or 1 Gbps, depending on the type of Ethernet LAN. However, because nothing is absolutely perfect, adapter A will not transmit the frame at exactly the target rate; there will always be some drift from the target rate, a drift which is not known a priori by the other adapters on the LAN. A receiving adapter can lock onto adapter A’s clock simply by locking onto the bits in the first 7 bytes of the preamble. The last 2 bits of the eighth byte of the preamble (the first two consecutive 1s) alert adapter B that the “important stuff” is about to come.

All of the Ethernet technologies provide **connectionless service** to the network layer. That is, when adapter A wants to send a datagram to adapter B, adapter A encapsulates the datagram in an Ethernet frame and sends the frame into the LAN, without first handshaking with adapter B.
Ethernet technologies provide an **unreliable service** to the network layer. Specifically, when adapter B receives a frame from adapter A, it runs the frame through a CRC check, but neither sends an acknowledgment when a frame passes the CRC check nor sends a negative acknowledgment when a frame fails the CRC check. When a frame fails the CRC check, adapter B simply discards the frame.
This lack of reliable transport (at the link layer) helps to make Ethernet simple and cheap. But it also means that the stream of datagrams passed to the network layer can have gaps.
If there are gaps, the behavior depends on the transport layer protocol (TCP or UDP)

In our discussion above, we’ve referred to Ethernet as if it were a single protocol standard. But in fact, Ethernet comes in many different flavors.

Ethernet is both a link-layer and a physical-layer specification and is carried over a variety of physical media including coaxial cable, copper wire, and fiber.

- *many* different Ethernet standards
  - common MAC protocol and frame format
  - different speeds: 2 Mbps, 10 Mbps, 100 Mbps, 1Gbps, 10 Gbps, 40 Gbps
  - different physical layer media: fiber, cable



In the days of bus topologies and hub-based star topologies, Ethernet was clearly a broadcast link (as defined in Section 6.3) in which frame collisions occurred when nodes transmitted at the same time. To deal with these collisions, the Ethernet standard included the CSMA/CD protocol, which is particularly effective for a wired broadcast LAN spanning a small geographical region.

Furthermore, modern switches are full-duplex, so that a switch and a node can each send frames to each other at the same time without interference. In other words, in a switch-based Ethernet LAN there are no collisions and, therefore, there is no need for a MAC protocol!

As we've seen, today's Ethernets are very different from the original Ethernet conceived by Metcalfe and Boggs more than 30 years ago—speeds have increased by three orders of magnitude, Ethernet frames are carried over a variety of media, switched-Ethernets have become dominant, and now even the MAC protocol is often unnecessary! Is all of this really still Ethernet? The answer, of course, is "yes, by definition." It is interesting to note, however, that through all of these changes, there has indeed been one enduring constant that has remained unchanged over 30 years: Ethernet's frame format. Perhaps this then is the one true and timeless centerpiece of the Ethernet standard.

Link-Layer switches

Up until this point, we have been purposefully vague about what a switch actually does and how it works. The role of the switch is to receive incoming link-layer frames and forward them onto outgoing links.

We'll see that the switch itself is **transparent** to the hosts and routers in the subnet; that is, a host/router addresses a frame to another host/router (rather than addressing the frame to the switch) and happily sends the frame into the LAN, unaware that a switch will be receiving the frame and forwarding it.

The rate at which frames arrive to any one of the switch's output interfaces may temporarily exceed the link capacity of that interface. To accommodate this problem, switch output

interfaces have buffers, in much the same way that router output interfaces have buffers for datagrams.
Filtering is the switch function that determines whether a frame should be forwarded to some interface or should just be dropped. **Forwarding** is the switch function that determines the interfaces to which a frame should be directed, and then moves the frame to those interfaces. Switch filtering and forwarding are done with a switch table. The switch table contains entries for some, but not necessarily all, of the hosts and routers on a LAN.

Address	Interface	Time
62-FE-F7-11-89-A3	1	9:32
7C-BA-B2-B4-91-10	3	9:36
....	....	....

An entry in the switch table contains:

1. a MAC address
2. the switch interface that leads toward that MAC address
3. the time at which the entry was placed in the table

The important distinction between switches and routers is that switches forward packets based on MAC addresses rather than on IP addresses.

To understand how switch filtering and forwarding work, suppose a frame with destination address DD-DD-DD-DD-DD-DD arrives at the switch on interface x. The switch indexes its table with the MAC address DD-DD-DD-DD-DD-DD. There are three possible cases:

1. There is no entry in the table for DD-DD-DD-DD-DD-DD. In this case, the switch forwards copies of the frame to the output buffers preceding all interfaces except for interface x. In other words, if there is no entry for the destination address, the switch broadcasts the frame.
2. There is an entry in the table, associating DD-DD-DD-DD-DD-DD with interface x. In this case, the frame is coming from a LAN segment that contains adapter DD-DD-DD-DD-DD-DD. There being no need to forward the frame to any of the other interfaces, the switch performs the filtering function by discarding the frame.
3. There is an entry in the table, associating DD-DD-DD-DD-DD-DD with interface y≠x. In this case, the frame needs to be forwarded to the LAN segment attached to interface y. The switch performs its forwarding function by putting the frame in an output buffer that precedes interface y.

A switch has the wonderful property (particularly for the already-overworked network administrator) that its table is built automatically, dynamically, and autonomously (without any intervention from a network administrator or from a configuration protocol). In other words, switches are **self-learning**.
This capability is accomplished as follows:

1. The switch table is initially empty.
2. For each incoming frame received on an interface, the switch stores in its table (1) the MAC address in the frame's source address field, (2) the interface from which the frame

arrived, and (3) the current time. In this manner the switch records in its table the LAN segment on which the sender resides. If every host in the LAN eventually sends a frame, then every host will eventually get recorded in the table.

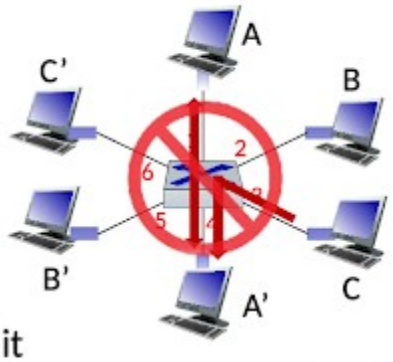
3. The switch deletes an address in the table if no frames are received with that address as the source address after some period of time (the aging time). In this manner, if a PC is replaced by another PC (with a different adapter), the MAC address of the original PC will eventually be purged from the switch table.

Switches are **plug-and-play devices** because they require no intervention from a network administrator or user. A network administrator wanting to install a switch need do nothing more than connect the LAN segments to the switch interfaces. The administrator need not configure the switch tables at the time of installation or when a host is removed from one of the LAN segments. Switches are also full-duplex, meaning any switch interface can send and receive at the same time.

Having described the basic operation of a link-layer switch, let's now consider their features and properties. We can identify several advantages of using switches, rather than broadcast links such as buses or hub-based star topologies:

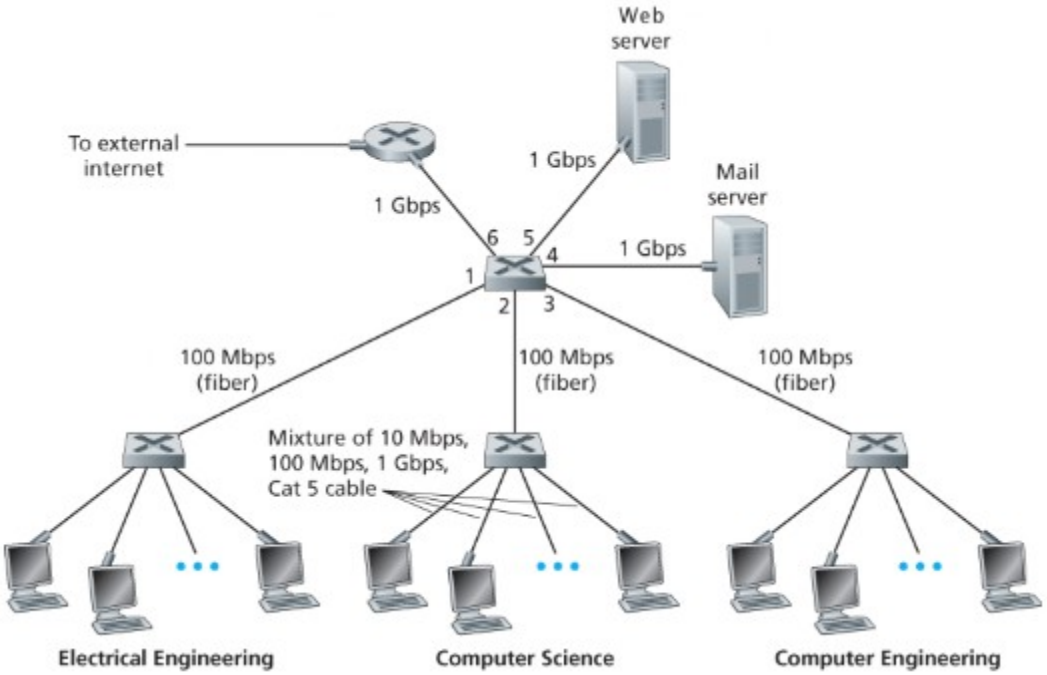
- **Elimination of collisions**
  In a LAN built from switches (and without hubs), there is no wasted bandwidth due to collisions! The switches buffer frames and never transmit more than one frame on a segment at any one time. As with a router, the maximum aggregate throughput of a switch is the sum of all the switch interface rates. Thus, switches provide a significant performance improvement over LANs with broadcast links.
- **Heterogeneous links**
  Because a switch isolates one link from another, the different links in the LAN can operate at different speeds and can run over different media
  Thus, a switch is ideal for mixing legacy equipment with new equipment
- **Management**
  In addition to providing enhanced security (see sidebar on Focus on Security), a switch also eases network management. For example, if an adapter malfunctions and continually sends Ethernet frames (called a jabbering adapter), a switch can detect the problem and internally disconnect the malfunctioning adapter.
  Switches also gather statistics on bandwidth usage, collision rates, and traffic types, and make this information available to the network manager. This information can be used to debug and correct problems, and to plan how the LAN should evolve in the future.

- hosts have dedicated, direct connection to switch
- switches buffer packets
- Ethernet protocol used on *each* incoming link, so:
  - no collisions; full duplex
  - each link is its own collision domain
- switching: A-to-A' and B-to-B' can transmit simultaneously, without collisions
  - but A-to-A' and C to A' can *not* happen simultaneously



switch with six interfaces (1,2,3,4,5,6)

As we learned, routers are store-and-forward packet switches that forward packets using network-layer addresses. Although a switch is also a store-and-forward packet switch, it is fundamentally different from a router in that it forwards packets using MAC addresses. Whereas a router is a layer-3 packet switch, a switch is a layer-2 packet switch.
Even though switches and routers are fundamentally different, network administrators must often choose between them when installing an interconnection device.



For example, for the network in the figure, the network administrator could just as easily have used a router instead of a switch to connect the department LANs, servers, and internet gateway router. Indeed, a router would permit interdepartmental communication without creating collisions. Given that both switches and routers are candidates for interconnection devices, what are the pros and cons of the two approaches?
Switches:

- are plug-and-play, a property that is cherished by all the overworked network administrators of the world
- can also have relatively high filtering and forwarding rates (switches have to process frames only up through layer 2, whereas routers have to process datagrams up through layer 3)
- to prevent the cycling of broadcast frames, the active topology of a switched network is restricted to a spanning tree
- a large switched network would require large ARP tables in the hosts and routers and would generate substantial ARP traffic and processing
- are susceptible to broadcast storms (if one host goes haywire and transmits an endless stream of Ethernet broadcast frames, the switches will forward all of these frames, causing the entire network to collapse)

Routers:

- Because network addressing is often hierarchical (and not flat, as is MAC addressing), packets do not normally cycle through routers even when the network has redundant paths
- packets are not restricted to a spanning tree and can use the best path between source and destination
- they provide firewall protection against layer-2 broadcast storms
- they are not plug-and-play (they and the hosts that connect to them need their IP addresses to be configured)
- often have a larger per-packet processing time than switches, because they have to process up through the layer-3 fields

	Hubs	Routers	Switches
Traffic isolation	No	Yes	Yes
Plug and play	Yes	No	Yes
Optimal routing	No	Yes	No

Typically, small networks consisting of a few hundred hosts have a few LAN segments. Switches suffice for these small networks, as they localize traffic and increase aggregate throughput without requiring any configuration of IP addresses. But larger networks consisting of thousands of hosts typically include routers within the network (in addition to switches). The routers provide a more robust isolation of traffic, control broadcast storms, and use more “intelligent” routes among the hosts in the network.

Data Center Networking

In recent years, Internet companies such as Google, Microsoft, Facebook, and Amazon (as well as their counterparts in Asia and Europe) have built massive data centers, each housing tens to hundreds of thousands of hosts, and concurrently supporting many distinct cloud applications (e.g., search, e-mail, social networking, and e-commerce). Each data center has its own **data**

center network that interconnects its hosts with each other and interconnects the data center with the Internet. In this section, we provide a brief introduction to data center networking for cloud applications.

The cost of a large data center is huge, exceeding $12 million per month for a 100,000 host data center:

- 45% attributed to the hosts themselves (which need to be replaced every 3–4 years)
- 25% to infrastructure, including transformers, uninterruptable power supplies (UPS) systems, generators for long-term outages, and cooling systems
- 15% for electric utility costs for the power draw
- 15% for networking, including network gear (switches, routers and load balancers), external links, and transit traffic costs

While networking is not the largest cost, networking innovation is the key to reducing overall cost and maximizing performance

The challenges are:

- There are multiple applications, each serving massive number of clients
- The reliability
- The managing and the balancing of the load, avoiding processing, networking and data bottlenecks

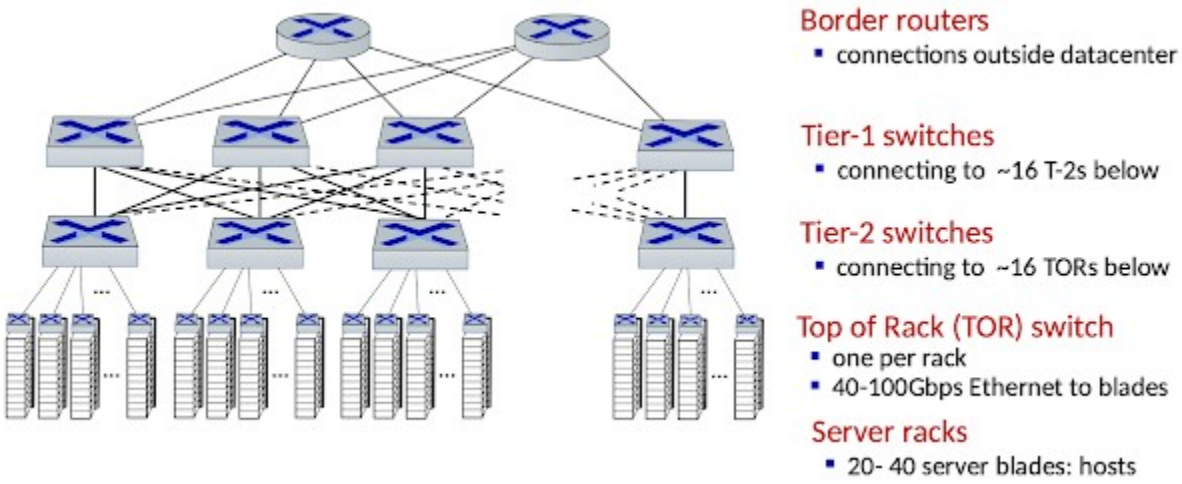
The worker bees in a data center are the hosts (they serve content, store e-mails and documents, and collectively perform massively distributed computations).
The hosts in data centers, called blades and resembling pizza boxes, are generally commodity hosts that include CPU, memory, and disk storage. The hosts are stacked in **racks**, with each rack typically having 20 to 40 blades.
At the top of each rack there is a switch, aptly named the **Top of Rack (TOR) switch**, that interconnects the hosts in the rack with each other and with other switches in the data center Specifically, each host in the rack has a network interface card that connects to its TOR switch, and each TOR switch has additional ports that can be connected to other switches. Today hosts typically have 40 Gbps Ethernet connections to their TOR switches

The data center network supports two types of traffic:

- traffic flowing between external clients and internal hosts
- traffic flowing between internal hosts.

To handle flows between external clients and internal hosts the data center network includes one or more **border routers**, connecting the data center network to the public Internet.
The data center network therefore interconnects the racks with each other and connects the racks to the border routers.

For a small data center housing only a few thousand hosts, a simple network consisting of a border router, a load balancer, and a few tens of racks all interconnected by a single Ethernet switch could possibly suffice. But to scale to tens to hundreds of thousands of hosts, a data center often employs a **hierarchy of routers and switches**



Each access router connects to a top-tier switch, and each top-tier switch connects to multiple second-tier switches and a load balancer. Each second-tier switch in turn connects to multiple racks via the racks' TOR switches (third-tier switches). All links typically use Ethernet for their link-layer and physical-layer protocols, with a mix of copper and fiber cabling. With such a hierarchical design, it is possible to scale a data center to hundreds of thousands of hosts.

Data center network design, the art of designing the interconnection network and protocols that connect the racks with each other and with the border routers, has become an important branch of computer networking research in recent years.

A cloud data center, such as a Google or Microsoft data center, provides many applications concurrently, such as search, e-mail, and video applications. To support requests from external clients, each application is associated with a publicly visible IP address to which clients send their requests and from which they receive responses. Inside the data center, the external requests are first directed to a **load balancer** whose job it is to distribute requests to the hosts, balancing the load across the hosts as a function of their current load.

Facebook F16 data center network topology:

